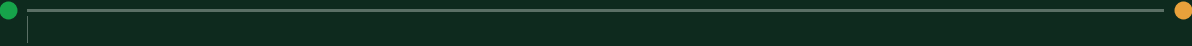


TURIFY



Referencia técnica del proyecto

Clases, funciones, métodos y variables de las dos aplicaciones, leídas del código y ordenadas para repasar.

Rama `main` · commit `72c8884`

2 de septiembre de 2026

Universidad Pascual Bravo

Índice

Cómo leer este documento	8
Qué es Turify	8
Los tres roles	8
El ciclo de vida de un viaje	9
La arquitectura	9
La estructura del repositorio	10
Cómo se levanta el proyecto	12
Con Docker	12
A mano	12
Variables de entorno	12
Convenciones del proyecto	13
El modelo de datos y el núcleo del backend	14
Modelos (<code>app/models.py</code>)	14
Esquemas Pydantic (<code>app/schemas.py</code>)	25
<code>app/security.py</code>	35
<code>app/audit.py</code>	36
<code>app/database.py</code>	38
<code>app/main.py</code>	39
<code>requirements.txt</code>	40
Los endpoints del backend	42
<code>app/routers/auth.py</code>	42
Router	42
Constantes de módulo	42
Funciones auxiliares privadas	42
<code>POST /users/register</code>	43
<code>POST /users/login</code>	43
<code>GET /users/me</code>	44
<code>GET /users/me/profile</code>	44
<code>PATCH /users/me/phone</code>	45
<code>PATCH /users/me/password</code>	45
<code>app/routers/drivers.py</code>	46
Router	46

Constantes de módulo	46
Funciones auxiliares (todas públicas, ninguna empieza con <code>_</code> en este archivo)	46
<code>POST /drivers/register-details</code>	47
<code>POST /drivers/upload-runt</code>	49
<code>GET /drivers/my-documents</code>	50
<code>GET /drivers/registration-status</code>	51
<code>PATCH /drivers/location</code>	51
<code>GET /drivers/vehicle</code>	52
<code>PATCH /drivers/vehicle</code>	52
<code>GET /drivers/earnings</code>	53
<code>GET /drivers/{driver_id}/public-profile</code>	54
<code>app/routers/admin.py</code>	55
Router	55
Esquemas locales (definidos en este archivo, no en <code>schemas.py</code>)	55
Dependencia de rol	56
Constantes de módulo	56
<code>GET /admin/drivers/pending</code>	56
<code>PATCH /admin/documents/{document_id}/verify</code>	56
<code>GET /admin/drivers/{user_id}/documents</code>	57
<code>GET /admin/documents/{document_id}/file</code>	58
<code>GET /admin/logs</code>	58
<code>app/routers/service_requests.py</code>	59
Router	59
Constantes de módulo	59
Funciones auxiliares	59
Estados de la máquina de negociación	61
<code>POST /api/service-requests/</code>	61
<code>GET /api/service-requests/pending</code>	63
<code>GET /api/service-requests/demand-zones</code>	64
<code>POST /api/service-requests/{request_id}/offers</code>	64
<code>GET /api/service-requests/driver/history</code>	65
<code>GET /api/service-requests/driver/my-offers</code>	66
<code>GET /api/service-requests/notifications</code>	66
<code>PATCH /api/service-requests/notifications/{notification_id}/read</code>	66
<code>GET /api/service-requests/assigned</code>	67
<code>GET /api/service-requests/{request_id}/receipt</code>	67
<code>GET /api/service-requests/{request_id}/offers</code>	68
<code>PATCH /api/service-requests/offers/{offer_id}/counter-offer</code>	69
<code>PATCH /api/service-requests/offers/{offer_id}/accept</code>	70
<code>PATCH /api/service-requests/offers/{offer_id}/resolve</code>	71

<code>PATCH /api/service-requests/{request_id}/cancel</code>	72
<code>PATCH /api/service-requests/{request_id}/start</code>	72
<code>PATCH /api/service-requests/{request_id}/complete</code>	73
<code>GET /api/service-requests/{request_id}/status</code>	73
<code>POST /api/service-requests/{request_id}/passengers</code>	74
<code>GET /api/service-requests/{request_id}/passengers</code>	75
<code>POST /api/service-requests/{request_id}/rating</code>	75
<code>GET /api/service-requests/{request_id}/rating</code>	76

El sistema de diseño y el arranque del frontend 77

Sistema de diseño (<code>diseno.jsx</code> + <code>index.css</code>)	77
El objeto <code>T</code> completo	77
El objeto <code>FIJO</code>	79
Paleta completa de <code>index.css</code> (claro vs. oscuro)	80
Reglas globales de <code>index.css</code>	81
Las tres familias tipográficas	81
El componente <code>Icono</code> y el catálogo completo	82
Componentes exportados	83
Tema: <code>leerTema</code> , <code>aplicarTema</code> , <code>useTema</code>	87
<code>MAPA_OSCURO</code>	88
Clases CSS globales	89
Arranque y enrutado	89
Punto de entrada	89
<code>App.jsx</code> — tabla de rutas	90
<code>AuthContext.jsx</code>	91
<code>AuthProvider.jsx</code>	91
<code>RutaPrivada.jsx</code>	92
<code>api.js</code>	93
Utilidades de interfaz	93
<code>Toast.jsx</code>	93
<code>Skeleton.jsx</code>	95

La pantalla del pasajero 97

<code>Dashboard.jsx</code> — Importaciones	97
Librerías externas	97
Contexto y componentes propios del proyecto	97
Del sistema de diseño (<code>./diseno</code>)	98
Constantes de módulo	98
Componentes auxiliares definidos en el archivo	99

<code>ModalErrorDireccion</code> (nivel de módulo, líneas 39–72)	99
<code>SelectorPasajero</code> (definido dentro de <code>Dashboard</code> , líneas 932–947)	99
El componente <code>Dashboard</code>	100
Estados (<code>useState</code>)	100
<code>useRef</code> , <code>useContext</code> y otros hooks	103
Constantes derivadas dentro del componente	103
Funciones definidas dentro del componente	104
<code>useEffect</code>	108
Suscripciones a Supabase Realtime	108
Estructura de vistas y pestañas	109
Modales y cuándo aparecen	111
Flujos completos, paso a paso	111
Publicar un viaje	111
Recibir y evaluar ofertas	112
Contraofertar	112
Aceptar una oferta	112
Registrar ocupantes (FUEC)	113
"Ver el FUEC"	113
Seguir al conductor en el mapa (HU43)	113
Calificar (HU46)	113
Cancelar	113
<code>InputDireccion.jsx</code>	114
Props	114
Estados y refs	114
Constantes y funciones	115
Efectos y render	115
<code>SelectorFechaHora.jsx</code>	115
Constantes y ayudantes de módulo	115
Props	116
Estados, refs y valores derivados	116
Funciones y efectos	117
Render	117
Las pantallas del conductor	118
<code>PanelConductor.jsx</code>	118
Importaciones y constantes de módulo	118
Componentes auxiliares definidos en el archivo	120
El componente principal <code>PanelConductor</code>	120
Estructura de la interfaz	128

<code>PerfilDrawer.jsx</code>	132
Importaciones y constantes de módulo	132
Componentes auxiliares definidos en el archivo	133
El componente principal <code>PerfilDrawer</code>	133
Estructura de la interfaz	136
<code>FormularioConductor.jsx</code>	138
Importaciones y constantes de módulo	138
Componentes auxiliares definidos en el archivo	139
El componente principal <code>FormularioConductor</code>	140
Estructura de la interfaz	142
Acceso, administración y perfil público	145
Pantallas de acceso	145
<code>Login.jsx</code>	145
<code>Registro.jsx</code>	147
Pantallas de administrador	149
<code>AdminConductores.jsx</code> (ruta <code>/admin/conductores</code>)	149
<code>AdminLogs.jsx</code> (ruta <code>/admin/logs</code>)	153
Perfil público del conductor	155
Diferencia entre los dos	155
<code>PerfilConductorPagina.jsx</code>	156
<code>PerfilConductorPublico.jsx</code>	157
Observaciones y deuda técnica	159
Backend · modelo de datos y núcleo	159
Código duplicado / muerto	159
Comentarios que contradicen el código	159
Inconsistencias de esquema/modelo	160
Seguridad	160
Versiones sospechosas en <code>requirements.txt</code>	161
Backend · endpoints	161
Código muerto y contradicciones dentro de los cuatro archivos	161
Endpoints sin consumidor en el frontend	163
Huecos de autorización que conviene conocer	163
Frontend · pantalla del pasajero	164
Código muerto o sin uso	164
Bugs probables	164
Inconsistencias de estilo y contenido	165
Cosas que el código no permite determinar	166

Frontend · pantallas del conductor	166
PanelConductor.jsx	166
PerfilDrawer.jsx	167
FormularioConductor.jsx	168
Transversal	169
Frontend · sistema de diseño y pantallas auxiliares	169

Cómo leer este documento

Esto es una referencia, no un tutorial. Está escrita a partir del código tal como está en la rama `main` (commit `72c8884`, 2 de septiembre de 2026), leyendo archivo por archivo: cada nombre de clase, función, variable, ruta y mensaje de error que aparece acá está copiado del código, no reconstruido de memoria.

Las siete partes se pueden leer en cualquier orden:

Parte	Qué encontrás
1	Las doce tablas de la base de datos, los esquemas de validación, la autenticación y la auditoría
2	Los 42 endpoints, uno por uno: quién puede llamarlos, qué hacen y qué devuelven
3	El sistema de diseño, el enrutado y el manejo de la sesión
4	La pantalla del pasajero: estados, funciones, efectos y los flujos completos
5	Las pantallas del conductor: panel de cinco pestañas, perfil y formulario de registro
6	Login, registro, las dos vistas de administrador y el perfil público
7	Lo que se encontró de paso: código muerto, contradicciones y huecos conocidos

Si vas a repasar para explicar el proyecto, el camino más corto es: esta introducción, después los flujos completos de la parte 4, y después la parte 2 para saber qué hace el servidor detrás de cada paso.

Una advertencia sobre la parte 7: no está ahí para señalar errores, sino porque en un proyecto real la deuda técnica es parte de lo que hay que saber. Un comentario que contradice al código engaña más que la ausencia de comentario.

Qué es Turify

Turify conecta pasajeros y conductores de transporte especial en Medellín y Antioquia, con un giro respecto a las aplicaciones de transporte conocidas: **el precio se negocia**. El pasajero publica el viaje que necesita, los conductores cercanos ofertan, y ambas partes pueden contraofertar hasta acordar.

Eso responde a un contexto concreto. En veredas y corregimientos la dirección exacta muchas veces no existe, el trayecto no es un recorrido urbano estándar y la tarifa depende del estado de la vía, los peajes y la temporada. Un precio fijo calculado por kilómetro no funciona ahí; una negociación sí.

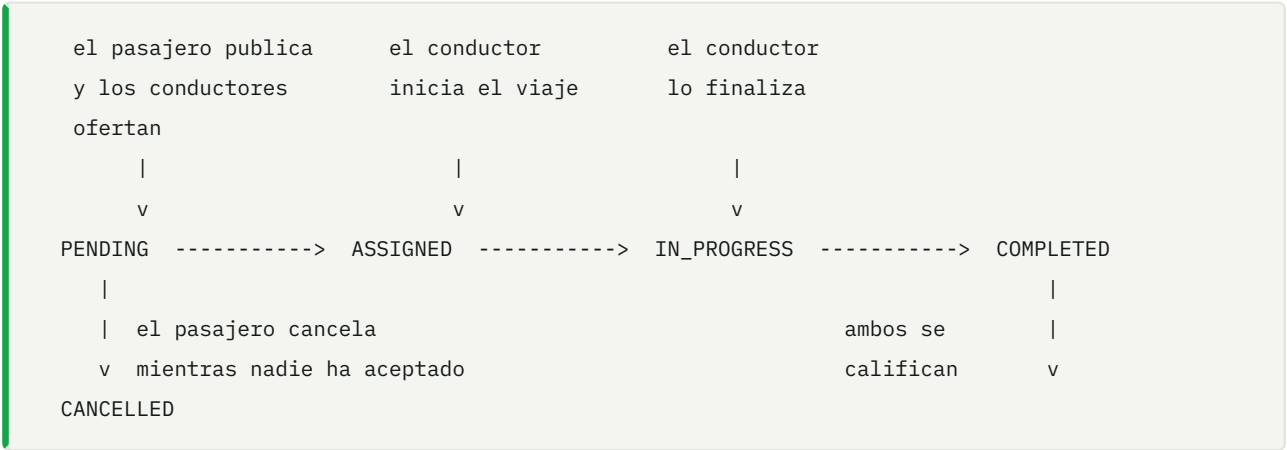
Los tres roles

Rol	Qué puede hacer
PASSENGER	Publica solicitudes de viaje, recibe ofertas, contraoferta, acepta, registra a los ocupantes y califica
DRIVER	Ve las solicitudes de su zona, oferta, responde contraofertas, inicia y finaliza el viaje, y califica

Rol	Qué puede hacer
ADMIN	Verifica los documentos de los conductores y consulta la bitácora de auditoría

Un usuario nace como `PASSENGER`. Para volverse `DRIVER` sube sus documentos y espera a que un administrador los apruebe: el cambio de rol lo hace el backend cuando todos los documentos quedan en `APPROVED`.

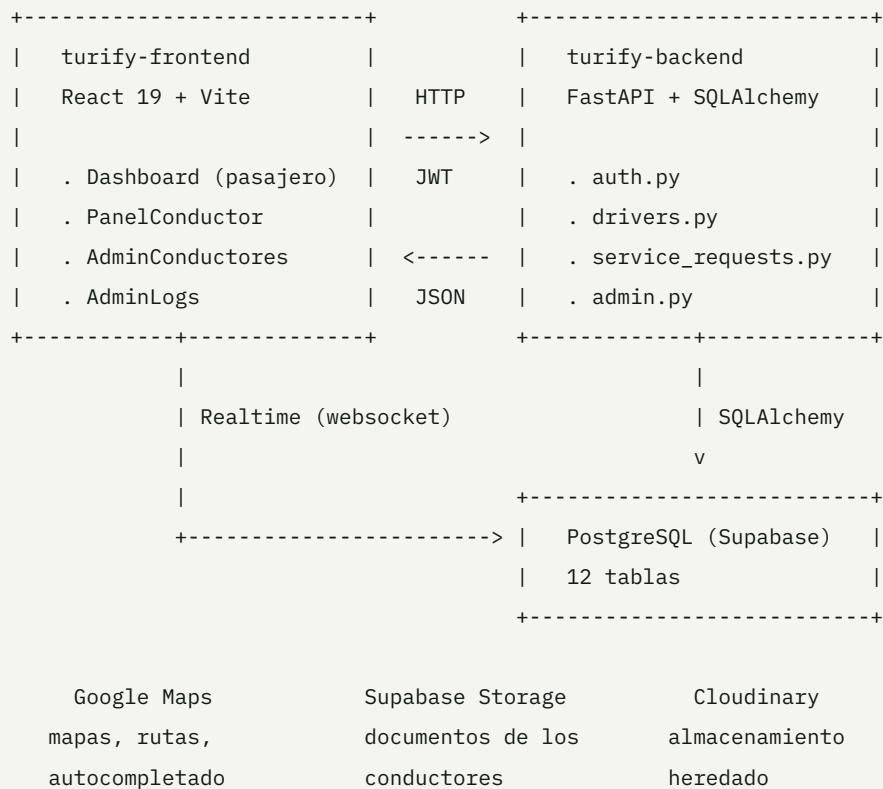
El ciclo de vida de un viaje



En paralelo, cada oferta del conductor tiene su propio estado: `PENDING` cuando llega, `COUNTERED` si el pasajero contraoferta, y `ACCEPTED` o `REJECTED` al cerrarse. La oferta aceptada es la que fija el precio final del viaje: el resto quedan rechazadas.

La arquitectura

Son dos aplicaciones separadas dentro de un mismo repositorio, más tres servicios externos.



Cómo se hablan las dos mitades. El frontend guarda el token JWT en `localStorage` y lo manda en la cabecera `Authorization: Bearer <token>` de cada petición. El backend lo decodifica en `get_current_user`, carga el usuario de la base de datos y lo inyecta en el endpoint. No hay sesión en el servidor: el token lleva el `user_id` en el campo `sub` y una fecha de expiración. Son 42 endpoints repartidos en cuatro routers: 6 de autenticación, 9 de conductor, 22 de viajes y 5 de administración.

Cómo se encuentran pasajero y conductor. No hay un radar de conductores disponibles. Cada solicitud guarda su propio punto de búsqueda y un radio de visibilidad — `RADIO_VISIBILIDAD_KM`, hoy 60 km, fijado por el backend al crear la solicitud— y `GET /api/service-requests/pending` le devuelve a cada conductor las solicitudes cuyo radio cubre su posición actual, ordenadas de más cerca a más lejos con la fórmula haversine. Un conductor que no reportó ubicación las ve todas, solo que sin ese orden.

Dónde entra el tiempo real. Dos cosas no esperan a que el usuario recargue: las notificaciones y la posición del conductor durante el viaje. Ambas usan Supabase Realtime, que abre un websocket contra la base de datos y avisa cuando cambia una fila. Si no hay credenciales de Supabase configuradas, el código cae a consultar cada quince segundos.

La estructura del repositorio

Un solo repositorio de git, con el `.git` en la raíz. Las dos carpetas no son repositorios separados: cualquier comando de git ejecutado dentro de una de ellas opera sobre el proyecto completo.

```

Turify/
|-- docker-compose.yml           postgres + backend + frontend
|-- README.md
|-- turify-backend/
|   |-- Dockerfile
|   |-- requirements.txt
|   |-- app/
|       |-- main.py              instancia FastAPI, CORS, routers
|       |-- database.py          conexion y sesion de SQLAlchemy
|       |-- models.py            las 12 tablas
|       |-- schemas.py           validacion y respuestas (Pydantic)
|       |-- security.py          hash de contraseñas y JWT
|       |-- audit.py             registrar_log
|       |-- routers/
|           |-- auth.py          registro, login, perfil
|           |-- drivers.py       documentos, vehiculo, ganancias
|           |-- service_requests.py viajes, ofertas, FUEC, calificaciones
|           |-- admin.py         verificacion y bitacora
|-- turify-frontend/
    |-- vite.config.js
    |-- scripts/
    |   |-- verificar-referencias.py
    |   |-- eslint.undef.mjs
    |-- src/
        |-- main.jsx             punto de entrada
        |-- App.jsx              rutas
        |-- AuthProvider.jsx     sesion y token
        |-- RutaPrivada.jsx      proteccion por rol
        |-- diseno.jsx           sistema de diseno completo
        |-- index.css            paletas clara y oscura
        |-- Dashboard.jsx        pantalla del pasajero
        |-- PanelConductor.jsx    pantalla del conductor
        |-- PerfilDrawer.jsx      cajon de perfil (ambos roles)
        |-- FormularioConductor.jsx
        |-- AdminConductores.jsx
        |-- AdminLogs.jsx
        |-- Login.jsx            Registro.jsx
        |-- InputDireccion.jsx    SelectorFechaHora.jsx
        |-- Toast.jsx            Skeleton.jsx
        |-- PerfilConductorPagina.jsx PerfilConductorPublico.jsx

```

Cómo se levanta el proyecto

Con Docker

```
docker-compose up --build      # levanta postgres, backend y frontend
docker-compose up -d           # en segundo plano
docker-compose logs backend    # ver los registros del backend
docker-compose down            # detener todo
```

Quedan en `localhost:3000` el frontend, `localhost:8000` el backend y `localhost:5432` la base de datos.

A mano

```
# Backend
cd turify-backend
python -m venv venv
venv\Scripts\activate          # Windows
pip install -r requirements.txt
uvicorn app.main:app --reload   # queda en http://127.0.0.1:8000

# Frontend
cd turify-frontend
npm install
npm run dev
```

La documentación interactiva de la API la genera FastAPI sola en `http://127.0.0.1:8000/docs`.

Variables de entorno

El backend necesita `SQLALCHEMY_DATABASE_URL` (o `DATABASE_URL`), `SECRET_KEY`, las tres de Cloudinary y las dos de Supabase. El frontend necesita `VITE_API_URL`, `VITE_GOOGLE_MAPS_API_KEY`, `VITE_SUPABASE_URL` y `VITE_SUPABASE_ANON_KEY`.

Ojo con esto. Hoy `docker-compose.yml` y el `README.md` tienen credenciales reales escritas en texto plano —clave de firma de los tokens, contraseña de la base, secreto de Cloudinary, un token de ngrok y una clave de Geoapify— y el repositorio es público. Cualquiera que clone el proyecto puede firmar tokens válidos. Conviene rotar todas esas credenciales y dejar solo referencias a variables de entorno. No se reproducen acá a propósito.

Convenciones del proyecto

La numeración de las historias de usuario. El proyecto arrastra dos tandas de HU. La primera, de los sprints iniciales, va de HU01 a HU17 y está toda cerrada. La segunda empezó otra vez desde HU01 y llegó a HU44, así que durante meses "HU09" señaló a dos historias distintas. El 2 de septiembre de 2026 la tanda nueva se corrió diecisiete lugares: HU01 pasó a ser HU18 y así hasta HU44, que pasó a HU61. Los comentarios del código ya usan la numeración nueva. Un comentario que diga HU10 o HU17 es de la tanda vieja; uno que diga HU26 o HU55, de la nueva.

El sistema de diseño manda. Si un color no está en `diseño.jsx`, no debería usarse. Los valores viven como variables CSS en `index.css`, en dos paletas, y `diseño.jsx` solo apunta a ellas. Cambiar el atributo `data-tema` del elemento `<html>` repinta la aplicación entera sin tocar una línea de JavaScript. La excepción es el objeto `FIJO`, con valores literales para los contextos donde las variables CSS no llegan: Google Maps recibe JSON, no CSS.

Verificar antes de dar por bueno. `npm run build` compila aunque una variable no exista: eso es un error de ejecución, no de compilación, y se manifiesta como pantalla en blanco. Por eso hay dos verificaciones:

```
npm run verificar      # referencias a componentes + eslint con no-undef
```

La primera busca componentes e iconos usados pero no definidos; la segunda es una configuración aparte de eslint que activa `no-undef`, la regla que la configuración principal del proyecto no trae encendida. Las dos existen porque las dos fallas ya ocurrieron.

El nombre de los archivos importa. Windows no distingue mayúsculas de minúsculas, Linux sí. Un `import` con la mayúscula corrida funciona en el equipo de desarrollo y rompe el build en Docker. Los archivos de `src/` se llaman exactamente como los importa quien los usa.

PARTE 1

El modelo de datos y el núcleo del backend

Las doce tablas, los esquemas de validación, la autenticación y la auditoría.

Modelos (`app/models.py`)

Importaciones relevantes: `from sqlalchemy import (Column, Integer, String, Enum, ForeignKey, DateTime, Boolean, Numeric, Text, Index, JSON), from sqlalchemy.dialects.postgresql import JSONB, from sqlalchemy import TIMESTAMP, from sqlalchemy.orm import relationship, from sqlalchemy.sql import func, from app.database import Base.`

Todas las clases heredan de `Base`.

AffiliatedCompany — `__tablename__ = "AffiliatedCompany"`

Columna	Tipo SQLAlchemy	Restricciones	Para qué sirve
<code>company_id</code>	<code>Integer</code>	<code>primary_key=True, index=True, autoincrement=True</code>	PK de la empresa afiliada
<code>name</code>	<code>String(100)</code>	<code>nullable=False</code>	Nombre de la empresa
<code>nit</code>	<code>String(50)</code>	<code>unique=True, nullable=False</code>	NIT (identificador tributario) único
<code>logo_url</code>	<code>Text</code>	— (nullable por defecto)	URL del logo
<code>created_at</code>	<code>TIMESTAMP(timezone=True)</code>	<code>server_default=func.now()</code>	Fecha de creación del registro

Relaciones: — `users = relationship("User", back_populates="company")` — `vehicles = relationship("Vehicle", back_populates="company")`

User — `__tablename__ = "User"`

Columna	Tipo SQLAlchemy	Restricciones	Para qué sirve
<code>user_id</code>	<code>Integer</code>	<code>primary_key=True, index=True, autoincrement=True</code>	PK del usuario

Columna	Tipo SQLAlchemy	Restricciones	Para qué sirve
full_name	String(100)	nullable=False	Nombre completo
email	String(100)	unique=True, nullable=False	Correo (identificador de login)
phone_number	String(20)	nullable=False	Teléfono de contacto
password_hash	String(255)	nullable=False	Hash bcrypt de la contraseña
role	Enum('PASSENGER', 'DRIVER', 'ADMIN', name='user_role')	default='PASSENGER'	Rol del usuario
status	Enum('ACTIVE', 'INACTIVE', name='user_status')	default='ACTIVE'	Estado de la cuenta
affiliated_company	Integer	ForeignKey("AffiliatedCompany.company_id", ondelete="SET NULL"), nullable=True	Empresa afiliada del usuario
profile_photo_url	Text	—	URL de la foto de perfil
age	Integer	—	Edad
current_lat	Numeric(10, 8)	—	Latitud actual del conductor (Épica 3)
current_lng	Numeric(11, 8)	—	Longitud actual del conductor (Épica 3)
is_online	Boolean	default=False	Si el conductor está conectado
rating_avg	Numeric(3, 2)	default=0.00	Promedio de calificaciones (Épica 7)
total_ratings	Integer	default=0	Cantidad total de calificaciones recibidas
conductor_verificado	Boolean	default=False	Badge de "conductor verificado" (HU38)
created_at	TIMESTAMP(timezone=True)	server_default=func.now()	Fecha de alta

Valores posibles de campos de estado: – role (enum user_role): 'PASSENGER', 'DRIVER', 'ADMIN'
 – status (enum user_status): 'ACTIVE', 'INACTIVE'

Comentario textual del código sobre conductor_verificado (HU38): «Badge de "conductor verificado": se activa cuando el admin aprueba el RUNT del conductor (experiencia declarada

verificada). Es independiente del rol DRIVER/documentos obligatorios de registro — el RUNT es opcional y posterior.»

Relaciones: - `company = relationship("AffiliatedCompany", back_populates="users")` -
`documents = relationship("Document", back_populates="owner", cascade="all, delete")` -
`vehicles = relationship("Vehicle", back_populates="owner", cascade="all, delete")`

Document — `__tablename__ = "Document"`

Columna	Tipo SQLAlchemy	Restricciones	Para qué sirve
document_id	Integer	primary_key=True, index=True, autoincrement=True	PK del documento
user_id	Integer	ForeignKey("User.user_id", ondelete="CASCADE"), nullable=False	Dueño del documento
document_type	Enum('SOAT', 'Licencia de Conduccion', 'Tarjeta de operacion', 'Tecnomecanica', 'Seguros Contractual y extracontractual', 'RUNT', name='doc_type')	nullable=False	Tipo de documento subido
file_url	Text	nullable=False	URL del archivo almacenado
verification_status	Enum('PENDING', 'APPROVED', 'REJECTED', 'AI_PRE_APPROVED', 'AI_PRE_REJECTED', name='verification_status')	default='PENDING'	Estado de verificación
ai_extracted_data	JSONB	—	Datos extraídos por el agente IA (Épica 4 HU61)
ai_expiry_date	DateTime	—	Fecha de vencimiento detectada por IA
ai_holder_name	String(100)	—	Nombre del titular detectado por IA
ai_confidence	Numeric(5, 2)	—	Confianza de la extracción IA
ai_observations	Text	—	Observaciones de la IA
years_experience	Integer	—	Años de experiencia (solo para RUNT)

Columna	Tipo SQLAlchemy	Restricciones	Para qué sirve
license_categories	String(50)	—	Categorías de licencia (solo para RUNT)
created_at	TIMESTAMP(timezone=True)	server_default=func.now()	Fecha de carga

Valores posibles: - document_type (enum doc_type): 'SOAT', 'Licencia de Conduccion', 'Tarjeta de operacion', 'Tecnomecanica', 'Seguros Contractual y extracontractual', 'RUNT' - verification_status (enum verification_status): 'PENDING', 'APPROVED', 'REJECTED', 'AI_PRE_APPROVED', 'AI_PRE_REJECTED'

Relaciones: - owner = relationship("User", back_populates="documents")

Vehicle — __tablename__ = "Vehicle"

Columna	Tipo SQLAlchemy	Restricciones	Para qué sirve
vehicle_id	Integer	primary_key=True, index=True, autoincrement=True	PK del vehículo
owner_id	Integer	ForeignKey("User.user_id", ondelete="CASCADE"), nullable=False	Conductor propietario
company_id	Integer	ForeignKey("AffiliatedCompany.company_id", ondelete="SET NULL"), nullable=True	Empresa afiliada del vehículo
plate	String(20)	unique=True, nullable=False	Placa
capacity	Integer	nullable=False	Capacidad declarada
capacidad_real	Integer	—	Capacidad real de pasajeros
vehicle_year	Integer	—	Año del vehículo
photo_url	Text	—	Foto del vehículo
tarifa_km_base	Numeric(10, 2)	—	Tarifa base por km (Épica 4)
tarifa_espera_hora	Numeric(10, 2)	—	Tarifa por hora de espera
tarifa_dia	Numeric(10, 2)	—	Tarifa por día
km_incluidos_por_dia	Integer	default=200	Km incluidos por día
recargo_dificil_acceso	Numeric(5, 2)	default=15.00	Recargo por acceso difícil

Columna	Tipo SQLAlchemy	Restricciones	Para qué sirve
tiene_ac	Boolean	default=False	Aire acondicionado (HU55)
tiene_wifi	Boolean	default=False	WiFi
tiene_bano	Boolean	default=False	Baño
tiene_musica	Boolean	default=False	Música
tiene_maletero_amplio	Boolean	default=False	Maletero amplio
tiene_sillas_bebe	Boolean	default=False	Sillas para bebé
acepta_mascotas	Boolean	default=False	Acepta mascotas
cargo_mascota	Numeric(10, 2)	default=0	Cargo extra por mascota
acepta_menores_2_anos	Boolean	default=True	Acepta menores de 2 años
created_at	TIMESTAMP(timezone=True)	server_default=func.now()	Fecha de registro

Relaciones: - `owner = relationship("User", back_populates="vehicles")` - `company = relationship("AffiliatedCompany", back_populates="vehicles")`

ServiceRequest — `__tablename__ = "ServiceRequest"`

Columna	Tipo SQLAlchemy	Restricciones	Para qué sirve
request_id	Integer	primary_key=True, autoincrement=True (sin index=True)	PK de la solicitud
passenger_id	Integer	ForeignKey("User.user_id", ondelete="CASCADE"), nullable=False	Pasajero que solicita
origin	String(255)	nullable=False	Dirección de origen
destination	String(255)	nullable=False	Dirección de destino
origin_lat	Numeric(10, 8)	—	Latitud de origen
origin_lng	Numeric(11, 8)	—	Longitud de origen
destination_lat	Numeric(10, 8)	—	Latitud de destino

Columna	Tipo SQLAlchemy	Restricciones	Para qué sirve
destination_lng	Numeric(11, 8)	—	Longitud de destino
trip_type	Enum('ONE_WAY', 'ROUND_TRIP', name='trip_type')	nullable=False, default='ONE_WAY'	Tipo de viaje
departure_time	TIMESTAMP(timezone=True)	nullable=False	Fecha/hora de salida
return_time	TIMESTAMP(timezone=True)	—	Fecha/hora de regreso
adults_count	Integer	nullable=False	Adultos
children_count	Integer	default=0	Niños
infants_count	Integer	default=0	Infantes
has_pets	Boolean	default=False	Viaja con mascotas
num_pets	Integer	default=0	Número de mascotas
status	Enum('PENDING', 'ASSIGNED', 'IN_PROGRESS', 'COMPLETED', 'CANCELLED', 'SCHEDULED', name='request_status')	default='PENDING'	Estado del viaje
distance_km	Numeric(10, 2)	—	Distancia calculada (Épica 4)
tolls_count	Integer	default=0	Cantidad de peajes
tolls_cost	Numeric(10, 2)	default=0	Costo de peajes
wait_time_hours	Numeric(5, 2)	default=0	Horas de espera
num_days	Integer	default=1	Días del servicio
tipo_via	Enum('PAVIMENTADA', 'DESTAPADA', 'MIXTA', name='via_type')	default='PAVIMENTADA'	Tipo de vía
is_peak_hour	Boolean	default=False	Hora pico
is_high_season	Boolean	default=False	Temporada alta
suggested_price	Numeric(10, 2)	—	Precio sugerido por la IA
suggested_price_min	Numeric(10, 2)	—	Mínimo del rango sugerido
suggested_price_max	Numeric(10, 2)	—	Máximo del rango sugerido

Columna	Tipo SQLAlchemy	Restricciones	Para qué sirve
price_explanation	Text	—	Explicación del precio
intermediate_stops	JSONB	—	Paradas intermedias (JSON)
search_lat	Numeric(10, 8)	—	Latitud del punto de búsqueda de conductores (HU26)
search_lng	Numeric(11, 8)	—	Longitud del punto de búsqueda (HU26)
search_radius_km	Numeric(5, 2)	default=15	Radio de búsqueda en km (HU26)
requiere_ac	Boolean	default=False	Exige aire acondicionado (HU55)
requiere_wifi	Boolean	default=False	Exige WiFi
requiere_bano	Boolean	default=False	Exige baño
requiere_musica	Boolean	default=False	Exige música
requiere_maletero_amplio	Boolean	default=False	Exige maletero amplio
requiere_sillas_bebe	Boolean	default=False	Exige sillas de bebé
requiere_acepta_mascotas	Boolean	default=False	Exige que acepte mascotas
created_at	TIMESTAMP(timezone=True)	server_default=func.now()	Fecha de creación

Valores posibles: - `trip_type` (enum `trip_type`): 'ONE_WAY', 'ROUND_TRIP' - `status` (enum `request_status`): 'PENDING', 'ASSIGNED', 'IN_PROGRESS', 'COMPLETED', 'CANCELLED', 'SCHEDULED' - `tipo_via` (enum `via_type`): 'PAVIMENTADA', 'DESTAPADA', 'MIXTA'

Comentarios textuales del código: - HU26: «Punto y radio de búsqueda de conductores. Ya NO lo elige el pasajero: se calculan automáticamente al crear el viaje (`search_lat/lng` = origen del viaje; `search_radius_km` = radio amplio fijo usado para que cualquier conductor que se conecte más tarde siga viendo la solicitud en su radar).» - HU55: «Comodidades del vehículo que el pasajero exige (filtro de búsqueda). Si todas quedan en False, no se filtra por comodidades.»

Relaciones: ninguna declarada (`relationship`) en este modelo.

DriverOffer — `__tablename__ = "DriverOffer"`

Columna	Tipo SQLAlchemy	Restricciones	Para qué sirve
offer_id	Integer	primary_key=True, index=True, autoincrement=True	PK de la oferta
request_id	Integer	ForeignKey("ServiceRequest.request_id", ondelete="CASCADE"), nullable=False	Solicitud ofertada
driver_id	Integer	ForeignKey("User.user_id", ondelete="CASCADE"), nullable=False	Conductor que oferta
vehicle_id	Integer	ForeignKey("Vehicle.vehicle_id", ondelete="CASCADE"), nullable=False	Vehículo ofrecido
offered_price	Numeric(10, 2)	nullable=False	Precio ofertado
status	Enum('DRIVER_OFFERED', 'PASSENGER_COUNTER_OFFERED', 'ACCEPTED', 'REJECTED', name='offer_status')	default='DRIVER_OFFERED'	Estado de la negociación
created_at	TIMESTAMP(timezone=True)	server_default=func.now()	Fecha de la oferta

Valores posibles de status (enum offer_status): 'DRIVER_OFFERED', 'PASSENGER_COUNTER_OFFERED', 'ACCEPTED', 'REJECTED'

Relaciones: ninguna declarada.

TripPassenger — `__tablename__ = "TripPassenger"`

Columna	Tipo SQLAlchemy	Restricciones	Para qué sirve
passenger_entry_id	Integer	primary_key=True, index=True, autoincrement=True	PK del pasajero del viaje
request_id	Integer	ForeignKey("ServiceRequest.request_id", ondelete="CASCADE"), nullable=False	Viaje al que pertenece
full_name	String(100)	nullable=False	Nombre del pasajero
document_type	Enum('CC', 'TI', 'CE', 'PA', name='doc_id_type')	default='CC'	Tipo de documento de identidad
document_number	String(20)	nullable=False	Número de documento
created_at	TIMESTAMP(timezone=True)	server_default=func.now()	Fecha de registro

Valores posibles de document_type (enum doc_id_type): 'CC', 'TI', 'CE', 'PA'

Relaciones: ninguna declarada.

AuditLog — __tablename__ = "AuditLog"

Columna	Tipo SQLAlchemy	Restricciones	Para qué sirve
log_id	Integer	primary_key=True, index=True, autoincrement=True	PK del log
user_id	Integer	ForeignKey("User.user_id", ondelete="SET NULL"), nullable=True	Usuario que ejecutó la acción
action	String(50)	nullable=False	Acción realizada
entity	String(50)	—	Entidad afectada
entity_id	Integer	—	ID de la entidad afectada
detail	Text	—	Detalle textual
ip_address	String(45)	—	IP de origen
created_at	TIMESTAMP(timezone=True)	server_default=func.now()	Fecha del evento

Relaciones: ninguna declarada.

Notification — __tablename__ = "Notification"

Columna	Tipo SQLAlchemy	Restricciones	Para qué sirve
notification_id	Integer	primary_key=True, index=True, autoincrement=True	PK de la notificación
user_id	Integer	ForeignKey("User.user_id", ondelete="CASCADE"), nullable=False	Destinatario
title	String(100)	nullable=False	Título
message	Text	nullable=False	Cuerpo del mensaje
type	Enum('NEW_OFFER', 'COUNTER_OFFER', 'TRIP_ACCEPTED', 'TRIP_REJECTED', 'TRIP_STARTED', 'TRIP_COMPLETED', 'SYSTEM', name='notification_type')	nullable=False	Tipo de notificación
is_read	Boolean	default=False	Si fue leída
related_offer_id	Integer	ForeignKey("DriverOffer.offer_id", ondelete="SET NULL"), nullable=True	Oferta relacionada

Columna	Tipo SQLAlchemy	Restricciones	Para qué sirve
created_at	TIMESTAMP(timezone=True)	server_default=func.now()	Fecha de emisión

Valores posibles de type (enum notification_type): 'NEW_OFFER', 'COUNTER_OFFER', 'TRIP_ACCEPTED', 'TRIP_REJECTED', 'TRIP_STARTED', 'TRIP_COMPLETED', 'SYSTEM'

Relaciones: ninguna declarada.

PriceHistory — `__tablename__ = "PriceHistory"`

Docstring: `"""Historial de precios para alimentar el agente IA (Épica 4)"""`

Columna	Tipo SQLAlchemy	Restricciones	Para qué sirve
history_id	Integer	primary_key=True, autoincrement=True	PK del histórico
request_id	Integer	ForeignKey("ServiceRequest.request_id", ondelete="SET NULL"), nullable=True	Solicitud asociada
vehicle_category	Enum('SEDAN', 'VAN', 'MICROBUS', 'BUS', 'BUS_GRANDE', name='vehicle_category')	nullable=False	Categoría del vehículo
distance_km	Numeric(10, 2)	nullable=False	Distancia del viaje
suggested_price	Numeric(10, 2)	nullable=False	Precio sugerido
final_price	Numeric(10, 2)	—	Precio final pactado
tolls_cost	Numeric(10, 2)	default=0	Costo de peajes
wait_time_hours	Numeric(5, 2)	default=0	Horas de espera
num_days	Integer	default=1	Días del servicio
is_peak_hour	Boolean	default=False	Hora pico
is_high_season	Boolean	default=False	Temporada alta
tipo_via	Enum('PAVIMENTADA', 'DESTAPADA', 'MIXTA', name='via_type')	sin default	Tipo de vía
has_ac	Boolean	default=False	Tenía aire acondicionado
has_wifi	Boolean	default=False	Tenía WiFi

Columna	Tipo SQLAlchemy	Restricciones	Para qué sirve
num_passengers	Integer	nullable=False	Número de pasajeros
origin_city	String(100)	—	Ciudad de origen
destination_city	String(100)	—	Ciudad de destino
created_at	TIMESTAMP(timezone=True)	server_default=func.now()	Fecha del registro

Valores posibles: - `vehicle_category` (enum `vehicle_category`): 'SEDAN', 'VAN', 'MICROBUS', 'BUS', 'BUS_GRANDE' - `tipo_via` (enum `via_type`): 'PAVIMENTADA', 'DESTAPADA', 'MIXTA'

Relaciones: ninguna declarada.

TripStop — `__tablename__ = "TripStop"`

Docstring: `"""Paradas intermedias en un viaje"""`

Columna	Tipo SQLAlchemy	Restricciones	Para qué sirve
stop_id	Integer	primary_key=True, autoincrement=True	PK de la parada
request_id	Integer	ForeignKey("ServiceRequest.request_id", ondelete="CASCADE"), nullable=False	Viaje al que pertenece
stop_order	Integer	nullable=False	Orden de la parada en la ruta
address	String(255)	nullable=False	Dirección de la parada
lat	Numeric(10, 8)	—	Latitud
lng	Numeric(11, 8)	—	Longitud
created_at	TIMESTAMP(timezone=True)	server_default=func.now()	Fecha de creación

Relaciones: ninguna declarada.

Rating — `__tablename__ = "Rating"`

Docstring: `"""Calificaciones bidireccionales pasajero ↔ conductor (Épica 7)"""`

Columna	Tipo SQLAlchemy	Restricciones	Para qué sirve
rating_id	Integer	primary_key=True, autoincrement=True	PK de la calificación

Columna	Tipo SQLAlchemy	Restricciones	Para qué sirve
request_id	Integer	ForeignKey("ServiceRequest.request_id", ondelete="CASCADE"), nullable=False	Viaje calificado
rater_id	Integer	ForeignKey("User.user_id", ondelete="CASCADE"), nullable=False	Quien califica
rated_id	Integer	ForeignKey("User.user_id", ondelete="CASCADE"), nullable=False	Quien es calificado
score	Integer	nullable=False	Puntaje (validado 1–5 en el schema Pydantic)
comment	Text	—	Comentario libre
created_at	TIMESTAMP(timezone=True)	server_default=func.now()	Fecha de la calificación

Relaciones: ninguna declarada.

Esquemas Pydantic (app/schemas.py)

Importaciones: from pydantic import BaseModel, EmailStr, Field, model_validator, from typing import Optional, from datetime import datetime, from enum import Enum.

Prefijos de routers (para ubicar los endpoints): - auth.py → prefix="/users", tags=["Authentication"] - drivers.py → prefix="/drivers", tags=["Modo Conductor"] - service_requests.py → prefix="/api/service-requests", tags=["Service Requests"] - admin.py → prefix="/admin", tags=["Admin"] - routes.py → prefix="/api/routes", tags=["Routes"] (router **no incluido** en main.py)

Enums

Clase	Base	Miembros (nombre = valor)
RoleEnum	str, Enum	PASSENGER = 'PASSENGER', DRIVER = 'DRIVER', ADMIN = 'ADMIN'
DocumentTypeEnum	str, Enum	SOAT = 'SOAT', Licencia = 'Licencia de Conduccion', TarjetaOperacion = 'Tarjeta de operacion', Tecnomecanica = 'Tecnomecanica', Seguros = 'Seguros Contractual y extracontractual', RUNT = 'RUNT'
VerificationStatusEnum	str, Enum	PENDING, APPROVED, REJECTED, AI_PRE_APPROVED, AI_PRE_REJECTED (todos con valor igual a su nombre)
TripType	str, Enum	ONE_WAY = "ONE_WAY", ROUND_TRIP = "ROUND_TRIP"

Comentario del código sobre DocumentTypeEnum: «Valores alineados con el enum 'doc_type' real de models.py — antes no coincidían (ej. aquí decía 'Licencia' pero en la BD/registro se guarda 'Licencia de Conduccion'), lo que rompía la validación de response_model en GET /drivers/my-documents para cualquier documento que no fuera exactamente 'SOAT'.»

Comentario sobre `DocumentTypeEnum.RUNT`: «HU37 — RUNT (experiencia del conductor), opcional y posterior al registro».

UserCreate

Campo	Tipo	Opcional / default
full_name	str	obligatorio
email	EmailStr	obligatorio
password	str	obligatorio, Field(..., min_length=8)
phone_number	str	obligatorio (comentario: «Ahora es obligatorio según tu SQL»)

Uso: cuerpo de POST `/users/register` (`auth.py:13`, `status_code=status.HTTP_201_CREATED`).

UserResponse — response_model

Campo	Tipo	Opcional / default
user_id	int	obligatorio
full_name	str	obligatorio
email	EmailStr	obligatorio
role	str	obligatorio
phone_number	Optional[str]	= None
profile_photo_url	Optional[str]	= None

```
class Config: from_attributes = True
```

Comentario del código: «El telefono se declara aca a proposito: `response_model` descarta todo campo que no este en el esquema, y el formulario de conductor lo necesita para precargarse. Optional porque hay usuarios antiguos sin el dato.»

Uso: `response_model` de GET `/users/me` (`auth.py:70`).

DocumentResponse — response_model

Campo	Tipo	Opcional / default
document_id	int	obligatorio
user_id	int	obligatorio
document_type	DocumentTypeEnum	obligatorio
file_url	str	obligatorio
verification_status	VerificationStatusEnum	obligatorio

Campo	Tipo	Opcional / default
years_experience	Optional[int]	= None (HU37/HU38, solo en documentos RUNT)
license_categories	Optional[str]	= None (solo en documentos RUNT)

```
class Config: from_attributes = True
```

Uso: `response_model=List[schemas.DocumentResponse]` en GET `/drivers/my-documents` (`drivers.py:316`).

TokenResponse — response_model

Campo	Tipo	Opcional / default
access_token	str	obligatorio
token_type	str	obligatorio

Uso: `response_model=schemas.TokenResponse` en POST `/users/login` (`auth.py:41`).

ServiceRequestCreate (primera definición, líneas 73-99) — sobrescrita

Campo	Tipo	Opcional / default
departure_time	datetime	obligatorio
trip_type	TripType	obligatorio
return_time	Optional[datetime]	= None
origin	str	obligatorio
destination	str	obligatorio

Validador: `@model_validator(mode='after')` `def validate_dates(self) -> 'ServiceRequestCreate':`
 1. `departure_time` no puede ser anterior a `datetime.now()` → "La fecha de salida no puede estar en el pasado."
 2. Si `trip_type == TripType.ROUND_TRIP`: `return_time` obligatorio → "El campo 'return_time' es obligatorio para viajes de ida y vuelta."; y `return_time` debe ser posterior a `departure_time` → "La fecha de regreso debe ser posterior a la de salida."
 3. Si es `ONE_WAY`, fuerza `self.return_time = None`.

Esta clase queda **redefinida** por la de la línea 101 (ver Observaciones).

ServiceRequestCreate (segunda definición, líneas 101-138) — la efectiva

Campo	Tipo	Opcional / default
origin	str	obligatorio
destination	str	obligatorio
departure_time	datetime	obligatorio

Campo	Tipo	Opcional / default
return_time	Optional[datetime]	= None
trip_type	TripType	obligatorio
adults_count	int	obligatorio
children_count	int	= 0
has_pets	bool	= False
origin_lat	Optional[float]	= None (HU25, Google Maps)
origin_lng	Optional[float]	= None
destination_lat	Optional[float]	= None
destination_lng	Optional[float]	= None
distance_km	Optional[float]	= None
tolls_count	Optional[int]	= None
tolls_cost	Optional[float]	= None
tipo_via	Optional[str]	= None — comentario: 'PAVIMENTADA' \ 'DESTAPADA' \ 'MIXTA'
requiere_ac	Optional[bool]	= False (HU55)
requiere_wifi	Optional[bool]	= False
requiere_bano	Optional[bool]	= False
requiere_musica	Optional[bool]	= False
requiere_maletero_amplio	Optional[bool]	= False
requiere_sillas_bebe	Optional[bool]	= False
requiere_acepta_mascotas	Optional[bool]	= False

Validador: `@model_validator(mode='after')` def `validate_dates(self)` `-> 'ServiceRequestCreate'`
— el cuerpo solo contiene el comentario `# ...` (mantén tu lógica de validación de fechas anterior) y la validación `if self.adults_count <= 0: raise ValueError("Debe haber al menos un adulto en la solicitud.")`.

Comentario HU55 del código: «comodidades que el pasajero exige del vehículo (filtro de búsqueda). Si no marca ninguna, no se filtra por comodidades: se notifica a todos los conductores cercanos, sea cual sea su vehículo.»

Uso: cuerpo (`request_data: schemas.ServiceRequestCreate`) de `POST /api/service-requests/` (`service_requests.py:20-22`, `status_code=status.HTTP_201_CREATED`).

ServiceRequestResponse(ServiceRequestCreate)

Hereda todos los campos de `ServiceRequestCreate` (segunda definición) y añade:

Campo	Tipo	Opcional / default
request_id	int	obligatorio
passenger_id	int	obligatorio
status	str	obligatorio
created_at	datetime	obligatorio

```
class Config: from_attributes = True
```

Uso: no aparece referenciada en ningún router (ver Observaciones).

DriverResponse

Campo	Tipo	Opcional / default
id	int	obligatorio
full_name	str	obligatorio
message	str	obligatorio

```
class Config: from_attributes = True
```

Uso: no aparece referenciada en ningún router (ver Observaciones).

ServiceRequestRead — response_model

Campo	Tipo	Opcional / default
request_id	int	obligatorio
passenger_id	int	obligatorio
origin	str	obligatorio
destination	str	obligatorio
departure_time	datetime	obligatorio
return_time	Optional[datetime]	= None
trip_type	str	obligatorio
adults_count	int	obligatorio
children_count	int	obligatorio
has_pets	bool	obligatorio
status	str	obligatorio
created_at	datetime	obligatorio
origin_lat	Optional[float]	= None (HU06, pin en el mapa del conductor)
origin_lng	Optional[float]	= None

Campo	Tipo	Opcional / default
requiere_ac	Optional[bool]	= False (HU55)
requiere_wifi	Optional[bool]	= False
requiere_bano	Optional[bool]	= False
requiere_musica	Optional[bool]	= False
requiere_maletero_amplio	Optional[bool]	= False
requiere_sillas_bebe	Optional[bool]	= False
requiere_acepta_mascotas	Optional[bool]	= False
comodidades_exigidas	Optional[int]	= None (solo para el CONDUCTOR)
comodidades_cumplidas	Optional[int]	= None
comodidades_faltantes	Optional[list[str]]	= None

```
class Config: from_attributes = True # Permite mapear desde modelos de SQLAlchemy
```

Comentario HU55: «filtro flexible: solo presentes para el CONDUCTOR (indican cuántas de las comodidades exigidas cumple su propio vehículo y cuáles le faltan)».

Uso: `response_model=list[schemas.ServiceRequestRead]` en GET `/api/service-requests/pending` (`service_requests.py:272`). Comentario en el router (línea 244): «`schemas.ServiceRequestRead` — se usa en `/pending` porque ahí se le agregan...».

OfferCreate

Campo	Tipo	Opcional / default
offered_price	float	obligatorio, <code>Field(..., gt=0, description="Precio propuesto por el conductor")</code>

Uso: cuerpo de POST `/api/service-requests/{request_id}/offers` (`service_requests.py:382-385`).

OfferRead

Campo	Tipo	Opcional / default
offer_id	int	obligatorio
request_id	int	obligatorio
driver_id	int	obligatorio
vehicle_id	int	obligatorio
offered_price	float	obligatorio
status	str	obligatorio
created_at	datetime	obligatorio
driver_name	Optional[str]	= None — «Datos del conductor (se añaden manualmente en el endpoint)»

Campo	Tipo	Opcional / default
driver_photo	Optional[str]	= None

```
class Config: from_attributes = True
```

Uso: no aparece como `response_model` en ningún endpoint (ver Observaciones); el endpoint relacionado sería `GET /api/service-requests/{request_id}/offers` (`service_requests.py:777`), que no declara `response_model`.

CounterOfferCreate

Campo	Tipo	Opcional / default
offered_price	float	obligatorio, Field(..., gt=0, description="Precio de contraoferta del pasajero")

Uso: cuerpo de `PATCH /api/service-requests/offers/{offer_id}/counter-offer` (`service_requests.py:872-875`).

ResolveOfferCreate

Campo	Tipo	Opcional / default
action	str	obligatorio — comentario: 'ACCEPT' \ 'REJECT'

Uso: cuerpo de `PATCH /api/service-requests/offers/{offer_id}/resolve` (`service_requests.py:1068-1071`).

TripPassengerItem (HU10 — FUEC)

Campo	Tipo	Opcional / default
full_name	str	obligatorio
document_type	str	= 'CC' — comentario: # CC, TI, CE, PA
document_number	str	obligatorio

Uso: anidado dentro de `TripPassengersCreate`.

TripPassengersCreate

Campo	Tipo	Opcional / default
passengers	list[TripPassengerItem]	obligatorio

Uso: cuerpo de `POST /api/service-requests/{request_id}/passengers` (`service_requests.py:1363-1366`, `status_code=201`).

UpdatePhoneRequest (HU16 – Perfil de usuario)

Campo	Tipo	Opcional / default
phone_number	str	obligatorio

Uso: cuerpo de PATCH `/users/me/phone` (auth.py:201-203).

UpdatePasswordRequest

Campo	Tipo	Opcional / default
current_password	str	obligatorio
new_password	str	obligatorio, Field(..., min_length=8)

Uso: cuerpo de PATCH `/users/me/password` (auth.py:215-217).

UserProfileResponse

Campo	Tipo	Opcional / default
user_id	int	obligatorio
full_name	str	obligatorio
email	EmailStr	obligatorio
phone_number	str	obligatorio
role	str	obligatorio
profile_photo_url	Optional[str]	= None
age	Optional[int]	= None

```
class Config: from_attributes = True
```

Uso: no aparece referenciada en ningún router; el endpoint natural sería GET `/users/me/profile` (auth.py:109), que **no** declara `response_model` (ver Observaciones).

DriverLocationUpdate (HU26 – Rango geográfico de conductores)

Campo	Tipo	Opcional / default
current_lat	Optional[float]	= None
current_lng	Optional[float]	= None
is_online	Optional[bool]	= None

Uso: cuerpo (payload: `schemas.DriverLocationUpdate`) de PATCH `/drivers/location` (drivers.py:364-366).

RatingCreate (HU46 – Calificaciones bidireccionales)

Campo	Tipo	Opcional / default
score	int	obligatorio, Field(..., ge=1, le=5, description="Calificación de 1 a 5 estrellas")
comment	Optional[str]	= None

Uso: cuerpo de POST `/api/service-requests/{request_id}/rating` (`service_requests.py:1454-1457`, `status_code=status.HTTP_201_CREATED`).

VehicleSettingsUpdate (HU55)

Comentario del código: «Comodidades del vehículo. Las tarifas (por km, espera, día, etc.) NO las decide el conductor — quedaron fuera de este schema a propósito, así que PATCH `/drivers/vehicle` no puede tocarlas aunque alguien las envíe en el body.»

Campo	Tipo	Opcional / default
capacidad_real	Optional[int]	Field(None, ge=1, le=60, description="Capacidad real de pasajeros")
tiene_ac	Optional[bool]	= None
tiene_wifi	Optional[bool]	= None
tiene_bano	Optional[bool]	= None
tiene_musica	Optional[bool]	= None
tiene_maletero_amplio	Optional[bool]	= None
tiene_sillas_bebe	Optional[bool]	= None
acepta_mascotas	Optional[bool]	= None
cargo_mascota	Optional[float]	Field(None, ge=0)
acepta_menores_2_anos	Optional[bool]	= None

Uso: cuerpo de PATCH `/drivers/vehicle` (`drivers.py:439-441`).

VehicleSettingsResponse — response_model

Campo	Tipo	Opcional / default
vehicle_id	int	obligatorio
plate	str	obligatorio
capacity	int	obligatorio
capacidad_real	Optional[int]	= None
categoria	str	obligatorio (campo calculado, no existe en el modelo)
tarifa_km_base	Optional[float]	= None

Campo	Tipo	Opcional / default
tarifa_km_rango	list[float]	obligatorio (campo calculado, no existe en el modelo)
tarifa_espera_hora	Optional[float]	= None
tarifa_dia	Optional[float]	= None
km_incluidos_por_dia	Optional[int]	= None
recargo_dificil_acceso	Optional[float]	= None
tiene_ac	bool	obligatorio
tiene_wifi	bool	obligatorio
tiene_bano	bool	obligatorio
tiene_musica	bool	obligatorio
tiene_maletero_amplio	bool	obligatorio
tiene_sillas_bebe	bool	obligatorio
acepta_mascotas	bool	obligatorio
cargo_mascota	Optional[float]	= None
acepta_menores_2_anos	bool	obligatorio

```
class Config: from_attributes = True
```

Uso: `response_model` de GET `/drivers/vehicle` (`drivers.py:398`) y de PATCH `/drivers/vehicle` (`drivers.py:439`).

Esquemas definidos fuera de `schemas.py`

En `app/routers/admin.py` hay "Schemas locales" (comentario en línea 19): `DocumentVerifyRequest` (línea 20), `DriverPendingOut` (usado como `response_model=List[DriverPendingOut]` en GET `/admin/drivers/pending`, línea 60) y `DocumentOut` (usado como `response_model=List[DocumentOut]` en GET `/admin/drivers/{user_id}/documents`, línea 141). En `app/routers/routes.py` está `ComputeRouteResponse` (`response_model` de POST `/api/routes/compute`, línea 37).

app/security.py**Importaciones y librerías**

```

from fastapi import Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer
from jose import JWTError, jwt
from sqlalchemy.orm import Session
from passlib.context import CryptContext
from datetime import datetime, timedelta
from typing import Optional
from app import models, schemas
from app.database import get_db
import os
from dotenv import load_dotenv

load_dotenv()

```

- **JWT:** python-jose (from jose import JWTError, jwt).
- **Hash de contraseñas:** passlib con esquema bcrypt.
- **Variables de entorno:** python-dotenv (load_dotenv() al importar el módulo).

Constantes

Constante	Valor exacto	Nota
SECRET_KEY	os.getenv("SECRET_KEY", "una_clave_muy_secreta_para_desarrollo")	Se lee de la variable de entorno SECRET_KEY; el fallback literal es "una_clave_muy_secreta_para_desarrollo"
ALGORITHM	"HS256"	Algoritmo de firma HMAC-SHA256
ACCESS_TOKEN_EXPIRE_MINUTES	30	Comentario: «¿Cuánto dura la sesión? 30 minutos es un buen estándar»
oauth2_scheme	OAuth2PasswordBearer(tokenUrl="users/login")	Comentario: «Ruta que usa Swagger para probar el login»
pwd_context	CryptContext(schemes=["bcrypt"], deprecated="auto")	Comentario: «Motor de encriptación de contraseñas»

Funciones

DEF GET_PASSWORD_HASH(PASSWORD: STR):

Devuelve `pwd_context.hash(password)`. Genera el hash bcrypt de una contraseña en texto plano; es lo que se guarda en `User.password_hash`. No tiene docstring.

DEF VERIFY_PASSWORD(PLAIN_PASSWORD, HASHED_PASSWORD):

Docstring: `"""Compara la contraseña en texto plano con el hash guardado"""`. Devuelve `pwd_context.verify(plain_password, hashed_password) → True / False`. Los parámetros no llevan anotación de tipo.

```
DEF CREATE_ACCESS_TOKEN(DATA: DICT, EXPIRES_DELTA: OPTIONAL[TIMEDELTA] = NONE):
```

Docstring: `"""Crea la 'Manilla VIP' (El Token)"""`. 1. `to_encode = data.copy()` 2. Si se pasa `expires_delta`: `expire = datetime.utcnow() + expires_delta`; en caso contrario `expire = datetime.utcnow() + timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)`. 3. `to_encode.update({"exp": expire})` — agrega el claim estándar `exp`. 4. `encoded_jwt = jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)` y lo devuelve.

```
DEF GET_CURRENT_USER(TOKEN: STR = DEPENDS(OAUTH2_SCHEME), DB: SESSION = DEPENDS(GET_DB)):
```

Docstring: `"""El Guardia de Seguridad: Valida el token y deja pasar al usuario"""`.

Define primero:

```
credentials_exception = HTTPException(
    status_code=status.HTTP_401_UNAUTHORIZED,
    detail="No se pudieron validar las credenciales",
    headers={"WWW-Authenticate": "Bearer"},
)
```

Pasos (numerados en los comentarios del código): 1. `payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])` — decodifica y verifica firma y expiración. 2. `user_id: str = payload.get("sub")`; si es `None`, lanza `credentials_exception`. 3. Si se produce `JWTError` (token expirado o inválido), lanza `credentials_exception`. 4. Consulta: `user = db.query(models.User).filter(models.User.user_id == user_id).first()`; si es `None`, lanza `credentials_exception`. 5. Devuelve el objeto `models.User` validado.

Flujo de autenticación con JWT

1. **Registro** (POST `/users/register`): la contraseña recibida en `UserCreate.password` se convierte con `get_password_hash()` y se guarda en `User.password_hash`.
2. **Login** (POST `/users/login`, `response_model=schemas.TokenResponse`): se busca al usuario por email, se comprueba la contraseña con `verify_password()` y, si es válida, se emite un token con `create_access_token()` cuyo claim `sub` contiene el identificador del usuario y cuyo claim `exp` es `utcnow() + 30 minutos`. La respuesta es `{access_token, token_type}`.
3. **Cliente**: envía el token en la cabecera `Authorization: Bearer <token>`. El esquema `OAuth2PasswordBearer(tokenUrl="users/login")` es lo que hace que Swagger UI muestre el botón "Authorize" apuntando a ese endpoint.
4. **Endpoints protegidos**: declaran `current_user = Depends(get_current_user)`. FastAPI extrae el token con `oauth2_scheme`, `get_current_user` lo decodifica con `SECRET_KEY / HS256`, lee `sub`, carga el `User` desde la base de datos y lo inyecta en el endpoint. Cualquier fallo (token ausente, inválido, expirado, `sub` nulo o usuario inexistente) produce `401 Unauthorized` con `detail="No se pudieron validar las credenciales"` y la cabecera `WWW-Authenticate: Bearer`.

app/audit.py

Importaciones: `from sqlalchemy.orm import Session, from app import models.`

Firma completa

```
def registrar_log(
    db: Session,
    action: str,
    user_id: int = None,
    entity: str = None,
    entity_id: int = None,
    detail: str = None,
    ip_address: str = None
):
```

Significado de cada parámetro

Parámetro	Tipo	Default	Significado
db	Session	— (obligatorio)	Sesión de SQLAlchemy donde se inserta el log
action	str	— (obligatorio)	Acción registrada; se guarda en AuditLog.action
user_id	int	None	ID del usuario que ejecutó la acción (AuditLog.user_id, FK con ondelete="SET NULL")
entity	str	None	Nombre de la entidad afectada (AuditLog.entity)
entity_id	int	None	ID de la entidad afectada (AuditLog.entity_id)
detail	str	None	Texto libre con el detalle del evento (AuditLog.detail)
ip_address	str	None	IP de origen de la petición (AuditLog.ip_address)

Comportamiento

Construye un `models.AuditLog(...)` con esos valores, hace `db.add(log)` y `db.commit()`. Todo va dentro de un `try/except Exception as e` que, ante cualquier fallo, hace `db.rollback()` e imprime `f"[AuditLog] Error al registrar log: {e}"`. Comentario del código: «El log nunca debe interrumpir el flujo principal».

Acciones documentadas en el docstring

- LOGIN : Login exitoso
- LOGIN_FAILED : Login fallido
- REGISTER : Registro de nuevo usuario
- CREATE_TRIP : Pasajero crea solicitud de viaje
- CREATE_OFFER : Conductor envía oferta
- VERIFY_DOCUMENT : Admin aprueba o rechaza documento
- REGISTER_DRIVER : Conductor envía documentos

app/database.py

Importaciones: `os`, `from sqlalchemy import create_engine`, `from sqlalchemy.ext.declarative import declarative_base`, `from sqlalchemy.orm import sessionmaker`, `from dotenv import load_dotenv`. Se ejecuta `load_dotenv()` al importar.

Construcción de la URL de conexión

```
SQLALCHEMY_DATABASE_URL = (
    os.getenv("SUPABASE_DB_URL") or
    os.getenv("SQLALCHEMY_DATABASE_URL") or
    "postgresql://postgres:1234@localhost:5432/turify_db"
)
```

Variables de entorno usadas (en orden de prioridad): 1. `SUPABASE_DB_URL` — cadena de conexión a la base Postgres de Supabase. 2. `SQLALCHEMY_DATABASE_URL` — alternativa. 3. Fallback literal: `"postgresql://postgres:1234@localhost:5432/turify_db"` (Postgres local).

Normalización posterior (comentario: «Convertir postgres:// a postgresql:// si es necesario»):

```
if SQLALCHEMY_DATABASE_URL.startswith("postgres://"):
    SQLALCHEMY_DATABASE_URL = SQLALCHEMY_DATABASE_URL.replace("postgres://", "postgresql://", 1)
```

Esto cubre el caso de proveedores que entregan la URL con el esquema antiguo `postgres://`, que SQLAlchemy 2.x ya no acepta.

Objetos creados

- `engine = create_engine(SQLALCHEMY_DATABASE_URL, pool_pre_ping=True)` — el `pool_pre_ping=True` prueba la conexión antes de usarla, evitando errores por conexiones cerradas por el servidor (típico en Supabase/pooler).
- `SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)` — fábrica de sesiones; sin autocommit ni autoflush, por lo que hay que llamar `db.commit()` explícitamente.
- `Base = declarative_base()` — clase base declarativa de la que heredan todos los modelos de `models.py`; su `Base.metadata` es lo que `main.py` usa para crear las tablas.

get_db

```
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

Generador usado como dependencia de FastAPI (`db: Session = Depends(get_db)`): abre una sesión por petición, la cede al endpoint y la cierra siempre en el `finally`, haya o no excepción.

app/main.py

```

from app.routers import auth, drivers, service_requests, admin
from fastapi import FastAPI, Request
from fastapi.middleware.cors import CORSMiddleware
from app.database import engine, Base, SessionLocal
from app.audit import registrar_log

# Crea las tablas automáticamente (incluye AuditLog)
Base.metadata.create_all(bind=engine)

app = FastAPI(title="Turify API")

```

Creación de tablas

`Base.metadata.create_all(bind=engine)` se ejecuta al importar el módulo, antes de instanciar la app. Comentario: «Crea las tablas automáticamente (incluye AuditLog)».

Instancia de FastAPI

`app = FastAPI(title="Turify API")` — único parámetro: `title="Turify API"`.

Middlewares

Un solo middleware, `CORSMiddleware`:

Parámetro	Valor
<code>allow_origins</code>	<code>["*"]</code> (todos los orígenes permitidos)
<code>allow_credentials</code>	<code>True</code>
<code>allow_methods</code>	<code>["*"]</code>
<code>allow_headers</code>	<code>["*"]</code>

Routers incluidos

Llamada	Prefijo efectivo	Tags
<code>app.include_router(auth.router)</code>	<code>/users</code>	<code>["Authentication"]</code>
<code>app.include_router(drivers.router)</code>	<code>/drivers</code>	<code>["Modo Conductor"]</code>
<code>app.include_router(service_requests.router)</code>	<code>/api/service-requests</code>	<code>["Service Requests"]</code>
<code>app.include_router(admin.router)</code>	<code>/admin</code>	<code>["Admin"]</code>

Ninguna llamada a `include_router` pasa `prefix` ni `tags`: cada prefijo viene del `APIRouter(prefix=...)` definido dentro de cada router.

Manejadores de excepciones

No hay ninguno. El archivo no define `@app.exception_handler(...)`, ni middlewares personalizados, ni eventos `startup/shutdown`. `Request`, `SessionLocal` y `registrar_log` se importan pero no se usan en este archivo.

requirements.txt

Paquete	Versión
annotated-types	0.7.0
anyio	4.13.0
bcrypt	4.0.1
certifi	2026.4.22
cffi	2.0.0
click	8.4.1
colorama	0.4.6
cryptography	48.0.0
ecdsa	0.19.2
fastapi	0.136.3
greenlet	3.5.1
h11	0.16.0
httpcore	10.9
httpx	0.28.1
idna	3.13
passlib	1.7.4
pyasn1	0.6.3
pycparser	3.0
pydantic	2.13.4
pydantic_core	2.46.4
PyMySQL	1.2.0
python-dotenv	1.2.2
python-jose	3.5.0
python-multipart	0.0.29
rsa	4.9.1
six	1.17.0
SQLAlchemy	2.0.50
starlette	1.1.0
typing-inspection	0.4.2
typing_extensions	4.15.0

Paquete	Versión
urllib3	2.7.0
uvicorn	0.48.0
psycpg2-binary	2.9.10
email-validator	2.2.0
supabase	2.15.1
reportlab	5.0.1

Total: 36 dependencias, todas fijadas con `==`.

PARTE 2

Los endpoints del backend

Los cuatro routers, endpoint por endpoint: quién puede llamarlos, qué hacen y qué devuelven.

Los cuatro routers documentados se registran en `app/main.py`:

```
app.include_router(auth.router)
app.include_router(drivers.router)
app.include_router(service_requests.router)
app.include_router(admin.router)
```

Todos los endpoints protegidos usan `get_current_user` (dependencia definida en `app/security.py`), que resuelve el usuario a partir del token JWT Bearer.

`app/routers/auth.py`

Router

```
router = APIRouter(prefix="/users", tags=["Authentication"])
```

Constantes de módulo

Este archivo no define constantes de módulo.

Funciones auxiliares privadas

`_resumen_calificaciones(db: Session, user_id: int) -> dict`

Docstring: "Promedio, cantidad y últimos comentarios recibidos por un usuario. Sirve igual para pasajero y conductor: las calificaciones son bidireccionales, así que ambos reciben y a ambos les interesa ver cómo los califican."

Pasos: 1. Consulta agregada `func.avg(models.Rating.score)` y `func.count(models.Rating.rating_id)` filtrando por `models.Rating.rated_id == user_id`, y toma `.first()` (devuelve la tupla (promedio,

cantidad)). 2. Consulta las últimas 5 calificaciones recibidas: filtro por `rated_id == user_id`, `order_by(models.Rating.created_at.desc())`, `.limit(5)`. 3. Devuelve el dict: - `rating_avg`: `round(float(promedio), 1)` o `None` si `promedio` is `None`. - `rating_count`: cantidad or 0. - `rating_comentarios`: lista de dicts `{"score", "comment", "created_at"}` — **solo se incluyen las filas que tienen comment no vacío** (for `r` in `recientes` if `r.comment`), por lo que la lista puede tener menos de 5 elementos, o estar vacía.

POST /users/register

- **Función:** `register_passenger(user: schemas.UserCreate, request: Request, db: Session = Depends(get_db))`
- **status_code:** `status.HTTP_201_CREATED (201)`. No declara `response_model`.
- **Quién puede llamarlo:** público, sin autenticación.
- **Cuerpo esperado** (`schemas.UserCreate`): `full_name: str`, `email: EmailStr`, `password: str` (mínimo 8 caracteres, `Field(..., min_length=8)`), `phone_number: str`.

Qué hace: 1. Busca `models.User` con `email == user.email`. Si existe → error 400. 2. Hashea la contraseña con `security.get_password_hash(user.password)`. 3. Crea `models.User` con `full_name`, `email`, `phone_number`, `password_hash` y **`role="PASSENGER"` fijo** (no se puede registrar un conductor ni un admin por esta vía; el rol `DRIVER` se otorga después desde el panel de admin). 4. `db.add` → `db.commit` → `db.refresh`. 5. Registra auditoría. 6. Devuelve el objeto ORM `new_user` completo (sin `response_model`, FastAPI lo serializa entero, **incluyendo password_hash** — ver Observaciones).

Respuesta: el modelo `User` serializado completo (todas las columnas de la tabla `User`).

Errores: - 400 — "Email is already registered in Turify"

Auditoría: `registrar_log(action="REGISTER", user_id=new_user.user_id, entity="User", entity_id=new_user.user_id, detail=f"Nuevo pasajero registrado: {new_user.email}", ip_address=request.client.host if request.client else None)`

Notificaciones: ninguna.

POST /users/login

- **Función:** `login(request: Request, form_data: OAuth2PasswordRequestForm = Depends(), db: Session = Depends(get_db))`
- **response_model:** `schemas.TokenResponse` (`access_token: str`, `token_type: str`). Sin `status_code` explícito (200).
- **Quién puede llamarlo:** público.
- **Formato del cuerpo:** `application/x-www-form-urlencoded` con `username` (que aquí es el email) y `password`, por usar `OAuth2PasswordRequestForm`.

Qué hace: 1. Obtiene la IP: `request.client.host if request.client else None`. 2. Busca el usuario por `models.User.email == form_data.username`. 3. Si no existe o `security.verify_password(form_data.password, user.password_hash)` es falso: - Registra auditoría `LOGIN_FAILED` (con `user_id=user.user_id if user else None`, es decir el log guarda el id cuando el usuario existe pero la contraseña es incorrecta). - Lanza 400. 4. Genera el token:

`security.create_access_token(data={"sub": str(user.user_id)})`. 5. Registra auditoría `LOGIN`. 6. Devuelve `{"access_token": access_token, "token_type": "bearer"}`.

Respuesta: `{"access_token", "token_type"}`.

Errores: - 400 — "Email o contraseña incorrectos" (mismo mensaje para usuario inexistente y contraseña errónea, lo cual es correcto para no filtrar qué emails existen).

Auditoría: - Fallo: `action="LOGIN_FAILED", entity="User", detail=f"Login fallido para: {form_data.username}"`, `ip_address=ip` (no envía `entity_id`). - Éxito: `action="LOGIN", entity="User", entity_id=user.user_id, detail=f"Login exitoso: {user.email} (rol: {user.role})"`, `ip_address=ip`.

GET /users/me

- **Función:** `get_my_profile(current_user: models.User = Depends(get_current_user))`
- **response_model:** `schemas.UserResponse` → claves `user_id`, `full_name`, `email`, `role`, `phone_number` (opcional), `profile_photo_url` (opcional).
- **Quién puede llamarlo:** cualquier usuario autenticado (cualquier rol). Sin token, `get_current_user` responde con el error de autenticación correspondiente.
- **Qué hace:** devuelve directamente `current_user`; el `response_model` recorta todo lo demás (por eso `phone_number` se declara explícitamente en el esquema — el comentario del código lo explica: el formulario de conductor lo necesita para precargarse).
- **No consulta la base de datos, no registra auditoría, no crea notificaciones, no lanza `HTTPException` propias.**

GET /users/me/profile

Comentario del código: `# SCRUM-113` — GET `/users/me` (enriquecido con datos completos según rol)

- **Función:** `get_full_profile(current_user: models.User = Depends(get_current_user), db: Session = Depends(get_db))`
- **response_model:** ninguno (devuelve un `dict` construido a mano). Sin `status_code` explícito.
- **Quién puede llamarlo:** cualquier usuario autenticado. La respuesta cambia según `current_user.role`.

Qué hace:

1. Construye el dict `base` común a todos los roles: `user_id`, `full_name`, `email`, `phone_number`, `role`, `profile_photo_url`, `age`.
2. **Si `role == 'PASSENGER'`:** - `historial_viajes` (SCRUM-116): consulta `ServiceRequest` con `passenger_id == current_user.user_id`, `order_by(created_at.desc())`, `.limit(20)`. Cada elemento: `request_id`, `origin`, `destination`, `departure_time` (ISO o None), `status`, `created_at` (ISO o None). **No filtra por estado**, así que incluye también los `CANCELLED`. - `ofertas_activas`: `count()` de `DriverOffer` unido con `ServiceRequest` por `request_id`, filtrando `ServiceRequest.passenger_id == current_user.user_id` y `DriverOffer.status.in(['DRIVER_OFFERED', 'PASSENGER_COUNTER_OFFERED'])`. Es decir: negociaciones abiertas sobre cualquiera de sus viajes.

3. Si `role == 'DRIVER'`: – documentos: todos los `Document` con `user_id == current_user.user_id`. Cada elemento: `document_id`, `document_type`, `verification_status`, `file_url`, `years_experience`, `license_categories` (los dos últimos, según el comentario, *"solo tienen valor real en el documento RUNT"*). – conductor_verificado: `current_user.conductor_verificado` or `False` (HU38 – badge de conductor verificado, RUNT aprobado por el admin). – vehiculo: primer `Vehicle` con `owner_id == current_user.user_id`; se expone `{"vehicle_id", "plate", "capacity", "photo_url"}`, o `None` si no tiene vehículo. – empresa_afiliada: si `current_user.affiliated_company` tiene valor, busca `AffiliatedCompany` por `company_id` y devuelve `{"company_id", "name"}`; si no, `None`.
4. Para ambos roles (y también para `ADMIN`, que no entra en ninguna rama):
`base.update(_resumen_calificaciones(db, current_user.user_id))`, lo que añade `rating_avg`, `rating_count` y `rating_comentarios`.

Respuesta (claves finales): – Siempre: `user_id`, `full_name`, `email`, `phone_number`, `role`, `profile_photo_url`, `age`, `rating_avg`, `rating_count`, `rating_comentarios`. – Pasajero, además: `historial_viajes`, `ofertas_activas`. – Conductor, además: `documentos`, `conductor_verificado`, `vehiculo`, `empresa_afiliada`.

Errores: ninguno propio. **Auditoría:** no. **Notificaciones:** no.

PATCH /users/me/phone

Comentario del código: # SCRUM-114 – PATCH /users/me – actualizar teléfono

- **Función:** `update_phone(payload: schemas.UpdatePhoneRequest, current_user: models.User = Depends(get_current_user), db: Session = Depends(get_db))`
- **Cuerpo** (`schemas.UpdatePhoneRequest`): `phone_number: str`.
- **Sin `response_model` ni `status_code`.**
- **Quién puede llamarlo:** cualquier usuario autenticado, sobre su propio registro.

Qué hace: asigna `current_user.phone_number = payload.phone_number`, hace `db.commit()` y registra auditoría. **No valida el formato del teléfono.**

Respuesta: `{"message": "Teléfono actualizado correctamente.", "phone_number": current_user.phone_number}`

Errores: ninguno propio.

Auditoría: `registrar_log(db, action="UPDATE_PROFILE", user_id=current_user.user_id, entity="User", entity_id=current_user.user_id, detail="Teléfono actualizado")`

PATCH /users/me/password

Comentario del código: # SCRUM-115 – PATCH /users/me/password – cambiar contraseña

- **Función:** `update_password(payload: schemas.UpdatePasswordRequest, current_user: models.User = Depends(get_current_user), db: Session = Depends(get_db))`
- **Cuerpo** (`schemas.UpdatePasswordRequest`): `current_password: str`, `new_password: str` (`Field(..., min_length=8)`).
- **Quién puede llamarlo:** cualquier usuario autenticado.

Qué hace: 1. Verifica `security.verify_password(payload.current_password, current_user.password_hash)`. Si falla → 400. 2. Reemplaza `current_user.password_hash` con `security.get_password_hash(payload.new_password)`. 3. `db.commit()` y auditoría.

Respuesta: `{"message": "Contraseña actualizada correctamente."}`

Errores: - 400 — "La contraseña actual es incorrecta."

Auditoría: `action="UPDATE_PASSWORD", entity="User", entity_id=current_user.user_id, detail="Contraseña cambiada"`.

app/routers/drivers.py

Router

```
router = APIRouter(prefix="/drivers", tags=["Modo Conductor"])
```

Constantes de módulo

Constante	Valor
SUPABASE_URL	<code>os.getenv("SUPABASE_URL")</code>
SUPABASE_SERVICE_KEY	<code>os.getenv("SUPABASE_SERVICE_KEY")</code> — comentario: # service_role key (no anon)
TIPOS_PERMITIDOS	<code>{"application/pdf": "pdf", "image/jpeg": "jpg", "image/jpg": "jpg", "image/png": "png", "image/webp": "webp"}</code>
TAMANO_MAXIMO_MB	5
RANGOS_CATEGORIA	<code>[(1, 4, "SEDAN", (1500, 3000)), (5, 10, "VAN", (2000, 4000)), (11, 19, "MICROBUS", (2500, 5000)), (20, 35, "BUS", (3000, 6000)), (36, 60, "BUS_GRANDE", (3500, 7000))]</code> — tuplas (capacidad mínima, capacidad máxima, categoría, rango COP/km)

Comentario sobre `RANGOS_CATEGORIA` (HU55): "Rangos en COP/km, orientativos — el conductor puede moverse dentro de su categoría pero no salirse de ella (evita tarifas absurdas por error)."

Funciones auxiliares (todas públicas, ninguna empieza con `_` en este archivo)

`get_supabase() -> Client`

Devuelve `create_client(SUPABASE_URL, SUPABASE_SERVICE_KEY)`. Si falta alguna de las dos variables de entorno lanza `HTTPException(500, "Supabase no configurado. Verifica SUPABASE_URL y SUPABASE_SERVICE_KEY en el .env")`.

`calcular_categoria(capacidad: int) -> str`

Recorre `RANGOS_CATEGORIA` y devuelve la categoría cuyo rango contiene `capacidad`. Si no cae en ninguno: "BUS_GRANDE" si `capacidad > 60`, si no "SEDAN" (este último caso cubre `capacidad <= 0`).

rango_tarifa_km(categoria: str)

Devuelve `list(rango)` de la categoría indicada, o `[1500, 3000]` (el rango de `SEDAN`) si la categoría no existe.

validar_archivo(file: UploadFile) -> str

Docstring: "Valida tipo MIME y devuelve la extensión. Lanza `HTTPException` si no es válido." Lee `file.content_type` or `""`; si no está en `TIPOS_PERMITIDOS` lanza 400 con `f"Tipo de archivo no permitido: {content_type}. Solo se aceptan PDF, JPG, PNG y WEBP."`. Devuelve la extensión mapeada.

async upload_to_supabase(supabase: Client, file: UploadFile, bucket: str, path: str) -> str

Docstring: "Sube un archivo a Supabase Storage y devuelve la URL firmada (expira en 15 min). Para acceso permanente del admin se usa signed URL con TTL largo." (El docstring contradice al código — ver Observaciones.)

Pasos: 1. `ext = validar_archivo(file)`. 2. `contenido = await file.read()`; calcula `tamano_mb = len(contenido) / (1024 * 1024)`. Si supera `TAMANO_MAXIMO_MB` → 400 con `f"El archivo supera el límite de {TAMANO_MAXIMO_MB}MB ({tamano_mb:.1f}MB)."`. 3. `file_path = f"{path}/{uuid.uuid4()}.{ext}"` — nombre aleatorio, nunca sobrescribe. 4.

`supabase.storage.from_(bucket).upload(path=file_path, file=contenido, file_options={"content-type": file.content_type})`. Cualquier excepción → 500 con `f"Error subiendo archivo a Supabase: {str(e)}"`. 5. `create_signed_url(file_path, expires_in=31536000)` (un año, no 15 minutos) y devuelve `signed["signedURL"]`. Excepción → 500 con `f"Error generando URL firmada: {str(e)}"`.

Buckets usados: `"turify-fotos"` (fotos de perfil y de vehículo) y `"turify-documentos"` (documentos legales).

POST /drivers/register-details

- Función:

```

async def register_driver_info(
    age: int = Form(...),
    affiliated_company: int = Form(...),
    plate: str = Form(...),
    capacity: int = Form(...),
    tiene_ac: bool = Form(False),
    tiene_wifi: bool = Form(False),
    tiene_bano: bool = Form(False),
    tiene_musica: bool = Form(False),
    tiene_maletero_amplio: bool = Form(False),
    tiene_sillas_bebe: bool = Form(False),
    acepta_mascotas: bool = Form(False),
    profile_photo: UploadFile = File(...),
    vehicle_photo: UploadFile = File(...),
    doc_soat: UploadFile = File(...),
    doc_licencia: UploadFile = File(...),
    doc_tarjeta_operacion: UploadFile = File(...),
    doc_tecnomecanica: UploadFile = File(...),
    doc_seguros: UploadFile = File(...),
    db: Session = Depends(get_db),
    current_user: models.User = Depends(get_current_user)
)

```

- Sin `response_model` ni `status_code` (200). Recibe `multipart/form-data`.
- **Quién puede llamarlo:** cualquier usuario autenticado. **No comprueba el rol** — la idea es que lo llame un `PASSENGER` que quiere convertirse en conductor; el rol `DRIVER` se lo dará el admin al aprobar los documentos.

Qué hace, paso a paso:

1. **Bloqueo por reenvío:** busca cualquier `Document` del usuario con `verification_status.in(["PENDING", "APPROVED"])`. Si existe alguno → 400. Es decir, solo se puede reenviar si **todos** sus documentos fueron rechazados (o si no tiene ninguno).
2. `supabase = get_supabase(); base_path = f"drivers/{current_user.user_id}"`.
3. **A. Datos del usuario:** recarga el `User` por id (si no existe → 404), asigna `age` y `affiliated_company`, y sube la foto de perfil a `turify-fotos` en `{base_path}/profile`, guardando la URL firmada en `profile_photo_url`.
4. **B. Vehículo:** busca `Vehicle` con `owner_id == current_user.user_id`. **Solo si no tiene uno**, sube la foto del vehículo a `turify-fotos` en `{base_path}/vehicle` y crea `models.Vehicle` con `owner_id`, `company_id=affiliated_company`, `plate=plate.upper()`, `capacity`, `photo_url` y las siete comodidades del formulario. Si ya tenía vehículo, **la placa, la capacidad y las comodidades enviadas se ignoran silenciosamente**.
5. **C. Documentos legales:** itera sobre la lista fija `docs_to_save`:

document_type guardado	campo del formulario	subcarpeta
"SOAT"	<code>doc_soat</code>	<code>soat</code>
"Licencia de Conduccion"	<code>doc_licencia</code>	<code>licencia</code>

document_type guardado	campo del formulario	subcarpeta
"Tarjeta de operacion"	doc_tarjeta_operacion	tarjeta_operacion
"Tecnomecanica"	doc_tecnomecanica	tecnomecanica
"Seguros Contractual y extracontractual"	doc_seguros	seguros

Para cada uno: busca el documento previo del mismo tipo y usuario. Si existe y su estado está en ["PENDING", "APPROVED"], hace `continue` ("Ya existe y no fue rechazado, no tocar"). En caso contrario sube el archivo a `turify-documentos` en `{base_path}/{nombre_clave}` y: - si había un documento previo (rechazado) → actualiza `file_url`, pone `verification_status = "PENDING"` y **limpia `ai_extracted_data`, `ai_confidence` y `ai_observations` a `None`**; - si no había → crea `models.Document(user_id, document_type, file_url, verification_status="PENDING")`. 6. `db.commit()`.

Todo el bloque 3–6 está dentro de un `try`. Cualquier `HTTPException` hace `db.rollback()` y se re-lanza; cualquier otra excepción hace `db.rollback()` y se convierte en `500`.

Respuesta: `{"status": "success", "message": "Documentos enviados exitosamente. El administrador los revisará pronto."}`

Errores: - `400` — "Ya tienes documentos enviados. Debes esperar la revisión del administrador o tener documentos rechazados para volver a enviar." - `404` — "Usuario no encontrado" - `400` — `f"Tipo de archivo no permitido: {content_type}. Solo se aceptan PDF, JPG, PNG y WEBP."` (desde `validar_archivo`) - `400` — `f"El archivo supera el límite de {TAMANO_MAXIMO_MB}MB ({tamano_mb:.1f}MB)."` - `500` — "Supabase no configurado. Verifica `SUPABASE_URL` y `SUPABASE_SERVICE_KEY` en el `.env`" - `500` — `f"Error subiendo archivo a Supabase: {str(e)}"` / `f"Error generando URL firmada: {str(e)}"` - `500` — `f"Error procesando el registro: {str(e)}"`

Auditoría: no registra ninguna, pese a que `app/audit.py` documenta una acción `REGISTER_DRIVER` (ver Observaciones).

Notificaciones: ninguna.

POST /drivers/upload-runt

Comentario del código: `#` — `HU37` — Subida de documento `RUNT` (`SCRUM-183`) — "A diferencia de los 5 documentos obligatorios de `/register-details`, el `RUNT` es opcional y se sube DESPUÉS del registro, para declarar/verificar años de experiencia. No afecta el rol `DRIVER` ya activo — solo habilita el badge de 'conductor verificado' (`HU38`) cuando el admin lo aprueba."

• Función:

```

async def upload_runt(
    years_experience: int = Form(...),
    license_categories: str = Form(None),
    doc_runt: UploadFile = File(...),
    db: Session = Depends(get_db),
    current_user: models.User = Depends(get_current_user)
)

```

- Sin `response_model` ni `status_code`.

- **Quién puede llamarlo:** solo **DRIVER**; si no → 403 "Solo los conductores pueden subir el RUNT."

Qué hace: 1. Valida el rol. 2. Valida $0 \leq \text{years_experience} \leq 80$; fuera de rango → 400. 3. Busca un Document de tipo "RUNT" del usuario con estado en ["PENDING", "APPROVED"]. Si existe → 400 (no se puede reenviar hasta que sea rechazado). 4. Sube el archivo a turify-documentos en drivers/{user_id}/runt. 5. Busca cualquier Document RUNT previo (sin filtro de estado, o sea el rechazado): - si existe: actualiza file_url, verification_status = "PENDING", years_experience, license_categories y limpia ai_extracted_data, ai_confidence, ai_observations; - si no: crea models.Document(user_id, document_type="RUNT", file_url, verification_status="PENDING", years_experience, license_categories). 6. db.commit().

Respuesta: {"status": "success", "message": "RUNT enviado correctamente. El administrador revisará tu experiencia pronto."}

Errores: - 403 — "Solo los conductores pueden subir el RUNT." - 400 — "Los años de experiencia declarados no son válidos." - 400 — "Ya tienes un RUNT enviado. Espera la revisión del administrador, o a que sea rechazado para volver a enviarlo." - Más los errores de get_supabase / validar_archivo / upload_to_supabase descritos arriba.

Auditoría: no. **Notificaciones:** no. Sin `try/except`: una excepción de base de datos sube sin `rollback` explícito.

GET /drivers/my-documents

- **Función:** get_my_documents(db: Session = Depends(get_db), current_user: models.User = Depends(get_current_user))

- **response_model:** List[schemas.DocumentResponse] → cada elemento: document_id, user_id, document_type (validado contra DocumentTypeEnum), file_url, verification_status (validado contra VerificationStatusEnum), years_experience, license_categories.

- **Quién puede llamarlo:** cualquier usuario autenticado; devuelve **solo sus propios documentos** (Document.user_id == current_user.user_id). No valida el rol.

- **Qué hace:** una sola consulta, sin filtros de estado ni orden.

- **Errores propios:** ninguno. Sí puede fallar con 500 de validación de Pydantic si un document_type o verification_status en la base no coincide con el enum (el comentario de schemas.py documenta exactamente ese bug ya corregido).

- **Auditoría / notificaciones:** no.

GET /drivers/registration-status• **Función:**

```
get_registration_status(db: Session = Depends(get_db), current_user: models.User = Depends(get_current_user))
```

• **Sin response_model.**• **Quién puede llamarlo:** cualquier usuario autenticado, sobre sus propios documentos.

Qué hace — máquina de estados de la respuesta, evaluada en este orden:

1. Consulta todos los `Document` del usuario. Si la lista está vacía → `{"estado": "SIN_DOCUMENTOS", "mensaje": "No has enviado documentos aún."}`
2. `estados = [d.verification_status for d in documentos]`.
3. Si **todos** son `"APPROVED"` → `{"estado": "APROBADO", "mensaje": "Todos tus documentos han sido aprobados. ¡Ya eres conductor!"}`
4. Si **alguno** es `"REJECTED"` → `{"estado": "RECHAZADO", "mensaje": "Algunos documentos fueron rechazados. Puedes volver a enviarlos.", "documentos_rechazados": [tipos rechazados]}`
5. En cualquier otro caso → `{"estado": "PENDIENTE", "mensaje": "Tus documentos están siendo revisados por el administrador.", "documentos": [{"tipo", "estado"}, ...]}`

Nota importante: **el RUNT cuenta en este cálculo**. Un conductor ya aprobado que suba un RUNT pasa a ver estado `"PENDIENTE"`, y si el admin le rechaza el RUNT, el estado global se vuelve `"RECHAZADO"` aunque sus cinco documentos obligatorios estén aprobados.

Errores: ninguno. **Auditoría / notificaciones:** no.

PATCH /drivers/location

Comentario del código: `# — HU26 — Rango geográfico de conductores`

• **Función:** `update_driver_location(payload: schemas.DriverLocationUpdate, db: Session = Depends(get_db), current_user: models.User = Depends(get_current_user))`• **Cuerpo** (`schemas.DriverLocationUpdate`): `current_lat: Optional[float]`, `current_lng: Optional[float]`, `is_online: Optional[bool]` — los tres opcionales.• **Docstring:** *"Permite al conductor actualizar su posición actual (current_lat/current_lng) y su estado online/offline. El frontend del conductor debe llamar este endpoint periódicamente mientras esté disponible/en viaje (Supabase Realtime hace el resto en el lado de Supabase, pero la fuente de verdad la actualizamos por este endpoint estándar de FastAPI)."*• **Quién puede llamarlo:** solo **DRIVER**; si no → 403 `"Solo los conductores pueden actualizar su ubicación."`

Qué hace: para cada campo, si viene distinto de `None`, lo asigna sobre `current_user` (patch parcial real: enviar `{}` no borra nada). `db.commit() + db.refresh(current_user)`.

Respuesta: `{"current_lat", "current_lng", "is_online"}` — las coordenadas convertidas con `float(...)` o `None`.

Errores: 403 — `"Solo los conductores pueden actualizar su ubicación."`

Auditoría / notificaciones: no.

GET /drivers/vehicle

Comentario del código: # — HU55 — Comodidades, capacidad real y tarifas del vehículo (SCRUM-207)

- **Función:** `get_my_vehicle(db: Session = Depends(get_db), current_user: models.User = Depends(get_current_user))`
- **response_model:** `schemas.VehicleSettingsResponse`.
- **Quién puede llamarlo:** solo **DRIVER**; si no → 403 "Solo los conductores tienen vehículo registrado."

Qué hace: 1. Busca `Vehicle` con `owner_id == current_user.user_id`. Si no hay → 404. 2. `capacidad_para_categoria = vehiculo.capacidad_real or vehiculo.capacity` — la capacidad real declarada por el conductor prevalece sobre la de registro. 3. `categoria = calcular_categoria(capacidad_para_categoria)`. 4. Devuelve el dict con las tarifas convertidas a float (o None) y `tarifa_km_rango = rango_tarifa_km(categoria)`.

Respuesta (claves exactas): `vehicle_id, plate, capacity, capacidad_real, categoria, tarifa_km_base, tarifa_km_rango, tarifa_espera_hora, tarifa_dia, km_incluidos_por_dia, recargo_dificil_acceso, tiene_ac, tiene_wifi, tiene_bano, tiene_musica, tiene_maletero_amplio, tiene_sillas_bebe, acepta_mascotas, cargo_mascota, acepta_menores_2_anos`.

Errores: - 403 — "Solo los conductores tienen vehículo registrado." - 404 — "Aún no tienes un vehículo registrado. Completa tu registro de conductor primero."

PATCH /drivers/vehicle

- **Función:** `update_my_vehicle(payload: schemas.VehicleSettingsUpdate, db: Session = Depends(get_db), current_user: models.User = Depends(get_current_user))`
- **response_model:** `schemas.VehicleSettingsResponse`.
- **Cuerpo** (`schemas.VehicleSettingsUpdate`): `capacidad_real (ge=1, le=60), tiene_ac, tiene_wifi, tiene_bano, tiene_musica, tiene_maletero_amplio, tiene_sillas_bebe, acepta_mascotas, cargo_mascota (ge=0), acepta_menores_2_anos`. Comentario del esquema: "Las tarifas (por km, espera, día, etc.) NO las decide el conductor — quedaron fuera de este schema a propósito, así que PATCH /drivers/vehicle no puede tocarlas aunque alguien las envíe en el body."
- **Quién puede llamarlo:** solo **DRIVER**; si no → 403 "Solo los conductores pueden editar su vehículo."

Qué hace: 1. Valida el rol y busca el vehículo (mismo 404 que el GET). 2. `datos = payload.model_dump(exclude_unset=True)` — solo los campos enviados explícitamente. 3. **Regla de negocio:** si se envía `cargo_mascota > 0` y el valor efectivo de `acepta_mascotas` (el enviado, o el actual del vehículo si no se envía) es `False` → 400 "No puedes configurar un cargo por mascota si no aceptas mascotas." 4. Aplica `setattr(vehiculo, campo, valor)` para cada campo enviado, `db.commit(), db.refresh(vehiculo)`. 5. Devuelve `get_my_vehicle(db=db, current_user=current_user)` — reutiliza el GET para recalcular `categoria` y `tarifa_km_rango` con la nueva capacidad.

Errores: - 403 — "Solo los conductores pueden editar su vehículo." - 404 — "Aún no tienes un vehículo registrado. Completa tu registro de conductor primero." - 400 — "No puedes configurar un cargo por mascota si no aceptas mascotas."

Auditoría / notificaciones: no.

GET /drivers/earnings

Comentario del código: # — HU52 — Panel de ganancias del conductor (SCRUM-204)

- **Función:** `get_driver_earnings(db: Session = Depends(get_db), current_user: models.User = Depends(get_current_user))`
- **Sin response_model.**
- **Quién puede llamarlo:** solo **DRIVER**; si no → 403 "Solo los conductores pueden ver su panel de ganancias."

Docstring: "Resumen de ganancias y rendimiento del conductor autenticado: ganancias de la semana y el mes (suma del precio aceptado de sus viajes COMPLETED), número total de viajes completados, horarios más activos (conteo por hora del día) y sus 3 rutas más frecuentes. La 'calificación promedio' del criterio de aceptación queda pendiente: todavía no existe HU46 (Calificaciones bidireccionales / SCRUM-194), así que el frontend debe mostrar 'Próximamente' para ese dato por ahora." (Esta última frase ya no aplica — ver Observaciones.)

Cálculo, paso a paso:

1. **Ventanas de tiempo:** `python ahora = datetime.now(timezone.utc) inicio_semana = ahora - timedelta(days=7) inicio_mes = ahora - timedelta(days=30)` Un comentario extenso explica por qué se usa `datetime.now(timezone.utc)` y no `datetime.utcnow()`: `departure_time` es `TIMESTAMP(timezone=True)`, y comparar naive contra aware lanzaba "can't compare offset-naive and offset-aware datetimes", lo que producía un 500 sin cabeceras CORS y el navegador lo veía como un fallo de red.
2. **Consulta de viajes:** `ServiceRequest JOIN DriverOffer` por `request_id`, seleccionando `request_id, origin, destination, departure_time, offered_price`, con filtros: - `ServiceRequest.status == "COMPLETED" - DriverOffer.driver_id == current_user.user_id - DriverOffer.status == "ACCEPTED"`

El comentario explica la clave del cálculo: "con el precio que realmente se aceptó (`DriverOffer.status == ACCEPTED` es el precio final del viaje, no `offered_price` de ofertas rechazadas/contraofertadas)". Como una contraoferta **sobrescribe** `offered_price` en la misma fila, el precio de la oferta `ACCEPTED` es siempre el precio final negociado.

1. **Acumulación** (bucle en Python, no en SQL): - `precio = float(v.offered_price or 0)`. - Si `fecha.tzinfo is None`, se le asigna UTC ("Cinturón y tirantes: si alguna fila llegara sin zona (SQLite en pruebas, o un dato viejo), se asume UTC en vez de tumbar la pestaña entera."). - `ganancias_semana += precio` si `fecha >= inicio_semana`; `ganancias_mes += precio` si `fecha >= inicio_mes`. **Ambas ventanas se miden sobre `departure_time`, no sobre la fecha de finalización**, y son ventanas móviles de 7 y 30 días, no "semana calendario" ni "mes calendario". Un viaje de los últimos 7 días suma en las dos. - `horarios[fecha.hour] += 1` — lista de 24 posiciones inicializada como `[0] * 24`. - `rutas[f"{v.origin} → {v.destination}"] += 1` — la clave es el texto completo de origen y destino concatenado con la flecha.

2. `top_rutas = sorted(rutas.items(), key=lambda x: x[1], reverse=True)[:3]`.
3. **Calificación (HU46):** agregada `func.avg(Rating.score) / func.count(Rating.rating_id)` filtrando `Rating.rated_id == current_user.user_id`. `calificacion_promedio = round(float(promedio_rating), 1)` o `None`. Nótese que **cuenta todas las calificaciones recibidas por el usuario, sin filtrar por viaje completado ni por rol del que califica**.

Respuesta (claves exactas):

```
ganancias_semana      (float, round 2)
ganancias_mes         (float, round 2)
viajes_completados    (len(viajes) – número de filas de la consulta)
calificacion_promedio (float 1 decimal o None)
calificacion_cantidad (int)
horarios_activos      (lista de 24 enteros, conteo por hora 0-23)
top_rutas              ([{"ruta": str, "viajes": int}] – máximo 3)
```

Errores: 403 — "Solo los conductores pueden ver su panel de ganancias."

Auditoría / notificaciones: no.

GET /drivers/{driver_id}/public-profile

Comentario del código: # — HU38 – Perfil público del conductor (lo ve el pasajero desde una oferta) –

- **Función:** `get_driver_public_profile(driver_id: int, db: Session = Depends(get_db), current_user: models.User = Depends(get_current_user))`
- **Sin response_model.**
- **Quién puede llamarlo:** cualquier usuario autenticado (no se valida el rol del que consulta). Solo devuelve datos si el `driver_id` corresponde a un usuario con `role == "DRIVER"`.

Qué hace: 1. Busca User con `user_id == driver_id` y `role == "DRIVER"`. Si no existe → 404 "Conductor no encontrado." 2. Calificación: `avg(Rating.score)` y `count(Rating.rating_id)` con `rated_id == driver_id`. 3. viajes_completados: `count()` sobre `ServiceRequest JOIN DriverOffer`, filtrando `ServiceRequest.status == "COMPLETED"`, `DriverOffer.driver_id == driver_id`, `DriverOffer.status == "ACCEPTED"` (misma definición que en `/earnings`). 4. Vehículo: primer `Vehicle` con `owner_id == driver_id`. Si existe, `capacidad = vehiculo.capacidad_real` or `vehiculo.capacity` y se expone `plate`, `capacity` (la efectiva), `categoria` (con `calcular_categoria`) y las siete comodidades. **No expone tarifas ni cargo_mascota**. 5. Empresa afiliada: solo `{"name": emp.name}` (a diferencia de `/users/me/profile`, aquí no se expone `company_id`). 6. **Años de experiencia:** solo se calcula si `conductor.conductor_verificado` es verdadero; entonces busca el `Document` de tipo "RUNT" con `verification_status == "APPROVED"` y toma su `years_experience`. Comentario: "El dato solo se expone si el RUNT está aprobado — es lo que respalda el sello". Si el conductor no está verificado, `years_experience` queda en `None`.

Respuesta (claves exactas): `user_id`, `full_name`, `profile_photo_url`, `rating_avg`, `rating_count`, `viajes_completados`, `vehiculo` (dict o `None`), `empresa_afiliada` (dict o `None`), `conductor_verificado` (bool), `years_experience` (int o `None`).

Errores: 404 — "Conductor no encontrado."

Auditoría / notificaciones: no.

app/routers/admin.py

Router

```
router = APIRouter(prefix="/admin", tags=["Admin"])
```

Esquemas locales (definidos en este archivo, no en `schemas.py`)

```
class DocumentVerifyRequest(BaseModel):
    verification_status: str # 'APPROVED' | 'REJECTED'
    years_experience: int | None = None
```

Comentario: "HU38 — el admin puede corregir/confirmar los años de experiencia declarados al aprobar un documento RUNT. Se ignora para cualquier otro tipo de documento."

```
class DocumentOut(BaseModel):
    document_id: int
    document_type: str
    file_url: str
    verification_status: str
    years_experience: int | None = None
    license_categories: str | None = None
    class Config: from_attributes = True
```

(Nótese que `document_type` y `verification_status` son `str` libres, sin enum — a diferencia de `schemas.DocumentResponse`.)

```
class DriverPendingOut(BaseModel):
    user_id: int
    full_name: str
    email: str
    phone_number: str
    profile_photo_url: str | None
    role: str
    conductor_verificado: bool = False
    documents: List[DocumentOut]
    class Config: from_attributes = True
```

Dependencia de rol

```
def get_admin_user(current_user: models.User = Depends(get_current_user)):
    if current_user.role != 'ADMIN':
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="Acceso denegado. Se requiere rol ADMIN."
        )
    return current_user
```

Todos los endpoints de este router dependen de `get_admin_user`, así que todos devuelven `403` "Acceso denegado. Se requiere rol ADMIN." a quien no sea `ADMIN`. No se repite en cada endpoint más abajo.

Constantes de módulo

Ninguna, más allá de `router`. (Se llama `load_dotenv()` a nivel de módulo.)

GET /admin/drivers/pending

- **Función:** `get_pending_drivers(db: Session = Depends(get_db), admin: models.User = Depends(get_admin_user))`
- **response_model:** `List[DriverPendingOut]`.

Qué hace: consulta `User JOIN Document` por `Document.user_id == User.user_id`, filtrando `Document.verification_status == 'PENDING'`, con `.distinct()`. Devuelve los usuarios ORM; el `response_model` serializa además la relación `documents`.

Punto importante: el JOIN filtra qué **usuarios** aparecen, pero la lista `documents` de cada usuario se resuelve por la relación del ORM y **contiene todos sus documentos**, no solo los `PENDING`. Así el admin ve la ficha completa del conductor.

Errores: solo el `403` de la dependencia. **Auditoría / notificaciones:** no.

PATCH /admin/documents/{document_id}/verify

- **Función:** `verify_document(document_id: int, payload: DocumentVerifyRequest, db: Session = Depends(get_db), admin: models.User = Depends(get_admin_user))`
- Sin **response_model** ni **status_code**.

Qué hace, paso a paso — este es el corazón de la verificación de documentos:

1. **Valida el estado enviado** contra `['APPROVED', 'REJECTED', 'PENDING']`. Si no coincide → `400` "Estado inválido. Use 'APPROVED', 'REJECTED' o 'PENDING'." (nótese que el comentario del esquema solo menciona `'APPROVED' | 'REJECTED'`, pero el código también acepta `'PENDING'`, útil para revertir una decisión).
2. Busca el `Document` por `document_id`. Si no existe → `404` "Documento no encontrado."

3. **HU38 — corrección de experiencia:** si `documento.document_type == 'RUNT'` y `payload.years_experience is not None`, sobrescribe `documento.years_experience` con el valor del admin. Para cualquier otro tipo de documento el campo se ignora.
4. Asigna `documento.verification_status = payload.verification_status`, `db.commit()`, `db.refresh(documento)`.
5. **Bifurcación por tipo de documento:**

a) Si es RUNT — comentario: *"el RUNT es opcional y posterior al registro: no toca el rol DRIVER, solo activa/desactiva el badge de 'conductor verificado'"*. Busca el `User` dueño del documento y le asigna: `python conductor.conductor_verificado = (payload.verification_status == 'APPROVED')` Es decir: aprobar activa el badge; rechazar o volver a `PENDING` lo desactiva. Segundo `db.commit()`.

b) Si NO es RUNT y `verification_status == 'APPROVED'` — activación del rol conductor. Consulta todos los `Document` del usuario con `document_type != 'RUNT'` (los obligatorios de registro). Si la lista no está vacía y **todos** están `'APPROVED'`, busca el `User` y le asigna `role = 'DRIVER'` y `status = 'ACTIVE'`, con un segundo `db.commit()`.

Es decir: **el rol DRIVER solo se otorga cuando el último documento obligatorio pendiente pasa a aprobado**, y el RUNT queda deliberadamente fuera de ese cómputo. Rechazar un documento **no** revierte el rol de un conductor ya activo.

1. Registra auditoría.

Respuesta:

```
{
  "message": f"Documento {payload.verification_status.lower()} correctamente.",
  "document_id": documento.document_id,
  "nuevo_estado": documento.verification_status
}
```

Errores: - 400 — "Estado inválido. Use 'APPROVED', 'REJECTED' o 'PENDING'." - 404 — "Documento no encontrado." - 403 — "Acceso denegado. Se requiere rol ADMIN."

Auditoría:

```
registrar_log(db, action="VERIFY_DOCUMENT", user_id=admin.user_id,
              entity="Document", entity_id=documento.document_id,
              detail=f"Documento {documento.document_type} {payload.verification_status.lower()} para
              usuario #{documento.user_id}")
```

(El `user_id` del log es el del **admin**, no el del conductor; el conductor queda identificado en `detail`.)

Notificaciones: ninguna — el conductor no recibe aviso de que su documento fue aprobado o rechazado; debe consultar `GET /drivers/registration-status`.

GET /admin/drivers/{user_id}/documents

- **Función:** `get_driver_documents(user_id: int, db: Session = Depends(get_db), admin: models.User = Depends(get_admin_user))`

- **response_model**: `List[DocumentOut]`.

Qué hace: verifica que exista un `User` con ese `user_id` (si no → 404 "Conductor no encontrado.") — nótese que **no comprueba que el usuario sea un conductor**, solo que exista — y devuelve todos sus `Document`.

Errores: 404 — "Conductor no encontrado."; 403 de la dependencia.

Auditoría / notificaciones: no.

GET /admin/documents/{document_id}/file

Comentario del código: "Proxy: descarga el archivo desde su URL (Supabase Storage) y lo sirve al frontend".

- **Función:** `async def proxy_documento(document_id: int, db: Session = Depends(get_db), admin: models.User = Depends(get_admin_user))`
- **Sin response_model**; devuelve un `fastapi.responses.Response` binario.

Qué hace: 1. Busca el documento; si no existe → 404 "Documento no encontrado." 2. `url_original = documento.file_url` (ya es una URL firmada de Supabase Storage; el comentario aclara: "no necesita firmarse aparte como pasaba con Cloudinary"). 3. `httpx.AsyncClient(timeout=30, follow_redirects=True)` hace un GET sobre esa URL. Si el código de respuesta no es 200 → 502 con `f"El almacenamiento devolvió {respuesta.status_code}"`. 4. Determina el `content_type`: toma `respuesta.headers.get("content-type", "application/pdf")`, y si la URL termina en **.pdf (en minúsculas) fuerza "application/pdf"**. 5. Devuelve `Response(content=respuesta.content, media_type=content_type, headers={"Content-Disposition": "inline"})` — para que el navegador lo muestre embebido en vez de descargarlo.

Errores: - 404 — "Documento no encontrado." - 502 — `f"El almacenamiento devolvió {respuesta.status_code}"` - 502 — `f"Error descargando documento: {str(e)}"` (cualquier otra excepción; las `HTTPException` se re-lanzan intactas gracias al `except HTTPException: raise` previo)

Auditoría / notificaciones: no.

GET /admin/logs

Comentario del código: "Vista de auditoría para el admin".

- **Función:** `get_audit_logs(limit: int = 100, action: str = None, db: Session = Depends(get_db), admin: models.User = Depends(get_admin_user))`
- **Parámetros de consulta:** `limit` (por defecto 100, **sin cota máxima**) y `action` (opcional, filtro exacto).
- **Sin response_model**.

Qué hace: `db.query(models.AuditLog).order_by(models.AuditLog.created_at.desc());` si `action` viene, añade `filter(models.AuditLog.action == action);` aplica `.limit(limit)`.

Respuesta: lista de dicts con claves `log_id`, `user_id`, `action`, `entity`, `entity_id`, `detail`, `ip_address`, `created_at` (ISO o None).

Errores: solo el 403 de la dependencia. **Auditoría:** no registra su propia consulta.

app/routers/service_requests.py

Router

```
router = APIRouter(prefix="/api/service-requests", tags=["Service Requests"])
```

Constantes de módulo

Constante	Valor	Propósito
RADIOS_NOTIFICACION_KM	[5, 10, 20, 40, 80]	Radios de expansión automática para la notificación inicial al publicar un viaje
RADIO_VISIBILIDAD_KM	60	Radio fijo que se guarda en ServiceRequest.search_radius_km y define quién ve el viaje en GET /pending
_ETIQUETAS_COMODIDAD	{ "tiene_ac": "Aire acondicionado", "tiene_wifi": "WiFi", "tiene_bano": "Baño", "tiene_musica": "Música", "tiene_maletero_amplio": "Maletero amplio", "tiene_sillas_bebe": "Sillas para bebé", "acepta_mascotas": "Acepta mascotas" }	Nombres legibles de las comodidades faltantes
_RANGOS_CATEGORIA_VEHICULO	[(1, 4, "SEDAN"), (5, 10, "VAN"), (11, 19, "MICROBUS"), (20, 35, "BUS"), (36, 60, "BUS_GRANDE")]	Copia local de la tabla de categorías de drivers.py, sin los rangos de tarifa

Comentario sobre los radios (HU26): *"se prueba 5km, si no hay nadie 10km, luego 20, 40 y por último 80km)"* y radio fijo de visibilidad guardado en cada solicitud para que el radar (GET /pending) no dependa de la ubicación de los conductores en el instante de publicar el viaje."

Funciones auxiliares

`_distancia_km(lat1, lon1, lat2, lon2)`

Docstring: *"Distancia en línea recta (fórmula Haversine) entre dos puntos lat/Ing."* Importa `math` dentro de la función. $R = 6371.0$ km. Devuelve $R * 2 * \text{atan2}(\text{sqrt}(a), \text{sqrt}(1 - a))$. Es distancia geodésica en línea recta, **no distancia por carretera**.

`_match_comodidades(db, conductor, service_request) -> dict`

Comentario (HU55): *"Filtro FLEXIBLE de comodidades (no excluyente): en vez de decir 'cumple / no cumple' a secas, dice CUÁNTAS de las comodidades pedidas cumple este conductor y cuáles le faltan,*

para que el pasajero decida con esa información (nunca se oculta un conductor solo por no tener el 100%)."

1. Construye la lista de pares (requiere_X del viaje, campo tiene_X del vehículo) para las siete comodidades.
2. exigidos = los campos cuyo requiere_X es verdadero. **Si no hay ninguno**, devuelve inmediatamente {"exigidas": 0, "cumplidas": 0, "cumple_todas": True, "faltantes": []} **sin consultar el vehículo**.
3. Busca Vehicle con owner_id == conductor.user_id.
4. Por cada campo exigido: tiene = bool(getattr(vehiculo, campo)) if vehiculo else False. Suma a cumplidas o añade _ETIQUETAS_COMODIDAD[campo] a faltantes.
5. Devuelve {"exigidas": len(exigidos), "cumplidas": cumplidas, "cumple_todas": cumplidas == len(exigidos), "faltantes": faltantes}.

_serializar_service_request(sr) -> dict

Comentario: "Convierte un ServiceRequest (ORM) en el dict base que espera schemas.ServiceRequestRead — se usa en /pending porque ahí se le agregan campos calculados (comodidades_) que no son columnas de la tabla."*

Claves devueltas: request_id, passenger_id, origin, destination, departure_time, return_time, trip_type, adults_count, children_count, has_pets, status, created_at, origin_lat (float o None), origin_lng (float o None), requiere_ac, requiere_wifi, requiere_bano, requiere_musica, requiere_maletero_amplio, requiere_sillas_bebe, requiere_acepta_mascotas.

_etiqueta_zona(origin: str) -> str

Docstring: "Heurística simple para sacar un nombre de zona legible del texto de origen (formato típico de Google Places: 'Calle X, Vereda Y, El Retiro, Antioquia, Colombia')." Parte el texto por comas, descarta vacíos y descarta las partes que en minúsculas son "colombia" o "antioquia"; si no queda nada devuelve "Zona sin nombre", y si queda algo devuelve **la última parte** (partes[-1]).

_categoria_vehiculo(capacidad: int) -> str

Igual que calcular_categoria de drivers.py, sobre _RANGOS_CATEGORIA_VEHICULO: devuelve la categoría del rango que contiene la capacidad, o "BUS_GRANDE" si capacidad > 60, o "SEDAN" en cualquier otro caso.

_promedio_calificacion(db, user_id: int)

Comentario: # HU46 — Calificaciones bidireccionales (SCRUM-194). Docstring: "Promedio y cantidad de calificaciones (Rating.score) recibidas por un usuario." Devuelve la **tupla**

(promedio_redondeado_1_decimal_o_None, cantidad_o_0), filtrando por Rating.rated_id == user_id.

crear_notificacion(db, user_id: int, title: str, message: str, tipo: str, offer_id: int = None)

Comentario: # --- Función auxiliar para crear notificaciones ---. Crea models.Notification(user_id, title, message, type=tipo, related_offer_id=offer_id) y hace db.add + db.commit(). Envuelta en try/except: ante cualquier excepción hace db.rollback() e imprime f"[Notificación] Error: {e}" — **nunca interrumpe el flujo del endpoint**, pero su rollback puede deshacer cambios no confirmados del endpoint que la llama (por eso todos los llamantes hacen db.commit() antes de notificar).

Tipos válidos según `models.Notification.type`: `'NEW_OFFER'`, `'COUNTER_OFFER'`, `'TRIP_ACCEPTED'`, `'TRIP_REJECTED'`, `'TRIP_STARTED'`, `'TRIP_COMPLETED'`, `'SYSTEM'`.

Estados de la máquina de negociación

ServiceRequest.status (enum `request_status`): `PENDING`, `ASSIGNED`, `IN_PROGRESS`, `COMPLETED`, `CANCELLED`, `SCHEDULED`.

DriverOffer.status (enum `offer_status`, por defecto `DRIVER_OFFERED`): `DRIVER_OFFERED`, `PASSENGER_COUNTER_OFFERED`, `ACCEPTED`, `REJECTED`.

Transiciones por endpoint:

Endpoint	Actor	ServiceRequest	DriverOffer
POST <code>/api/service-requests/</code>	Pasajero	→ <code>PENDING</code> (creación)	—
POST <code>/api/{request_id}/offers</code>	Conductor	(exige <code>PENDING</code>)	→ <code>DRIVER_OFFERED</code> (creación)
PATCH <code>/api/offers/{offer_id}/counter-offer</code>	Pasajero	(sin cambio)	<code>DRIVER_OFFERED</code> → <code>PASSENGER_COUNTER_OFFERED</code> (y sobrescribe <code>offered_price</code>)
PATCH <code>/api/offers/{offer_id}/accept</code>	Pasajero	→ <code>ASSIGNED</code>	esa oferta → <code>ACCEPTED</code> ; todas las demás del viaje → <code>REJECTED</code>
PATCH <code>/api/offers/{offer_id}/resolve (ACCEPT)</code>	Conductor	→ <code>ASSIGNED</code>	<code>PASSENGER_COUNTER_OFFERED</code> → <code>ACCEPTED</code> ; las demás → <code>REJECTED</code>
PATCH <code>/api/offers/{offer_id}/resolve (REJECT)</code>	Conductor	(sin cambio, sigue <code>PENDING</code>)	<code>PASSENGER_COUNTER_OFFERED</code> → <code>REJECTED</code>
PATCH <code>/api/{request_id}/cancel</code>	Pasajero	<code>PENDING</code> → <code>CANCELLED</code>	ofertas en <code>DRIVER_OFFERED</code> / <code>PASSENGER_COUNTER_OFFERED</code> → <code>REJECTED</code>
PATCH <code>/api/{request_id}/start</code>	Conductor asignado	<code>ASSIGNED</code> → <code>IN_PROGRESS</code>	(sin cambio)
PATCH <code>/api/{request_id}/complete</code>	Conductor asignado	<code>IN_PROGRESS</code> → <code>COMPLETED</code>	(sin cambio)

El estado `SCHEDULED` existe en el enum del modelo pero **ningún endpoint lo asigna ni lo consulta**.

POST `/api/service-requests/`

- **Función:** `create_service_request(request_data: schemas.ServiceRequestCreate, db: Session = Depends(get_db), current_user: models.User = Depends(security.get_current_user))`
- **status_code:** `status.HTTP_201_CREATED` (201). **Sin response_model.**
- **Quién puede llamarlo:** cualquier usuario autenticado. **No valida el rol** — el `passenger_id` siempre es `current_user.user_id`, así que un conductor podría publicar un viaje a su propio nombre.

- **Cuerpo** (`schemas.ServiceRequestCreate`, la segunda definición del archivo, que es la que queda vigente): `origin`, `destination`, `departure_time`, `return_time`, `trip_type` (`ONE_WAY` / `ROUND_TRIP`), `adults_count`, `children_count` (defecto 0), `has_pets` (defecto False), `origin_lat`, `origin_lng`, `destination_lat`, `destination_lng`, `distance_km`, `tolls_count`, `tolls_cost`, `tipo_via`, y los siete `requiere_*`. El `model_validator` del esquema valida que `adults_count > 0` ("Debe haber al menos un adulto en la solicitud.").

Qué hace, paso a paso:

1. **Validación de fecha:** si `request_data.departure_time.replace(tzinfo=None) < datetime.now()` → 400 "La fecha y hora de salida no puede estar en el pasado." El comentario explica el `.replace(tzinfo=None)`: *"por si el frontend envía fechas con zona horaria"*.
2. `trip_type_str = request_data.trip_type.value if hasattr(request_data.trip_type, 'value') else request_data.trip_type`.
3. **Validaciones de ida y vuelta:** – Si `trip_type_str == "ROUND_TRIP"`: `return_time` es obligatorio (400 si falta) y debe ser posterior a `departure_time` (400 si no). – En cualquier otro caso (`ONE_WAY`): se fuerza `request_data.return_time = None`.
4. **Creación de `models.ServiceRequest`** con `passenger_id=current_user.user_id`, `status="PENDING"`, los datos de ruta de Google Maps (`origin_lat/lng`, `destination_lat/lng`, `distance_km`, `tolls_count` or 0, `tolls_cost` or 0, `tipo_via` or "PAVIMENTADA") y los siete `requiere_*` or False.

Campos de búsqueda (HU26) — comentario clave: *"búsqueda de conductores 100% automática: ya no la elige el pasajero. El centro de búsqueda es siempre el origen del viaje, y el radio guardado es amplio y fijo (`RADIO_VISIBILIDAD_KM`) para que la solicitud siga siendo visible en el radar (`GET` / `pending`) de cualquier conductor que se conecte más tarde, sin importar qué tan lejos esté el conductor más cercano en el momento exacto de publicar el viaje."* `python search_lat =`

`request_data.origin_lat search_lng = request_data.origin_lng search_radius_km =`
`RADIO_VISIBILIDAD_KM # 60 5. db.add / db.commit / db.refresh.` 6. **Auditoría:** `action="CREATE_TRIP"`,
`entity="ServiceRequest"`, `entity_id=new_request.request_id`, `detail=f"Viaje creado:`

`{request_data.origin} → {request_data.destination}"`. 7. **Notificación con expansión de radios**

(HU26 + HU55) — solo si `search_lat` y `search_lng` no son None: 1. Consulta **todos** los User con `role == "DRIVER"`, `current_lat.isnot(None)` y `current_lng.isnot(None)`. La variable se llama `conductores_online` pero **el filtro `is_online` no se aplica**; el comentario lo explica: *"Ya no se exige `is_online`: los viajes son premeditados y al conductor le sirve enterarse aunque no tenga la app abierta en ese momento. El único requisito es tener una ubicación conocida, que es lo que permite ordenar por cercanía."* 2. Recorre `RADIOS_NOTIFICACION_KM` **en orden ascendente** (5, 10, 20, 40, 80). Para cada radio filtra los conductores cuya `_distancia_km(search_lat, search_lng, c.current_lat, c.current_lng) <= radio`. **Al primer radio que produzca al menos un candidato, se queda con ese conjunto y hace `break`** — nunca se acumulan radios. 3. Dentro de ese radio, ordena por comodidades cumplidas de forma descendente: `sorted(candidatos_en_radio, key=lambda c: -`

`_match_comodidades(db, c, new_request)["cumplidas"])`. Comentario: *"las comodidades ya NO excluyen a nadie de la notificación inicial (filtro flexible) [...]. El filtro solo se usa para PRIORIZAR el orden en que se notifica dentro de cada radio (primero los que más coinciden), no para dejar a nadie fuera."* 4. A cada conductor del conjunto le crea una notificación: – `title: "Nuevo viaje disponible cerca de ti"` – `message: f"{request_data.origin} → {request_data.destination}"` – `tipo: "SYSTEM"` – sin `offer_id`. 5. `conductores_notificados = len(conductores_a_notificar)`.

Si no hay coordenadas de origen, `conductores_notificados = 0` y no se notifica a nadie. 8. **Respuesta:** `{c.name: getattr(new_request, c.name) for c in new_request.__table__.columns}` — **todas las**

columnas de la tabla `ServiceRequest` —, más la clave añadida `conductores_notificados`. Comentario: *"Se agrega 'conductores_notificados' a la respuesta para que el frontend pueda avisarle al pasajero si, con los filtros de comodidades que eligió, no había ningún conductor conectado en este momento (HU55)."*

Errores: - 400 — "La fecha y hora de salida no puede estar en el pasado." - 400 — "Para viajes de ida y vuelta (ROUND_TRIP), la fecha de regreso es obligatoria." - 400 — "La fecha de regreso debe ser posterior a la fecha de salida." - 500 — `f"Error al procesar la solicitud: {str(e)}"` (el `try/except` `Exception` envuelve la creación, la auditoría y **también toda la lógica de notificación**, con `db.rollback()` previo) - 422 de Pydantic si `adults_count <= 0`.

GET `/api/service-requests/pending`

- **Función:** `get_pending_requests(db: Session = Depends(get_db), current_user: models.User = Depends(security.get_current_user))`
- **response_model:** `list[schemas.ServiceRequestRead]`.
- **Docstring:** *"Retorna la lista de viajes con estado 'PENDING'. - Conductores: ven las solicitudes pendientes dentro del radio de visibilidad del viaje, ordenadas de más cerca a más lejos."*

Qué hace, paso a paso:

1. Consulta base: `ServiceRequest.status == "PENDING"`.
2. **Filtro por rol:** si `current_user.role != "DRIVER"`, añade `filter(ServiceRequest.passenger_id == current_user.user_id)` — un pasajero (o un admin) solo ve **sus propias** solicitudes pendientes.
3. `order_by(models.ServiceRequest.created_at.desc())` — el más reciente primero. Este es el orden final para los no conductores.
4. **Rama de conductor: - Radio de visibilidad y orden por cercanía (HU26):** solo se aplica si el conductor tiene `current_lat` y `current_lng`. Para cada solicitud:
 - si `sr.search_lat is None or sr.search_lng is None or not sr.search_radius_km` → se añade con distancia 0 y **continue** (las solicitudes sin coordenadas siempre se muestran, y quedan primeras en el orden, empatadas en distancia 0);
 - si no, calcula `distancia = _distancia_km(current_user.current_lat, current_user.current_lng, sr.search_lat, sr.search_lng)` y **solo la incluye si distancia <= float(sr.search_radius_km)** (es decir, contra los 60 km guardados en la propia solicitud, no contra un radio del conductor).
 - Ordena la lista por distancia ascendente: `con_distancia.sort(key=lambda par: par[0])`.

Si el conductor **no** tiene coordenadas, se salta todo esto: ve todas las solicitudes `PENDING` de la plataforma en orden cronológico inverso. - **Anotación de comodidades (HU55)** — comentario: *"filtro flexible: se anota, no se oculta"*. Para cada solicitud llama a `_match_comodidades(db, current_user, sr)` y añade al dict serializado: - `comodidades_exigidas = match["exigidas"]` - `comodidades_cumplidas = match["cumplidas"]` - `comodidades_faltantes = match["faltantes"]` 5. Para no conductores devuelve `[_serializar_service_request(sr) for sr in pending_requests]`, sin las claves `comodidades_*` (que en el esquema son `Optional` y saldrán como `null`).

Respuesta: lista de `schemas.ServiceRequestRead` (claves: `request_id`, `passenger_id`, `origin`, `destination`, `departure_time`, `return_time`, `trip_type`, `adults_count`, `children_count`, `has_pets`,

`status`, `created_at`, `origin_lat`, `origin_lng`, los siete `requiere_*`, y `comodidades_exigidas` / `comodidades_cumplidas` / `comodidades_faltantes`).

Errores: - 500 — `f"Error al obtener solicitudes pendientes: {str(e)}"` (todo el cuerpo está en un `try/except Exception`).

Auditoría / notificaciones: no.

GET /api/service-requests/demand-zones

Comentario del código: *"HU23 — Zonas de demanda agregadas (SCRUM-164). A diferencia de /pending, esto NO no depende de ningún estado de conexión: el objetivo es que pueda ver dónde hay más flujo de solicitudes y decidir dónde posicionarse ANTES de conectarse. Por privacidad no se exponen ubicaciones exactas de pasajeros individuales, solo centroides agrupados por zona (grilla ~3km) con un conteo."*

- **Función:** `get_demand_zones(db: Session = Depends(get_db), current_user: models.User = Depends(security.get_current_user))`
- Sin `response_model`.
- **Quién puede llamarlo:** solo **DRIVER**; si no → 403 "Solo los conductores pueden ver las zonas de demanda."

Qué hace: 1. Consulta `origin`, `origin_lat`, `origin_lng` de los `ServiceRequest` con `status == "PENDING"` y ambas coordenadas no nulas. **No filtra por distancia al conductor:** son todas las zonas de la plataforma. 2. Agrupa en una grilla de ~0.03° (≈3 km): `clave = (round(lat / 0.03), round(lng / 0.03))`. 3. Por cada celda acumula `lats`, `lngs`, un `label` (el `_etiqueta_zona(s.origin)` de la **primera** solicitud que cayó en esa celda) y `count`. 4. Devuelve un centroide por celda: `lat` y `lng` son la media aritmética de las coordenadas del grupo.

Respuesta: lista de `{"lat", "lng", "count", "label"}`.

Errores: 403 — "Solo los conductores pueden ver las zonas de demanda."

Auditoría / notificaciones: no.

POST /api/service-requests/{request_id}/offers

- **Función:** `async def create_driver_offer(request_id: int, payload: schemas.OfferCreate, db: Session = Depends(get_db), current_user: models.User = Depends(get_current_user))`
- **status_code:** `status.HTTP_201_CREATED` (201). Sin `response_model`.
- **Cuerpo** (`schemas.OfferCreate`): `offered_price: float = Field(..., gt=0, description="Precio propuesto por el conductor")`.
- **Quién puede llamarlo:** solo **DRIVER**; si no → 403 "Solo los conductores registrados pueden enviar ofertas."

Qué hace, paso a paso (los números son los del propio código): 1. Valida `current_user.role != 'DRIVER'` → 403. 2. Busca el `ServiceRequest`. Si no existe → 404. Si `status != 'PENDING'` → 400 con el estado actual en el mensaje. 3. Obtiene el `vehicle_id` automáticamente: primer `Vehicle` con `owner_id == current_user.user_id`. Si no tiene → 400. El comentario aclara: *"Se extrae del token automáticamente" / "Se extrae de la consulta anterior"* — el cliente **solo envía el precio**. 4. Crea

```
models.DriverOffer(request_id, driver_id=current_user.user_id, vehicle_id=vehicle.vehicle_id,
offered_price=payload.offered_price, status='DRIVER_OFFERED'), commit, refresh. 5. Auditoría.
```

Respuesta: `{"message": "Oferta registrada exitosamente", "offer_id": new_offer.offer_id, "offered_price": new_offer.offered_price}`

Errores: - 403 — "Solo los conductores registrados pueden enviar ofertas." - 404 — "La solicitud de viaje no existe." - 400 — `f"No se pueden hacer ofertas a viajes cerrados. Estado actual: {service_request.status}"` - 400 — "No tienes un vehículo registrado para realizar esta oferta." - 500 — `f"Error al guardar la oferta: {str(e)}"` (con `db.rollback()`)

Auditoría: `action="CREATE_OFFER", entity="DriverOffer", entity_id=new_offer.offer_id, detail=f"Oferta de ${new_offer.offered_price} para solicitud #{request_id}"`.

Notificaciones: *ninguna* — el pasajero no recibe aviso de que llegó una oferta nueva, pese a que el enum de `Notification` tiene un tipo `'NEW_OFFER'` sin usar (ver Observaciones). **Tampoco impide que un mismo conductor oferte varias veces sobre el mismo viaje.**

GET /api/service-requests/driver/history

Comentario del código: *"Historial de viajes del conductor. El conductor no tenía dónde ver lo que ya manejó: /assigned filtra por passenger_id y /driver/my-offers solo trae negociaciones abiertas. Este trae los viajes que efectivamente hizo, es decir aquellos donde su oferta quedó ACCEPTED, con el pasajero y el precio final."*

- **Función:**

```
get_driver_trip_history(db: Session = Depends(get_db), current_user: models.User =
Depends(security.get_current_user))
```

- **Sin response_model.**

- **Quién puede llamarlo: solo DRIVER;** si no → 403 "Solo los conductores pueden ver su historial."

Qué hace: 1. Consulta pares (`ServiceRequest`, `DriverOffer`) unidos por `request_id`, filtrando `DriverOffer.driver_id == current_user.user_id`, `DriverOffer.status == 'ACCEPTED'` y `ServiceRequest.status.in(['ASSIGNED', 'IN_PROGRESS', 'COMPLETED', 'CANCELLED'])`. Orden: `ServiceRequest.departure_time.desc()`. 2. Por cada fila, busca el `User` pasajero y comprueba si ya lo calificó: existe un `Rating` con `request_id == viaje.request_id` y `rater_id == current_user.user_id` (comentario: *"Sirve para no ofrecer calificar dos veces"*). **Aquí ya_califico se calcula sin importar el estado del viaje.**

Respuesta: lista de dicts con claves `request_id`, `origin`, `destination`, `departure_time`, `trip_status` (el estado del `ServiceRequest`), `precio` (`float(oferta.offered_price)` o `0`), `pasajero_nombre` (o `"Pasajero"` si no se encuentra), `pasajero_foto`, `asientos` (`(adults_count or 1) + (children_count or 0)`), `ya_califico`.

Errores: 403 — "Solo los conductores pueden ver su historial."

GET /api/service-requests/driver/my-offers

- **Función:** `get_driver_active_offers(db: Session = Depends(get_db), current_user: models.User = Depends(security.get_current_user))`
- **Sin response_model.**
- **Quién puede llamarlo:** solo **DRIVER**; si no → 403 "Solo los conductores pueden ver sus ofertas."

Qué hace: 1. Consulta `DriverOffer` con `driver_id == current_user.user_id` y `status.in(['DRIVER_OFFERED', 'PASSENGER_COUNTER_OFFERED', 'ACCEPTED'])` — es decir, negociaciones abiertas más los viajes ya ganados; **excluye las REJECTED**. Orden: `created_at.desc()`. 2. Por cada oferta busca su `ServiceRequest` (puede ser `None`, y el código lo contempla con valores por defecto "" / `None`). 3. `ya_califico` (HU46): **solo se calcula si el viaje existe y `viaje.status == 'COMPLETED'`**; en cualquier otro caso queda `False`.

Respuesta: lista de dicts con `offer_id`, `request_id`, `origin`, `destination`, `departure_time` (ISO o `None`), `offered_price` (`float`), `status` (el de la oferta), `trip_status` (el del viaje), `created_at` (ISO o `None`), `passenger_id`, `ya_califico`.

Errores: 403 — "Solo los conductores pueden ver sus ofertas."

GET /api/service-requests/notifications

Comentario del código: `# SCRUM-90/91 — GET /api/service-requests/notifications — Obtener notificaciones del usuario actual`

- **Función:** `get_notifications(db: Session = Depends(get_db), current_user: models.User = Depends(security.get_current_user))`
- **Sin response_model.**
- **Quién puede llamarlo:** cualquier usuario autenticado; solo ve las suyas (`Notification.user_id == current_user.user_id`).

Qué hace: `order_by(models.Notification.created_at.desc()).limit(20)`. **No filtra por leídas/no leídas** y el límite de 20 es fijo, sin parámetro de paginación.

Respuesta: lista de dicts con `notification_id`, `title`, `message`, `type`, `is_read`, `related_offer_id`, `created_at` (ISO o `None`).

Errores: ninguno propio.

PATCH /api/service-requests/notifications/{notification_id}/read

- **Función:** `mark_notification_read(notification_id: int, db: Session = Depends(get_db), current_user: models.User = Depends(security.get_current_user))`
- **Quién puede llamarlo:** cualquier usuario autenticado, sobre **sus propias** notificaciones (el filtro incluye `Notification.user_id == current_user.user_id`, así que marcar la de otro devuelve 404, no 403).

Qué hace: pone `notif.is_read = True` y hace `commit`.

Respuesta: `{"message": "Notificación marcada como leída."}`

Errores: 404 — "Notificación no encontrada."

GET /api/service-requests/assigned

Comentario del código: *"Pasajero ve sus viajes activos (ASSIGNED, IN_PROGRESS, COMPLETED) con datos del conductor asignado".*

- **Función:** `get_assigned_requests(db: Session = Depends(get_db), current_user: models.User = Depends(security.get_current_user))`
- **Sin `response_model`.**
- **Quién puede llamarlo:** cualquier usuario autenticado; siempre filtra por `ServiceRequest.passenger_id == current_user.user_id`, así que un conductor que lo llame verá una lista vacía (salvo que además sea pasajero de algún viaje).

Qué hace: 1. Consulta `ServiceRequest` con `passenger_id == current_user.user_id` y `status.in(['ASSIGNED', 'IN_PROGRESS', 'COMPLETED'])`, `order_by(created_at.desc())`. **No incluye PENDING ni CANCELLED.** 2. Por cada viaje busca la `DriverOffer` con `request_id` del viaje y `status == 'ACCEPTED'`. Si la encuentra, extrae `driver_id`, el nombre y la foto del conductor y `precio_acordado = float(oferta_aceptada.offered_price)`. Si no la encuentra, deja los valores por defecto: `conductor_nombre = 'Conductor asignado', conductor_foto = None, precio_acordado = 0, driver_id = None`. 3. **HU43 — ubicación en vivo:** `conductor_lat / conductor_lng` solo se rellenan si `v.status == 'IN_PROGRESS'` ("solo tiene sentido mientras el viaje está en curso"); en ASSIGNED o COMPLETED van como None. 4. **HU46:** `ya_califico` solo se calcula si `v.status == 'COMPLETED'`, buscando un `Rating` con ese `request_id` y `rater_id == current_user.user_id`.

Respuesta: lista de dicts con `request_id`, `origin`, `destination`, `departure_time`, `return_time`, `trip_type`, `adults_count`, `children_count`, `has_pets`, `status`, `created_at`, `conductor_nombre`, `conductor_foto`, `precio_acordado`, `driver_id`, `conductor_lat`, `conductor_lng`, `ya_califico`.

Errores: ninguno propio.

GET /api/service-requests/{request_id}/receipt

Comentario del código: # HU42 — Recibo en PDF de un viaje completado (SCRUM-190). Este es el endpoint que genera el documento tipo FUEC.

- **Función:** `descargar_recibo(request_id: int, db: Session = Depends(get_db), current_user: models.User = Depends(security.get_current_user))`
- **Sin `response_model`**; devuelve un `StreamingResponse` con el PDF.
- **Quién puede llamarlo:** el **pasajero dueño** del viaje o un `ADMIN`. Un conductor, incluso el asignado, recibe 403.

Qué hace, paso a paso: 1. Busca el viaje; si no existe → 404. 2. Autorización: `if viaje.passenger_id != current_user.user_id and current_user.role != 'ADMIN' → 403`. 3. Estado: si `viaje.status != 'COMPLETED' → 400`. 4. Busca la `DriverOffer` `ACCEPTED` del viaje y, a partir de ella, el `User` conductor. Ambas búsquedas toleran el None. 5. Busca el `User` pasajero. 6. **Número de referencia:** `numero_interno = f"TFY-{request_id:06d}"` (por ejemplo `TFY-000042`). El comentario es explícito:

"Turify no está conectado al registro oficial de FUEC (Épica futura) — este número es un identificador interno de Turify, no un FUEC gubernamental real." 7. **Generación del PDF con ReportLab**

(canvas.Canvas(buffer, pagesize=letter)): – Encabezado: rectángulo verde de marca (setFillColorRGB(0.086, 0.639, 0.290)) de 2.6 cm de alto a todo el ancho, con el texto "Turify" en Helvetica-Bold 22 y "Recibo de viaje" en Helvetica 11, ambos en blanco. – Título: f"Referencia interna N.º {numero_interno}" en Helvetica-Bold 13. – Función interna campo(label, valor) que usa nonlocal y para ir bajando 0.75 cm por línea, con la etiqueta a 2 cm y el valor a 6 cm del margen izquierdo. – Campos, en orden: "Pasajero:" (nombre o "-"), "Conductor:" (nombre o "No disponible"), "Origen:", "Destino:", "Fecha de salida:" (formato '%d/%m/%Y %H:%M' o "-"), "Tipo de viaje:" ("Ida y vuelta" si trip_type == 'ROUND_TRIP', si no "Solo ida"), "Pasajeros:" ((adults_count or 0) + (children_count or 0)), "Precio pagado:" (f"\${precio:.0f} COP", donde precio es float(oferta_aceptada.offered_price) o 0). – Línea separadora gris y dos líneas de pie en Helvetica-Oblique 8: "Este recibo es un comprobante interno de Turify, no un FUEC oficial ante autoridad de transporte." y f"Generado el {datetime.now().strftime('%d/%m/%Y %H:%M')}".

Nota: el PDF **no incluye la lista de ocupantes** registrada por POST /{request_id}/passengers, ni la placa del vehículo, que son datos propios de un FUEC real. 8. Devuelve StreamingResponse(buffer, media_type="application/pdf", headers={"Content-Disposition": f'attachment; filename="turify_recibo_{request_id}.pdf"'}) — se descarga, no se muestra embebido.

Errores: – 404 — "Viaje no encontrado." – 403 — "No tienes permiso para ver el recibo de este viaje." – 400 — "El recibo solo está disponible para viajes completados."

Auditoría / notificaciones: no.

GET /api/service-requests/{request_id}/offers

Comentario del código:

SCRUM-77 – GET /api/service-requests/{request_id}/offers – El pasajero ve todas las ofertas de su viaje con datos del conductor

- **Función:** get_offers_for_request(request_id: int, db: Session = Depends(get_db), current_user: models.User = Depends(security.get_current_user))
- **Sin response_model.**
- **Quién puede llamarlo:** el **pasajero dueño** del viaje o un **ADMIN**; cualquier otro → 403.

Qué hace, paso a paso: 1. Busca el ServiceRequest; si no existe → 404 "Solicitud de viaje no encontrada." 2. Autorización: if service_request.passenger_id != current_user.user_id and current_user.role != 'ADMIN' → 403. 3. Consulta las DriverOffer del viaje con status.in(['DRIVER_OFFERED', 'PASSENGER_COUNTER_OFFERED']) — es decir, **solo las negociaciones abiertas**; las ACCEPTED y REJECTED no aparecen aquí (por eso la pantalla de ofertas queda vacía una vez el pasajero acepta una). Sin orden explícito. 4. pasajeros_totales = (adults_count or 0) + (children_count or 0). 5. Para cada oferta: – Busca el User conductor. – **Calificación real (HU46):** promedio, cantidad = _promedio_calificacion(db, oferta.driver_id). Comentario: "antes venía simulada en el frontend". – Busca el Vehicle por vehicle_id de la oferta (no por owner_id). – **Bloque comodidades** (solo si hay vehículo): capacidad = vehiculo.capacidad_real or vehiculo.capacity, match = _match_comodidades(db, conductor, service_request) (o el dict neutro si no hay conductor), y se arma el dict con categoria (vía _categoria_vehiculo), capacidad, las siete comodidades, cargo_mascota, acepta_menores_2_anos y comodidades_exigidas / comodidades_cumplidas / comodidades_faltantes. – **Cálculo de recomendado** — la regla de negocio más sutil del endpoint.

Empieza en `False` y solo se evalúa si hay vehículo: `python if pasajeros_totales > 0 and capacidad >= pasajeros_totales: recomendado = capacidad <= max(pasajeros_totales * 3, pasajeros_totales + 3) if service_request.has_pets and not vehiculo.acepta_mascotas: recomendado = False if match["exigidas"] > 0 and not match["cumple_todas"]:` recomendado = False O sea: el vehículo debe caber al grupo sin sobredimensionar de forma exagerada (el tope es el mayor entre el triple del grupo y el grupo más tres asientos), debe aceptar mascotas si el viaje lleva mascotas, y debe cumplir **todas** las comodidades exigidas. Comentario: *"Si le falta alguna, la oferta se sigue mostrando igual — solo no se destaca como 'recomendada'."*

Respuesta: lista de dicts con `offer_id`, `request_id`, `driver_id`, `driver_name` (o "Conductor"), `driver_photo`, `driver_verificado` (bool, badge HU38), `vehicle_id`, `offered_price` (float), `status`, `created_at` (ISO o None), `driver_rating`, `driver_rating_count`, `comodidades` (dict o None), `recomendado` (bool).

Errores: - 404 — "Solicitud de viaje no encontrada." - 403 — "No tienes permiso para ver estas ofertas."

PATCH /api/service-requests/offers/{offer_id}/counter-offer

Comentario del código: # SCRUM-78 — PATCH /api/offers/{offer_id}/counter-offer — El pasajero envía una contraoferta — cambia el estado a `PASSENGER_COUNTER_OFFERED` (la ruta real lleva el prefijo `/api/service-requests`).

- **Función:** `counter_offer(offer_id: int, payload: schemas.CounterOfferCreate, db: Session = Depends(get_db), current_user: models.User = Depends(security.get_current_user))`
- **Cuerpo** (`schemas.CounterOfferCreate`): `offered_price: float = Field(..., gt=0, description="Precio de contraoferta del pasajero")`.
- **Quién puede llamarlo:** solo el pasajero dueño del viaje. No se comprueba el rol, se comprueba la propiedad: `service_request.passenger_id != current_user.user_id → 403`. Un ADMIN **no** está exceptuado aquí.

Qué hace: 1. Busca la `DriverOffer` por `offer_id`; si no existe → 404. 2. Busca el `ServiceRequest` asociado; si no existe o el pasajero no es el dueño → 403. 3. **Transición válida:** `if oferta.status not in ['DRIVER_OFFERED'] → 400`. Solo se puede contraofertar una vez sobre la oferta original: una oferta ya en `PASSENGER_COUNTER_OFFERED` no admite una segunda contraoferta del pasajero. 4. **Sobrescribe** `oferta.offered_price = payload.offered_price` (el precio original del conductor se pierde: no hay historial de negociación) y pone `oferta.status = 'PASSENGER_COUNTER_OFFERED'`. `commit + refresh`. 5. Auditoría. 6. Busca el nombre del pasajero (`nombre_pasajero = pasajero.full_name if pasajero else "El pasajero"`) y notifica al conductor.

Respuesta:

```
{
  "message": "Contraoferta enviada al conductor.",
  "offer_id": oferta.offer_id,
  "nuevo_precio": float(oferta.offered_price),
  "nuevo_estado": oferta.status
}
```

Errores: - 404 — "Oferta no encontrada." - 403 — "No tienes permiso para contraofertar en este viaje." - 400 — f"No se puede contraofertar una oferta en estado: {oferta.status}"

Auditoría: action="COUNTER_OFFER", entity="DriverOffer", entity_id=oferta.offer_id, detail=f"Contraoferta de \${payload.offered_price} para oferta #{offer_id}".

Notificación — al conductor (user_id=oferta.driver_id): - title: "Nueva contraoferta recibida" - message: f"{nombre_pasajero} propuso \${payload.offered_price:,.0f} para el viaje de {service_request.origin} → {service_request.destination}." - tipo: "COUNTER_OFFER", offer_id=oferta.offer_id.

PATCH /api/service-requests/offers/{offer_id}/accept

Comentario del código: # SCRUM-81 — PATCH /api/offers/{offer_id}/accept — El pasajero acepta una oferta — pasa a ACCEPTED y el viaje a ASSIGNED

- **Función:** accept_offer(offer_id: int, db: Session = Depends(get_db), current_user: models.User = Depends(security.get_current_user)) — **sin cuerpo.**
- **Quién puede llamarlo:** solo el pasajero dueño del viaje; si no → 403.

Qué hace, paso a paso: 1. Busca la oferta; si no existe → 404 "Oferta no encontrada." 2. Busca el viaje; si no existe o el pasajero no es el dueño → 403. 3. **Transición válida:** if oferta.status not in ['DRIVER_OFFERED', 'PASSENGER_COUNTER_OFFERED'] → 400. Es decir, el pasajero puede aceptar tanto el precio original del conductor como el precio de su propia contraoferta **sin esperar la respuesta del conductor** (caso poco intuitivo pero permitido por el código). 4. oferta.status = 'ACCEPTED'. 5. **Rechazo en cascada:** consulta todas las DriverOffer con el mismo request_id y offer_id != offer_id, y les pone status = 'REJECTED'. **El filtro no excluye las que ya estaban REJECTED**, pero el efecto es idempotente. 6. service_request.status = 'ASSIGNED'. 7. db.commit() — un solo commit para toda la transición. 8. Auditoría. 9. Notifica al conductor ganador y luego, en bucle, a cada conductor rechazado.

Respuesta:

```
{
  "message": "¡Oferta aceptada! El viaje ha sido asignado al conductor.",
  "offer_id": oferta.offer_id,
  "request_id": oferta.request_id,
  "estado_oferta": "ACCEPTED",
  "estado_viaje": "ASSIGNED"
}
```

Errores: - 404 — "Oferta no encontrada." - 403 — "No tienes permiso para aceptar esta oferta." - 400 — f"No se puede aceptar una oferta en estado: {oferta.status}"

Auditoría: action="ACCEPT_OFFER", entity="DriverOffer", entity_id=oferta.offer_id, detail=f"Oferta #{offer_id} aceptada. Viaje #{oferta.request_id} asignado al conductor #{oferta.driver_id}".

Notificaciones: 1. Al conductor ganador (`user_id=oferta.driver_id`): - `title: "¡Tu oferta fue aceptada!"` - `message: f"{nombre_pasajero} aceptó tu oferta de ${float(oferta.offered_price):.0f}`. Viaje de `{service_request.origin}` → `{service_request.destination}`." - tipo: `"TRIP_ACCEPTED"`, `offer_id=oferta.offer_id`. 2. A cada conductor cuya oferta fue rechazada (`user_id=otra.driver_id`, uno por oferta): - `title: "Oferta no seleccionada"` - `message: f"El pasajero eligió otra oferta para el viaje de {service_request.origin} → {service_request.destination}."` - tipo: `"TRIP_REJECTED"`, `offer_id=otra.offer_id`.

PATCH /api/service-requests/offers/{offer_id}/resolve

Comentario del código: # SCRUM-82 – PATCH /api/service-requests/offers/{offer_id}/resolve – El conductor responde a una contraoferta: acepta o rechaza

- **Función:** `resolve_counter_offer(offer_id: int, payload: schemas.ResolveOfferCreate, db: Session = Depends(get_db), current_user: models.User = Depends(security.get_current_user))`
- **Cuerpo** (`schemas.ResolveOfferCreate`): `action: str` – 'ACCEPT' o 'REJECT' (un `str` libre, la validación es manual dentro del endpoint).
- **Quién puede llamarlo:** solo **DRIVER** (403 "Solo los conductores pueden responder contraoferta.") y además **solo sobre sus propias ofertas**: la consulta filtra por `DriverOffer.driver_id == current_user.user_id`, así que la oferta de otro conductor devuelve 404, no 403.

Qué hace: 1. Valida el rol. 2. Busca la oferta por `offer_id` y `driver_id == current_user.user_id`; si no aparece → 404 "Oferta no encontrada o no te pertenece." 3. **Transición válida:** `if oferta.status != 'PASSENGER_COUNTER_OFFERED' → 400`. 4. Busca el `ServiceRequest` asociado (sin comprobar que exista – ver Observaciones). 5. **Rama `payload.action == 'ACCEPT'`:** - `oferta.status = 'ACCEPTED'`; `service_request.status = 'ASSIGNED'`. - Rechaza en cascada las demás ofertas del mismo viaje (`offer_id != oferta.offer_id → 'REJECTED'`). - `db.commit()`. - Notifica **al pasajero** (`user_id=service_request.passenger_id`): `title="¡Viaje confirmado!"`, `message=f"El conductor aceptó tu contraoferta de ${oferta.offered_price}`. Tu viaje está confirmado.", `tipo="TRIP_ACCEPTED"`, `offer_id=oferta.offer_id`. (Aquí el precio se interpola sin formato de miles, a diferencia de otros mensajes que usan `:.0f`.) - Auditoría: `action="ACCEPT_COUNTER_OFFER"`, `entity="DriverOffer"`, `entity_id=oferta.offer_id`, `detail=f"Conductor #{current_user.user_id} aceptó contraoferta de ${oferta.offered_price}"`. - Respuesta: `{"message": "Contraoferta aceptada. El viaje ha sido confirmado.", "offer_id": ..., "estado_oferta": "ACCEPTED", "estado_viaje": "ASSIGNED"}` - **No** se notifica a los conductores cuyas ofertas quedaron `REJECTED` en la cascada (a diferencia de /accept). 6. **Rama `payload.action == 'REJECT'`:** - `oferta.status = 'REJECTED'`; `db.commit()`. **El viaje sigue en PENDING**, así que el pasajero puede aceptar otra oferta. - Notifica al pasajero: `title="Conductor rechazó la contraoferta"`, `message="Un conductor rechazó tu contraoferta. Puedes aceptar otra oferta disponible."`, `tipo="TRIP_REJECTED"`, `offer_id=oferta.offer_id`. - Auditoría: `action="REJECT_COUNTER_OFFER"`, `detail=f"Conductor #{current_user.user_id} rechazó contraoferta"`. - Respuesta: `{"message": "Has rechazado la contraoferta. Tu oferta fue cancelada.", "offer_id": ..., "estado_oferta": "REJECTED"}` 7. **Cualquier otro valor de `action` → 400.**

Nota sobre el flujo: el conductor **no puede recontraofertar**. Solo tiene dos salidas: aceptar el precio del pasajero o rechazar y quedar fuera de esa negociación.

Errores: - 403 — "Solo los conductores pueden responder contraoferta." - 404 — "Oferta no encontrada o no te pertenece." - 400 — f"Solo puedes resolver ofertas en estado PASSENGER_COUNTER_OFFERED. Estado actual: {oferta.status}" - 400 — "Acción inválida. Use 'ACCEPT' o 'REJECT'."

PATCH /api/service-requests/{request_id}/cancel

Comentario del código: "HU26 — [...] El pasajero cancela la búsqueda mientras el viaje sigue PENDING (todavía no aceptó ninguna oferta). Una vez el viaje pasa a ASSIGNED ya no se puede cancelar desde aquí — a esa altura hay un conductor comprometido y eso necesitaría su propio flujo (no forma parte de este cambio)."

- **Función:** `cancel_service_request(request_id: int, db: Session = Depends(get_db), current_user: models.User = Depends(security.get_current_user))`
- **Quién puede llamarlo:** solo el pasajero dueño; ni siquiera el ADMIN está exceptuado.

Qué hace: 1. Busca el viaje; si no existe → 404. 2. `if viaje.passenger_id != current_user.user_id → 403`. 3. `if viaje.status != 'PENDING' → 400` con el estado actual. 4. `viaje.status = 'CANCELLED'`. 5. Consulta las ofertas del viaje con `status.in(['DRIVER_OFFERED', 'PASSENGER_COUNTER_OFFERED'])` y a todas les pone 'REJECTED'. 6. `db.commit()`. 7. Notifica a cada conductor afectado. 8. Auditoría.

Respuesta: `{"message": "Búsqueda cancelada.", "request_id": viaje.request_id, "status": "CANCELLED"}`

Errores: - 404 — "Viaje no encontrado." - 403 — "No tienes permiso para cancelar este viaje." - 400 — f"Solo puedes cancelar la búsqueda mientras el viaje está pendiente. Estado actual: {viaje.status}"

Auditoría: `action="CANCEL_TRIP", entity="ServiceRequest", entity_id=viaje.request_id, detail=f"Pasajero #{current_user.user_id} canceló la búsqueda del viaje #{request_id}"`.

Notificación — a cada conductor con oferta activa (`user_id=oferta.driver_id`): - `title: "Viaje cancelado por el pasajero"` - `message: f"El pasajero canceló la solicitud de {viaje.origin} → {viaje.destination}"` - `tipo: "TRIP_REJECTED"`, `offer_id=oferta.offer_id`.

PATCH /api/service-requests/{request_id}/start

Comentario del código: # HU17 — PATCH /api/service-requests/{request_id}/start — Conductor inicia el viaje: ASSIGNED → IN_PROGRESS

- **Función:** `start_trip(request_id: int, db: Session = Depends(get_db), current_user: models.User = Depends(security.get_current_user))`
- **Quién puede llamarlo:** solo DRIVER y, además, solo el conductor asignado.

Qué hace: 1. `if current_user.role != 'DRIVER' → 403` "Solo el conductor puede iniciar el viaje." 2. Busca el viaje; si no existe → 404 "Viaje no encontrado." 3. `if viaje.status != 'ASSIGNED' → 400`. 4. **Verificación de asignación:** busca una DriverOffer con ese request_id, `driver_id == current_user.user_id` y `status == 'ACCEPTED'`. Si no existe → 403 "No eres el conductor asignado a este viaje." 5. `viaje.status = 'IN_PROGRESS'`; `db.commit()`. 6. Notifica al pasajero y registra auditoría.

Respuesta: `{"message": "¡Viaje iniciado! El pasajero ha sido notificado.", "request_id": viaje.request_id, "status": "IN_PROGRESS"}`

Errores: - 403 — "Solo el conductor puede iniciar el viaje." - 404 — "Viaje no encontrado." - 400 — `f"El viaje debe estar en estado ASSIGNED para iniciarlo. Estado actual: {viaje.status}"` - 403 — "No eres el conductor asignado a este viaje."

Auditoría: `action="TRIP_STARTED", entity="ServiceRequest", entity_id=viaje.request_id, detail=f"Viaje #{request_id} iniciado por conductor #{current_user.user_id}"`.

Notificación — al pasajero (`user_id=viaje.passenger_id`): - `title: "¡Tu viaje ha comenzado!"` - `message: f"El conductor está en camino. Viaje de {viaje.origin} a {viaje.destination}"` - `tipo: "TRIP_STARTED", offer_id=oferta_aceptada.offer_id`.

PATCH /api/service-requests/{request_id}/complete

Comentario del código: # HU17 — PATCH /api/service-requests/{request_id}/complete — Conductor finaliza el viaje: IN_PROGRESS → COMPLETED

- **Función:** `complete_trip(request_id: int, db: Session = Depends(get_db), current_user: models.User = Depends(security.get_current_user))`
- **Quién puede llamarlo:** solo **DRIVER** y solo el conductor asignado. Estructura idéntica a /start.

Qué hace: valida rol → busca viaje → exige `status == 'IN_PROGRESS'` → verifica que exista la oferta `ACCEPTED` propia → `viaje.status = 'COMPLETED'` → `commit` → notifica → auditoría.

Respuesta: `{"message": "¡Viaje completado! Gracias por usar Turify.", "request_id": viaje.request_id, "status": "COMPLETED"}`

Errores: - 403 — "Solo el conductor puede finalizar el viaje." - 404 — "Viaje no encontrado." - 400 — `f"El viaje debe estar EN CURSO para finalizarlo. Estado actual: {viaje.status}"` - 403 — "No eres el conductor asignado a este viaje."

Auditoría: `action="TRIP_COMPLETED", entity="ServiceRequest", entity_id=viaje.request_id, detail=f"Viaje #{request_id} completado por conductor #{current_user.user_id}"`.

Notificación — al pasajero: - `title: "¡Viaje completado!"` - `message: f"Tu viaje de {viaje.origin} a {viaje.destination} ha finalizado. ¡Gracias por usar Turify!"` - `tipo: "TRIP_COMPLETED", offer_id=oferta_aceptada.offer_id`.

Al llegar a `COMPLETED` se habilitan tres cosas: el recibo (`GET /{request_id}/receipt`), la calificación bidireccional (`POST /{request_id}/rating`) y el conteo de ganancias del conductor (`GET /drivers/earnings`).

GET /api/service-requests/{request_id}/status

Comentario del código: # HU17 — GET /api/service-requests/{request_id}/status — Pasajero consulta el estado actual de su viaje

- **Función:** `get_trip_status(request_id: int, db: Session = Depends(get_db), current_user: models.User = Depends(security.get_current_user))`

- **Quién puede llamarlo:** el pasajero dueño o cualquier usuario con rol **DRIVER** (if viaje.passenger_id != current_user.user_id and current_user.role != 'DRIVER' → 403) — nótese que **no exige ser el conductor asignado**. Un ADMIN que no sea el pasajero recibe 403.

Respuesta: {"request_id", "status", "origin", "destination", "departure_time"} (esta última en ISO o None).

Errores: - 404 — "Viaje no encontrado." - 403 — "No tienes permiso para ver este viaje."

POST /api/service-requests/{request_id}/passengers

Comentario del código: # HU10 — POST /api/service-requests/{request_id}/passengers — Pasajero registra los ocupantes del viaje (FUEC simulado)

- **Función:** registrar_ocupantes(request_id: int, payload: schemas.TripPassengersCreate, db: Session = Depends(get_db), current_user: models.User = Depends(security.get_current_user))
- **status_code:** 201.
- **Cuerpo** (schemas.TripPassengersCreate): passengers: list[TripPassengerItem], donde cada TripPassengerItem es {full_name: str, document_type: str = 'CC', document_number: str}. El comentario del esquema indica los valores esperados de document_type: # CC, TI, CE, PA (el enum del modelo TripPassenger.document_type es 'CC', 'TI', 'CE', 'PA' con defecto 'CC').
- **Quién puede llamarlo:** solo el pasajero dueño del viaje (403 en cualquier otro caso, incluido ADMIN y el conductor asignado).

Qué hace: 1. Busca el viaje; si no existe → 404. 2. if viaje.passenger_id != current_user.user_id → 403. 3. if viaje.status not in ['ASSIGNED', 'IN_PROGRESS'] → 400. Es decir: **solo se pueden registrar ocupantes después de aceptar una oferta y antes de que el viaje termine**; no se puede en PENDING ni en COMPLETED. 4. **Reemplazo completo:** db.query(models.TripPassenger).filter(TripPassenger.request_id == request_id).delete() — comentario: "Eliminar registros previos si el pasajero actualiza la lista". Cada llamada sustituye la lista entera; enviar {"passengers": []} borra todos los ocupantes. 5. Crea un models.TripPassenger(request_id, full_name, document_type, document_number) por cada elemento del payload. 6. db.commit() y auditoría.

Respuesta: {"message": f"{len(payload.passengers)} ocupante(s) registrado(s) correctamente.", "request_id": request_id}

Errores: - 404 — "Viaje no encontrado." - 403 — "Solo el pasajero del viaje puede registrar ocupantes." - 400 — f"Solo puedes registrar ocupantes en viajes confirmados. Estado actual: {viaje.status}"

Auditoría: action="REGISTER_FUEC", entity="ServiceRequest", entity_id=request_id, detail=f"FUEC registrado con {len(payload.passengers)} ocupante(s) para viaje #{request_id}".

Notificaciones: ninguna (el conductor no se entera de que ya hay lista de ocupantes).

Validaciones ausentes: no se comprueba que el número de ocupantes coincida con adults_count + children_count, ni que quepan en la capacidad del vehículo, ni que los documentos no estén repetidos.

GET /api/service-requests/{request_id}/passengers

Comentario del código:

HU10 – GET /api/service-requests/{request_id}/passengers – Conductor y pasajero consultan los ocupantes registrados

- **Función:** `get_ocupantes(request_id: int, db: Session = Depends(get_db), current_user: models.User = Depends(security.get_current_user))`
- **Quién puede llamarlo:** tres perfiles, evaluados así:
 - `es_pasajero = viaje.passenger_id == current_user.user_id;`
 - `es_conductor_asignado: solo si current_user.role == 'DRIVER' y existe una DriverOffer con ese request_id, driver_id == current_user.user_id y status == 'ACCEPTED';`
 - `o current_user.role == 'ADMIN'.`

Si no se cumple ninguno → 403.

Qué hace: consulta todos los `TripPassenger` con `request_id == request_id`.

Respuesta: lista de dicts con `passenger_entry_id`, `full_name`, `document_type`, `document_number`.

Errores: - 404 – "Viaje no encontrado." - 403 – "No tienes permiso para ver estos datos."

POST /api/service-requests/{request_id}/rating

Comentario del código: # — HU46 – Calificaciones bidireccionales (SCRUM-194)

- **Función:** `calificar_viaje(request_id: int, payload: schemas.RatingCreate, db: Session = Depends(get_db), current_user: models.User = Depends(security.get_current_user))`
- **status_code:** `status.HTTP_201_CREATED (201)` — **también cuando actualiza una calificación existente.**
- **Cuerpo** (`schemas.RatingCreate`): `score: int = Field(..., ge=1, le=5, description="Calificación de 1 a 5 estrellas")`, `comment: Optional[str] = None`.
- **Docstring:** *"Pasajero y conductor se califican mutuamente (1-5 estrellas + comentario opcional) una vez el viaje está COMPLETED. Cada uno solo puede calificar una vez por viaje — un segundo envío actualiza su calificación anterior."*
- **Quién puede llamarlo:** solo los dos participantes del viaje (el pasajero dueño y el conductor de la oferta `ACCEPTED`); cualquier otro, incluido un `ADMIN`, recibe 403 "No participaste en este viaje."

Qué hace: 1. Busca el viaje; si no existe → 404. 2. `if viaje.status != 'COMPLETED' → 400 "Solo puedes calificar viajes ya completados."` 3. Busca la `DriverOffer` `ACCEPTED` del viaje; si no hay → 400 "Este viaje no tiene un conductor asignado." 4. **Determina la dirección de la calificación:** - si `current_user.user_id == viaje.passenger_id → rated_id = oferta_aceptada.driver_id`; - si `current_user.user_id == oferta_aceptada.driver_id → rated_id = viaje.passenger_id`; - en cualquier otro caso → 403. 5. Busca un `Rating` previo con ese `request_id` y `rater_id == current_user.user_id`. Si existe, **actualiza** `score` y `comment` (un `comment` ausente lo sobrescribe con `None`); si no, crea `models.Rating(request_id, rater_id, rated_id, score, comment)`. 6. `db.commit()` y auditoría.

Respuesta: `{"message": "¡Gracias por tu calificación!", "score": payload.score}`

Errores: - 404 — "Viaje no encontrado." - 400 — "Solo puedes calificar viajes ya completados." - 400 — "Este viaje no tiene un conductor asignado." - 403 — "No participaste en este viaje."

Auditoría: `action="CREATE_RATING", entity="Rating", entity_id=request_id` (es el id del viaje, no el `rating_id`), `detail=f"Calificación de {payload.score}★ para usuario #{rated_id} (viaje #{request_id})"`.

Notificaciones: ninguna.

Efecto sobre el resto del sistema: los `Rating` alimentan `_promedio_calificacion` (usado en `GET /{request_id}/offers`), `_resumen_calificaciones` (en `GET /users/me/profile`), `calificacion_promedio` en `GET /drivers/earnings` y `rating_avg` en `GET /drivers/{driver_id}/public-profile`. Todos esos cálculos usan `Rating.rated_id`, así que un usuario ve el promedio de lo que **recibió**.

GET /api/service-requests/{request_id}/rating

- **Función:** `obtener_mi_calificacion(request_id: int, db: Session = Depends(get_db), current_user: models.User = Depends(security.get_current_user))`
- **Docstring:** *"Indica si el usuario autenticado ya calificó este viaje (y con qué puntaje)."*
- **Quién puede llamarlo:** cualquier usuario autenticado. **No verifica que el viaje exista ni que el usuario haya participado;** simplemente consulta si hay un `Rating` con ese `request_id` y `rater_id == current_user.user_id`, así que para un tercero siempre devuelve la respuesta negativa (no filtra información ajena).

Respuesta: `{"ya_califico": bool, "score": int | None, "comment": str | None}`

Errores: ninguno. **Auditoría / notificaciones:** no.

PARTE 3

El sistema de diseño y el arranque del frontend

Los tokens de color y tipografía, el catálogo de iconos, el enrutado y la sesión.

Sistema de diseño (diseno.jsx + index.css)

`diseno.jsx` es el único lugar donde viven color, tipografía, radios, iconos y componentes compartidos. Su comentario de cabecera lo declara explícitamente:

«Un solo lugar para color, tipografía, radios y los componentes compartidos. Si un color no está aquí, no debería usarse en la app.»

La estrategia es de dos capas:

1. `index.css` define los valores reales como variables CSS, en dos paletas (`:root` y `:root[data-tema="oscuro"]`).
2. `diseno.jsx` exporta el objeto `T`, cuyos campos son *punteros* a esas variables (`'var(--t-...)'`). Así, cambiar el atributo `data-tema` de `<html>` repinta toda la aplicación sin tocar JS.

El objeto `T` completo

Comentario en el código: «Los valores viven en `index.css` (paletas clara y oscura). Acá solo se apunta a la variable, así todo lo que use `T` cambia de tema sin tocar una línea más.»

Familia monte (verdes oscuros de fondo)

Propiedad de T	Variable CSS
monte	<code>var(--t-monte)</code>
monteAlto	<code>var(--t-monte-alto)</code>
monteLinea	<code>var(--t-monte-linea)</code>

Familia ruta (verde de marca / acción primaria)

Propiedad de T	Variable CSS
ruta	<code>var(--t-ruta)</code>

Propiedad de T	Variable CSS
rutaHover	var(--t-ruta-hover)

Familia musgo (verde suave, estados de éxito)

Propiedad de T	Variable CSS
musgo	var(--t-musgo)
musgoLinea	var(--t-musgo-linea)
musgoTexto	var(--t-musgo-texto)

Familia superficies

Propiedad de T	Variable CSS
niebla	var(--t-niebla)
niebla2	var(--t-niebla-2)
papel	var(--t-papel)

Familia chiva (dorado/ámbar; lo verificado, lo pendiente)

Propiedad de T	Variable CSS
chiva	var(--t-chiva)
chivaSuave	var(--t-chiva-suave)
chivaLinea	var(--t-chiva-linea)
chivaTexto	var(--t-chiva-texto)

Familia neutros (texto y bordes)

Propiedad de T	Variable CSS
tinta	var(--t-tinta)
piedra	var(--t-piedra)
piedraClara	var(--t-piedra-clara)
linea	var(--t-linea)

Familia cielo (azul informativo)

Propiedad de T	Variable CSS
cielo	var(--t-cielo)
cieloSuave	var(--t-cielo-suave)
cieloLinea	var(--t-cielo-linea)
cieloTexto	var(--t-cielo-texto)

Familia alerta (naranja/rojo de error y rechazo)

Propiedad de T	Variable CSS
alerta	<code>var(--t-alerta)</code>
alertaSuave	<code>var(--t-alerta-suave)</code>
alertaLinea	<code>var(--t-alerta-linea)</code>
alertaTexto	<code>var(--t-alerta-texto)</code>

Total: 27 tokens de color.

Familia tipografía

Propiedad de T	Valor literal
display	<code>"'Syne', system-ui, sans-serif"</code>
ui	<code>"'DM Sans', system-ui, sans-serif"</code>
dato	<code>"'IBM Plex Mono', ui-monospace, monospace"</code>

Familia radios

Propiedad de T	Valor
rDato	<code>'4px'</code>
rControl	<code>'10px'</code>
rTarjeta	<code>'14px'</code>
rChip	<code>'999px'</code>

Sombras

No existe ninguna familia de sombras en T. Cada componente escribe su `boxShadow` a mano con `rgba(...)` literal (por ejemplo `'0 2px 6px rgba(0,0,0,.18)'` en `BotonCentrarMapa`, `'0 10px 36px rgba(5,46,22,0.14)'` en `AdminConductores`). Es la única categoría del sistema que quedó fuera de la tokenización.

El objeto FIJO

```
export const FIJO = { ruta: '#16A34A', chiva: '#E9A13B', monte: '#0E2A1E', tinta: '#131A16' };
```

Cuatro colores hexadecimales literales, idénticos a los valores del **modo claro** de `--t-ruta`, `--t-chiva`, `--t-monte` y `--t-tinta`.

Por qué existe. El comentario lo explica: «Colores fijos — SOLO para donde no llega el CSS: marcadores de Google Maps, el favicon y cualquier export a imagen. El logo NO los usa: sigue el tema, porque un verde de marca distinto al de los botones se lee como un error.»

Es decir: hay contextos que no son DOM con CSS aplicado y por tanto no resuelven `var(--t-...)`:

- Los marcadores e íconos que se le pasan a la API de **Google Maps** (reciben strings/JSON, no hojas de estilo).
- El **favicon** y cualquier exportación a imagen.

`FIJO` se importa en `Dashboard.jsx` y `PanelConductor.jsx` (las dos pantallas con mapa). No se usa en ninguno de los archivos documentados aquí.

Paleta completa de `index.css` (claro vs. oscuro)

Comentario del bloque oscuro: «Modo oscuro — no es negro puro: es el mismo verde monte, bajado de luz. La marca (ruta y chiva) sube un punto de brillo para sostener el contraste.»

Variable	Modo claro (:root)	Modo oscuro (:root[data-tema="oscuro"])
color-scheme	light	dark
--t-monte	#0E2A1E	#081A11
--t-monte-alto	#1A3C2B	#12291F
--t-monte-linea	#2C503C	#274534
--t-ruta	#16A34A	#22C55E
--t-ruta-hover	#128A3E	#34D871
--t-musgo	#E6F1E8	#102E1D
--t-musgo-linea	#C9E3D0	#22553A
--t-musgo-texto	#146B36	#7EE0A3
--t-niebla	#F1F2ED	#0B1F16
--t-niebla-2	#EDEEE8	#102A1E
--t-papel	FFFFFF	#12291F
--t-chiva	#E9A13B	#F0B45E
--t-chiva-suave	#FBF1DE	#2E2211
--t-chiva-linea	#EBD5A9	#57411D
--t-chiva-texto	#8A5A12	#F0B45E
--t-tinta	#131A16	#E8EFE9
--t-piedra	#6B7A72	#93A69A
--t-piedra-claro	#9AA79F	#6F8177
--t-linea	#E2E4DC	#22402F
--t-cielo	#2563EB	#60A5FA
--t-cielo-suave	#EAF1FE	#13253F
--t-cielo-linea	#BFD3F7	#294676

Variable	Modo claro (:root)	Modo oscuro (:root[data-tema="oscuro"])
--t-cielo-texto	#1E40AF	#9CC0F8
--t-alerta	#C2410C	#F97316
--t-alerta-suave	#FBEDE7	#2E1A12
--t-alerta-linea	#EED4C7	#5A3222
--t-alerta-texto	#9A3412	#F5A97A

Puntos de interés:

- En modo oscuro `--t-chiva` y `--t-chiva-texto` valen **lo mismo** (`#F0B45E`); en claro son distintos (`#E9A13B` vs `#8A5A12`).
- `--t-papel` en oscuro (`#12291F`) coincide exactamente con `--t-monte-alto` en oscuro. El "papel" del modo oscuro es el monte elevado.
- `--t-monte-alto` claro (`#1A3C2B`) y `--t-monte-linea` oscuro (`#274534`) refuerzan que el monte es oscuro en ambos temas: por eso varios archivos definen constantes locales para el texto sobre monte (ver A.9).

Reglas globales de `index.css`

```
html, body, #root {
  margin: 0; padding: 0;
  width: 100%; min-height: 100vh;
  box-sizing: border-box;
  background-color: var(--t-niebla);
  color: var(--t-tinta);
  font-family: 'DM Sans', system-ui, -apple-system, sans-serif;
  -webkit-font-smoothing: antialiased;
  transition: background-color .25s ease, color .25s ease;
}
*, ::before, ::after { box-sizing: inherit; }
```

La transición de `.25s` sobre `background-color` y `color` es lo que hace que el cambio de tema se vea como un fundido y no como un corte.

Las tres familias tipográficas

Se cargan desde una única URL exportada como `FUENTES_URL`:

```
https://fonts.googleapis.com/css2?
family=Syne:wght@600;700;800&family=DM+Sans:opsz,wght@9..40,400;9..40,500;9..40,700&family=IBM+Plex+Mono:wght@4
```

(La misma URL está duplicada como `<link>` en `index.html`.)

Token	Familia	Pesos cargados	Rol
T.display	Syne	600, 700, 800	Voz de marca. Titulares (h1, h2, h3), el wordmark TURIFY, nombres propios grandes. Siempre con <code>fontWeight: 800</code> y <code>letterSpacing</code> negativo (-.01em / -.02em) en los títulos, y muy abierto (.24em) en el logo.
T.ui	DM Sans	400, 500, 700 (eje óptico 9..40)	Texto de interfaz: párrafos, etiquetas de formulario, botones, navegación. Es también la fuente por defecto de body en <code>index.css</code> .
T.dato	IBM Plex Mono	400, 500, 600	"Dato oficial": placas, FUEC, identificadores, horas, IPs, contadores numéricos, rótulos en mayúsculas espaciadas y el TableroRuta. Su función semántica es señalar <i>esto es un dato verificable</i> , no decorar.

El componente **Icono** y el catálogo completo

```
export const Icono = ({ children, size = 16, color = 'currentColor', grosor = 1.7, style, ...rest }) => (
  <svg width={size} height={size} viewBox="0 0 24 24" fill="none" stroke={color}
    strokeWidth={grosor} strokeLinecap="round" strokeLinejoin="round" style={style} {...rest}>
    {children}
  </svg>
);
```

Prop	Tipo	Por defecto	Efecto
children	nodos SVG	—	Los path, circle, rect... del dibujo
size	número	16	Se aplica a width y height (siempre cuadrado)
color	string	'currentColor'	Va a stroke; por defecto hereda el color del texto contenedor
grosor	número	1.7	strokeWidth
style	objeto	—	Estilo en línea
...rest	—	—	Se derrama sobre el <svg>

Todos los iconos son de trazo (`fill="none"`), con `viewBox="0 0 24 24"`, extremos y uniones redondeados. Comentario del bloque: «Iconos de trazo — un solo grosor, sin relleno, sin emojis.»

Cada icono exportado es un componente de una línea con la forma `(p) => <Icono {...p}>...</Icono>`, así que **todos aceptan exactamente las mismas props que `Icono`**.

Catálogo completo de iconos exportados (46), agrupado por tema

Tiempo, estado y confirmación `IconReloj` · `IconVisto` · `IconBandera` · `IconAlerta` · `IconEquis`

Personas y transporte `IconAuto` · `IconPersonas` · `IconPersona` · `IconGorro`

Ubicación y navegación en mapa `IconRadar` · `IconPin` · `IconCentrar`

Calendario, dinero y documentos de viaje `IconCalendario` · `IconRecibo` · `IconPrecio` · `IconEstrella` · `IconClipboard`

Flechas e interacción `IconFlecha` · `IconIntercambio` · `IconOjo` · `IconOjoTachado` · `IconCampana` · `IconIdea`

Comodidades del vehículo (bloque comentado `// — Comodidades del vehículo —`) `IconAire` · `IconWifi` · `IconBano` · `IconMusica` · `IconMaleta` · `IconBebe` · `IconMascota` · `IconEmpresa`

Suelto, sin agrupar `IconSenal`

Documentos oficiales (comentario: «se repiten entre el registro del conductor y la verificación del admin, así que viven acá y no sueltos en cada pantalla») `IconEscudo` · `IconTarjeta` · `IconLlave` · `IconDocumento` · `IconDescargar`

Navegación y acciones de la barra del admin `IconFlechaIzq` · `IconSalir` · `IconRecargar` · `IconLupa`

Bitácora de auditoría `IconCandado` · `IconProhibido` · `IconMapa` · `IconChincheta` · `IconBandeja`

No exportados (privados del módulo): `IconSol` e `IconLuna`, usados solo por `BotonTema`.

Componentes exportados

`EstilosBase`

Componente sin props. Renderiza una única etiqueta `<style>` con:

- `@import url(FUENTES_URL)` — las tres familias.
- Cuatro `@keyframes`: `t-girar` (rotación `0→360°`), `t-latido` (opacidad `1→.45` y escala `1→.72` en el 50%), `t-entra` (`fade + translateY(6px)→0`), `t-flap` (rotación en X hasta `-88deg` con `filter: brightness(.35)` en el medio — el efecto de un cartel abatible).
- La clase `.t-foco` (ver A.10).
- Un bloque `@media (prefers-reduced-motion: reduce)` que anula la animación de cualquier elemento con clase `.t-anim` (`animation: none !important`).

Comentario: «Estilos globales — se monta una vez por pantalla.» Efectivamente cada pantalla raíz lo monta: `Login`, `Registro`, `AdminConductores`, `AdminLogs` (y `Dashboard / PanelConductor`).

`BotonTema`

```
<BotonTema tema alternar compacto={false} />
```

Prop	Tipo	Por defecto
<code>tema</code>	<code>'claro' 'oscuro'</code>	— (requerida)
<code>alternar</code>	función	— (requerida)
<code>compacto</code>	booleano	<code>false</code>

Renderiza un `<button type="button" className="t-foco">` con `title` y `aria-label` iguales: `'Cambiar a modo claro'` si `tema === 'oscuro'`, `'Cambiar a modo oscuro'` si no. Muestra `<IconSol size={15} />` en modo oscuro y `<IconLuna size={15} />` en modo claro. Con `compacto={false}` añade el texto `('Modo claro' / 'Modo oscuro')` y `gap: '8px'`, `padding: '9px 12px'`; con `compacto` el `gap` es `0` y el

padding: '8px'. Fondo T.papel, borde 1px solid T.linea, radio T.rControl, color T.piedra, fontFamily: T.ui, 12px, peso 600.

Comentario: «Interruptor de tema — cabe en una barra lateral sin robar atención.»

BotonCentrarMapa

```
<BotonCentrarMapa onClick titulo="Centrar el mapa" deshabilitado={false} style />
```

Prop	Tipo	Por defecto
onClick	función	—
titulo	string	'Centrar el mapa' (va a title y a aria-label)
deshabilitado	booleano	false (va a disabled)
style	objeto	— (se derrama al final, permite sobrescribir la posición)

Botón cuadrado de 40×40 px, borderRadius: '10px' (literal, no T.rControl), posicionado absolute a right: '10px', bottom: '96px', zIndex: 20. Contiene <IconCentrar size={19} />. Deshabilitado: color T.piedraClara, cursor: 'not-allowed', opacity: 0.55.

Comentario: «flota sobre el mapa, arriba de los controles de zoom, que es donde el ojo ya lo busca. Cada pantalla decide qué significa "centrar": el pasajero encuadra su ruta, el conductor vuelve a su posición.»

TableroRuta — el elemento firma

```
<TableroRuta origen destino oscuro={false} size={12.5} style />
```

Prop	Tipo	Por defecto
origen	nodo/string	—
destino	nodo/string	—
oscuro	booleano	false
size	número	12.5 (se usa como fontSize: \\${size}px)
style	objeto	—

Renderiza un flex con gap: '9px', en T.dato, peso 600, textTransform: 'uppercase', letterSpacing: '.09em'. Estructura de tres :

1. **Origen** — truncado con elipsis; color '#EAF2EC' si oscuro, si no T.tinta.
2. **La línea** — flex: 1, alto 1 px, fondo T.monteLinea (oscuro) o T.linea; contiene dos puntos absolutos de 4×4 px: a la izquierda uno verde T.ruta, a la derecha uno dorado T.chiva.
3. **Destino** — truncado; color T.chiva si oscuro, si no T.chivaTexto.

Comentario: «El cartón del parabrisas: origen y destino en mayúsculas, una línea que los une.»

Boton

```
<Boton variante="primario" ancho children style {...rest} />
```

Prop	Tipo	Por defecto
variante	string	'primario'
ancho	booleano (truthy)	undefined → si es truthy, width: '100%'
style	objeto	—
...rest	—	— (se derrama al <button>: type, onClick, disabled...)

Base común: `inline-flex` centrado, `gap: '8px'`, `borderRadius: T.rControl`, `padding: '12px 18px'`, `fontFamily: T.ui`, peso 700, 13.5px, `cursor: 'pointer'`, `border: '1px solid transparent'`, transición de .18s en `background`, `border-color` y `color`. Siempre lleva `className="t-foco"`.

Cinco variantes:

Variante	Estilos
primario	<code>background: T.ruta</code> , <code>color: '#fff'</code>
fantasma	<code>background: 'transparent'</code> , <code>color: T.tinta</code> , <code>borderColor: T.linea</code>
monte	<code>background: T.monte</code> , <code>color: '#fff'</code>
peligro	<code>background: T.alertaSuave</code> , <code>color: T.alertaTexto</code> , <code>borderColor: T.alertaLinea</code>
inactivo	<code>background: 'var(--t-niebla-2)'</code> , <code>color: T.piedraClara</code> , <code>cursor: 'not-allowed'</code>

Nota: `inactivo` escribe la variable CSS a mano en vez de usar `T.niebla2`, que apunta a la misma variable.

Chip

```
<Chip tono="neutro" children style />
```

Prop	Tipo	Por defecto
tono	string	'neutro'
children	nodo	—
style	objeto	—

Base: `inline-flex`, `gap: '5px'`, `borderRadius: T.rChip` (píldora), `padding: '3px 9px'`, `fontSize: '10.5px'`, `fontWeight: 500`, `border: '1px solid'`.

Tono	fondo / borde / texto	Peso
neutro	<code>T.niebla</code> / <code>T.linea</code> / <code>T.piedra</code>	500 (base)
verde	<code>T.musgo</code> / <code>T.musgoLinea</code> / <code>T.musgoTexto</code>	700
chiva	<code>T.chivaSuave</code> / <code>T.chivaLinea</code> / <code>T.chivaTexto</code>	700

Tono	fondo / borde / texto	Peso
cielo	T.cieloSuave / T.cieloLinea / T.cieloTexto	700
alerta	T.alertaSuave / T.alertaLinea / T.alertaTexto	700

No acepta `...rest`: solo `tono`, `children` y `style`.

Dato

```
<Dato children style />
```

`<span>` con `fontFamily`: `T.dato`, peso 600, `letterSpacing`: `'.08em'`, `fontSize`: `'11.5px'`.

Comentario: «Dato oficial: placa, FUEC, identificador, hora.»

Rotulo

```
<Rotulo children style />
```

`<div>` con `fontFamily`: `T.dato`, 10.5px, peso 500, `textTransform`: `'uppercase'`, `letterSpacing`: `'.16em'`, `color`: `T.piedraClara`. Comentario: «Rótulo de sección — mayúsculas espaciadas, en monoespaciada.»

LogoWordmark

```
<LogoWordmark alto={22} oscuro={false} />
```

Prop	Tipo	Por defecto
alto	número	22 (es el <code>fontSize</code> en px del texto; el resto se deriva de él)
oscuro	booleano	false

Columna flex con `gap`: `alto * 0.32 px`:

1. La palabra `TURIFY` en `T.display`, peso 800, `letterSpacing`: `'.24em'`, `textIndent`: `'.24em'` (compensa el tracking del último carácter), `lineHeight`: 1, `color` `'#EAF2EC'` si `oscuro` o `T.tinta` si no.
2. Debajo, una barra de alto `Math.max(alto * 0.2, 5) px` que contiene la línea de 1.5 px (`T.monteLinea` u `T.linea`) y dos círculos de `alto * 0.19 px`: izquierdo `T.ruta`, derecho `T.chiva`.

Comentario: «A · WORDMARK CON RUTA — el nombre con la ruta corriendo por debajo, del ancho exacto de la palabra. Es el logo más callado y el más fiel al sistema.»

MarcaTurify

```
<MarcaTurify alto={32} oscuro={false} plano={false} />
```

Prop	Tipo	Por defecto
alto	número	32 (ancho y alto del SVG; <code>viewBox="0 0 40 40"</code>)

Prop	Tipo	Por defecto
oscuro	booleano	false
plano	booleano	false

SVG con `aria-hidden="true"`. Si **no** es plano, dibuja primero un `<rect>` de 40×40 con `rx="11"` relleno de `'#0A1F16'` (oscuro) o `T.tinta` (claro), con `stroke 'rgba(255,255,255,.12)'` solo en oscuro. Encima, un trazo diagonal de 12.5,27.5 a 27.5,12.5 (`strokeWidth="2"`, opacidad `.45` si plano, `.55` si no) y dos círculos de radio 5: el inferior izquierdo en `T.ruta`, el superior derecho en `T.chiva`.

Comentario: «los dos puntos son la marca. Sin nombre, quedan ellos y el tramo que los une, en diagonal para que llenen el cuadro y se separen bien a tamaño chico.»

LogoMonograma (propuesta B)

```
<LogoMonograma alto={40} oscuro={false} conTexto={true} />
```

Fila flex con `gap: alto * 0.32 px`. SVG 40×40 con una **T** cuyo travesaño horizontal (`M9 13h22`) es la ruta y el astil vertical (`M20 13v18`) la baja; en los extremos del travesaño, círculos de radio 4 (`T.ruta` a la izquierda, `T.chiva` a la derecha). Con `conTexto`, añade **TURIFY** en `T.display` peso 800, `fontSize: alto * 0.44`, `letterSpacing: '.2em'`, `textIndent: '.2em'`.

LogoBifurcacion (propuesta C)

```
<LogoBifurcacion alto={40} oscuro={false} conTexto={true} />
```

Misma estructura que `LogoMonograma`, pero el SVG dibuja una **Y**: un tronco vertical (`M20 33V22`) y dos ramales curvos que suben a izquierda y derecha, cada uno terminado en un círculo de radio 4 (`T.ruta` en 11.4,12.6; `T.chiva` en 28.6,12.6).

Comentario: «la Y de Turify leída como lo que es en una carretera: una bifurcación. Cada ramal termina en un destino posible. Es el logo que habla de elegir, que es justo lo que hace el pasajero con las ofertas.»

Regla de uso de la marca (documentada en comentario)

- **LogoWordmark** (con nombre) — donde hay ancho y la marca tiene que presentarse: pantalla de entrada y registro, cabecera de la barra lateral del pasajero, cabecera del panel del conductor y del admin, documentos generados (recibo, FUEC).
- **MarcaTurify** (compacta) — donde el nombre no cabe o sobra: favicon e icono de la app, barra lateral colapsada y vistas móviles angostas, avatar, marcador del mapa, estados de carga, cualquier caja de menos de ~120 px de ancho.
- Se usa **una, nunca las dos juntas**.

Tema: `leerTema`, `aplicarTema`, `useTema`

Comentario del bloque: «TEMA — claro / oscuro. Se guarda la preferencia y se marca `<html data-tema>`. Los tokens de arriba apuntan a variables CSS, así que con eso basta.»

leerTema() → 'claro' | 'oscuro'

1. Intenta `localStorage.getItem('turify-tema')` dentro de un `try`. Si el valor es exactamente 'oscuro' o 'claro', lo devuelve.
2. Si el almacenamiento está bloqueado, el `catch` no hace nada (comentario: «almacenamiento bloqueado: seguimos con el del sistema»).
3. Fallback: `window.matchMedia('(prefers-color-scheme: dark)').matches ? 'oscuro' : 'claro'`.

aplicarTema(tema)

1. `document.documentElement.setAttribute('data-tema', tema)` — esto es lo que activa el selector `:root[data-tema="oscuro"]` de `index.css`.
2. `localStorage.setItem('turify-tema', tema)` dentro de un `try/catch` (comentario: «sin persistencia»).

useTema() → [tema, alternar]

```
const [tema, setTema] = useState(() => (typeof window === 'undefined' ? 'claro' : leerTema()));
useEffect(() => { aplicarTema(tema); }, [tema]);
return [tema, () => setTema(t => (t === 'oscuro' ? 'claro' : 'oscuro'))];
```

- Inicializa perezosamente; en SSR (`typeof window === 'undefined'`) asume 'claro'.
- Un efecto aplica el tema cada vez que cambia — también en el primer render, así el atributo queda puesto al montar.
- El segundo elemento del array es una función que alterna. **Nota:** esa función se recrea en cada render (no está memorizada).

Dónde se persiste: clave 'turify-tema' en `localStorage`, valores 'claro' / 'oscuro'.

Dónde se usa: en `AdminConductores` y `AdminLogs` como `const [tema, alternarTema] = useTema();`, pasándolo a `<BotonTema tema={tema} alternar={alternarTema} compacto />`. Cada pantalla llama a su propio `useTema()`: no hay un contexto de tema compartido, pero como el estado real vive en el atributo del `<html>` y en `localStorage`, el resultado visual es coherente.

MAPA_OSCURO

Array de 14 objetos de estilo con la forma que espera la opción `styles` de Google Maps (`{ featureType, elementType, stylers: [{ color }] }`).

#	featureType	elementType	styler
1	—	geometry	color: '#0B1F16'
2	—	labels.text.stroke	color: '#0B1F16'
3	—	labels.text.fill	color: '#7E9187'
4	administrative	geometry	color: '#22402F'
5	administrative.locality	labels.text.fill	color: '#A8BBAF'
6	poi	labels	visibility: 'off'

#	featureType	elementType	styler
7	poi.park	geometry	color: '#102A1E'
8	road	geometry	color: '#1B3527'
9	road	labels.text.fill	color: '#8DA396'
10	road.highway	geometry	color: '#274534'
11	road.highway	geometry.stroke	color: '#12291F'
12	transit	labels	visibility: 'off'
13	water	geometry	color: '#07160F'
14	water	labels.text.fill	color: '#3F5A4B'

Por qué hace falta, según el comentario del bloque: «no es el "dark" genérico de Google: el suelo y el agua se llevan al verde monte de la marca, y las vías quedan apenas más claras que el fondo para que la ruta verde siga siendo lo que más resalta. **Los valores van fijos a propósito: Google Maps recibe JSON, no lee variables CSS.**»

Es decir, es el mismo motivo que justifica FIJO: el mapa está fuera del alcance del CSS. Se consume en `Dashboard.jsx` y `PanelConductor.jsx` como `styles: tema === 'oscuro' ? MAPA_OSCURO : undefined`.

Clases CSS globales

Las clases globales no viven en `index.css` sino dentro de `<EstilosBase />` (`diseño.jsx`):

Clase	Definición	Qué hace
.t-foco	<code>.t-foco:focus-visible { outline: 2px solid var(--t-ruta); outline-offset: 2px; }</code>	Anillo de foco unificado para navegación por teclado, en el verde de marca. Se aplica a prácticamente todo elemento interactivo del sistema (Boton, BotonTema, BotonCentrarMapa, enlaces, inputs, pestañas). Usa <code>:focus-visible</code> , así que no aparece al hacer clic con el ratón.
.t-anim	<code>@media (prefers-reduced-motion: reduce) { .t-anim { animation: none !important; } }</code>	Marcador para elementos animados: quien la lleve pierde su animación cuando el sistema pide movimiento reducido.

En `index.css` no hay clases: solo el bloque `html, body, #root` y el reset de `box-sizing`. En `Skeleton.jsx` se definen aparte `.sk` y `.sk-dark` (ver sección C).

Arranque y enrutado

Punto de entrada

`main.jsx` monta `<App />` dentro de `<StrictMode>` sobre `#root`, importando `./index.css`. `index.html` precarga los dominios de Google Fonts y enlaza la misma hoja de fuentes que `FUENTES_URL`; el favicon es `/favicon.svg`.

Dependencias relevantes de `package.json`: `react@^19.2.4`, `react-router-dom@^7.14.1`, `framer-motion@^12.38.0`, `@react-google-maps/api@^2.20.8`, `@supabase/supabase-js@^2.112.4`, `vite@^8.0.1`.

App.jsx — tabla de rutas

Jerarquía: `<AuthProvider>` envuelve a `<BrowserRouter>`, que envuelve a `<Routes>`. Es decir, **el contexto de autenticación está por fuera del router**, así que está disponible para todas las rutas pero no puede usar hooks de navegación.

Path	Componente renderizado	Protección
/login	<code><LoginConNavegacion /></code>	Pública
/registro	<code><RegistroConNavegacion /></code>	Pública
/dashboard	<code><Dashboard /></code>	<code><RutaPrivada /></code> (token)
/registro-conductor	<code><FormularioConductor /></code>	<code><RutaPrivada /></code> (token)
/conductor/:driverId	<code><PerfilConductorPagina /></code>	<code><RutaPrivada /></code> (token)
/admin/conductores	<code><AdminConductores /></code>	<code><RutaAdmin /></code> (token + role === 'ADMIN')
/admin/logs	<code><AdminLogs /></code>	<code><RutaAdmin /></code> (token + role === 'ADMIN')
/	<code><Navigate to="/dashboard" replace /></code>	Redirección

No hay ruta comodín `*`: una URL desconocida no renderiza nada.

Dos envoltorios locales existen para inyectar navegación en componentes que no la conocen:

```
const LoginConNavegacion = () => {
  const navigate = useNavigate();
  const { iniciarSesion } = useContext(AuthContext);
  return (
    <Login
      irARegistro={() => navigate('/registro')}
      onLoginSuccess={(token) => { iniciarSesion(token); navigate('/dashboard'); }}
    />
  );
};

const RegistroConNavegacion = () => {
  const navigate = useNavigate();
  return <Registro irALogin={() => navigate('/login')} />;
};
```

Así `Login` y `Registro` quedan desacoplados del router: reciben callbacks (`irARegistro`, `onLoginSuccess`, `irALogin`) en vez de importar `useNavigate`.

Obsérvese que tras iniciar sesión siempre se navega a `/dashboard`, incluso si el usuario es ADMIN.

AuthContext.jsx

```
import { createContext } from 'react';
export const AuthContext = createContext();
```

Tres líneas. El comentario explica el motivo de que esté en su propio archivo: «Solo exporta el contexto — sin componentes. Esto es necesario para que Vite Fast Refresh funcione correctamente.» (Un módulo que exporta a la vez un componente y un no-componente rompe el HMR de React Refresh.)

Se crea **sin valor por defecto**, así que fuera del proveedor su valor es `undefined` y desestructurarlo lanzaría un error.

AuthProvider.jsx**Helpers de módulo (no exportados):**

```
const decodeJWT = (token) => {
  try {
    const payload = token.split('.')[1];
    return JSON.parse(atob(payload));
  } catch { return null; }
};

const tokenEstaExpirado = (token) => {
  const decoded = decodeJWT(token);
  if (!decoded?.exp) return false;
  return decoded.exp * 1000 < Date.now();
};
```

- `decodeJWT` toma el segundo segmento del JWT (el payload), lo decodifica con `atob` y lo parsea. Ante cualquier fallo devuelve `null`. No verifica la firma (no puede: es cliente).
- `tokenEstaExpirado` multiplica `exp` por 1000 porque el claim está en segundos y `Date.now()` en milisegundos. **Si el token no tiene `exp`, devuelve `false`** — es decir, se lo considera vigente.

Estados:

Estado	Inicialización
token	<code>useState(() => localStorage.getItem('token') null)</code> — lazy, leído del almacenamiento
usuario	<code>useState(null)</code> — se rellena con la respuesta de <code>/users/me</code>

Funciones:

```
function cerrarSesion() {
  localStorage.removeItem('token');
  setToken(null);
  setUsuario(null);
}

function iniciarSesion(nuevoToken) {
  localStorage.setItem('token', nuevoToken);
  setToken(nuevoToken);
}
```

`iniciarSesion` no toca `usuario`: el perfil llega por el efecto que dispara el cambio de `token`.

Efecto 1 — cargar el perfil (dependencia `[token]`):

- Si no hay token, `setUsuario(null)` y sale.
- Si lo hay, `fetch(\`${API_BASE_URL}/users/me\`)` con cabeceras `Authorization: Bearer` y `ngrok-skip-browser-warning: 'true'`.
- Si la respuesta es `401`, llama a `cerrarSesion()` y devuelve `null`.
- Si no es `ok`, lanza `(throw new Error())`.
- Si llegan datos, `setUsuario(data)`.
- En el `catch`, solo cierra sesión **si el token está expirado** (`if (tokenEstaExpirado(token)) cerrarSesion();`). Así un fallo de red transitorio no expulsa al usuario.

Efecto 2 — vigilancia de expiración (dependencia `[token]`):

```
const intervalo = setInterval(() => { if (tokenEstaExpirado(token)) cerrarSesion(); }, 60000);
return () => clearInterval(intervalo);
```

Cada 60 000 ms comprueba la caducidad y cierra sesión automáticamente. Se limpia al desmontar o al cambiar el token.

Valor del contexto: `{ token, usuario, iniciarSesion, cerrarSesion }` — objeto literal recreado en cada render (sin `useMemo`).

RutaPrivada.jsx

Dos componentes de guardia, ambos basados en `<Outlet />` (rutas anidadas de React Router v6/v7):

```
const RutaPrivada = () => {
  const { token } = useContext(AuthContext);
  if (!token) return <Navigate to="/login" replace />;
  return <Outlet />;
};

export const RutaAdmin = () => {
  const { token, usuario } = useContext(AuthContext);
  if (!token) return <Navigate to="/login" replace />;
  // Mientras usuario carga, no redirigir
  if (!usuario) return null;
  if (usuario.role !== 'ADMIN') return <Navigate to="/dashboard" replace />;
  return <Outlet />;
};

export default RutaPrivada;
```

- **RutaPrivada** (exportación **por defecto**): basta con que exista `token`.
- **RutaAdmin** (exportación **nombrada**): además exige `usuario.role === 'ADMIN'`. El `return null` intermedio es deliberado — mientras `/users/me` no responde, `usuario` es `null` y redirigir sería expulsar a un admin legítimo. El efecto secundario es que la pantalla queda en blanco durante esa espera (sin indicador de carga).
- Ambas usan `replace`, así que la redirección no deja entrada en el historial.

api.js

```
const API_BASE_URL = import.meta.env.VITE_API_URL || 'http://127.0.0.1:8000';
export default API_BASE_URL;
```

Base de la API leída de la variable de entorno de Vite `VITE_API_URL`, con `http://127.0.0.1:8000` como valor por defecto para desarrollo local (puerto típico de FastAPI/uvicorn). Se importa como `import API_BASE_URL from './api'`; en todos los archivos que hacen `fetch`.

La cabecera `'ngrok-skip-browser-warning': 'true'`, presente en **todas** las peticiones de la app, revela que el backend se expone a veces mediante un túnel de ngrok: esa cabecera salta la página intersticial de advertencia de ngrok.

Utilidades de interfaz

Toast.jsx

Depende de `T`, `Icono`, `IconVisto`, `IconEquis`, `IconAlerta` de `diseno`, y define localmente un `IconInfo` (círculo con una `i`).

Tabla CONFIG — los cuatro tipos:

Tipo	bg	border	color	Ico	barColor
success	T.musgo	T.ruta	T.musgoTexto	IconVisto	T.ruta
error	T.alertaSuave	T.alerta	T.alertaTexto	IconEquis	T.alerta
warning	T.chivaSuave	T.chivaLinea	T.chivaTexto	IconAlerta	T.chiva
info	T.cieloSuave	T.cieloLinea	T.cieloTexto	IconInfo	T.cielo

ToastItem (privado, no exportado) — props { id, type, message, duration, onRemove }:

- Estados locales: `visible` (empieza en `false`) y `progress` (empieza en `100`); `intervalRef` para el temporizador de la barra.
- En el montaje (`useEffect` con array de dependencias vacío):
- `requestAnimationFrame(() => setVisible(true))` — dispara la animación de entrada en el siguiente fotograma.
- Un `setInterval` cada 50 ms resta `step = 100 / (duration / 50)` a `progress`, deteniéndose en 0.
- Un `setTimeout` a los `duration` ms pone `visible` en `false` y, 300 ms después, llama a `onRemove(id)`.
- La limpieza cancela ambos temporizadores.
- Visual: entra deslizándose desde `translateX(120%)` hasta `translateX(0)` con `cubic-bezier(0.34, 1.56, 0.64, 1)` (rebote suave) en 0.3 s. Caja de `minWidth: '300px'`, `maxWidth: '380px'`, `borderRadius: '10px'`, borde de 1 px con **borde izquierdo de 4 px** del color del tipo. Icono a 15 px, mensaje a 14 px peso 500, botón de cierre con el carácter `×` (opacidad 0.5). Abajo, una barra de progreso absoluta de 3 px de alto, ancho ``${progress}%``, opacidad 0.6.
- El cierre manual repite el patrón: `setVisible(false)` y `onRemove` 300 ms después, para que se vea salir.

ToastContainer (exportado) — props { toasts, onRemove }:

Contenedor `position: 'fixed'`, `top: '80px'`, `right: '20px'`, `zIndex: 99999`, columna alineada a la derecha, con `pointerEvents: 'none'` en el contenedor y `pointerEvents: 'all'` en cada envoltorio de toast — así el área vacía de la columna no bloquea clics sobre la interfaz de abajo. Mapea `toasts` por `t.id`.

useToast() (exportado) — devuelve { toasts, removeToast, toast }:

```
const addToast = useCallback((message, type = 'info', duration = 4000) => {
  const id = Date.now() + Math.random();
  setToasts(prev => [...prev, { id, message, type, duration }]);
}, []);

const removeToast = useCallback((id) => setToasts(prev => prev.filter(t => t.id !== id)), []);

const toast = {
  success: (msg, dur) => addToast(msg, 'success', dur),
  error: (msg, dur) => addToast(msg, 'error', dur || 5000),
  warning: (msg, dur) => addToast(msg, 'warning', dur),
  info: (msg, dur) => addToast(msg, 'info', dur),
};
```

- El `id` es `Date.now() + Math.random()` para evitar colisiones entre toasts creados en el mismo milisegundo.
- Duración por defecto **4000 ms**, salvo `error`, que usa **5000 ms** cuando no se pasa `dur`.
- **Nota:** `addToast` no se exporta directamente; el objeto `toast` es la API pública. `toast` se recrea en cada render (no memorizado), a diferencia de las dos funciones de `useCallback`.

Uso: `Dashboard.jsx` y `PanelConductor.jsx` — `const { toasts, removeToast, toast } = useToast();` y luego `<ToastContainer toasts={toasts} onRemove={removeToast} />`. No se usa en ninguna de las pantallas de admin ni de acceso.

Skeleton.jsx

Cabecera: «Skeleton.jsx — HU15 Mejoras de interfaz. Componente de pantalla de carga animada para Turify».

CSS compartido (`pulseStyle`), inyectado como `<style>` por cada componente que lo necesita:

- `@keyframes shimmer` — mueve `background-position` de `-200% 0` a `200% 0`.
- `.sk` — degradado de 90° `var(--t-linea) 25% → var(--t-niebla-2) 50% → var(--t-linea) 75%`, `background-size: 200% 100%`, `animation: shimmer 1.4s infinite`, `border-radius: 6px`. Es la versión que sigue el tema.
- `.sk-dark` — mismo patrón con `rgba(255,255,255,0.06 / 0.12 / 0.06)`, pensado para superficies oscuras.

Componentes exportados:

Componente	Props	Qué renderiza	Dónde se usa
<code>SkeletonBlock</code>	<code>width = '100%', height = 16, dark = false, style = {}</code>	Un solo <code><div></code> con clase <code>sk</code> o <code>sk-dark</code>	En ningún sitio (solo en el <code>export default</code>)

Componente	Props	Qué renderiza	Dónde se usa
SkeletonTarjetaViaje	dark = false	Maqueta de una tarjeta de solicitud (radar del conductor): dos líneas de texto (70 % y 55 %), una píldora de 80×22, dos chips de 120 y 80, y una barra de acción de 34 px al 100 %. Fondo #fff o rgba(255,255,255,0.04) según dark; borde var(--t-linea) o rgba(255,255,255,0.08)	PanelConductor.jsx (tres instancias seguidas mientras carga el radar)
SkeletonTarjetaConfirmado	ninguna	Maqueta de tarjeta de viaje confirmado del pasajero: cabecera con etiqueta y píldora, tres líneas de texto decrecientes, y un bloque interior sobre var(--t-niebla) con avatar circular de 36 px y dos líneas. Fondo var(--t-papel)	Dashboard.jsx
SkeletonDashboard	ninguna	Pantalla completa (position: fixed; inset: 0, zIndex: 99999) sobre var(--t-monte): un anillo de 64 px que gira (border: 3px solid rgba(34,197,94,0.2) con borderTop: 3px solid var(--t-ruta), animation: spin 0.8s linear infinite), la palabra Turify en Syne 20 px peso 700 color var(--t-ruta), y el texto Cargando tu sesión.... Define su propio @keyframes spin	Dashboard.jsx ({cargandoInicial && <SkeletonDashboard />})
ErrorConexion	onReintentar	Pantalla completa de error sobre var(--t-monte): tarjeta var(--t-monte-alto) con borde rgba(239,68,68,0.3), icono SVG de wifi tachado en un círculo var(--t-alerta-suave), titular «Sin conexión al servidor» en Syne, el texto «No se pudo conectar con el backend de Turify. Verifica que el servidor esté activo e intenta de nuevo.» y, solo si se pasó onReintentar , un botón «Reintentar»	Dashboard.jsx

Exportación por defecto: un objeto literal { SkeletonBlock, SkeletonTarjetaViaje, SkeletonTarjetaConfirmado, SkeletonDashboard, ErrorConexion }. Nadie lo importa: todos los consumidores usan las exportaciones nombradas.

PARTE 4

La pantalla del pasajero

Dashboard.jsx en detalle, más los dos componentes de entrada que usa.

Dashboard.jsx – Importaciones

Librerías externas

Import	Origen	Para qué
useState, useEffect, useContext, useRef, useCallback	react	Hooks base del componente.
motion, AnimatePresence	framer-motion	Animaciones de modales, paneles laterales, badges y botones (whileHover, whileTap, initial/animate/exit).
GoogleMap, MarkerF, PolylineF, useJsApiLoader	@react-google-maps/api	Mapa principal, marcadores de origen/destino, polilínea de la ruta y carga única del SDK JS de Google Maps.
useNavigate	react-router-dom	Navegación programática (solo se usa para /registro-conductor).

Contexto y componentes propios del proyecto

Import	Origen	Para qué
AuthContext	./AuthContext	Provee token y usuario.
PanelConductor	./PanelConductor	Vista completa que reemplaza al dashboard cuando usuario.role === 'DRIVER'.
AdminConductores	./AdminConductores	Vista completa cuando usuario.role === 'ADMIN'.
InputDireccion	./InputDireccion	Campo de dirección con autocompletado de Google Places.
SelectorFechaHora	./SelectorFechaHora	Calendario + selector de hora propio.
PerfilDrawer	./PerfilDrawer	Cajón lateral de perfil (HU16).
ToastContainer, useToast	./Toast	Sistema de notificaciones efímeras.

Import	Origen	Para qué
SkeletonDashboard, SkeletonTarjetaConfirmado, ErrorConexion	./Skeleton	Estados de carga y de error de conexión.
API_BASE_URL	./api (default export)	Base de todas las llamadas HTTP. En api.js: import.meta.env.VITE_API_URL \\ 'http:// 127.0.0.1:8000'.

Del sistema de diseño (./diseno)

Un único import agrupa tokens, iconos y componentes compartidos:

- **Tokens / utilidades:** T (objeto de tokens: T.ui, T.display, T.dato, T.ruta, T.monte, T.monteAlto, T.piedra, T.piedraClara, T.tinta, T.linea, T.musgo, T.musgoTexto, T.chiva, T.chivaTexto, T.chivaSuave, T.cieloTexto, T.cieloSuave, T.alertaTexto, T.alertaSuave, T.niebla2, T.rControl, T.rTarjeta), FIJO (colores hex literales: { ruta: '#16A34A', chiva: '#E9A13B', monte: '#0E2A1E', tinta: '#131A16' }), MAPA_OSCURO (estilos de Google Maps para tema oscuro).
- **Componentes:** EstilosBase, TableroRuta, Icono, MarcaTurify, LogoWordmark, BotonTema, BotonCentrarMapa.
- **Hook:** useTema.
- **Iconos:** IconReloj, IconVisto, IconBandera, IconAuto, IconCalendario, IconPersonas, IconRadar, IconRecibo, IconEstrella, IconClipboard, IconEquis, IconFlecha, IconAlerta, IconGorro, IconPin, IconPersona, IconCampana, IconPrecio, IconIntercambio, IconIdea.

Constantes de módulo

Constante	Valor	Para qué sirve
BRAND_GREEN	T.ruta (que en diseno.jsx es la cadena 'var(--t-ruta)')	Verde de marca; se usa en botones primarios, badges, pestaña activa, precios, etc. Se declara en la línea 14, antes del import { T, ... } de la línea 16 (funciona por hoisting de imports ES, pero es un orden inusual).
GOOGLE_MAPS_LIBRARIES	['places', 'geometry']	Librerías de Google Maps a cargar: places para el autocompletado de InputDireccion, geometry para decodePath del polyline. El comentario del código advierte que debe ser constante fuera del componente para no remontar el script en cada render.
GOOGLE_MAPS_API_KEY	import.meta.env.VITE_GOOGLE_MAPS_API_KEY	Clave de API del SDK de Google Maps.
mapContainerStyle	{ width: '100%', height: '100%' }	Estilo del contenedor del <GoogleMap> principal.

Constante	Valor	Para qué sirve
<code>centroDefaultColombia</code>	<code>{ lat: 4.6097, lng: -74.0817 }</code>	Centro por defecto del mapa (coordenadas de Bogotá) cuando aún no hay origen. También sirve de fallback en <code>handleContinuarConDireccionManual</code> .
<code>IconTrazo</code>	Icono (alias)	Alias del componente Icono del sistema de diseño; se usa para dibujar iconos ad hoc (flecha del desplegable "Quién", icono de refrescar).

Otras variables de entorno leídas **dentro** de los `useEffect` (no como constantes de módulo):

`VITE_SUPABASE_URL` y `VITE_SUPABASE_ANON_KEY`.

Componentes auxiliares definidos en el archivo

`ModalErrorDireccion` (nivel de módulo, líneas 39–72)

Prop	Tipo esperado	Uso
<code>textoDireccion</code>	<code>string</code>	Texto que el usuario escribió y que Google no encontró; se interpola en el título <code>No encontramos "{textoDireccion}"</code> .
<code>onCerrar</code>	función	Se ejecuta al hacer clic en el fondo (backdrop) y en el botón "Quiero corregir la dirección".
<code>onContinuar</code>	función	Se ejecuta en el botón "Continuar con esta dirección".

Qué renderiza: un fragmento con dos `motion.div`: 1. Un backdrop `position: absolute`, `height: '100vh'`, `color rgba(14,42,30,0.52)`, `zIndex: 3000`. 2. Una tarjeta centrada (`translate(-50%, -50%)`, `ancho 390px`, `zIndex: 3001`) con: icono `IconPin`, título, párrafo explicativo, botón verde "Continuar con esta dirección" (con `IconVisto`), botón secundario "Quiero corregir la dirección", y una caja de consejos con `IconIdea` y una lista de 4 sugerencias (barrio, comuna, lugar cercano, municipio).

`SelectorPasajero` (definido dentro de `Dashboard`, líneas 932–947)

Prop	Uso
<code>título</code>	Texto principal ("Adultos", "Niños").
<code>subtítulo</code>	Texto secundario ("15 años o más", "Edades 0 – 14").
<code>tipo</code>	Clave dentro del estado pasajeros ('adultos' o 'ninos').

Qué renderiza: una fila con título/subtítulo a la izquierda y a la derecha un stepper: `motion.button -` (llama `actualizarPasajeros(tipo, 'restar')`), el número `pasajeros[tipo]`, y `motion.button +` (llama `actualizarPasajeros(tipo, 'sumar')`), deshabilitado cuando `totalAsientos >= 44` (constante local `deshabilitado`). Lee `totalAsientos`, `pasajeros` y `actualizarPasajeros` por cierre léxico, no por props.

El componente Dashboard

Estados (useState)

Carga general y errores

Variable	Setter	Valor inicial	Representa
cargandoInicial	setCargandoInicial	true	Muestra <code><SkeletonDashboard /></code> hasta que la primera carga de viajes termina.
errorConexion	setErrorConexion	false	Muestra <code><ErrorConexion onReintentar={...} /></code> si falló la carga inicial.

Búsqueda de ruta y mapa

Variable	Setter	Valor inicial	Representa
tipoViaje	setTipoViaje	'ida'	'ida' o 'redondo'; controla si se pide <code>return_time</code> y qué <code>trip_type</code> se envía.
busqueda	setBusqueda	{ origen: '', destino: '', departure_time: '', return_time: '' }	Los cuatro campos del formulario de búsqueda.
mostrarPasajeros	setMostrarPasajeros	false	Abre/cierra el desplegable de conteo de pasajeros.
pasajeros	setPasajeros	{ adultos: 1, ninos: 0, mascotas: false }	Composición del grupo de viaje.
cargandoMapa	setCargandoMapa	false	Indicador de "Buscando ruta..." en el botón de submit.
datosMapa	setDatosMapa	{ origen: null, destino: null, ruta: [] }	Coordenadas {lat, lng} de origen y destino y el array de puntos del polyline decodificado.
comodidadesFiltro	setComodidadesFiltro	{ tiene_ac: false, tiene_wifi: false, tiene_bano: false, tiene_musica: false, tiene_maletero_amplio: false, tiene_sillas_bebe: false, acepta_mascotas: false }	HU55 — comodidades que el pasajero exige del vehículo.
infoRuta	setInfoRuta	null	{ distancia, tiempo, datosParaPrecio }; su presencia es lo que hace aparecer el modal "Resumen del viaje".
marcandoEnMapa	setMarcandoEnMapa	null	'origen', 'destino' o null: modo "tocar el mapa para fijar el punto".

Variable	Setter	Valor inicial	Representa
errorDireccion	setErrorDireccion	null	{ campo: 'origen' 'destino' }; dispara ModalErrorDireccion.

Publicación de viajes

Variable	Setter	Valor inicial	Representa
enviandoSolicitud	setEnviandoSolicitud	false	Bloquea el botón "Confirmar y Publicar Viaje" mientras se hace el POST.

Lista de viajes

Variable	Setter	Valor inicial	Representa
mostrarMisSolicitudes	setMostrarMisSolicitudes	false	Abre el panel lateral derecho "Mis viajes".
listaSolicitudes	setListaSolicitudes	[]	Viajes en estado PENDING con sus ofertas anidadas.
viajesConfirmados	setViajesConfirmados	[]	Viajes ASSIGNED / IN_PROGRESS / COMPLETED.
pestanasViajes	setPestanasViajes	'activos'	Pestaña activa: 'activos' 'confirmados' 'completados'.
viajeSeleccionado	setViajeSeleccionado	null	Viaje cuyo detalle de ofertas se está viendo dentro del panel.
actualizandoViajes	setActualizandoViajes	false	Anima el botón "Actualizar" (icono girando).
confirmandoCancelarId	setConfirmandoCancelarId	null	ID del viaje que muestra la confirmación inline "¿Cancelar?".
cancelandoId	setCancelandoId	null	ID del viaje cuya cancelación está en curso.

Ofertas y contraofertas

Variable	Setter	Valor inicial	Representa
modalContraoferta	setModalContraoferta	null	{ viajeId, ofertaId }; su presencia abre el modal de contraoferta.
precioContraoferta	setPrecioContraoferta	''	Valor del input numérico de la contraoferta.
enviandoContraoferta	setEnviandoContraoferta	false	Bloquea el botón "Enviar oferta".

Notificaciones

Variable	Setter	Valor inicial	Representa
notificaciones	setNotificaciones	[]	Lista de notificaciones del backend (notification_id, type, title, message, is_read, created_at).

Variable	Setter	Valor inicial	Representa
mostrarNotificaciones	setMostrarNotificaciones	false	Abre el panel lateral izquierdo de notificaciones.

Modales / perfil / tema

Variable	Setter	Valor inicial	Representa
mostrarPerfil	setMostrarPerfil	false	Abre PerfilDrawer.
tema, alternarTema	(hook useTema(), no useState)	—	Tema actual ('oscuro' se compara explícitamente) y función para alternarlo.

Calificaciones (HU46)

Variable	Setter	Valor inicial	Representa
modalCalificar	setModalCalificar	null	request_id del viaje a calificar; abre el modal.
estrellasCalificar	setEstrellasCalificar	0	Puntaje 1–5 elegido.
comentarioCalificar	setComentarioCalificar	''	Comentario opcional.
enviandoCalificacion	setEnviandoCalificacion	false	Bloquea el botón "Enviar calificación".

FUEC / ocupantes (HU10)

Variable	Setter	Valor inicial	Representa
modalFuec	setModalFuec	null	request_id del viaje cuyos ocupantes se registran; abre el modal.
ocupantesFuec	setOcupantesFuec	[{ full_name: '', document_type: 'CC', document_number: '' }]	Lista editable de ocupantes.
enviandoFuec	setEnviandoFuec	false	Bloquea "Confirmar ocupantes".
fuecEnviado	setFuecEnviado	{}	Mapa { [request_id]: true } para cambiar el texto/color del botón. Solo vive en memoria: se pierde al recargar.

Recibo (HU42)

Variable	Setter	Valor inicial	Representa
descargandoRecibo	setDescargandoRecibo	null	request_id cuyo PDF se está generando. Declarado en la línea 603, en medio de las funciones.

useRef, useContext y otros hooks

Hook	Declaración	Uso
useContext	<code>const { token, usuario } = useContext(AuthContext)</code>	token para el header Authorization; usuario para user_id, role y profile_photo_url.
useNavigate	<code>const navigate = useNavigate()</code>	Solo <code>navigate('/registro-conductor')</code> .
useToast	<code>const { toasts, removeToast, toast } = useToast()</code>	<code>toast(...)</code> , <code>toast.success</code> , <code>toast.warning</code> , <code>toast.error</code> .
useJsApiLoader	<code>const { isLoading: mapsLoaded } = useJsApiLoader({ id: 'google-map-script', googleMapsApiKey: GOOGLE_MAPS_API_KEY, libraries: GOOGLE_MAPS_LIBRARIES })</code>	Carga única del SDK; mapsLoaded se pasa como prop a InputDireccion.
useRef	<code>const mapRef = useRef(null)</code>	Instancia del mapa de Google, para <code>fitBounds</code> , <code>panTo</code> , <code>setZoom</code> .
useRef	<code>const coordsBuffer = useRef({})</code>	Guarda { lat1, lon1 } del origen ya resuelto mientras el modal de error de dirección espera decisión sobre el destino.
useTema	<code>const [tema, alternarTema] = useTema()</code>	Tema claro/oscuro; alimenta MAPA_OSCURO y BotonTema.

Constantes derivadas dentro del componente

Nombre	Valor	Uso
BOUNDS_ANTIOQUIA	<code>{ south: 5.4, west: -77.2, north: 8.9, east: -73.9 }</code>	Sesgo (no filtro) geográfico para la geocodificación. Está dentro del componente (se recrea en cada render); el mismo valor está duplicado en InputDireccion.jsx.
totalAsientos	<code>pasajeros.adultos + pasajeros.ninos</code>	Límite de 44 y texto del selector.
textoViajeros	<code>`\${totalAsientos} viajero\${totalAsientos > 1 ? 's' : ''}`</code>	Texto del campo "Quién".
MINUTOS_AVISO_SIN_OFERTAS	15	Umbral para el aviso "Seguimos buscando conductores en tu zona".
viajesActivosConfirmados	<code>viajesConfirmados.filter(v => v.trip_status !== 'COMPLETED')</code> ordenado: primero IN_PROGRESS, luego por departure_time ascendente	Contenido de la pestaña "Confirmados".
viajesCompletadosOrdenados	<code>viajesConfirmados.filter(v => v.trip_status === 'COMPLETED')</code> ordenado por departure_time descendente	Contenido de la pestaña "Completados".

Nombre	Valor	Uso
sidebarStyle, sidebarTopStyle, fieldBoxStyle, fieldLabelStyle, dividerStyle, inputStyle, navBtnStyle	objetos de estilo inline	Estética del panel lateral izquierdo. inputStyle está declarado pero no se usa .

Funciones definidas dentro del componente

Mapa y geocodificación

onMapLoad(map) — useCallback(..., []) Guarda la instancia del mapa: `mapRef.current = map`. Se pasa como `onLoad` del `<GoogleMap>` principal.

encuadrarMapa() — useCallback(..., [datosMapa.origen, datosMapa.destino]) 1. Si no hay `mapRef.current` o no hay `window.google`, sale. 2. Si hay origen y destino: crea `new window.google.maps.LatLngBounds()`, extiende con ambos y llama `mapRef.current.fitBounds(bounds, 50)`. 3. Si solo hay origen: `panTo(datosMapa.origen)` y `setZoom(15)`. 4. Si no hay nada y existe `navigator.geolocation`: pide la posición actual y hace `panTo + setZoom(14)`. No llama a ningún endpoint. Es el `onClick` de `BotonCentrarMapa`.

geocodificar(texto) — devuelve `Promise<{lat, lon} | null>` Si `!mapsLoaded || !window.google` resuelve `null` inmediatamente. Crea `new window.google.maps.Geocoder()` y llama `geocode({ address: ${texto}, Colombia, componentRestrictions: { country: 'co' }, bounds: <LatLngBounds de BOUNDS_ANTIQUA> })`. Si `status === 'OK'` resuelve `{ lat: loc.lat(), lon: loc.lng() }` (nótese la clave **lon**, no **lng**); si no, hace `console.warn('[Geocoding] Google no encontró "..."` — `status: ...')` y resuelve `null`. Sin proveedor de respaldo.

geocodificarInverso(lat, lng) — devuelve `Promise<string | null>` Geocodificación inversa con `geocode({ location: { lat, lng } })`; resuelve `results[0].formatted_address` o `null` con `console.warn`.

nombreDelPunto({ lat, lng }) — useCallback(..., []), async Usa la API basada en promesas del `geocoder` (`const { results } = await geocoder.geocode({ location })`). Devuelve `results[0].formatted_address` o, en su defecto, `'Punto en el mapa (${lat.toFixed(5)}, ${lng.toFixed(5)})'`.

alTocarMapa(evento) — useCallback(..., [marcandoEnMapa, nombreDelPunto]), async 1. Si `marcandoEnMapa` es `null`, no hace nada. 2. Extrae `lat/lng` de `evento.latLng`, guarda `campo = marcandoEnMapa` y desactiva el modo (`setMarcandoEnMapa(null)`). 3. Obtiene el texto con `nombreDelPunto` y actualiza `busqueda[campo]` y `datosMapa[campo]`. 4. Muestra `toast('Origen|Destino marcado en el mapa.', 'success')`. Es el `onClick` del `<GoogleMap>`.

handleBusqueda(e) `setBusqueda(prev => ({ ...prev, [e.target.name]: e.target.value })).` Es el `onChange` de ambos `InputDireccion`.

handleUbicacionActual({ texto, coords }) Recibe el callback de `InputDireccion` cuando el usuario usa el botón de ubicación actual. Convierte `coords` (array `[lat, lng]`) al formato de Google: `setDatosMapa(prev => ({ ...prev, origen: { lat: coords[0], lng: coords[1] } })).` **No usa texto.**

actualizarPasajeros(tipo, operacion) Reglas dentro del `setPasajeros`: si `operacion === 'sumar'` y `prev.adultos + prev.ninos >= 44`, no cambia nada; calcula nuevo; si `tipo === 'adultos'` y `nuevo < 1`, no cambia (mínimo 1 adulto); si `nuevo < 0`, no cambia; si no, devuelve el objeto actualizado.

trazarRutaConCoords(lat1, lon1, lat2, lon2) — devuelve `Promise<leg | null>` 1. Si el SDK no está listo: `toast.error('El mapa todavía se está cargando, intenta de nuevo en un momento.')` y resuelve `null`. 2. `new window.google.maps.DirectionsService().route({ origin, destination, travelMode: DRIVING })`. 3. Con `status === 'OK'`: toma `result.routes[0]` y `ruta.legs[0]`; decodifica `ruta.overview_polyline` con `window.google.maps.geometry.encoding.decodePath` y lo mapea a `{lat, lng}`. 4. Heurística de vía: concatena `leg.steps[].instructions` en minúsculas y evalúa `/trocha|sin pavimentar|camino rural|unnamed road/` → `posibleTrocha`. 5. `setDatosMapa({ origen, destino, ruta: pathDecodificado })`. 6. `setInfoRuta({ distancia: '<km> km', tiempo: '<min> min', datosParaPrecio: { distancia_metros, duracion_segundos, posible_trocha, tiene_estimacion_peajes: false } })`. 7. Resuelve `leg`. En caso contrario: `toast.warning('No se pudo trazar una ruta entre estas dos ubicaciones.')` y resuelve `null`.

trazarRutaConductor(origen, destino) — `async` Geocodifica ambos textos en paralelo (`Promise.all`), llama `trazarRutaConCoords` y después hace `setInfoRuta(null)` (traza la ruta sin abrir el modal de resumen). **Nunca se invoca en este archivo** (código muerto).

buscarRuta(e) — `async`, `onSubmit` del formulario 1. `e.preventDefault()`. 2. Valida `busqueda.origen`, `busqueda.destino` y `busqueda.departure_time` → `toast.warning('Por favor completa origen, destino y fecha de salida.')`. 3. Si `tipoViaje === 'redondo'` y falta `return_time` → `toast.warning('Por favor selecciona una fecha de regreso.')`. 4. `setCargandoMapa(true)`, `setInfoRuta(null)`, `setMostrarPasajeros(false)`. 5. Origen: si ya hay `datosMapa.origen` y el texto contiene 'ubicación', 'ubicacion' o 'mi ubicación', reutiliza esas coordenadas; si no, geocodifica. Si la geocodificación falla: limpia `coordsBuffer.current`, `setErrorDireccion({ campo: 'origen' })` y sale. 6. Destino: geocodifica. Si falla: guarda `coordsBuffer.current = { lat1, lon1 }`, `setErrorDireccion({ campo: 'destino' })` y sale. 7. `await trazarRutaConCoords(...)`. 8. `catch: toast.error('Error de conexión con el servidor de rutas.')`. `finally: setCargandoMapa(false)`. No llama endpoints propios (todo es SDK de Google).

handleContinuarConDireccionManual() — `async` Se ejecuta desde `ModalErrorDireccion`. Cierra el modal, activa `cargandoMapa` y bifurca: - Si `errorDireccion.campo === 'origen'`: usa `datosMapa.origen` o `centroDefaultColombia` como origen, geocodifica el destino y, si lo encuentra, actualiza `datosMapa` y traza la ruta; si no, `setInfoRuta({ distancia: 'No disponible', tiempo: 'No disponible' })`. - Si el campo fallido fue el destino: recupera `{ lat1, lon1 }` de `coordsBuffer.current`, fija solo el origen en `datosMapa` y deja `infoRuta` en "No disponible".

Publicación del viaje

crearViaje() — `async` 1. Sin token: `toast.warning('Debes iniciar sesión para publicar un viaje.')` y sale. 2. `setEnviandoSolicitud(true)`. 3. Arma payload: `origin, destination, departure_time, return_time` (`null` si no es redondo), `trip_type` (`'ROUND_TRIP'` | `'ONE_WAY'`), `adults_count, children_count, has_pets`. 4. Si hay `datosMapa.origen` y `datosMapa.destino`, añade `origin_lat, origin_lng, destination_lat, destination_lng`, y si existe `infoRuta.datosParaPrecio` añade `distance_km` (redondeado a 2 decimales) y `tipo_via` (`'DESTAPADA'` si `posible_trocha`, si no `'PAVIMENTADA'`). 5. Añade las 7 banderas HU55: `requiere_ac, requiere_wifi, requiere_bano, requiere_musica, requiere_maletero_amplio, requiere_sillas_bebe, requiere_acepta_mascotas`. 6. **POST** ``${API_BASE_URL}/api/service-requests/` con headers `Content-Type: application/json`, `Authorization: Bearer <token>`, `ngrok-skip-browser-warning: true`. 7. Si `!res.ok`, lanza `err.detail`

```
|| 'Error al publicar el viaje'. 8. Lee el cuerpo; si data.conductores_notificados === 0, muestra un toast.warning explicando que no hay conductores conectados. 9. Limpia: setInfoRuta(null), setDatosMapa({ origen: null, destino: null, ruta: [] }), setComodidadesFiltro(...todo false), setBusqueda(...vacío). 10. Si sí había conductores: toast.success('¡Viaje publicado exitosamente! Los conductores podrán hacerte ofertas.'). 11. finally: setEnviandoSolicitud(false).
```

Notificaciones

cargarNotificaciones() — async Sin token, sale. **GET ``${API_BASE_URL}/api/service-requests/notifications`** (headers Authorization, ngrok-skip-browser-warning). Si res.ok, setNotificaciones(await res.json()). catch {} silencioso.

marcarLeida(notifId) — async **PATCH ``${API_BASE_URL}/api/service-requests/notifications/${notifId}/read`**. Actualiza localmente notificaciones poniendo is_read: true en la que coincida por notification_id. catch {} silencioso.

marcarTodasLeidas() Filtra las no leídas y llama marcarLeida(n.notification_id) por cada una (una petición PATCH por notificación).

Ocupantes / FUEC

enviarFuec() — async 1. Valida que todos los ocupantes tengan full_name y document_number no vacíos → toast.warning('Completa nombre y número de documento de todos los ocupantes.'). 2. setEnviandoFuec(true). 3. **POST ``${API_BASE_URL}/api/service-requests/${modalFuec}/passengers`** con body { passengers: ocupantesFuec }. 4. Si falla, lanza err.detail. 5. Éxito: setFuecEnviado(prev => ({ ...prev, [modalFuec]: true })), setModalFuec(null), toast.success('Ocupantes registrados. El conductor ya puede ver la lista.').

agregarOcupante() — añade { full_name: '', document_type: 'CC', document_number: '' } al final.

quitarOcupante(idx) — no hace nada si solo queda uno; si no, filtra por índice.

actualizarOcupante(idx, campo, valor) — mapea y reemplaza campo en el ocupante idx.

Calificación

enviarCalificacion() — async 1. Si estrellasCalificar < 1 → toast.warning('Elige de 1 a 5 estrellas.'). 2. **POST ``${API_BASE_URL}/api/service-requests/${modalCalificar}/rating`** con body { score: estrellasCalificar, comment: comentarioCalificar || null }. 3. Éxito: marca ya_califico: true en el viaje correspondiente de viajesConfirmados, cierra el modal, resetea estrellas y comentario, toast.success('Gracias por tu calificación.').

Recibo

descargarRecibo(requestId) — async 1. setDescargandoRecibo(requestId). 2. **GET ``${API_BASE_URL}/api/service-requests/${requestId}/receipt`** con Authorization (por eso no puede ser un <a href> plano). 3. Si falla, intenta leer detail del JSON; si no, usa 'No se pudo generar el recibo.'. 4. Convierte a blob, crea window.URL.createObjectURL, genera un <a> con download = turify\_recibo\_\${requestId}.pdf`, lo inserta, hace click(), lo quita y libera el object URL. 5. finally : setDescargandoRecibo(null).

Carga de viajes

cargarMisViajes() — async 1. Headers comunes: Authorization: Bearer <token>, ngrok-skip-browser-warning: true. 2. **GET ``${API_BASE_URL}/api/service-requests/pending`**. Si no está ok, retorna sin tocar nada. 3. Por cada viaje pendiente, en paralelo (Promise.all + map async), **GET ``${API_BASE_URL}/api/service-requests/${requestId}`**

{API_BASE_URL}/api/service-requests/\${v.request_id}/offers y mapea cada oferta a: { id: o.offer_id, driverId: o.driver_id, conductor: o.driver_name || 'Conductor #<id>', foto: o.driver_photo || null, verificado: !!o.driver_verificado, calificacion: o.driver_rating ?? null, calificacionCantidad: o.driver_rating_count || 0, precio: o.offered_price, estado: o.status, comodidades: o.comodidades || null, recomendado: !!o.recomendado }. 4. Cada viaje pendiente queda como: { id: v.request_id, origin, destination, departure_time, seats_needed: (adults_count||1)+(children_count||0), estado: ofertas.length > 0 ? 'Oferta recibida' : 'Buscando conductor', fechaCreacion, created_at, ofertas }. 5. **GET {API_BASE_URL}/api/service-requests/assigned** → setViajesConfirmados(...) mapeando: id, origin, destination, departure_time, seats_needed, fechaCreacion, conductor_nombre (|| 'Conductor asignado'), conductor_foto, precio_acordado (|| 0), trip_status (v.trip_status || v.status || 'ASSIGNED'), driver_id, conductor_lat, conductor_lng, ya_califico. 6. Finalmente setListaSolicitudes(viajesConOfertas). 7. catch: console.error('Error al cargar los viajes:', error).

refrescarViajes() — async setActualizandoViajes(true) → await cargarMisViajes() → finally setActualizandoViajes(false). Es el botón circular de refrescar junto a las pestañas.

Ofertas

handleAceptarOferta(viajeId, ofertaId) — async **PATCH {API_BASE_URL}/api/service-requests/offers/\${ofertaId}/accept**. Éxito: toast.success('¡Viaje aceptado! El conductor ha sido notificado.'). setViajeSeleccionado(null) y cargarMisViajes(). **El parámetro viajeId no se usa.**

handleRechazarOferta(viajeId, ofertaId) **No llama a ningún endpoint.** Solo local: quita la oferta de listaSolicitudes (recalculando estado a 'Oferta recibida' o 'Buscando conductor') y de viajeSeleccionado. Al próximo cargarMisViajes() la oferta reaparece.

handleContraoferta(viajeId, ofertaId) Abre el modal: setModalContraoferta({ viajeId, ofertaId }) y setPrecioContraoferta('').

enviarContraoferta() — async 1. Valida que precioContraoferta sea numérico y > 0 → toast.warning('Ingresa un precio válido mayor a 0.'). 2. **PATCH {API_BASE_URL}/api/service-requests/offers/\${modalContraoferta.ofertaId}/counter-offer** con body { offered_price: Number(precioContraoferta) }. 3. Éxito: toast.success('¡Contraoferta de \$X enviada al conductor!'), cierra el modal y cargarMisViajes().

cancelarBusqueda(viajeId) — async 1. setCancelandoId(viajeId). 2. **PATCH {API_BASE_URL}/api/service-requests/\${viajeId}/cancel**; si falla, err.detail || 'No se pudo cancelar la búsqueda.'. 3. Éxito: quita el viaje de listaSolicitudes, limpia viajeSeleccionado si era ese, toast.success('Búsqueda cancelada.'). 4. finally: setCancelandoId(null) y setConfirmandoCancelarId(null).

Utilidades de presentación

abrirPerfilConductor(driverId) — si hay id, window.open(/conductor/\${driverId}, '_blank', 'noopener,noreferrer') (HU38: página aparte, no modal).

tiempoRelativo(fechaStr) — devuelve '', 'hace un momento' (<60 s), 'hace N min' (<1 h), 'hace Nh' (<1 d) o 'hace Nd'.

minutosTranscurridos(fechaStr) — minutos enteros desde fechaStr; 0 si no hay fecha.

useEffect

#	Línea	Dependencias	Qué hace	Limpieza
1	128	[encuadrarMapa]	Llama encuadrarMapa(). Como encuadrarMapa es un useCallback que depende de datosMapa.origen/ datosMapa.destino, se reencuadra el mapa cada vez que cambian esos puntos.	Ninguna.
2	213	[mapsLoaded]	Si el SDK ya cargó y hay navigator.geolocation, pide la posición actual, fija datosMapa.origen y hace geocodificación inversa para poner el texto en busqueda.origen (fallback 'Mi ubicación'). Si el usuario deniega: console.log('El usuario denegó la ubicación').	Ninguna.
3	508	[token, usuario]	Llama cargarNotificaciones() y luego: si hay VITE_SUPABASE_URL y VITE_SUPABASE_ANON_KEY, importa dinámicamente @supabase/supabase-js y se suscribe al canal de notificaciones; si no, hace polling setInterval(cargarNotificaciones, 15000).	Solo la rama de polling limpia (clearInterval). La rama de Supabase devuelve el cleanup dentro del .then() , así que React nunca lo ejecuta.
4	825	[token, usuario]	Carga inicial de viajes con manejo de skeleton/error (setErrorConexion(true) en catch, setCargandoInicial(false) en finally) y suscripción Realtime a los cambios del propio pasajero; si no hay Supabase, polling de 15 s.	Mismo problema: clearInterval sí; removeChannel dentro del .then(), no efectivo.
5	864	[JSON.stringify(viajesConfirmados.filter(v => v.trip_status === 'IN_PROGRESS').map(v => v.driver_id))]	HU43 — Tracking. Calcula driverIdsEnCurso; si está vacío o falta configuración de Supabase, sale. Importa el cliente y crea un canal por conductor. Usa un flag cancelado y un array canales.	Sí: pone cancelado = true y llama ch.unsubscribe() en cada canal.

Suscripciones a Supabase Realtime

Canal	Tabla	Evento	Filtro	Acción al recibir
`notif-dashboard-\${usuario.user_id}`	public.Notification	INSERT	user_id=eq.\$ {usuario.user_id}	cargarNotificaciones() (recarga completa de la lista).
`viajes-pasajero-\${usuario.user_id}`	public.ServiceRequest	UPDATE	passenger_id=eq.\$ {usuario.user_id}	cargarMisViajes() (recarga pendientes + asignados).

Canal	Tabla	Evento	Filtro	Acción al recibir
`tracking-conductor-` conductor-` {driverId}` (uno por conductor con viaje IN_PROGRESS)	public.User	UPDATE	user_id=eq.\$ {driverId}	Lee payload.new.current_lat / current_lng; si alguno es null ignora; si no, actualiza solo conductor_lat/conductor_lng de los viajes con ese driver_id en viajesConfirmados (sin recargar la lista).

En los tres casos el cliente se crea con `createClient(SUPABASE_URL, SUPABASE_ANON_KEY)` tras un `import('@supabase/supabase-js')` dinámico.

Estructura de vistas y pestañas

Bifurcación por rol (antes del render principal, después de todos los hooks)

- `usuario?.role === 'DRIVER' →` devuelve un contenedor con `<PanelConductor /> + <ToastContainer />`. El dashboard de pasajero nunca se muestra (HU23).
- `usuario?.role === 'ADMIN' →` devuelve directamente `<AdminConductores />` (sin `ToastContainer`).
- Cualquier otro rol → dashboard de pasajero.

Layout del dashboard de pasajero

Contenedor `display: flex, height: 100vh, overflow: hidden:`

A. <aside> — panel lateral izquierdo (470 px fijos, scroll vertical) – Cabecera: `<LogoWordmark alto={17} />`, campana de notificaciones con badge del conteo de no leídas (`#C2410C`), y avatar (abre `PerfilDrawer`). – Título "¿A dónde viajas hoy?" y subtítulo. – Formulario (`onSubmit={buscarRuta}`): – Conmutador "Solo ida" / "Ida y vuelta". – Campo **Origen**: `<InputDireccion esOrigen ancho="260px" /> +` botón "Marcar en el mapa" / "Tocá el mapa...". – Campo **Destino**: igual, sin `esOrigen`. – `<SelectorFechaHora label="Salida" required />` y, solo si `tipoViaje === 'redondo'`, `<SelectorFechaHora label="Regreso" min={busqueda.departure_time} />` animado con `AnimatePresence`. – Campo **Quién**: muestra `textoViajeros` y despliega el popover con dos `SelectorPasajero` (Adultos, Niños), un checkbox de Mascotas, el aviso "Límite máximo alcanzado." si `totalAsientos >= 44` y un botón "Cerrar". – Botón submit: "Buscar ruta" (con `IconRadar`) o "Buscando ruta..." si `cargandoMapa`. – Botón "Mis viajes" (llama `cargarMisViajes()` y abre el panel) con badge = suma total de ofertas de todos los viajes pendientes. – Botón "Quiero ser conductor" (`navigate('/registro-conductor')`), oculto para `DRIVER` y `ADMIN`. – Pie "Apariencia" con `<BotonTema tema={tema} alternar={alternarTema} />`.

B. <main> — panel del mapa – `<GoogleMap>` con `center = datosMapa.origen || centroDefaultColombia, zoom = datosMapa.origen ? 15 : 6, disableDefaultUI: true, zoomControl: true, styles: MAPA_OSCURO` si el tema es oscuro, y `draggableCursor: 'crosshair'` si `marcandoEnMapa`. – `MarkerF` de origen y destino; `PolylineF` con `strokeColor: FIJO.ruta (#16A34A)` y `strokeWeight: 4`. – Aviso flotante "Tocá en el mapa el punto de partida/destino" con botón de cierre, cuando `marcandoEnMapa`. – `<BotonCentrarMapa onClick={encuadrarMapa} />` con título dinámico. – Si `!mapsLoaded`: recuadro "Cargando mapa...". – **Modal "Resumen del viaje"** (aparece cuando `infoRuta` no es `null`): tiempo y distancia, aviso ámbar si la distancia es 'No disponible', texto sobre búsqueda

automática de conductores, grilla de 7 checkboxes de comodidades (HU55), y botón "Confirmar y Publicar Viaje" / "Procesando...".

C. Panel lateral derecho "Mis viajes" (`mostrarMisSolicitudes`) Cabecera con botón "Volver" (si hay `viajeSeleccionado`), título dinámico ("Mis viajes" o "Ofertas del viaje" con la ruta debajo) y botón de cierre.

Pestañas (solo cuando no hay `viajeSeleccionado`), definidas en un array `[{ id: 'activos', label: 'Buscando', icon: IconReloj, count: listaSolicitudes.length }, { id: 'confirmados', label: 'Confirmados', icon: IconVisto, count: viajesActivosConfirmados.length }, { id: 'completados', label: 'Completados', icon: IconBandera, count: viajesCompletadosOrdenados.length }]`, más el botón circular de refrescar.

Pestaña	Qué muestra
activos ("Buscando")	Vacío: círculo con <code>IconRadar</code> + "Sin viajes activos". Con datos: una tarjeta <code>.viaje-pasaje</code> por viaje pendiente con <code>TableroRuta</code> , contador de ofertas (o reloj si no hay), fecha (<code>toLocaleString('es-CO', ...)</code>) y número de asientos; aviso "Seguimos buscando conductores en tu zona." si pasaron ≥ 15 min sin ofertas; y "Cancelar búsqueda" con confirmación inline ("¿Cancelar?" / "No" / "Sí, cancelar"). Al hacer clic en la tarjeta se abre el detalle de ofertas.
confirmados	Vacío: círculo con <code>IconVisto</code> + "Sin viajes confirmados". Con datos: tarjetas de <code>viajesActivosConfirmados</code> .
completados	Vacío: círculo con <code>IconBandera</code> + "Aún no tienes viajes completados". Con datos: <code>viajesCompletadosOrdenados</code> .

Tarjeta de viaje confirmado/completado (compartida por las dos últimas pestañas): -

`cfgEstadoViaje` define color, fondo del badge, icono y etiqueta para `ASSIGNED` ("Confirmado"), `IN_PROGRESS` ("En camino", con punto azul pulsante) y `COMPLETED` ("Completado"); el fallback es `ASSIGNED`. - `TableroRuta`, fecha y asientos. - **Línea de vida** (IIFE): tres pasos (`ASSIGNED` → "Confirmado" / "El conductor aceptó tu viaje."; `IN_PROGRESS` → "En camino" / "Va hacia el punto de recogida."; `COMPLETED` → "Completado" / "Viaje finalizado."). El detalle solo se muestra en el paso actual; el riel vertical se colorea en los pasos completados. - Tarjeta del conductor (foto o `IconAuto`, nombre subrayado si hay `driver_id`, precio acordado); al hacer clic abre `/conductor/:driverId` en pestaña nueva. - Si `IN_PROGRESS` y hay `conductor_lat` / `conductor_lng` y `mapsLoaded`: mini `<GoogleMap>` de 160 px de alto, `gestureHandling`: 'none', con un `MarkerF` circular azul (`SymbolPath.CIRCLE`, `fillColor`: '#2563eb'). Si faltan coordenadas: "Esperando la ubicación del conductor...". - Botón "Registrar ocupantes del viaje" / "Ocupantes registrados — Actualizar" si el estado es `ASSIGNED` o `IN_PROGRESS`. - Si `COMPLETED`: botón "Calificar viaje" (deshabilitado y con texto "Ya calificaste este viaje" si `ya_califico`) y botón "Descargar recibo (PDF)".

Detalle de ofertas (cuando hay `viajeSeleccionado`): una IIFE busca el viaje actualizado en `listaSolicitudes`; si no existe o no tiene ofertas, muestra "Esperando ofertas de conductores...". Por cada oferta renderiza: foto/ `IconPersonas`, nombre (clic → perfil del conductor), `IconGorro` si verificado, calificación con `IconEstrella` y cantidad entre paréntesis o "Sin calificaciones aún", `{oferta.vehiculo}`, precio en grande; chips de comodidades (incluye "Buen ajuste para tu grupo" si `recomendado` y un chip con `categoria` y `capacidad`); aviso ámbar "No tiene: ..." si `comodidades.comodidades_faltantes` tiene elementos; línea "Oferta enviada" si hay `created_at`; y tres botones: **Aceptar**, **Contra ofertar** y una X de **rechazar**.

D. Panel lateral izquierdo de notificaciones (`mostrarNotificaciones`) Entra desde la izquierda (`x: '-100%'`). Cabecera con "Notificaciones", conteo de no leídas, botón "Leer todas" (solo si hay no leídas)

y cierre. Lista con vacío ("Sin notificaciones") y, por cada notificación, un `cfgTipo` que asigna icono y colores según `notif.type`: `NEW_OFFER` (`IconPrecio`), `COUNTER_OFFER` (`IconIntercambio`), `TRIP_ACCEPTED` (`IconVisto`), `TRIP_REJECTED` (`IconEquis`), `TRIP_STARTED` (`IconAuto`), `TRIP_COMPLETED` (`IconBandera`), `SYSTEM` (`IconCampana`, también el fallback). Clic en una no leída → `marcarLeida`.

Modales y cuándo aparecen

Modal	Condición	Contenido / acciones
Resumen del viaje	<code>infoRuta !== null</code>	Distancia/tiempo, comodidades HU55, botón "Confirmar y Publicar Viaje" → <code>crearViaje</code> .
ModalErrorDireccion	<code>errorDireccion !== null</code>	Texto no geocodificable; "Continuar con esta dirección" → <code>handleContinuarConDireccionManual</code> ; "Quiero corregir la dirección" → <code>setErrorDireccion(null)</code> .
Contraoferta (SCRUM-80)	<code>modalContraoferta !== null</code>	Input numérico con prefijo \$ y <code>autoFocus</code> ; "Cancelar" / "Enviar oferta" → <code>enviarContraoferta</code> .
FUEC / Registrar ocupantes (HU10)	<code>modalFuec !== null</code>	Backdrop position: fixed (zIndex: 9000) y tarjeta oscura (- -t-monte) de 500 px con cabecera fija, cuerpo con scroll (una caja por ocupante: nombre, select de tipo CC/TI/CE/PA, número, botón "Quitar" si hay más de uno), botón "+ Agregar otro ocupante" y pie fijo "Cancelar" / "Confirmar ocupantes".
Calificar viaje (HU46 / SCRUM-194)	<code>modalCalificar !== null</code>	Cinco botones de estrella (<code>IconEstrella size={30}</code>), textarea de comentario opcional, "Cancelar" / "Enviar calificación" (deshabilitado si <code>estrellasCalificar < 1</code>).
PerfilDrawer (HU16)	Siempre montado; <code>abierto={mostrarPerfil}</code>	Cajón de perfil del usuario.

Flujos completos, paso a paso

Publicar un viaje

1. Al montar, si el SDK de Google ya cargó, el efecto de geolocalización centra el mapa en la posición del usuario y rellena `busqueda.origen` con la dirección obtenida por geocodificación inversa (o `'Mi ubicación'`).
2. El usuario elige "Solo ida" o "Ida y vuelta" (`tipoViaje`).
3. Escribe origen y destino en los `InputDireccion` (con autocompletado de Google Places) • pulsa "Marcar en el mapa", lo que activa `marcandoEnMapa` y hace que el siguiente clic en el mapa dispare `alTocarMapa` (fija coordenadas + nombre legible y muestra un toast).
4. Elige fecha/hora de salida (y de regreso si es redondo) en `SelectorFechaHora`, confirmando con "Confirmar horario".
5. Abre "Quién" y ajusta adultos, niños y mascotas; mínimo 1 adulto, máximo 44 asientos entre adultos y niños.
6. Pulsa "Buscar ruta" → `buscarRuta valida`, geocodifica ambos extremos (reutilizando las coordenadas si el origen dice "mi ubicación") y llama `trazarRutaConCoords`, que dibuja la polilínea y llena `infoRuta`. – Si una dirección no se puede geocodificar, aparece `ModalErrorDireccion`; el usuario

puede continuar igualmente (`handleContinuarConDireccionManual`) y el viaje quedará con distancia/tiempo "No disponible".

7. Aparece el modal "Resumen del viaje". El usuario marca opcionalmente las comodidades que necesita.
8. Pulsa "Confirmar y Publicar Viaje" → `crearViaje` hace `POST /api/service-requests/` con direcciones, fechas, tipo de viaje, conteo de pasajeros, coordenadas, `distance_km`, `tipo_via` y las 7 banderas `requiere_*`.
9. Si el backend responde `conductores_notificados === 0`, se avisa que no hay conductores conectados; si no, "¡Viaje publicado exitosamente!". En ambos casos se limpian formulario, mapa, ruta y filtros.

Recibir y evaluar ofertas

1. El backend inserta una `Notification`; el canal `notif-dashboard-<user_id>` dispara `cargarNotificaciones()` y el badge de la campana sube. Si no hay Supabase, el polling de 15 s hace lo mismo.
2. Cualquier `UPDATE` en `ServiceRequest` del pasajero dispara `cargarMisViajes()`, que recarga `/pending` y, por cada viaje, `/request_id/offers`.
3. El usuario abre "Mis viajes"; en la pestaña "Buscando" cada tarjeta muestra el número de ofertas recibidas (o un reloj y, tras 15 minutos sin ofertas, el aviso "Seguimos buscando conductores en tu zona.>").
4. Al tocar la tarjeta se pasa al detalle (`viajeSeleccionado`), donde cada oferta muestra conductor, insignia de verificado, calificación real, precio, chips de comodidades, la marca "Buen ajuste para tu grupo" y, si aplica, qué comodidades le faltan (el filtro es informativo, no excluyente).
5. El usuario puede abrir el perfil público del conductor en una pestaña nueva (`/conductor/:driverId`).
6. La `X` roja rechaza la oferta, pero **solo en el cliente**: `handleRechazarOferta` no llama a ningún endpoint.

Contraofertar

1. En una oferta, el usuario pulsa "Contra ofertar" → `handleContraoferta` guarda `{ viajeId, ofertaId }` y abre el modal con el input vacío.
2. Escribe el precio; se valida que sea numérico y mayor que 0.
3. "Enviar oferta" → `PATCH /api/service-requests/offers/{ofertaId}/counter-offer` con `{ offered_price }`.
4. Éxito: toast con el monto formateado, se cierra el modal y se recargan los viajes (la oferta vuelve con el nuevo estado que devuelva el backend).

Aceptar una oferta

1. "Aceptar" → `PATCH /api/service-requests/offers/{ofertaId}/accept`.
2. Toast "¡Viaje aceptado! El conductor ha sido notificado.", se cierra el detalle (`setViajeSeleccionado(null)`) y se llama `cargarMisViajes()`.
3. El viaje deja de venir en `/pending` y aparece en `/assigned` con `trip_status: 'ASSIGNED'`, es decir, se mueve a la pestaña "Confirmados".

Registrar ocupantes (FUEC)

1. En un viaje `ASSIGNED` o `IN_PROGRESS`, el usuario pulsa "Registrar ocupantes del viaje". El `onClick` **resetea** `ocupantesFuec` a un ocupante vacío y fija `modalFuec = viaje.id`.
2. En el modal añade tantos ocupantes como necesite (" + Agregar otro ocupante"), completando nombre, tipo de documento (`CC`, `TI`, `CE`, `PA`) y número.
3. "Confirmar ocupantes" → validación local de campos no vacíos → `POST /api/service-requests/{request_id}/passengers` con `{ passengers: [...] }`.
4. Éxito: `fuecEnviado[request_id] = true`, se cierra el modal y el botón pasa a "Ocupantes registrados — Actualizar".

"Ver el FUEC"

En este archivo no existe un flujo de visualización o descarga del FUEC. El pasajero solo *registra* los ocupantes (`/passengers`), y el texto del modal dice que "el conductor ya puede ver la lista". El único PDF descargable desde el dashboard es el **recibo** del viaje (`/receipt`), disponible únicamente cuando `trip_status === 'COMPLETED'`, mediante `descargarRecibo` (fetch autenticado → blob → `<a download>` sintético). Si existe una vista del FUEC, vive en otro componente (presumiblemente `PanelConductor`), fuera del alcance de este archivo.

Seguir al conductor en el mapa (HU43)

1. Cuando el conductor inicia el viaje, `trip_status` pasa a `IN_PROGRESS` (llega vía Realtime en `ServiceRequest` → `cargarMisViajes()`).
2. El efecto de tracking detecta `driver_id` de los viajes `IN_PROGRESS` y abre un canal `tracking-conductor-<driverId>` por cada uno.
3. Cada `UPDATE` de la fila `User` de ese conductor trae `current_lat` / `current_lng` en `payload.new`; se actualizan solo `conductor_lat` y `conductor_lng` de esa tarjeta.
4. La tarjeta muestra un mini mapa (no interactivo) con un punto azul centrado en el conductor. Mientras no llegan coordenadas se ve "Esperando la ubicación del conductor...".
5. Al cambiar el conjunto de conductores en curso, la limpieza del efecto desuscribe los canales anteriores.

Calificar (HU46)

1. Con el viaje en `COMPLETED`, aparece el botón "Calificar viaje" (o "Ya calificaste este viaje", deshabilitado, si `ya_califico`).
2. El `onClick` resetea estrellas y comentario y abre el modal.
3. El usuario elige de 1 a 5 estrellas y opcionalmente escribe un comentario.
4. "Enviar calificación" → `POST /api/service-requests/{request_id}/rating` con `{ score, comment: comentario || null }`.
5. Éxito: el viaje se marca `ya_califico: true` en memoria, se cierra el modal y se muestra "Gracias por tu calificación".

Cancelar

- **Cancelar la búsqueda (viaje pendiente):** en la pestaña "Buscando", "Cancelar búsqueda" → confirmación inline "¿Cancelar?" con "No" / "Sí, cancelar" →

PATCH `/api/service-requests/{viajeId}/cancel` → el viaje se elimina de `listaSolicitudes` sin esperar al siguiente refresco y se muestra "Búsqueda cancelada".

- **No hay** ninguna acción para cancelar un viaje ya confirmado (`ASSIGNED` / `IN_PROGRESS`) desde este componente.

InputDireccion.jsx

Props

Prop	Valor por defecto	Uso
<code>name</code>	—	Atributo name del input; se usa al sintetizar eventos <code>onChange({ target: { name, value } })</code> .
<code>placeholder</code>	—	Texto de marcador.
<code>value</code>	—	Valor controlado (viene de <code>busqueda.origen</code> / <code>busqueda.destino</code>).
<code>onChange</code>	—	Recibe eventos reales del input o sintéticos con la misma forma.
<code>esOrigen</code>	<code>false</code>	Si es <code>true</code> , muestra el botón de "Usar mi ubicación actual".
<code>onUbicacionActual</code>	—	Callback (<code>{ texto, coords }</code>) con <code>coords</code> como array [<code>latitude</code> , <code>longitude</code>].
<code>ancho</code>	<code>'110px'</code>	Ancho del <code><input></code> (el Dashboard pasa <code>"260px"</code>).
<code>mapsLoaded</code>	<code>false</code>	Indica si el SDK de Google ya cargó; el componente no carga su propio script.

Estados y refs

Nombre	Inicial	Representa
<code>sugerencias</code>	<code>[]</code>	Predicciones de <code>AutocompleteService</code> .
<code>mostrar</code>	<code>false</code>	Visibilidad del dropdown.
<code>cargando</code>	<code>false</code>	Muestra el spinner <code>IconGirar</code> mientras se consulta.
<code>sinResultados</code>	<code>false</code>	Cambia el dropdown al panel "No encontramos esa dirección".
<code>cargandoUbicacion</code>	<code>false</code>	Deshabilita el botón de geolocalización mientras resuelve.
<code>debounceRef</code>	<code>null</code>	<code>setTimeout</code> del debounce (350 ms).
<code>wrapperRef</code>	<code>null</code>	Contenedor, para detectar clics fuera.
<code>autocompleteServiceRef</code>	<code>null</code>	Instancia perezosa de <code>google.maps.places.AutocompleteService</code> .
<code>geocoderRef</code>	<code>null</code>	Instancia perezosa de <code>google.maps.Geocoder</code> .
<code>sessionTokenRef</code>	<code>null</code>	<code>AutocompleteSessionToken</code> para agrupar la facturación de Places; se resetea a <code>null</code> al elegir una sugerencia (nueva sesión).

Constantes y funciones

- `IconIdea` e `IconGirar`: iconos locales contruidos sobre `Icono`. `IconIdea` se declara **antes** del `import` que trae `Icono` (línea 2 vs. línea 3).
- `BRAND_GREEN = 'var(--t-ruta)'`.
- `BOUNDS_ANTIOQUIA = { south: 5.4, west: -77.2, north: 8.9, east: -73.9 }` — duplicado del valor de `Dashboard.jsx`.
- `boundsAntioquia()` — construye un `LatLngBounds` con esos límites.
- **`buscarSugerencias(texto)`**: limpia el timeout previo y `sinResultados`; si el texto tiene menos de 3 caracteres o el SDK no está listo, oculta el dropdown; si no, a los 350 ms crea (si hace falta) el servicio y el session token y llama `getPlacePredictions({ input, componentRestrictions: { country: 'co' }, locationBias: boundsAntioquia(), sessionToken })`. Con `PlacesServiceStatus.OK` y predicciones, las muestra; en cualquier otro caso muestra el panel de "sin resultados" y, si el status no es `ZERO_RESULTS`, deja un `console.warn`.
- **`handleChange(e)`**: propaga `onChange(e)` y dispara `buscarSugerencias(e.target.value)`.
- **`seleccionarSugerencia(prediction)`**: emite `onChange({ target: { name, value: prediction.description } })`, limpia el dropdown y anula el session token.
- **`usarDireccionManual()`**: solo cierra el dropdown (deja el texto tal cual lo escribió el usuario).
- **`usarUbicacionActual()`**: si no hay `navigator.geolocation`, `alert('Tu navegador no soporta geolocalización.')`. Si la hay, pide la posición con `{ enableHighAccuracy: true, timeout: 8000 }`; con el SDK disponible geocodifica inversamente y usa `formatted_address` (o 'Mi ubicación'), emite el `onChange` sintético y llama `onUbicacionActual({ texto, coords: [latitude, longitude] })`. En error: `alert('No pudimos acceder a tu ubicación. Verifica los permisos del navegador.')`.

Efectos y render

- Un único `useEffect` con `[]` registra `mousedown` en `document` para cerrar el dropdown al hacer clic fuera del `wrapperRef`; limpia el listener al desmontar.
- Render: un `<input>` sin borde (estilo "campo embebido"), el spinner mientras carga, y el botón de ubicación (solo si `esOrigen` && `!cargando`) con un SVG de diana. El dropdown (`position: absolute, zIndex: 3000`) muestra las predicciones con `structured_formatting.main_text / secondary_text` (usando `onMouseDown` en lugar de `onClick` para adelantarse al blur), o bien el panel "No encontramos esa dirección" con la opción Usar "{value}" de todas formas y la lista de consejos.

SelectorFechaHora.jsx

Constantes y ayudantes de módulo

Nombre	Valor / función
<code>BRAND_GREEN</code>	<code>'var(--t-ruta)'</code> .
<code>DIAS_SEMANA</code>	<code>['L', 'M', 'X', 'J', 'V', 'S', 'D']</code> .
<code>MESES</code>	Los 12 nombres en español.

Nombre	Valor / función
MINUTOS_DISPONIBLES	[0, 15, 30, 45].
aPartesLocal(valor)	Convierte "YYYY-MM-DDTHH:mm" a { anio, mes (0-based), dia, hora, minuto }; null si no hay valor.
aValor(anio, mes, dia, hora, minuto)	Vuelve al formato "YYYY-MM-DDTHH:mm" con relleno de ceros.
formatearBonito(valor)	Texto visible, p. ej. "Lun 5 Sep · 08:30" (usa toLocaleDateString('es-CO', { weekday: 'short' })).
mismodDia(a, b)	Compara año/mes/día (nombre con errata: mismodDia).
redondearAMinutosValidos(p)	Redondea hacia arriba al siguiente múltiplo de 15 min, con % 24 en la hora.

Props

Prop	Por defecto	Uso
label	—	Etiqueta pequeña sobre el valor ("Salida", "Regreso").
value	—	Valor controlado en formato "YYYY-MM-DDTHH:mm".
onChange	—	Se llama solo al confirmar, con la cadena resultante (no con un evento).
min	—	Límite inferior; si no se pasa, se usa el momento actual.
placeholder	'Elige fecha y hora'	Texto cuando no hay valor.
required	false	Se aplica a un <input> oculto para que la validación nativa del formulario funcione.
ancho	—	Ancho del contenedor (por defecto '100%').

Estados, refs y valores derivados

Nombre	Inicial	Representa
abierto	false	Visibilidad del popover del calendario.
contenedorRef	null	Para cerrar al hacer clic fuera.
mesVisible	valorInicial().mes	Mes mostrado en la grilla.
anioVisible	valorInicial().anio	Año mostrado.
borrador	valorInicial()	Selección provisional (día + hora + minuto) que aún no se confirmó.

Derivados: `minPartes` (de `min` o de `new Date()`), `valorInicial()` (partes de `value`, o `minPartes` redondeado a 15 min), `primerDiaMes`, `offsetInicio` (`(getDay() + 6) % 7` para empezar en lunes), `diasEnMes`, `celdas` (huecos `null` + días), `fechaMinima`, `esDiaDeshabilitado(dia)`, `esMesMinimo`, `horaMinimaHoy` (minutos desde medianoche, solo si el borrador cae en el día mínimo), `bonito` (texto formateado del `value` confirmado).

Funciones y efectos

- **cambiarMes(delta)** : avanza o retrocede, ajustando año; no permite ir antes del mes mínimo.
- **elegirDia(dia)** : ignora días deshabilitados; si el día elegido es el mínimo y la hora del borrador quedó por debajo de la hora mínima, sube la hora al mínimo y la redondea a un múltiplo de 15.
- **confirmar()** : llama `onChange(aValor(...))` con el borrador y cierra el popover.
- `useEffect` [abierto] (1): al abrir, reinicia `borrador`, `mesVisible` y `anioVisible` desde `valorInicial()`. Lleva `// eslint-disable-next-line react-hooks/exhaustive-deps`.
- `useEffect` [abierto] (2): mientras está abierto, escucha `mousedown` para cerrar al hacer clic fuera; limpia el listener.

Render

Campo cerrado: caja con la etiqueta, un icono de calendario en SVG y bonito `||` placeholder; dentro va un `<input type="text">` oculto (opacidad 0, tamaño 0, `readOnly`, `tabIndex={-1}`) con el `required` para el submit del formulario. Popover (`AnimatePresence + motion.div`, ancho 290 px, `zIndex: 2000`): cabecera con `<` (deshabilitada en el mes mínimo), `MESES[mesVisible]` `anioVisible` y `>`; fila de días de la semana; grilla de 7 columnas con los días (el seleccionado en verde); selector de hora con dos `<select>` (horas 0–23 filtradas por `h * 60 + 45 >= horaMinimaHoy`, minutos de `MINUTOS_DISPONIBLES` filtrados por `borrador.hora * 60 + m >= horaMinimaHoy`); y los botones "Cancelar" y "Confirmar horario".

PARTE 5

Las pantallas del conductor

El panel de cinco pestañas, el cajón de perfil y el formulario de registro.

Contexto común a los tres archivos:

- `API_BASE_URL` viene de `./api` y se define como `import.meta.env.VITE_API_URL || 'http://127.0.0.1:8000'`.
- Todas las peticiones envían la cabecera `'ngrok-skip-browser-warning': 'true'` además de `Authorization: Bearer <token>`.
- El sistema de diseño vive en `./diseno` (objeto `T` con tokens, iconos `Icon*`, `Icono`, `TableroRuta`, `Boton`, `Rotulo`, `EstilosBase`, `LogoWordmark`, `BotonTema`, `useTema`, `MAPA_OSCURO`, `FIJO`, `BotonCentrarMapa`).

PanelConductor.jsx

Archivo de 1517 líneas. Es la pantalla completa del conductor: panel lateral con cinco pestañas + mapa de Google a la derecha, más tres modales/paneles flotantes (notificaciones, ocupantes FUEC, calificar pasajero) y el `PerfilDrawer` incrustado.

Importaciones y constantes de módulo

Importaciones

Import	Origen	Uso
<code>useState, useEffect, useContext, useRef, useCallback</code>	<code>react</code>	Hooks del componente.
<code>T, TableroRuta, Icono, Rotulo</code>	<code>./diseno</code>	<code>T</code> = tokens de tema (colores, radios, familias). <code>TableroRuta</code> pinta el par <code>origen→destino</code> . <code>Icono</code> es el envoltorio <code><svg></code> base para definir iconos locales. <code>Rotulo</code> no se usa (ver Observaciones).

Import	Origen	Uso
IconReloj, IconVisto, IconEquis, IconBandera, IconAuto, IconCalendario, IconPersonas, IconPersona, IconRadar, IconPin, IconEstrella, IconClipboard, IconAlerta, IconCampana, IconPrecio, IconIntercambio, IconRecibo	./diseno	Iconografía de estados, pestañas y tarjetas. IconRecibo no se usa .
LogoWordmark, BotonTema, useTema, MAPA_OSCURO, FIJO, BotonCentrarMapa	./diseno	Marca en la cabecera oscura, interruptor de tema, hook de tema, estilo de mapa nocturno, paleta fija (para el marcador, que no puede usar variables CSS) y botón flotante de centrado.
motion, AnimatePresence	framer-motion	Animaciones de tarjetas, modales y paneles deslizantes.
GoogleMap, MarkerF, useJsApiLoader	@react-google-maps/api	Mapa y marcadores.
AuthContext	./AuthContext	Provee token y usuario.
ToastContainer, useToast	./Toast	Sistema de avisos. useToast() devuelve { toasts, removeToast, toast } con toast.success/error/warning/info.
SkeletonTarjetaViaje	./Skeleton	Esqueleto de carga del radar.
PerfilDrawer	./ PerfilDrawer	Drawer lateral de perfil (documentado en la sección 2).
API_BASE_URL	./api	Base de todas las URLs.

Iconos definidos en el propio archivo

Todos son componentes de una línea que reciben las props `p` y las pasan a `<Icono {...p}>`:

Nombre	Dibujo / propósito
IconGirar	Flechas circulares — botones "Actualizar" / "Verificar de nuevo" / "Reintentar".
IconFiltros	Tres líneas decrecientes — botón "Filtros" del radar.
IconMascota	Huella (cuatro elipses + almohadilla) — filtro y chips de mascotas.
IconGrafico	Ejes con barras — pestaña "Ganancias".
IconMapa	Mapa plegado — aviso "Ruta trazada en el mapa".

Constantes de módulo

Constante	Valor	Para qué sirve
BRAND_GREEN	'var(--t-ruta)'	Verde de marca usado en estilos en línea.
GOOGLE_MAPS_LIBRARIES	['places', 'geometry']	Debe coincidir con las de Dashboard.jsx para que useJsApiLoader reutilice el script ya inyectado (el comentario del código lo explica: evita el warning "google maps already loaded").

Constante	Valor	Para qué sirve
GOOGLE_MAPS_API_KEY	import.meta.env.VITE_GOOGLE_MAPS_API_KEY	Clave del API.
mapContainerStyle	{ width: '100%', height: '100%' }	Contenedor del GoogleMap.
centroDefaultAntioquia	{ lat: 6.2442, lng: -75.5812 }	Medellín; centro por defecto cuando aún no hay ubicación del conductor.

Componentes auxiliares definidos en el archivo

No hay componentes auxiliares con lógica: solo los cinco iconos de la tabla anterior (`IconGirar`, `IconFiltros`, `IconMascota`, `IconGrafico`, `IconMapa`), cuya única prop es el spread `p` (típicamente `size`, `color`, `grosor`, `style`).

El componente principal `PanelConductor`

Firma: `const PanelConductor = ({ onVerRuta }) => { ... }`.

- `onVerRuta` es una función opcional del padre; se llama como `onVerRuta(sol.origin, sol.destination)` para que el contenedor trace la ruta.

Contextos y hooks propios

Hook	Qué devuelve	Uso
<code>useContext(AuthContext)</code>	{ token, usuario }	token para todas las peticiones; <code>usuario.full_name</code> para el saludo y <code>usuario.user_id</code> para el canal de Realtime.
<code>useToast()</code>	{ toasts, removeToast, toast }	<code>toast.success/error/warning</code> en las acciones; <code>toasts</code> y <code>removeToast</code> alimentan <code><ToastContainer></code> .
<code>useTema()</code>	[tema, alternarTema]	tema vale 'claro' u 'oscuro'; controla <code>MAPA_OSCURO</code> en las opciones del mapa y alimenta <code><BotonTema></code> .
<code>useJsApiLoader({ id: 'google-map-script', googleMapsApiKey: GOOGLE_MAPS_API_KEY, libraries: GOOGLE_MAPS_LIBRARIES })</code>	{ isLoading: mapsLoaded }	Gate para renderizar el mapa o el texto "Cargando mapa..."

`useRef`

Ref	Valor inicial	Para qué
<code>mapaRef</code>	<code>useRef(null)</code>	Se asigna en <code>onLoad={mapa => { mapaRef.current = mapa; }}</code> . Permite <code>panTo/setZoom</code> sin forzar la prop <code>center</code> en cada render (el comentario indica que forzar <code>center</code> impediría que el conductor mueva el mapa a mano).

Todos los `useState`, agrupados por tema

Navegación y presentación general

Variable	Setter	Valor inicial	Representa
pestanasActiva	setPestanasActiva	'radar'	Pestaña visible: 'radar', 'activos', 'historial', 'ganancias', 'vehiculo'.
mostrarPerfil	setMostrarPerfil	false	Abre/cierra el PerfilDrawer (HU37: desde ahí el conductor sube su RUNT).
mostrarNotifPanel	setMostrarNotifPanel	false	Abre/cierra el panel lateral de notificaciones.

Radar (solicitudes pendientes)

Variable	Setter	Valor inicial	Representa
solicitudes	setSolicitudes	[]	Solicitudes pendientes devueltas por el backend.
cargando	setCargando	false	Carga "visible" del radar (muestra esqueletos).
error	setError	null	Mensaje de error del radar.
filtros	setFiltros	{ tipo: 'todos', pasajeros: 'todos', mascotas: false }	Filtros del cliente sobre solicitudes.
mostrarFiltros	setMostrarFiltros	false	Despliega/pliega el panel de filtros.
tarjetaRutaId	setTarjetaRutaId	null	request_id de la tarjeta seleccionada (la que tiene la ruta trazada).

Oferta de precio (modal)

Variable	Setter	Valor inicial	Representa
solicitudModal	setSolicitudModal	null	Objeto de la solicitud sobre la que se está ofertando.
precio	setPrecio	''	Precio escrito por el conductor.
errorPrecio	setErrorPrecio	''	Error de validación o del backend al ofertar.
enviandoOferta	setEnviandoOferta	false	Bloquea el botón "Enviar".
alertaExito	setAlertaExito	false	Banner "Oferta enviada" (se apaga solo a los 3500 ms).

Ofertas / viajes activos

Variable	Setter	Valor inicial	Representa
viajesActivos	setViajesActivos	[]	Ofertas del conductor con su estado (status) y el estado real del viaje (trip_status).
resolviendoOferta	setResolviendoOferta	null	Clave offer_id + action en curso ('...ACCEPT' / '...REJECT').
gestionandoViaje	setGestionandoViaje	null	Clave request_id + accion en curso ('...start' / '...complete').
ocupantesPorViaje	setOcupantesPorViaje	{}	Diccionario { [request_id]: ocupantes[] }.

Variable	Setter	Valor inicial	Representa
modalOcupantesId	setModalOcupantesId	null	request_id cuyo modal FUEC de ocupantes está abierto.

Notificaciones

Variable	Setter	Valor inicial	Representa
notificaciones	setNotificaciones	[]	Lista de notificaciones (notification_id, type, title, message, is_read, created_at).

Ubicación / geolocalización

Variable	Setter	Valor inicial	Representa
ubicacionActual	setUbicacionActual	null	{ lat, lng } del conductor.
sincronizado	setSincronizado	false	Marca que ya terminó el primer intento de reportar la ubicación (con permiso o sin él). Hasta que no es true no se carga el radar.

Calificaciones (HU46)

Variable	Setter	Valor inicial	Representa
modalCalificar	setModalCalificar	null	request_id del viaje que se está calificando.
estrellasCalificar	setEstrellasCalificar	0	Puntaje 1–5 elegido.
comentarioCalificar	setComentarioCalificar	''	Comentario opcional.
enviandoCalificacion	setEnviandoCalificacion	false	Bloquea el envío.

Historial

Variable	Setter	Valor inicial	Representa
historial	setHistorial	[]	Viajes históricos del conductor.
cargandoHistorial	setCargandoHistorial	false	Estado de carga.
errorHistorial	setErrorHistorial	null	Mensaje de error.

Ganancias (HU52 / SCRUM-204)

Variable	Setter	Valor inicial	Representa
ganancias	setGanancias	null	Objeto con ganancias_semana, ganancias_mes, viajes_completados, calificacion_promedio, horarios_activos, top_rutas.
cargandoGanancias	setCargandoGanancias	false	Estado de carga.
errorGanancias	setErrorGanancias	null	Mensaje de error (diferencia 401, 403 y fallo de red).

Vehículo (HU55 / SCRUM-207)

Variable	Setter	Valor inicial	Representa
vehiculo	setVehiculo	null	Copia "servidor" del vehículo.
formVehiculo	setFormVehiculo	null	Copia editable que alimenta el formulario.
cargandoVehiculo	setCargandoVehiculo	false	Estado de carga.
errorVehiculo	setErrorVehiculo	null	Mensaje de error.
guardandoVehiculo	setGuardandoVehiculo	false	Bloquea "Guardar cambios".

Todas las funciones definidas dentro del componente**CENTRARENMIUBICACION — USECALLBACK(() => {...}, [UBICACIONACTUAL])**

1. Si no hay `mapaRef.current`, sale.
2. Si ya hay `ubicacionActual`, hace `mapaRef.current.panTo(ubicacionActual)` y `setZoom(12)` y termina.
3. Si no, y si existe `navigator.geolocation`, pide `getCurrentPosition`: construye `punto = { lat, lng }`, hace `setUbicacionActual(punto)` y centra el mapa con zoom 12.

Endpoint: ninguno. Estado: `ubicacionActual`.

ENVIARCALIFICACION — ASYNC () => {}

1. Si `estrellasCalificar < 1`, muestra `toast.warning('Elige de 1 a 5 estrellas.')` y sale.
2. `setEnviandoCalificacion(true)`.
3. **POST ``${API_BASE_URL}/api/service-requests/${modalCalificar}/rating`** con cuerpo `{ score: estrellasCalificar, comment: comentarioCalificar || null }`.
4. Si la respuesta no es ok, lanza `new Error(err.detail)`.
5. Marca localmente el viaje como calificado: `setViajesActivos(prev => prev.map(v => v.request_id === modalCalificar ? { ...v, ya_califico: true } : v))`.
6. Cierra el modal (`setModalCalificar(null)`), limpia `estrellasCalificar` y `comentarioCalificar`, y muestra `toast.success('Gracias por calificar a tu pasajero.')`.
7. En error: `toast.error(\`Error: ${e.message}\`)`. En finally : `setEnviandoCalificacion(false)`.

CARGARNOTIFICACIONES — ASYNC () => {}

- **GET ``${API_BASE_URL}/api/service-requests/notifications`**.
- Si `res.ok`, `setNotificaciones(await res.json())`. El `catch {}` es vacío (falla en silencio).

MARCARLEIDA(NOTIFID) — ASYNC

- **PATCH ``${API_BASE_URL}/api/service-requests/notifications/${notifId}/read`**.
- Actualiza el estado local: `setNotificaciones(prev => prev.map(n => n.notification_id === notifId ? { ...n, is_read: true } : n))`. `catch {}` vacío. No comprueba `res.ok`.

MARCARTODASLEIDAS()

- Recorre `notificaciones.filter(n => !n.is_read)` y llama a `marcarLeida(n.notification_id)` por cada una (una petición por notificación).

ACTUALIZARUBICACIONBACKEND(PAYLOAD) — ASYNC

- **PATCH ``${API_BASE_URL}/drivers/location``** con `body: JSON.stringify(payload)` (HU26).
- Devuelve el JSON si `res.ok`; si no, devuelve `null`. `catch {}` vacío.
- El comentario dice que envía `current_lat/current_lng/is_online`, pero en la práctica todos los llamadores solo pasan `{ current_lat, current_lng }`.

REPORTARUBICACION — USECALLBACK(() => {...}, [TOKEN])

1. Si no hay `navigator.geolocation`: `setSincronizado(true)` y sale.
2. `getCurrentPosition` con `{ enableHighAccuracy: true }`: - éxito: `setUbicacionActual({ lat, lng })`,
`await actualizarUbicacionBackend({ current_lat: latitude, current_lng: longitude })`,
`setSincronizado(true)`. - error/denegado: `setSincronizado(true)` (el radar igual muestra los viajes).

CARGARSOLICITUDES(SILENCIOSO = FALSE) — ASYNC

- Si `!silencioso`: `setCargando(true)` y `setError(null)`.
- **GET ``${API_BASE_URL}/api/service-requests/pending``**. El comentario aclara que el radio de búsqueda lo resuelve el backend (HU26) y que este ya ordena por cercanía.
- Éxito: `setSolicitudes(data)`.
- Error: si `!silencioso`, `setError('No se pudieron cargar las solicitudes.')`.
- `finally`: si `!silencioso`, `setCargando(false)`.
- El parámetro `silencioso` existe para que el refresco automático de fondo no oculte la lista ni muestre esqueletos.

CARGAROCUPANTES(REQUESTID) — ASYNC

- **GET ``${API_BASE_URL}/api/service-requests/${requestId}/passengers``** (SCRUM-83 / HU10).
- Éxito: `setOcupantesPorViaje(prev => ({ ...prev, [requestId]: data })).catch {}` vacío.

CARGARVIAJESACTIVOS() — ASYNC

- **GET ``${API_BASE_URL}/api/service-requests/driver/my-offers``**.
- Éxito: `setViajesActivos(data)`. Error: `console.error('Error cargando ofertas activas:', e)`.

CARGARHISTORIAL() — ASYNC

1. `setCargandoHistorial(true)`, `setErrorHistorial(null)`.
2. **GET ``${API_BASE_URL}/api/service-requests/driver/history``**.
3. Si no es `ok`, intenta leer `detail` del cuerpo y lanza `new Error(detalle || \ El servidor respondió $ {res.status}.')`.
4. Éxito: `setHistorial(await res.json())`.
5. Error: `setErrorHistorial(e instanceof TypeError ? 'No pudimos conectarnos al servidor.' : e.message || 'No pudimos cargar tu historial.')`.
6. `finally`: `setCargandoHistorial(false)`.

CARGARGANANCIAS() — ASYNC

1. `setCargandoGanancias(true)`, `setErrorGanancias(null)`.
2. **GET ``${API_BASE_URL}/drivers/earnings``**.

3. Si no es ok: extrae detail; mapea 401 → 'Tu sesión expiró. Volvé a entrar.', 403 → detalle || 'Esta sección es solo para conductores.', resto → detalle || \ El servidor respondió \$ {res.status}.\`.
4. Éxito: setGanancias(await res.json()).
5. Error: si e instanceof TypeError (sin red) → mensaje de conexión; si no, e.message. Además console.error('[ganancias]', e).
6. finally: setCargandoGanancias(false).

CARGARVEHICULO() — ASYNC

1. setCargandoVehiculo(true), setErrorVehiculo(null).
2. GET \${API_BASE_URL}/drivers/vehicle.
3. Si no es ok, lanza new Error(err.detail || 'No pudimos cargar tu vehículo.').
4. Éxito: setVehiculo(data) y setFormVehiculo(data) (misma referencia).
5. Error: setErrorVehiculo(e.message). finally: setCargandoVehiculo(false).

GUARDARVEHICULO() — ASYNC

1. setGuardandoVehiculo(true).
2. Arma payload con exactamente estas claves: capacidad_real (null si es '' o null, si no Number(...)), tiene_ac, tiene_wifi, tiene_bano, tiene_musica, tiene_maletero_amplio, tiene_sillas_bebe, acepta_mascotas, cargo_mascota (null si es '', si no Number(...)), acepta_menores_2_anos. Un comentario aclara que las tarifas (por km, espera, día) quedan fuera a propósito: ya no las decide el conductor.
3. PATCH \${API_BASE_URL}/drivers/vehicle con ese cuerpo.
4. Si no es ok, lanza new Error(err.detail || 'No se pudo guardar. Verifica los valores.').
5. Éxito: setVehiculo(data), setFormVehiculo(data), toast.success('Cambios guardados en tu vehículo.').
6. Error: toast.error(e.message). finally: setGuardandoVehiculo(false).

GESTIONARVIAJE(REQUESTID, ACCION) — ASYNC (HU17)

1. setGestionandoViaje(requestId + accion).
2. PATCH \${API_BASE_URL}/api/service-requests/\${requestId}/\${accion} — accion vale 'start' o 'complete' según el botón. No lleva cuerpo.
3. Si no es ok, lanza new Error(err.detail).
4. Éxito: toast.success(data.message) y cargarViajesActivos().
5. Error: toast.error(\ Error: \${e.message}). finally : setGestionandoViaje(null)\`.

RESOLVERCONTRAOFERTA(OFFERID, ACTION) — ASYNC (SCRUM-82 / SCRUM-84)

1. setResolviendoOferta(offerId + action).
2. PATCH \${API_BASE_URL}/api/service-requests/offers/\${offerId}/resolve con cuerpo { action } ('ACCEPT' o 'REJECT').
3. Si no es ok, lanza new Error(err.detail).
4. Éxito: toast.success(data.message) y cargarViajesActivos().
5. Error: toast.error(...). finally: setResolviendoOferta(null).

VALIDARPRECIO(VAL) → BOOLEAN

- `'' / falsy` → `setErrorPrecio('El precio no puede estar vacío.')`, devuelve `false`.
- `isNaN(val) || Number(val) <= 0` → `setErrorPrecio('El precio debe ser mayor a 0.')`, devuelve `false`.
- En caso válido, limpia `errorPrecio` y devuelve `true`.

ENVIAROFERTA() — ASYNC (SCRUM-76)

1. Si `!validarPrecio(precio)`, sale.
2. `setEnviandoOferta(true)`.
3. **POST** `${API_BASE_URL}/api/service-requests/${solicitudModal.request_id}/offers` con cuerpo `{ offered_price: Number(precio) }`.
4. Si no es `ok`, lanza `new Error(err.detail || 'Error al enviar oferta')`.
5. Éxito: inserta de forma optimista en `viajesActivos` un objeto `{ request_id, origin, destination, departure_time, offered_price, estado: 'ESPERANDO' }`; cierra el modal (`setSolicitudModal(null)`), limpia `precio`, activa `alertaExito` y programa `setTimeout(() => setAlertaExito(false), 3500)`; llama a `cargarSolicitudes()` (versión visible).
6. Error: `setErrorPrecio(err.message)`. `finally: setEnviandoOferta(false)`.

HANDLECLICKTARJETA(SOL) (SCRUM-77)

- Alterna la selección: si `tarjetaRutaId === sol.request_id`, la `deselecciona (null)`; si no, la `selecciona` y, si existe `onVerRuta`, llama `onVerRuta(sol.origin, sol.destination)`.

FORMATEARFECHA(F)

- `new Date(f).toLocaleString('es-CO', { day: '2-digit', month: 'short', year: 'numeric', hour: '2-digit', minute: '2-digit' })`; si falla, devuelve `f` tal cual.

ESTADOCONFIG(ESTADO) — NO SE INVOKA EN NINGÚN LUGAR

- Mapa `ESPERANDO` / `CONTRAOFERTA` / `RECHAZADO` / `ACEPTADO` a `{ bg, color, Ico, label }`, con `fallback { bg: T.niebla2, color: T.piedra, Ico: IconReloj, label: estado }`. Ver Observaciones.

Valores derivados (no son estado)

Nombre	Cálculo
<code>viajeEnCurso</code>	<code>viajesActivos.some(v => v.trip_status === 'IN_PROGRESS')</code> .
<code>solicitudesFiltradas</code>	Filtra solicitudes por <code>filtros.tipo</code> (compara con <code>sol.trip_type</code>), por total de pasajeros (<code>sol.adults_count 1</code>) + (<code>sol.children_count 0</code>) con los tramos '1', '2-4', '5+', y por <code>filtros.mascotas && !sol.has_pets</code> .
<code>filtrosActivos</code>	<code>filtros.tipo !== 'todos' filtros.pasajeros !== 'todos' filtros.mascotas</code> .
<code>solicitudesConUbicacion</code>	<code>solicitudes.filter(sol => sol.origin_lat !== null && sol.origin_lng !== null)</code> (HUO6, pines del mapa). Nota: parte de solicitudes, no de <code>solicitudesFiltradas</code> .

Todos los `useEffect`

#	Dependencias	Qué hace	Limpieza
1	[token, usuario]	Si falta token o usuario, sale. Llama a <code>cargarNotificaciones()</code> y luego, si existen <code>VITE_SUPABASE_URL</code> y <code>VITE_SUPABASE_ANON_KEY</code> , importa dinámicamente <code>@supabase/supabase-js</code> y suscribe el canal de Realtime (detalle abajo). Si no hay credenciales, cae a <i>polling</i> con <code>setInterval(cargarNotificaciones, 15000)</code> .	La rama de polling devuelve <code>() => clearInterval(intervalo)</code> . La rama de Supabase devuelve <code>() => supabase.removeChannel(channel)</code> dentro del <code>.then()</code> , no desde el <code>useEffect</code> , por lo que React nunca la ejecuta (ver Observaciones).
2	[token, reportarUbicacion]	<code>if (token) reportarUbicacion();</code> — primer reporte de posición al abrir el panel.	Ninguna.
3	[token]	Si no hay <code>navigator.geolocation</code> , sale. Cada 20 000 ms pide la posición, hace <code>setUbicacionActual</code> y <code>actualizarUbicacionBackend({ current_lat, current_lng })</code> . Mantiene el orden por cercanía al día.	<code>clearInterval(intervalo)</code> .
4	[token, sincronizado]	<code>if (token && sincronizado) cargarSolitudes();</code> — carga visible del radar. El comentario explica que se espera a <code>sincronizado</code> para evitar un bug visual en el que el radar mostraba todas las solicitudes un instante y luego se vaciaba.	Ninguna.
5	[token]	<code>setInterval(() => cargarSolitudes(true), 15000)</code> — refresco silencioso del radar (antes eran 3 s).	<code>clearInterval(intervalo)</code> .
6	[token, pestanaActiva]	<code>if (token && pestanaActiva === 'activos') cargarViajesActivos();</code>	Ninguna.
7	[token, pestanaActiva]	<code>if (token && pestanaActiva === 'historial') cargarHistorial();</code>	Ninguna.
8	[token, pestanaActiva]	<code>if (token && pestanaActiva === 'ganancias') cargarGanancias();</code>	Ninguna.
9	[token, pestanaActiva]	<code>if (token && pestanaActiva === 'vehiculo') cargarVehiculo();</code>	Ninguna.
10	[token]	HU43 — sondeo de fondo independiente de la pestaña: <code>cargarViajesActivos()</code> inmediato + <code>setInterval(cargarViajesActivos, 15000)</code> , para saber si hay un viaje <code>IN_PROGRESS</code> .	<code>clearInterval(intervalo)</code> .
11	[viajeEnCurso, token]	HU43 — mientras <code>viajeEnCurso</code> sea verdadero y exista geolocalización, define <code>reportar()</code> (posición → <code>setUbicacionActual</code> + <code>actualizarUbicacionBackend</code>), lo ejecuta una vez y luego cada 3 000 ms, para que el pasajero vea al conductor moverse.	<code>clearInterval(intervalo)</code> .

Suscripción a Supabase Realtime

Aspecto	Valor
Cliente	<code>createClient(SUPABASE_URL, SUPABASE_ANON_KEY)</code> con <code>VITE_SUPABASE_URL / VITE_SUPABASE_ANON_KEY</code> , importado dinámicamente con <code>import('@supabase/supabase-js')</code> .
Canal	<code>`notif-conductor-\${usuario.user_id}`</code>
Evento	<code>.on('postgres_changes', { event: 'INSERT', schema: 'public', table: 'Notification', filter: `user_id=eq.\${usuario.user_id}` }, () => cargarNotificaciones())</code>
Tabla	Notification (esquema public)
Filtro	<code>user_id=eq.<usuario.user_id></code>
Efecto	Ante cada INSERT propio, vuelve a llamar a <code>cargarNotificaciones()</code> (recarga completa por HTTP; no usa el payload del evento).
Fallback	Sin credenciales de Supabase: <code>setInterval(cargarNotificaciones, 15000)</code> .

Estructura de la interfaz

Layout general

- Bloque `<style>` inyectado con clases `.fuec-input`, `.fuec-select` (estilos para campos sobre fondo oscuro) y reglas responsivas: por debajo de `1000px`, `.pc-raiz` pasa a `flex-direction: column`, `.pc-lateral` ocupa el 100 % y `.pc-mapa` toma `min-height: 340px`. El comentario explica el bug que corrige: con el panel de 420 px fijos el mapa quedaba con ancho cero en ventanas angostas.
- `.pc-raiz` es un flex con `<aside className="pc-lateral">` de 480px y `<main className="pc-mapa">` que toma el resto.

Cabecera del panel (fondo `var(--t-monte)`)

- `<LogoWordmark alto={13} oscuro />`.
- `<BotonTema tema={tema} alternar={alternarTema} compacto />`.
- Botón redondo con `IconPersona` → abre `PerfilDrawer` (`setMostrarPerfil(true)`), título "Mi perfil".
- Botón campana (`IconCampana`) → abre el panel de notificaciones; muestra una burbuja con `notificaciones.filter(n => !n.is_read).length` cuando hay alguna sin leer.
- Saludo: `Hola, {usuario?.full_name?.split(' ')[0] || 'Conductor'}`.
- Subtítulo condicional: con `ubicacionActual`, "Recibís solicitudes de tu zona" más, si hay, "· N viaje(s) en curso"; sin ubicación, "Activá la ubicación para ver primero los viajes más cercanos" y un botón "Usar mi ubicación" (`IconPin`) que llama a `reportarUbicacion`.

Barra de pestañas

Cinco botones generados a partir de un arreglo; cada uno muestra su icono, su etiqueta y una burbuja con count cuando `count > 0`:

id	Ico	label	count
radar	IconRadar	Radar	<code>solicitudes.length</code>
activos	IconClipboard	Ofertas	<code>viajesActivos.length</code>

id	Ico	label	count
historial	IconCalendario	Historial	0 (fijo)
ganancias	IconGrafico	Ganancias	0 (fijo)
vehiculo	IconAuto	Vehículo	0 (fijo)

Elementos comunes a todas las pestañas (dentro de la "hoja clara")

- **Banner de éxito** (`alertaExito`): "Oferta enviada. Esperando la respuesta del pasajero." con `IconVisto`.
- **Modal de oferta** (`solicitudModal`): tarjeta en línea con origen (`IconPin`), destino en verde, número de pasajeros (`adults_count || 1`) + (`children_count || 0`) y `IconMascota` si `has_pets`; pregunta "¿Cuánto cobrarías por este viaje?" con un `input type="number"` prefijado por \$, mensaje de `errorPrecio` en #C2410C y botón "Enviar" que llama a `enviarOferta`. El botón × limpia `solicitudModal`, `precio` y `errorPrecio`. Este bloque se renderiza siempre (no depende de la pestaña activa).

Pestaña 1 — Radar (`pestañaActiva === 'radar'`)

Qué carga: `cargarSolicitudes()` desde `GET /api/service-requests/pending`, disparada por el efecto #4 (cuando `sincronizado`) y refrescada silenciosamente cada 15 s por el efecto #5.

Contenido: 1. **Barra de filtros:** botón "Filtros" (`IconFiltros`) que alterna `mostrarFiltros`; se pinta en verde cuando `filtrosActivos`. A su derecha, "Limpiar filtros" (solo si hay filtros activos) que restablece `{ tipo: 'todos', pasajeros: 'todos', mascotas: false }`. 2. **Panel de filtros desplegable** (animado con `AnimatePresence`): – *Tipo de viaje:* `todos` / `ONE_WAY` ("→ Solo ida") / `ROUND_TRIP` ("↔ Ida y vuelta"). – *Pasajeros:* `todos` / `1` / `2-4` / `5+`. – *Solo con mascotas:* interruptor personalizado que alterna `filtros.mascotas`. 3. **Contador:** "N de M viaje(s) coinciden con tus filtros" cuando hay filtros activos y no está cargando. 4. **Estados:** – Cargando: tres `<SkeletonTarjetaViaje />`. – Error: mensaje con `IconAlerta` y botón "Reintentar" → `cargarSolicitudes`. – Vacío total (`solicitudes.length === 0`): ilustración SVG de radar, "No hay viajes en tu zona", texto explicativo y botón "Verificar de nuevo" (`IconGirar`) → `cargarSolicitudes`. – Vacío por filtros (`solicitudes.length > 0 && solicitudesFiltradas.length === 0`): "Sin resultados con estos filtros" y botón "Limpiar filtros". 5. **Tarjetas** (`solicitudesFiltradas.map`): cuerpo clicable (`handleClickTarjeta`) con `TableroRuta`, chip de tipo de viaje, fecha formateada (`IconCalendario`), pasajeros (`IconPersonas` + `IconMascota` si aplica) y el aviso de comodidades HU55: si `sol.comodidades_exigidas > 0`, muestra "El pasajero pidió comodidades que te faltan: ..." (a partir de `sol.comodidades_faltantes.join( ', ' )`) o bien "Cumplís las N comodidad(es) que pidió el pasajero". Al seleccionarla aparece "Ruta trazada en el mapa — hacé clic para ocultarla". Debajo, botón "**Hacer oferta**" que abre el modal de precio.

Pestaña 2 — Ofertas (`pestañaActiva === 'activos'`)

Qué carga: `cargarViajesActivos()` desde `GET /api/service-requests/driver/my-offers` (efecto #6 al entrar en la pestaña, y además el sondeo de fondo del efecto #10 cada 15 s).

Contenido: – Botón "Actualizar" (`IconGirar`) → `cargarViajesActivos`. – Vacío: "Aún no has enviado ofertas". – Por cada oferta (`key={viaje.offer_id}`) se calculan `esContraoferta` (`status === 'PASSENGER_COUNTER_OFFERED'`), `esAceptado` (`'ACCEPTED'`) y `esRechazado` (`'REJECTED'`), más el `estadoViaje = viaje.trip_status`. – Mapa `cfgEstado` sobre `viaje.status`:

status	Etiqueta
DRIVER_OFFERED	Esperando respuesta del pasajero
PASSENGER_COUNTER_OFFERED	Contraoferta recibida — respondé
ACCEPTED	Confirmado — listo para iniciar
REJECTED	Rechazado

- Mapa `cfgViaje` sobre `trip_status` (ASSIGNED "Viaje confirmado", IN_PROGRESS "Viaje en curso", COMPLETED "Viaje completado") — **declarado pero nunca usado en el JSX**.
- Cabecera de la tarjeta: `TableroRuta`, fecha, precio `{Number(viaje.offered_price).toLocaleString()}` y la aclaración "Precio del pasajero" (si es contraoferta) o "Tu oferta".
- **Acciones según estado:**
 - Contraoferta: botones "Aceptar precio" → `resolverContraoferta(viaje.offer_id, 'ACCEPT')` y "Rechazar" → `resolverContraoferta(viaje.offer_id, 'REJECT')`.
 - ACCEPTED + (ASSIGNED o IN_PROGRESS): botón "Ver ocupantes del viaje" → `await cargarOcupantes(viaje.request_id)` y luego `setModalOcupantesId(viaje.request_id)` (HU10).
 - ACCEPTED + ASSIGNED: aviso "Viaje confirmado. Cuando recojas al pasajero, iniciá el viaje." y botón azul "Iniciar viaje" → `gestionarViaje(viaje.request_id, 'start')`.
 - ACCEPTED + IN_PROGRESS: aviso "Viaje en curso. Finalizá cuando llegues al destino." y botón "Finalizar viaje" → `gestionarViaje(viaje.request_id, 'complete')`.
 - ACCEPTED + COMPLETED: aviso "Viaje completado." y botón "Calificar pasajero" (deshabilitado y con el texto "Ya calificaste a este pasajero" si `viaje.ya_califico`) que abre el modal de calificación reiniciando `estrellasCalificar` y `comentarioCalificar`.

Pestaña 3 — Historial (`pestañaActiva === 'historial'`)

Qué carga: `cargarHistorial()` desde `GET /api/service-requests/driver/history` (efecto #7).

Contenido: – Cargando: "Cargando tu historial...". – Error: caja de alerta con `IconAlerta` y el texto de `errorHistorial`. – Vacío: "Todavía no manejaste ningún viaje" con `IconCalendario`. – Tarjetas: chip de estado según `v.trip_status` (ASSIGNED "Confirmado", IN_PROGRESS "En curso", COMPLETED "Completado", CANCELLED "Cancelado"; fallback con `IconReloj` y el propio `trip_status`), precio `{Number(v.precio).toLocaleString()}`, `TableroRuta`, fecha `v.departure_time` en formato corto, `v.pasajero_nombre` (`IconPersona`) y `v.asientos` (`IconPersonas`). – Esta pestaña es de solo lectura: no ofrece acciones.

Pestaña 4 — Ganancias (`pestañaActiva === 'ganancias'`, HU52 / SCRUM-204)

Qué carga: `cargarGanancias()` desde `GET /drivers/earnings` (efecto #8).

Contenido: – Cargando: "Cargando...". Error: mensaje + botón "Reintentar" (`IconGirar`). – Dos tarjetas grandes: "Esta semana" con `ganancias.ganancias_semana.toLocaleString('es-CO')` y "Este mes" con `ganancias.ganancias_mes.toLocaleString('es-CO')`. – Dos tarjetas pequeñas: `ganancias.viajes_completados` ("Viajes completados") y `ganancias.calificacion_promedio` con `IconEstrella` (si es `null` muestra – y la etiqueta "Calificación (próximamente)"). – Si `ganancias.viajes_completados === 0`: mensaje vacío con `IconGrafico`. – Si hay viajes: – **"Horarios más activos"**: gráfico de barras construido con `ganancias.horarios_activos.map((cant, hora) => ...)`; la altura es `Math.max((cant / max) * 100, 4)%` con `max =`

`Math.max(...ganancias.horarios_activos, 1);` cada barra tiene `title={\ ${hora}:OO — ${cant} viaje(s)}`. Eje etiquetado 12am / 6am / 12pm / 6pm / 11pm. - ****Rutas más frecuentes****: `ganancias.top_rutas.map(r => ...)` con `r.ruta` y `r.viajes``. - Solo lectura: no ofrece acciones más allá de "Reintentar".

Pestaña 5 — Vehículo (`pestañaActiva === 'vehiculo', HU55 / SCRUM-207`)

Qué carga: `cargarVehiculo()` desde `GET /drivers/vehicle` (efecto #9). Guarda con `PATCH /drivers/vehicle`.

Contenido: - Cargando / error con botón "Reintentar". - Cabecera verde: "Placa `{formVehiculo.plate}` · Categoría `{formVehiculo.categoria}`" y la explicación de que la categoría se calcula automáticamente a partir de la capacidad real. - **Capacidad real (pasajeros)**: `input type="number" (min="1" max="60")` sobre `formVehiculo.capacidad_real`, con `placeholder={\ Registrada: ${formVehiculo.capacity}}`. - ****Aviso condicional****: si ninguna de `tiene_ac`, `tiene_wifi`, `tiene_bano`, `tiene_musica`, `tiene_maletero_amplio`, `tiene_sillas_bebe` está marcada, aparece una caja ámbar recordando que los pasajeros filtran por comodidades. - ****Comodidades****: rejilla de seis casillas - `tiene_ac` "Aire acondicionado", `tiene_wifi` "WiFi", `tiene_bano` "Baño", `tiene_musica` "Música", `tiene_maletero_amplio` "Maletero amplio", `tiene_sillas_bebe` "Sillas para bebé". - ****Mascotas****: casilla "Acepto mascotas" (`acepta_mascotas`); al desmarcarla pone `cargo_mascota: 0`. Si está marcada, aparece "Cargo adicional por mascota (COP)" (`input type="number" min="0"`). - ****Menores****: casilla "Acepto menores de 2 años" (`acepta_menores_2_anos`). - Botón "Guardar cambios" → `guardarVehiculo` (texto "Guardando..." mientras `guardandoVehiculo``).

Mapa (`<main className="pc-mapa">`)

- Si `mapsLoaded: <GoogleMap>` con `center={ubicacionActual || centroDefaultAntioquia}`, `zoom={ubicacionActual ? 12 : 9}`, `onLoad` que guarda `mapaRef`, y `options={{ disableDefaultUI: true, zoomControl: true, styles: tema === 'oscuro' ? MAPA_OSCURO : undefined }}`.
- Marcador propio (si hay `ubicacionActual`): círculo `scale: 9`, `fillColor: FIJO.ruta`, borde blanco, título "Tu posición".
- Marcadores de solicitudes: uno por cada elemento de `solicitudesConUbicacion`, círculo `scale: 7`, `fillColor: '#f59e0b'`, con `title "origen → destino"`; al hacer clic abre el modal de oferta para esa solicitud.
- `<BotonCentrarMapa onClick={centrarEnMiUbicacion} titulo={ubicacionActual ? 'Centrar en mi ubicación' : 'Buscar mi ubicación'} />`.
- Si no está cargado: caja gris con "Cargando mapa..."

Paneles y modales flotantes (fuera del layout de dos columnas)

1. **Panel de notificaciones** (`mostrarNotifPanel`): fondo oscurecido + panel deslizante de 420px desde la derecha (`zIndex: 3001`). Cabecera con "Notificaciones", "N sin leer", botón "Leer todas" (`marcarTodasLeidas`) y cierre. Lista con estado vacío ("Sin notificaciones") y una tarjeta por notificación; el color e icono salen de un mapa por `notif.type: NEW_OFFER` (`IconPrecio`), `COUNTER_OFFER` (`IconIntercambio`), `TRIP_ACCEPTED` (`IconVisto`), `TRIP_REJECTED` (`IconEquis`), `TRIP_STARTED` (`IconAuto`), `TRIP_COMPLETED` (`IconBandera`), `SYSTEM` (`IconCampana`), con fallback a `IconCampana`. Al hacer clic en una no leída se llama a `marcarLeida(notif.notification_id)`.
2. **Modal de ocupantes FUEC** (`modalOcupantesId`, `zIndex: 9001`): fondo `T.monteAlto`, rótulo "Documento de viaje" y título "Ocupantes registrados". Si `ocupantesPorViaje[modalOcupantesId]`

está vacío o ausente: "El pasajero aún no ha registrado los ocupantes.". Si hay datos, una fila por ocupante con `oc.full_name`, "Ocupante N" y el chip `{oc.document_type} {oc.document_number}`. Botón "Cerrar".

3. **Modal de calificación** (`modalCalificar`, HU46 / SCRUM-194): título "¿Cómo estuvo tu pasajero?" y subtítulo "Tu calificación queda registrada en su perfil."; cinco botones de estrella (`IconEstrella`, `aria-label={\ Calificar con ${n} de 5}`) que fijan `estrellasCalificar`; `textarea` con `placeholder` "Cuéntanos más (opcional)"; botones "Cancelar" y "Enviar calificación" (deshabilitado si `enviandoCalificacion || estrellasCalificar < 1`).
4. `<PerfilDrawer abierto={mostrarPerfil} onClose={() => setMostrarPerfil(false)} />`.
5. `<ToastContainer toasts={toasts} onRemove={removeToast} />`.

PerfilDrawer.jsx

Archivo de 537 líneas. Cajón lateral derecho con tres secciones (Perfil, Historial/Mis documentos, Seguridad) y el botón de cerrar sesión al pie.

Importaciones y constantes de módulo

Importaciones

Import	Origen	Uso
<code>useState, useEffect, useContext</code>	<code>react</code>	Hooks.
<code>motion, AnimatePresence</code>	<code>framer-motion</code>	Fondo oscurecido y deslizamiento del cajón.
<code>AuthContext</code>	<code>./AuthContext</code>	<code>{ token, usuario, cerrarSesion }</code> .
<code>useNavigate</code>	<code>react-router-dom</code>	Redirigir a <code>/login</code> tras cerrar sesión.
<code>API_BASE_URL</code>	<code>./api</code>	Base de las URLs.
<code>T, Icono, IconPersona, IconAuto, IconReloj, IconVisto, IconBandera, IconEquis, IconCalendario, IconClipboard, IconGorro, IconRecibo, IconAlerta, IconEstrella, TableroRuta</code>	<code>./diseno</code>	Tokens, envoltorio de iconos e iconografía.

Iconos definidos en el archivo

`IconIdea` (bombilla, usado en el bloque "¿Cómo obtengo mi RUNT?"), `IconEmpresa` (edificio, para la empresa afiliada), `IconEscudo` (SOAT y rol ADMIN), `IconTarjeta` (licencia), `IconLlave` (tecnomecánica), `IconCandado` (pestaña Seguridad), `IconLapiz` (botón "Editar" del teléfono) e `IconSalir` (cerrar sesión).

Constantes de módulo

Constante	Contenido	Para qué
<code>BRAND_GREEN</code>	<code>'var(--t-ruta)'</code>	Verde de marca.

Constante	Contenido	Para qué
ICONO_DOC	{ 'SOAT': IconEscudo, 'Licencia de Conduccion': IconTarjeta, 'Tarjeta de operacion': IconClipboard, 'Tecnomecanica': IconLlave, 'Seguros Contractual y extracontractual': IconRecibo, 'RUNT': IconGorro }	Icono por document_type. Fallback: IconClipboard.
LABEL_DOC	{ 'SOAT': 'SOAT Vigente', 'Licencia de Conduccion': 'Licencia de Conducción', 'Tarjeta de operacion': 'Tarjeta de Operación', 'Tecnomecanica': 'Tecnomecánica', 'Seguros Contractual y extracontractual': 'Seguros', 'RUNT': 'RUNT (experiencia)' }	Etiqueta legible por document_type (las claves están sin tildes, como llegan del backend).
ESTADO_DOC	PENDING → { bg: T.chivaSuave, color: T.chivaTexto, Ico: IconReloj, label: 'Pendiente' }; APPROVED → { bg: T.musgo, color: T.musgoTexto, Ico: IconVisto, label: 'Aprobado' }; REJECTED → { bg: T.alertaSuave, color: T.alertaTexto, Ico: IconEquis, label: 'Rechazado' }	Chip de estado de cada documento; fallback a ESTADO_DOC.PENDING.
ESTADO_VIAJE	PENDING → "Buscando conductor" (IconReloj); ASSIGNED → "Confirmado" (IconVisto); COMPLETED → "Completado" (IconBandera); CANCELLED → "Cancelado" (IconEquis)	Chip de estado en el historial del pasajero; fallback { color: 'var(--t-piedra)', label: v.status } (sin Ico, por eso el JSX comprueba est.Ico &&).

Componentes auxiliares definidos en el archivo

Ninguno con lógica: solo los ocho iconos listados arriba. El estilo compartido de los `input` se define como el objeto `inputStyle` **dentro** del componente principal (no es una constante de módulo).

El componente principal `PerfilDrawer`

Firma: `const PerfilDrawer = ({ abierto, onCerrar }) => { ... }`.

- `abierto` (booleano) controla el montaje dentro de `<AnimatePresence>`; `onCerrar` es el callback de cierre (fondo, botón `×` y tras cerrar sesión).

Contexto y hooks

- `useContext(AuthContext)` → { `token`, `usuario`, `cerrarSesion` }.
- `useNavigate()` → `navigate`.
- No usa `useRef`, `useTema` ni `useToast` (los errores se muestran con `alert()` o con banners locales).

Todos los `useState`

Carga y navegación interna

Variable	Setter	Valor inicial	Representa
<code>perfil</code>	<code>setPerfil</code>	<code>null</code>	Respuesta de <code>/users/me/profile</code> .
<code>cargando</code>	<code>setCargando</code>	<code>false</code>	Muestra "Cargando perfil..."
<code>seccion</code>	<code>setSeccion</code>	<code>'perfil'</code>	Pestaña activa: <code>'perfil'</code> , <code>'historial'</code> o <code>'seguridad'</code> .

Edición del teléfono

Variable	Setter	Valor inicial	Representa
editandoTelefono	setEditandoTelefono	false	Alterna entre vista y edición.
nuevoTelefono	setNuevoTelefono	''	Valor del campo (se precarga con <code>data.phone_number</code> \\ '').
guardandoTelefono	setGuardandoTelefono	false	Bloquea el botón "Guardar".

Cambio de contraseña

Variable	Setter	Valor inicial	Representa
passwordForm	setPasswordForm	{ current: '', new: '', confirm: '' }	Los tres campos del formulario.
guardandoPassword	setGuardandoPassword	false	Bloquea el envío.
errorPassword	setErrorPassword	''	Mensaje de error (validación local o detail del backend).
exitoPassword	setExitoPassword	false	Banner "Contraseña actualizada." (se apaga a los 3000 ms).

Subida de RUNT (HU37)

Variable	Setter	Valor inicial	Representa
mostrarFormRunt	setMostrarFormRunt	false	Alterna entre la invitación y el formulario.
archivoRunt	setArchivoRunt	null	Archivo seleccionado.
aniosExperiencia	setAniosExperiencia	''	Años declarados.
categoriasLicencia	setCategoriasLicencia	''	Categorías de licencia (opcional).
subiendoRunt	setSubiendoRunt	false	Bloquea "Enviar RUNT".
errorRunt	setErrorRunt	''	Mensaje de error de la subida.

Funciones definidas dentro

CARGARPERFIL() — ASYNC

1. Si no hay `token`, sale. `setCargando(true)`.
2. **GET ``${API_BASE_URL}/users/me/profile``**.
3. Si `res.ok`: `setPerfil(data)` y `setNuevoTelefono(data.phone_number || '')`.
4. `catch`: `console.error('Error cargando perfil:', e)`. `finally`: `setCargando(false)`. – Nota: si la respuesta no es `ok`, no hay mensaje al usuario; `perfil` queda en `null` y el cuerpo del cajón se ve vacío.

GUARDARTELEFONO() — ASYNC

1. Si `!nuevoTelefono.trim()`, sale. `setGuardandoTelefono(true)`.
2. **PATCH ``${API_BASE_URL}/users/me/phone``** con `{ phone_number: nuevoTelefono }`.
3. Si no es `ok`, lanza `new Error()` (sin mensaje).

4. Éxito: `setPerfil(prev => ({ ...prev, phone_number: nuevoTelefono })))` y `setEditandoTelefono(false)`.
5. `catch: alert('Error al actualizar el teléfono.'). finally: setGuardandoTelefono(false)`.

CAMBIARPASSWORD() — ASYNC

1. Limpia `errorPassword` y `exitoPassword`.
2. Validaciones locales: si `passwordForm.new !== passwordForm.confirm` → "Las contraseñas nuevas no coinciden."; si `passwordForm.new.length < 8` → "Mínimo 8 caracteres.". En ambos casos corta.
3. `setGuardandoPassword(true)`.
4. **PATCH `/${API_BASE_URL}/users/me/password`** con `{ current_password: passwordForm.current, new_password: passwordForm.new }`.
5. Si no es `ok`, lanza `new Error(err.detail)`.
6. Éxito: `setExitoPassword(true)`, limpia `passwordForm` y programa `setTimeout(() => setExitoPassword(false), 3000)`.
7. `catch: setErrorPassword(e.message || 'Error al cambiar la contraseña.'). finally: setGuardandoPassword(false)`.

SUBIRRUNT() — ASYNC (HU37)

1. Limpia `errorRunt`.
2. Validaciones: si `archivoRunt` → "Selecciona el archivo de tu RUNT (PDF o imagen)."; `aniosExperiencia === '' || Number(aniosExperiencia) < 0` → "Ingresa tus años de experiencia".
3. `setSubiendoRunt(true)`. Arma un `FormData` con `years_experience`, opcionalmente `license_categories` (solo si `categoriasLicencia.trim()` no está vacío) y `doc_runt` (el archivo).
4. **POST `/${API_BASE_URL}/drivers/upload-runt`** con ese `FormData` (sin `Content-Type` manual, correcto para multipart).
5. Si no es `ok`, lanza `new Error(err.detail || 'No se pudo enviar el RUNT.')`.
6. Éxito: cierra el formulario, limpia `archivoRunt`, `aniosExperiencia` y `categoriasLicencia`, y hace `await cargarPerfil()` para refrescar el estado de documentos.
7. `catch: setErrorRunt(e.message || 'No se pudo enviar el RUNT.'). finally: setSubiendoRunt(false)`.

HANDLECERRARSESION()

- `cerrarSesion(); onCerrar(); navigate('/login');`.

FORMATEARFECHA(F)

- `new Date(f).toLocaleDateString('es-CO', { day: '2-digit', month: 'short', year: 'numeric' })`; si falla, devuelve `f`.

INPUTSTYLE

- No es función sino un objeto de estilo compartido por todos los `input` del cajón: `{ width: '100%', padding: '10px 12px', border: '1px solid var(--t-linea)', borderRadius: '8px', fontSize: '14px', boxSizing: 'border-box', outline: 'none' }`.

useEffect

Dependencias	Qué hace	Limpieza
[abierto]	if (abierto) cargarPerfil(); — recarga el perfil cada vez que se abre el cajón.	Ninguna.

Es el único efecto del archivo. **No hay suscripciones a Supabase Realtime en PerfilDrawer.jsx.**

Estructura de la interfaz

Envoltorio

`<AnimatePresence>` que, cuando `abierto`, monta: (a) un fondo `rgba(0,0,0,0.4)` a pantalla completa (`zIndex: 2000`) que cierra al hacer clic, y (b) el cajón de 460px (`maxWidth: '100%', zIndex: 2001`) que entra desde `x: '100%'` con `transition: { type: 'tween', duration: 0.3 }`.

Cabecera (fondo `T.monte` con las curvas SVG decorativas)

- Avatar circular de 52 px: muestra `<img src={perfil.profile_photo_url}>` si existe; si no, `<IconPersona size={24} />`.
- Nombre: `perfil?.full_name || usuario?.full_name`. Correo: `perfil?.email || usuario?.email`.
- **Chip de rol:** DRIVER → `IconAuto + "Conductor"`; ADMIN → `IconEscudo + "Admin"`; cualquier otro → `IconPersona + "Pasajero"`.
- **Chip "Verificado"** (`IconGorro`) solo si `perfil?.role === 'DRIVER' && perfil?.conductor_verificado`.
- Botón `×` de cierre.
- **Pestañas subrayadas** (no píldoras): `perfil` "Perfil" (`IconPersona`); `historial` cuya etiqueta e icono dependen del rol — `'Mis documentos' + IconClipboard` para DRIVER, `'Historial' + IconCalendario` para el resto; `seguridad` "Seguridad" (`IconCandado`).

Sección "Perfil" (`seccion === 'perfil'`)

Común a todos los roles: – **Teléfono**, único campo editable: en modo lectura muestra `perfil.phone_number` con el botón "Editar" (`IconLapiz`); en edición, un `input`, el botón "Guardar" (`guardarTelefono`, texto "...") mientras guarda) y un botón `IconEquis` que cancela. – Campos de solo lectura generados desde un arreglo: "Nombre completo" (`perfil.full_name`), "Correo electrónico" (`perfil.email`) y, solo si `perfil.age` es verdadero, "Edad" con `${perfil.age} años`. – **Tus calificaciones** — el comentario del código aclara que esta sección ya no vive dentro del ramal de conductor porque la reciben pasajeros y conductores por igual: – Si `perfil.rating_count > 0`: tarjeta ámbar con `IconEstrella` relleno, `Number(perfil.rating_avg).toFixed(1)` seguido de `"/ 5"`, y "N calificación / calificaciones" (singular/plural resuelto con `perfil.rating_count === 1 ? 'ón' : 'ones'`). – Debajo, si `perfil.rating_comentarios?.length > 0`, una tarjeta por comentario con cinco `IconEstrella` (rellenas hasta `c.score`, el resto con `opacity: .3`) y el texto `c.comment`. – Si `rating_count` es 0: "Todavía no te han calificado. Al terminar un viaje, la otra persona puede dejarte una calificación."

Solo para `perfil.role === 'DRIVER'`: – **"Empresa afiliada"** (si `perfil.empresa_afiliada`): `IconEmpresa + perfil.empresa_afiliada.name`. – **"Vehículo"** (si `perfil.vehiculo`): foto (`perfil.vehiculo.photo_url`) si existe, más `IconAuto + perfil.vehiculo.plate` en monoespaciada y `${perfil.vehiculo.capacity}` asientos.

Solo para `perfil.role === 'PASSENGER'`: – Si `perfil.ofertas_activas !== undefined`, banner verde: "Tenés **N** oferta(s) activa(s)" con `IconVisto`, con pluralización a partir de `perfil.ofertas_activas !== 1`.

Sección "Historial / Mis documentos" (`seccion === 'historial'`)

Rama pasajero (`role === 'PASSENGER'`) – "Historial de viajes": – Vacío (`!perfil.historial_viajes || length === 0`): icono `IconCalendario` y "Aún no has solicitado viajes." – Por cada viaje de `perfil.historial_viajes`: fecha de creación con `formatFecha(v.created_at)`, chip de estado desde `ESTADO_VIAJE[v.status]`, `TableroRuta` `origen={v.origin}` `destino={v.destination}` y la fecha de salida `v.departure_time` formateada en `es-CO`.

Rama conductor (`role === 'DRIVER'`) – "Estado de documentos": – Vacío: "No tienes documentos registrados." – Por cada `doc` de `perfil.documentos`: icono desde `ICONO_DOC[doc.document_type]` (fallback `IconClipboard`), etiqueta desde `LABEL_DOC[doc.document_type]` || `doc.document_type`, línea extra `"{doc.years_experience} años de experiencia declarados"` cuando `doc.document_type === 'RUNT' && doc.years_experience !== null`, y chip de estado desde `ESTADO_DOC[doc.verification_status]`. –

Bloque de subida de RUNT (IIFE): busca `runt = perfil.documentos?.find(d => d.document_type === 'RUNT')` y calcula `puedeSubir = !runt || runt.verification_status === 'REJECTED'`. Si no puede subir, no renderiza nada. – Estado plegado: "¿Querés verificar tu experiencia?" (`IconGorro`), "Sube tu RUNT para obtener el badge de conductor verificado." y un botón cuyo texto es "Volver a enviar RUNT" si ya existe un RUNT, o "Subir RUNT" si no. – Estado desplegado: caja informativa "¿Cómo obtengo mi RUNT?" (`IconIdea`) explicando que se descarga el "Extracto de conductor" en `runt.gov.co` con la cédula; banner de `errorRunt` con `IconAlerta`; campo "Años de experiencia" (`type="number" min="0" max="80"`); campo "Categorías de licencia (opcional)" con placeholder "Ej: C2, C3"; campo de archivo "Archivo del RUNT (PDF, JPG o PNG)" con `accept=".pdf, .jpg, .jpeg, .png, .webp"`; botones "Enviar RUNT" (`subirRunt`, texto "Enviando..." mientras sube) y "Cancelar" (cierra y limpia `errorRunt`).

Sección "Seguridad" (`seccion === 'seguridad'`)

- Título "Cambiar contraseña".
- Banner verde "Contraseña actualizada." si `exitoPassword`; banner rojo con `IconAlerta` y `errorPassword` si hay error.
- Tres `input type="password"` generados a partir del arreglo `[{ label: 'Contraseña actual', key: 'current' }, { label: 'Nueva contraseña', key: 'new' }, { label: 'Confirmar nueva contraseña', key: 'confirm' }]`; cada `onChange` limpia `errorPassword` y actualiza `passwordForm[campo.key]`.
- Botón "Actualizar contraseña" (`cambiarPassword`, texto "Actualizando..." mientras guarda).
- Esta sección se renderiza con `!cargando && seccion === 'seguridad'` – a diferencia de las otras dos, **no** exige que `perfil` esté cargado.

Pie

- Botón rojo suave "Cerrar sesión" (`IconSalir`) → `handleCerrarSesion`.

Sobre la subida de foto

En `PerfilDrawer.jsx` no existe ninguna subida de foto de perfil. El archivo solo *muestra* `perfil.profile_photo_url` en el avatar de la cabecera (y `perfil.vehiculo.photo_url` en la tarjeta de vehículo); no hay `input type="file"` para la foto, ni endpoint asociado. La única subida de archivos del

cajón es la del RUNT (`POST /drivers/upload-runt`). La carga de `profile_photo` y `vehicle_photo` ocurre en `FormularioConductor.jsx`, durante el registro (ver sección 3).

FormularioConductor.jsx

Archivo de 533 líneas. Pantalla completa (ruta propia) para que un usuario se registre como conductor: consulta primero su estado de registro y, según el resultado, muestra el formulario o la vista de estado.

Importaciones y constantes de módulo

Importaciones

Import	Origen	Uso
<code>useState, useEffect, useContext</code>	<code>react</code>	Hooks.
<code>motion</code>	<code>framer-motion</code>	Entrada animada de las tarjetas (<code>AnimatePresence</code> no se usa aquí).
<code>useNavigate</code>	<code>react-router-dom</code>	Botón "Volver al mapa" → / dashboard.
<code>AuthContext</code>	<code>./AuthContext</code>	{ <code>token</code> , <code>usuario</code> }.
<code>API_BASE_URL</code>	<code>./api</code>	Base de las URLs.
<code>T, EstilosBase, Boton, Rotulo, LogoWordmark, Icono, IconReloj, IconVisto, IconEquis, IconAlerta, IconPersona, IconAuto, IconClipboard, IconRecibo, IconIdea</code>	<code>./diseno</code>	Tokens, reset/estilos globales, botón del sistema, rótulo, marca e iconografía.

Iconos definidos en el archivo

`IconEscudo` (SOAT), `IconTarjeta` (licencia), `IconLlave` (tecnomecánica), `IconSubir` (icono por defecto del `DropZone`), `IconFlechaIz` (botón "Volver al mapa"). Además, dentro del render de carga se instancia un `<Icono>` en línea con un trazo de flecha circular y `animation: 't-girar .9s linear infinite'`.

Constantes de módulo

Constante	Valor	Para qué
<code>BRAND_GREEN</code>	<code>T.ruta</code> (valor directo, no la variable CSS)	Verde de marca.
<code>ETIQUETA_DOCUMENTO</code>	{ <code>'SOAT': 'SOAT Vigente'</code> , <code>'Licencia de Conduccion': 'Licencia de Conducción'</code> , <code>'Tarjeta de operacion': 'Tarjeta de Operación'</code> , <code>'Tecnomecanica': 'Revisión Tecnomecánica'</code> , <code>'Seguros Contractual y extracontractual': 'Seguros (Contractual / Extracontractual)'</code> }	Traduce los tipos de documento del backend. No incluye 'RUNT'.

Constante	Valor	Para qué
<code>pesoLegible(bytes)</code>	Función	Devuelve '' si bytes es falsy y distinto de 0; $\text{\${bytes} B bajo 1024; \${Math.round(bytes/1024)} KB bajo 1 MB; \${(bytes/(1024*1024)).toFixed(1)} MB en adelante.}$ Se usa para confirmar visualmente el archivo cargado.
<code>estiloEntrada</code>	Objeto de estilo	Padding 12/14, borde T.linea, radio T.rControl, fondo T.papel, fontSize: '15px'.
<code>estiloBloqueado</code>	<code>{ ...estiloEntrada, background: T.niebla2, color: T.piedra, cursor: 'not-allowed' }</code>	Campos readOnly (nombre, correo, teléfono).

Componentes auxiliares definidos en el archivo

DropZone

`({ label, name, onChange, file, Ico = IconSubir })`

- Estado interno propio: `const [arrastrando, setArrastrando] = useState(false)`. Derivado: `const cargado = Boolean(file)`.
- Maneja `onDragOver` (previene el default y marca `arrastrando`), `onDragLeave` y `onDrop`: si hay `e.dataTransfer.files?.length > 0`, sintetiza un evento compatible — `onChange({ target: { name, type: 'file', files: e.dataTransfer.files } })` — para reusar el mismo manejador que el `input`.
- Renderiza un `<input type="file" name={name} onChange={onChange} aria-label={label}>` transparente que cubre toda la caja (`position: absolute; inset: 0; opacity: 0`).
- Muestra tres estados visuales: vacío (borde punteado, fondo `T.niebla`, icono `Ico`), arrastrando (borde `BRAND_GREEN`, fondo `T.musgo`) y cargado (borde sólido `T.musgoLinea`, `IconVisto`).
- Texto secundario: `'Arrastrá el archivo o hacé clic'` cuando está vacío; `{file.name} · {pesoLegible(file.size)}` cuando hay archivo.
- **No valida** ni el tipo MIME ni el tamaño; tampoco declara `accept`.

Seccion

`({ n, titulo, descripcion, Ico, completa, children })`

- Cabecera de sección con línea inferior: cuadro de 30 px que muestra `IconVisto` (fondo `T.musgo`) si `completa`, o el icono `Ico` si no; `<h3>` con `titulo`; párrafo opcional `descripcion`; y a la derecha el número en monoespaciada con `String(n).padStart(2, '0')` ("01", "02", "03").
- Debajo renderiza `children`.

Campo

`({ etiqueta, hijo, ancho })`

- Envoltorio de campo con etiqueta visible siempre (el comentario explica que el placeholder solo no basta porque desaparece al escribir). Si `ancho` es verdadero, aplica `gridColumn: 'span 2'`.
- El contenido llega por la prop `hijo`, no por `children`.

VistaEstadoDocumentos

```
{ estadoData, onVolver }
```

- Objeto `config` interno con tres entradas por `estadoData.estado`:

Clave	Título	Descripción
PENDIENTE	"Documentos en revisión"	"Un administrador está revisando tus documentos. Te avisamos apenas haya respuesta." (IconReloj, paleta chiva)
APROBADO	"Ya sos conductor"	"Tus documentos quedaron aprobados. Ya podés recibir solicitudes de viaje." (IconVisto, paleta musgo)
RECHAZADO	"Documentos rechazados"	"Algunos documentos no pasaron la revisión. Corregilos y volvé a enviarlos." (IconEquis, paleta alerta)

Fallback: `config.PENDIENTE`. – Función interna `estadoDoc(e): 'APPROVED' → "Aprobado" (musgo, IconVisto); 'REJECTED' → "Rechazado" (alerta, IconEquis); cualquier otro → "Pendiente" (chiva, IconReloj)`. – Renderiza `<Cabecera titulo="Estado de tu solicitud" onVolver={onVolver} />`, la tarjeta grande de estado, y —si existe `estadoData.documentos`— la lista "Estado por documento" con `ETIQUETA_DOCUMENTO[doc.tipo] || doc.tipo` y el chip de `estadoDoc(doc.estado)`. – Si `estadoData.documentos_rechazados?.length > 0`, añade una caja roja "Tenés que volver a enviar:" con una línea `<etiqueta>` por cada tipo rechazado.

Cabecera

```
{ titulo, onVolver }
```

- Franja `T.monte` con las curvas SVG decorativas, `<LogoWordmark alto={113} oscuro />`, el `titulo` en `<h2>` y el botón "Volver al mapa" (`IconFlechaIz`) que dispara `onVolver`.
- Se declara **después** de `VistaEstadoDocumentos`, que la usa; funciona por *hoisting* de `const` en tiempo de render (no en tiempo de módulo), así que no es un error, pero el orden de lectura es confuso.

El componente principal FormularioConductor

Firma: `const FormularioConductor = () => { ... }` — no recibe props.

Contexto y hooks

- `useNavigate()` → `navigate`.
- `useContext(AuthContext)` → `{ token, usuario }`.
- No usa `useRef`, `useTema` ni `useToast`.

Todos los useState**Máquina de estados de la pantalla**

Variable	Setter	Valor inicial	Representa
<code>estadoCarga</code>	<code>setEstadoCarga</code>	<code>'cargando'</code>	Comentado en el código como <code>'cargando' \ 'mostrar_formulario' \ 'mostrar_estado'</code> . Decide cuál de las tres vistas se renderiza.

Variable	Setter	Valor inicial	Representa
estadoData	setEstadoData	null	Respuesta de /drivers/registration-status (o el objeto sintético que se arma tras enviar).

Envío

Variable	Setter	Valor inicial	Representa
enviando	setEnviando	false	Bloquea el botón de envío.
errorEnvio	setErrorEnvio	' '	Mensaje de error mostrado junto al botón (antes era un alert(), según el comentario).

Datos del formulario

Variable	Setter	Valor inicial	Representa
formConductor	setFormConductor	{ age: '', affiliated_company: '', profile_photo: null, plate: '', capacity: '', vehicle_photo: null, doc_soat: null, doc_licencia: null, doc_tarjeta_operacion: null, doc_tecnomecanica: null, doc_seguros: null, tiene_ac: false, tiene_wifi: false, tiene_bano: false, tiene_musica: false, tiene_maletero_amplio: false, tiene_sillas_bebe: false, acepta_mascotas: false }	Todo el formulario en un solo objeto: datos personales, vehículo, archivos y comodidades HU55.

Valores derivados

Nombre	Cálculo
datosUsuario	{ full_name: usuario?.full_name ' ', email: usuario?.email ' ', phone_number: usuario?.phone_number ' ' } — alimenta los tres campos readOnly.
documentos	['doc_soat', 'doc_licencia', 'doc_tarjeta_operacion', 'doc_tecnomecanica', 'doc_seguros'].
seccionPersonalLista	Boolean(formConductor.age && formConductor.affiliated_company).
seccionVehiculoLista	Boolean(formConductor.plate && formConductor.capacity).
documentosCargados	documentos.filter(d => formConductor[d]).length.
seccionDocsLista	documentosCargados === documentos.length (los cinco).
listas	[seccionPersonalLista, seccionVehiculoLista, seccionDocsLista].filter(Boolean).length — de 0 a 3.

Funciones definidas dentro

CONSULTARESTADO() — ASYNC, DEFINIDA DENTRO DEL USEEFFECT

1. Si no hay token, sale.
2. GET ``${API_BASE_URL}/drivers/registration-status``.
3. Si no es ok, lanza `new Error()` (vacío).

4. Si `data.estado === 'SIN_DOCUMENTOS'` o `'RECHAZADO'`: `setEstadoData(data)` y `setEstadoCarga('mostrar_formulario')`.
5. En cualquier otro caso: `setEstadoData(data)` y `setEstadoCarga('mostrar_estado')`.
6. `catch`: `setEstadoCarga('mostrar_formulario')` — si la consulta falla, se muestra el formulario de todas formas (y `estadoData` queda en `null`).

HANDLEINPUTCONDUCTOR(E)

- Extrae `{ name, value, type, files, checked }` del `e.target`.
- Si hay `errorEnvio`, lo limpia.
- Actualiza `formConductor[name]` con `files[0]` si `type === 'file'`, con `checked` si `type === 'checkbox'`, y con `value` en el resto.
- Es el único manejador: lo comparten los `input`, el `select`, los `checkbox` y todos los `DropZone` (incluido el evento sintético del `drop`).

ENVIARFORMULARIOCONDUCTOR(E) — ASYNC

1. `e.preventDefault()`, `setEnviando(true)`, `setErrorEnvio('')`.
2. Arma un `FormData`. Campos **siempre** añadidos: `age`, `affiliated_company`, `plate`, `capacity`, `tiene_ac`, `tiene_wifi`, `tiene_bano`, `tiene_musica`, `tiene_maletero_amplio`, `tiene_sillas_bebe`, `acepta_mascotas` (los booleanos viajan como los strings `"true"` / `"false"`).
3. Campos **condicionales** (solo si el archivo existe): `profile_photo`, `vehicle_photo`, `doc_soat`, `doc_licencia`, `doc_tarjeta_operacion`, `doc_tecnomecanica`, `doc_seguros`.
4. **POST** `$/API_BASE_URL/drivers/register-details` con `body`: `formData` (sin Content-Type manual).
5. Si no es ok, lanza `new Error(errorData.detail || 'No se pudo enviar la solicitud.')`.
6. Éxito: fija localmente `setEstadoData({ estado: 'PENDIENTE', mensaje: 'Tus documentos están siendo revisados por el administrador.', documentos: null })` y `setEstadoCarga('mostrar_estado')`; no vuelve a consultar el backend.
7. `catch`: `setErrorEnvio(error.message)`. `finally`: `setEnviando(false)`.

useEffect

Dependencias	Qué hace	Limpieza
<code>[token]</code>	Define y ejecuta <code>consultarEstado()</code> (ver arriba): decide entre formulario y vista de estado a partir de <code>GET /drivers/registration-status</code> .	Ninguna.

Es el único efecto del archivo. **No hay suscripciones a Supabase Realtime en `FormularioConductor.jsx`.**

Estructura de la interfaz

El componente tiene **tres renders mutuamente excluyentes** según `estadoCarga`:

`estadoCarga === 'cargando'`

`<EstilosBase />` + pantalla completa centrada con el icono giratorio (`animation: 't-girar .9s linear infinite'`) y el texto "Verificando tu estado de registro..."

```
estadoCarga === 'mostrar_estado'
```

`<EstilosBase /> + <VistaEstadoDocumentos estadoData={estadoData} onVolver={() => navigate('/dashboard')} />` sobre fondo `T.niebla`. Es la pantalla que ve un conductor con documentos ya enviados (PENDIENTE / APROBADO) o el que acaba de enviarlos.

```
estadoCarga === 'mostrar_formulario' (render por defecto)
```

Estructura de arriba abajo, dentro de una tarjeta de `maxWidth: '820px'`:

1. **<style> local:** `.fc-entrada:focus` (borde verde + halo `rgba(22,163,74,.12)`), `.fc-comodidad:hover` (borde `T.piedraClara`) y la regla responsiva `@media (max-width: 720px) { .fc-rejilla { grid-template-columns: 1fr !important; } }`.
2. **<Cabecera titulo="Registrate como conductor" onVolver={() => navigate('/dashboard')} />**.
3. **Barra de progreso:** una barra de 4 px cuyo ancho es `$(listas / 3) * 100%` con `transition: width .3s ease`, más el texto `{listas}` de 3 secciones en monoespaciada. Es puramente informativa: no bloquea el envío. Una sección cuenta como completa cuando cumple su condición derivada (ver 3.3.3): personales = edad + empresa; vehículo = placa + capacidad; documentos = los cinco archivos cargados.
4. **Banner de rechazo** (solo si `estadoData?.estado === 'RECHAZADO'`): "Algunos documentos fueron rechazados. Podés volver a enviarlos acá abajo." con la lista `estadoData.documentos_rechazados` traducida por `ETIQUETA_DOCUMENTO`.
5. **<form onSubmit={enviarFormularioConductor}>** con tres secciones:

Sección 01 — "Tus datos" (`Ico={IconPersona}`, `completa={seccionPersonalLista}`, descripción "Nombre, correo y teléfono vienen de tu cuenta."): - "Nombre completo", "Correo" y "Teléfono": `readOnly` con `estiloBloqueado`, alimentados por `datosUsuario`. - "Edad": `input type="number" name="age" required min="18"`, placeholder "Mínimo 18". - "Empresa afiliada" (ancho completo): `select name="affiliated_company" required` con `<option value="">Elegí tu empresa...</option>`, `<option value="1">Departour</option>` y `<option value="2">Transporte Real</option>` — la lista está **codificada en duro**, no viene del backend. - `DropZone` "Tu foto de perfil" (`name="profile_photo"`), a lo ancho de las dos columnas.

Sección 02 — "Tu vehículo" (`Ico={IconAuto}`, `completa={seccionVehiculoLista}`, sin descripción): - "Placa": `input type="text" name="plate" required`, placeholder "ABC-123", con `textTransform: 'uppercase'` (solo visual, el valor enviado conserva lo escrito). - "Capacidad": `input type="number" name="capacity" required min="1" max="44"`, placeholder "Asientos disponibles". - `DropZone` "Foto de tu vehículo" (`name="vehicle_photo"`), ancho completo. - **Bloque "Comodidades (opcional)"** (`IconIdea`) con el texto "Algunos pasajeros filtran su búsqueda por esto. Podés dejarlas sin marcar y configurarlas después desde tu panel.", y siete casillas: `tiene_ac` "Aire acondicionado", `tiene_wifi` "WiFi", `tiene_bano` "Baño", `tiene_musica` "Música", `tiene_maletero_amplio` "Maletero amplio", `tiene_sillas_bebe` "Sillas para bebé", `acepta_mascotas` "Acepta mascotas". Cada `<label>` cambia de fondo y borde cuando está marcada.

Sección 03 — "Documentación reglamentaria" (`Ico={IconClipboard}`, `completa={seccionDocsLista}`, descripción dinámica "En PDF o imagen. `$(documentosCargados)` de `$(documentos.length)` cargados."): - `DropZone` "SOAT vigente" (`doc_soat`, `IconEscudo`), "Licencia de conducción" (`doc_licencia`, `IconTarjeta`), "Tarjeta de operación" (`doc_tarjeta_operacion`, `IconClipboard`), "Revisión tecnomecánica" (`doc_tecnomecanica`, `IconLlave`) y, a dos columnas, "Seguros (contractual y extracontractual)" (`doc_seguros`, `IconRecibo`).

1. **Banner de errorEnvio** (si existe) con `IconAlerta`.

2. `<Boton type="submit" ancho disabled={enviando} variante={enviando ? 'inactivo' : 'primario'}>` con el texto "Enviar solicitud" / "Enviando documentos...".
3. Nota al pie: "Un administrador revisa tus documentos antes de habilitarte. Te avisamos cuando haya respuesta."

Validación – resumen

Nivel	Qué valida
HTML nativo	age (required, min="18"), affiliated_company (required), plate (required), capacity (required, min="1", max="44"). El navegador bloquea el submit si falta alguno.
JavaScript	Ninguna. enviarFormularioConductor no comprueba nada antes de armar el FormData.
Archivos	Sin validación: los cinco documentos y las dos fotos son opcionales desde el punto de vista del formulario (se añaden al FormData solo si existen). No se valida extensión, MIME ni tamaño, y el DropZone no declara accept.
Progreso	seccionDocsLista exige los cinco archivos, pero eso solo mueve la barra: no impide enviar un formulario sin ningún documento.
Servidor	Único filtro real sobre los archivos; su detail acaba en errorEnvio.

PARTE 6

Acceso, administración y perfil público

Login y registro, las dos vistas de administrador y el perfil público del conductor.

Pantallas de acceso

Login.jsx

Props: { `irARegistro`, `onLoginSuccess` } (inyectadas por `LoginConNavegacion` en `App.jsx`).

Constantes de módulo

- `const LOGO = LogoWordmark;` — indirección deliberada. Comentario: «Logo en uso. Alternativas: `LogoMonograma` | `LogoBifurcacion`». Los tres se importan; solo `LogoWordmark` se usa.
- `COPY_POR_VISTA` — una frase por pestaña, en JSX con `<br/>` y un `<span>` que el CSS pinta de `T.chiva`:
- `viajar`: «Llegamos hasta donde / *el bus se devuelve.*»
- `conducir`: «Caminos que otros / *no recorren.*»
- `quienes`: «Movilidad para / *toda Antioquia.*»
- En comentario quedan tres alternativas descartadas.
- `RUTAS` — seis pares reales de Antioquia: `['Palmitas', 'Medellín']`, `['Santa Elena', 'Rionegro']`, `['El Retiro', 'La Ceja']`, `['Urrao', 'Concordia']`, `['San Vicente', 'Guarne']`, `['Támesis', 'Jericó']`.

Subcomponente `TableroEnVivo`

Estados `indice` (0) y `visible` (`true`). Un `useEffect` de montaje:

- Si `window.matchMedia('(prefers-reduced-motion: reduce)').matches`, sale sin registrar nada — el tablero se queda fijo en la primera ruta.
- Si no, un `setInterval` de **4200 ms** pone `visible` en `false`, y **260 ms** después avanza el índice (`i + 1`) % `RUTAS.length` y vuelve a `true`.

Renderiza `<TableroRuta oscuro size={19}>` con opacidad animada (`transition: 'opacity .26s ease'`), y debajo el texto en `T.dato` «412 rutas activas en Antioquia».

Estados del componente `Login`

Estado	Inicial	Función
<code>vista</code>	<code>'viajar'</code>	Pestaña activa: <code>'viajar'</code> <code> </code> <code>'conducir'</code> <code> </code> <code>'quienes'</code>
<code>formData</code>	<code>{ email: '', password: '' }</code>	Campos del formulario
<code>errorBackend</code>	<code>''</code>	Mensaje de error del servidor o de red
<code>isLoading</code>	<code>false</code>	Petición en curso
<code>showPassword</code>	<code>false</code>	Alterna type del input entre password y text
<code>campoActivo</code>	<code>null</code>	Nombre del campo enfocado, para pintar el borde

Validación

```
const isValidForm = formData.email.includes('@') && formData.password.length > 0;
```

Validación mínima y **solo en el cliente para habilitar el botón**: basta con que el correo contenga una @ y que la contraseña no esté vacía. El `<input type="email">` añade la validación nativa del navegador.

`handleChange` actualiza `formData` y limpia `errorBackend` si lo había (el error desaparece al empezar a corregir).

Envío y endpoint

```
const urlEncodedData = new URLSearchParams();
urlEncodedData.append('username', formData.email);
urlEncodedData.append('password', formData.password);

fetch(`${API_BASE_URL}/users/login`, {
  method: 'POST',
  headers: { 'Content-Type': 'application/x-www-form-urlencoded', 'ngrok-skip-browser-warning': 'true' },
  body: urlEncodedData
})
```

- **POST /users/login**, cuerpo `application/x-www-form-urlencoded` con los campos `username` y `password` — es el formato que espera `OAuth2PasswordRequestForm` de FastAPI. Nótese que el correo viaja en el campo `username`.
- Si `response.ok`: guarda `data.access_token` en `localStorage` bajo la clave `'token'` y llama a `onLoginSuccess(data.access_token)` si existe.
- Si no: `setErrorBackend(errorData.detail || 'Correo o contraseña incorrectos.')`.
- `catch`: `'Error de conexión. Verifica que el backend esté activo.'`
- `finally`: `setIsLoading(false)`.

Estructura visual

Layout de dos columnas con CSS embebido en un `<style>`:

- **.login-izq** (flex: 1.6 1 0) — fondo `T.monte`. Contiene un SVG decorativo de seis curvas de nivel (stroke="#86EFAC", opacity: 0.18) — comentario: «Topografía de montaña — curvas de nivel, no una foto de stock». Encima: cabecera con `<LOGO alto={24} oscuro />` y una `<nav>` de tres pestañas (Viajar / Conducir / Quiénes somos); centro con el `<h1>` de la frase (Syne, clamp(34px, 4.2vw, 56px)) y `<TableroEnVivo />`; pie con «Transporte especial habilitado · © 2026 Turify».
- **.login-der** (flex: 0 0 400px) — fondo `T.papel`, borde izquierdo `T.linea`. Su contenido depende de vista:
 - **viajar**: el formulario. Rotulo «Bienvenido de nuevo», `<h2>` «Entrar», campos Correo y Contraseña (con botón de ojo `IconOjo / IconOjoTachado`, `tabIndex={-1}`), bloque de error con `IconAlerta`, botón de envío cuya variante es `'primario'` o `'inactivo'` según `isFormValid && !isLoading` y cuyo texto alterna entre `'Entrando…'` y `'Entrar'`; separador con la letra «o» y un botón fantasma «Crear cuenta» que llama a `irARegistro`.
 - **conducir**: propuesta de valor para conductores — tres viñetas (`.login-punto`, con un punto verde generado por `::before`) y botón «Crear cuenta y empezar» → `irARegistro`; enlace «Entrá acá» que devuelve a la pestaña `viajar`.
 - **quienes**: misión de la empresa, tres viñetas y botón fantasma «Volver a entrar».
 - **Responsive**: por debajo de 900px, `.login-izq` se oculta (`display: none`) y `.login-der` pasa a `flex: 1` sin borde.

Estilos de campo centralizados en la función `estiloCampo(nombre):` cuando `campoActivo === nombre`, el borde pasa a `T.ruta` y se añade `boxShadow: '0 0 0 3px rgba(22,163,74,.12)'` (el verde del modo claro, escrito a mano en rgba).

Registro.jsx

Props: `{ irALogin }`.

Estados

Estado	Inicial
formData	<code>{ full_name: '', email: '', password: '', confirmPassword: '', phone_number: '' }</code>
errores	<code>{}</code> — errores por campo
isLoading	<code>false</code>
showPassword	<code>false</code>
showConfirm	<code>false</code>
mensajeExito	<code>''</code>
errorBackend	<code>''</code>
campoActivo	<code>null</code>

Validación de contraseña (en vivo)

```
const hasMinLength = formData.password.length >= 8;
const hasUppercase = /[A-Z]/.test(formData.password);
const hasNumber = /[0-9]/.test(formData.password);
const isPasswordValid = hasMinLength && hasUppercase && hasNumber;
```

Y la validez global del formulario:

```
const isFormValid =
  formData.full_name.trim() !== '' &&
  formData.email.includes('@') &&
  isPasswordValid &&
  formData.confirmPassword === formData.password &&
  formData.password !== '';
```

`phone_number` **no participa** en la validación: es opcional.

`validarFormulario()` — mensajes al enviar

Condición	Campo	Mensaje
<code>!formData.full_name.trim()</code>	<code>full_name</code>	'El nombre es obligatorio.'
<code>!formData.email.includes('@')</code>	<code>email</code>	'Correo inválido.'
<code>!isPasswordValid</code>	<code>password</code>	'La contraseña no cumple los requisitos.'
<code>formData.password !== formData.confirmPassword</code>	<code>confirmPassword</code>	'Las contraseñas no coinciden.'

`handleSubmit` ejecuta esta validación primero; si devuelve algún error, hace `setErrores(...)` y **retorna sin llamar a la API**.

Endpoint

```
const { confirmPassword, ...payload } = formData;
fetch(`${API_BASE_URL}/users/register`, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json', 'ngrok-skip-browser-warning': 'true' },
  body: JSON.stringify(payload)
})
```

POST /users/register con JSON. La desestructuración descarta `confirmPassword`: el cuerpo enviado lleva `full_name`, `email`, `password` y `phone_number`.

- Éxito: `setMensajeExito('Tu cuenta quedó creada. Te llevamos al inicio de sesión...')` y un `setTimeout` de **1800 ms** que llama a `irALogin()`.
- Error del servidor: `errorData.detail || 'Error en el registro. Inténtalo de nuevo.'`
- Error de red: `'Error de conexión. Verifica que el backend esté activo.'`

Subcomponentes internos

- **Requisito({ ok, label })** — fila con un círculo de 14 px que, cuando `ok` es verdadero, se rellena con `T.musgo`, borde `T.ruta` y un `<IconVisto size={9} color={T.ruta} grosor={2.6} />`; el texto pasa de `T.piedraClara` a `T.musgoTexto`. Transición de `.2s`.
- **ojo(visible, alternar)** — no es un componente sino una función que devuelve el botón de mostrar/ocultar contraseña; se reutiliza para los dos campos.

Estructura visual

- **.reg-izq** (`flex: 0 0 400px`, fondo `T.monte`) — SVG de cinco curvas de nivel (`opacity: 0.16`), `<LogoWordmark alto={14} oscuro />`, titular «Sumate a la red **Turify**.» (la segunda línea en `T.chiva`), párrafo, un `<TableroRuta origen="Vos" destino="Tu destino" oscuro size={11} />`, y tres pasos numerados (`.reg-paso-n`, círculos de 24 px con el número en `T.dato` sobre `rgba(22,163,74,.16)`): **1** Creá tu cuenta, **2** Publicá tu viaje, **3** Elegí y viajá. Pie «© 2026 Turify».
- **.reg-der** (`flex: 1`, fondo `T.papel`, `overflow-y: auto`) — botón «← Volver a entrar», Rotulo «Nuevo usuario», `<h2>` «Crear cuenta», y el formulario en una rejilla `.reg-rejilla` de dos columnas: *Nombre completo* ocupa la fila entera (`.reg-completo`), y luego pares *Correo/Teléfono* y *Contraseña/Confirmar contraseña*.
- El campo *Teléfono* se estiliza con `fontFamily: T.dato` y `letterSpacing: '.06em'` — es un dato, no texto de interfaz. Además se le pasa siempre `false` como `hayError`, así que nunca se pinta en rojo.
- El panel de requisitos (`8+ caracteres`, `Mayúscula`, `Número`, `Coinciden`) solo aparece cuando `formData.password.length > 0`.
- Debajo del botón se muestran, si existen, el bloque de `errorBackend` (con `IconAlerta`, tonos alerta) y el de `mensajeExit` (con `IconVisto`, tonos musgo).
- **Responsive:** por debajo de `900px` se oculta `.reg-izq` y la rejilla pasa a una sola columna.

Pantallas de administrador

AdminConductores.jsx (ruta `/admin/conductores`)

Es el punto de entrada del administrador. Comentario en la cabecera: «este panel ES el punto de entrada del admin (no hay dashboard de pasajero de por medio), así que acá van la marca y las acciones de cuenta en vez de un botón "volver"».

Constantes de módulo

ETIQUETA_DOCUMENTO — traduce la clave del backend a texto legible:

Clave del backend	Etiqueta
'SOAT'	'SOAT Vigente'
'Licencia de Conduccion'	'Licencia de Conducción'
'Tarjeta de operacion'	'Tarjeta de Operación'
'Tecnomecanica'	'Revisión Tecnomecánica'
'Seguros Contractual y extracontractual'	'Seguros (Contractual y Extracontractual)'

Clave del backend	Etiqueta
'RUNT'	'RUNT (experiencia)'

ICONO_DOCUMENTO — `SOAT` → `IconEscudo`, `Licencia de Conduccion` → `IconTarjeta`, `Tarjeta de operacion` → `IconClipboard`, `Tecnomecanica` → `IconLlave`, `Seguros...` → `IconRecibo`, `RUNT` → `IconGorro`. El componente `IconoDocumento({ tipo, size = 17 })` resuelve el icono y cae en `IconClipboard` si el tipo es desconocido.

Constantes de contraste sobre monte (comentario: «el monte es oscuro en los dos temas, así que acá los claros van fijos, igual que en el panel del conductor»):

```
const CLARO = '#EAF2EC';
const claro = (a) => `rgba(234,242,236,${a})`;
const SOBRE_MONTE = { fondo: 'rgba(255,255,255,0.07)', linea: T.monteLinea };
```

Subcomponentes

- **BadgeEstado({ estado })** — mapea el estado del documento a un `<Chip>`: `PENDING` → tono `chiva`, `IconReloj`, «Pendiente»; `APPROVED` → tono `verde`, `IconVisto`, «Aprobado»; `REJECTED` → tono `alerta`, `IconEquis`, «Rechazado». Cualquier otro valor cae en `PENDING`.
- **esPDF(url)** — `url?.toLowerCase().includes('.pdf')`.
- **getPDFcomoImagen(url)** — trucos de Cloudinary: sustituye `/raw/upload/` por `/image/upload/` y cambia la extensión `.pdf` por `.jpg`, para obtener la primera página del PDF como imagen.
- **getUrlPreview(url)** — más conservador: solo sustituye `/raw/upload/` por `/image/upload/` cuando además la URL es PDF.
- **VisorPDF({ url, documentId, token })** — muestra `<img src={getPDFcomoImagen(url)}>`; si el `onError` dispara, cambia a un estado de fallo con `IconDocumento`, el texto «No se pudo mostrar la vista previa.» y un enlace «Descargar el documento» a la URL original. **documentId y token se reciben pero no se usan.**
- **ModalDocumento({ doc, onCerrar, token })** — capa fija a pantalla completa (`zIndex: 9999`) con un velo `rgba(0,0,0,0.75)` animado por `framer-motion` y un panel de `82vw × 82vh` (máx. 960 px). Cabecera en `T.monte` con el icono y la etiqueta del documento, un enlace «Descargar» (`target="_blank" rel="noopener noreferrer"`) y un botón circular de cierre. El cuerpo usa `VisorPDF` si es PDF, o una `<img>` directa si no.
- **Contador({ valor, etiqueta, color, Ico })** — celda del panel lateral con el número en `T.dato` a 19 px y la etiqueta en mayúsculas espaciadas. Comentario: «el número es dato, va en monoespaciada».

Estados del componente principal

Estado	Inicial	Función
<code>conductores</code>	<code>[]</code>	Lista de la cola de verificación
<code>cargando</code>	<code>true</code>	Petición de lista en curso
<code>error</code>	<code>null</code>	Mensaje de error de la carga
<code>conductorSeleccionado</code>	<code>null</code>	Conductor mostrado en el panel derecho

Estado	Inicial	Función
procesando	null	document_id cuya verificación está en vuelo
alerta	null	{ tipo, mensaje } para la banda superior
docPreview	null	Documento abierto en el modal
experienciaEditada	{}	Mapa document_id → texto de los años que el admin está escribiendo (HU38)

Además: `const { token, cerrarSesion } = useContext(AuthContext);, useNavigate()` y `const [tema, alternarTema] = useTema();`.

Funciones y endpoints

mostrarAlerta(tipo, mensaje) — fija `alerta` y la borra con un `setTimeout` de **3500 ms**.

cargarConductores()

```
GET {API_BASE_URL}/admin/drivers/pending
Authorization: Bearer <token>
ngrok-skip-browser-warning: true
```

Si no es `ok`, lanza `'No se pudieron cargar los conductores.'`, que acaba en el estado `error`.

verificarDocumento(documentId, nuevoEstado, yearsExperience)

```
PATCH {API_BASE_URL}/admin/documents/{documentId}/verify
Content-Type: application/json
body: { verification_status: nuevoEstado[, years_experience: <int>] }
```

`years_experience` solo se incluye si el argumento no es `undefined`, `null` ni cadena vacía, y se convierte con `parseInt(yearsExperience, 10)`.

Tras el éxito muestra `data.message` como alerta de éxito y actualiza el estado local **sin recargar la lista**: recorre `conductores`, y para el conductor que contiene ese documento recalcula `conductor_verificado` (solo si el documento afectado es de tipo `'RUNT'`, en cuyo caso pasa a `nuevoEstado === 'APPROVED'`) y sustituye el documento con su nuevo `verification_status` y `years_experience`.

Efectos:

- `useEffect(() => { if (token) cargarConductores(); }, [token])` — carga inicial.
- `useEffect(..., [conductores])` — si hay un conductor seleccionado, lo vuelve a buscar por `user_id` en la lista recién actualizada y lo re-selecciona, para que el panel derecho refleje los cambios.

Derivados:

```
const todosAprobados = (conductor) =>
  conductor.documents.filter(d => d.document_type !== 'RUNT').every(d => d.verification_status
=== 'APPROVED');
const contarPendientes = (conductor) => conductor.documents.filter(d => d.verification_status
=== 'PENDING').length;
const totalPendientes = conductores.reduce((acc, c) => acc + contarPendientes(c), 0);
const totalVerificados = conductores.filter(c => c.conductor_verificado).length;
```

El RUNT se excluye de `todosAprobados` — comentario: «El RUNT es opcional y posterior al registro — no cuenta para "conductor habilitado"».

Estructura visual

1. `<EstilosBase />` y el `<AnimatePresence>` del `ModalDocumento`.
2. **Cabecera oscura** (`T.monte`): `<LogoWordmark alto={13} oscuro />`, Rotulo «Panel de administración», y a la derecha: botón «Ver la bitácora» (`→ navigate('/admin/logs')`), `<BotonTema ... compacto />` y «Cerrar sesión» (llama a `cerrarSesion()` y `navigate('/login')`, en color `T.chiva`).
3. **Banda de alerta** animada con `framer-motion`, en tonos musgo (éxito) o alerta (error).
4. **Shell de dos paneles** (`borderRadius: '18px', minHeight: '640px'`): — **<aside> de 330 px, fondo monte** — la *cola de verificación*: Rotulo, `<h1>` «Verificación», tres `Contador` (Pendientes / En cola / Verificados), la lista desplazable de conductores y, al pie, un botón «Actualizar».
 - Cada fila de conductor: avatar (foto o `IconPersona`), nombre (con `IconGorro` dorado si `conductor_verificado`), correo, y a la derecha una píldora con el número de pendientes o un `IconVisto` si no hay ninguno. La selección se marca con `color-mix(in srgb, ${T.ruta} 16%, transparent)` y borde `T.ruta`; volver a pulsar deselecciona.
 - Cuatro estados: cargando, error (con botón «Reintentar»), vacío («Todo al día / No hay documentos pendientes.») y lista.
 - **Hoja clara `T.papel`** — el detalle. Sin selección muestra el estado vacío «Elegí un conductor». Con selección: cabecera con foto, nombre, correo y teléfono (en `T.dato`), más los chips «Conductor habilitado» (verde, si `todosAprobados`) y «Experiencia verificada» (chiva, si `conductor_verificado`).
 - **Lista de documentos**: cada tarjeta lleva un fondo dependiente del estado — si es RUNT, siempre `T.chivaSuave` con borde `T.chivaLinea`; si no, `T.musgo` (aprobado), `T.alertaSuave` (rechazado) o `T.papel` (pendiente). Contiene el icono del tipo, la etiqueta, un botón «Ver el documento» (abre el modal) y, **solo para el RUNT**, un `<input type="number" min="0" max="80">` de años de experiencia, deshabilitado cuando el estado ya no es `PENDING`, cuyo valor es `experienciaEditada[doc.document_id] ?? doc.years_experience ?? ''`.
 - A la derecha, el `BadgeEstado` y las acciones: si `PENDING`, botones «Aprobar» (verde, envía también los años) y «Rechazar» (alerta); si no, un botón «Revertir» que vuelve a poner `'PENDING'`. Durante `procesando === doc.document_id` el texto pasa a «Guardando», se deshabilita y baja a `opacity: 0.7`.
 - Si el conductor tiene algún RUNT, aparece una nota: «Al aprobar el RUNT, el conductor recibe el sello dorado en su perfil público y en sus ofertas — sin afectar su rol de conductor ya activo.»
 - **Resumen final**: tres contadores (Pendientes / Aprobados / Rechazados) en `T.dato` a 23 px, con Rotulo debajo.

AdminLogs.jsx (ruta `/admin/logs`)**Constantes de módulo**

Reutiliza `CLARO`, `claro(a)` y `SOBRE_MONTE` idénticos a los de `AdminConductores`.

TONOS — mapa de cinco tonos a `{ fondo, linea, texto }`:

Tono	fondo	linea	texto
verde	T.musgo	T.musgoLinea	T.musgoTexto
alerta	T.alertaSuave	T.alertaLinea	T.alertaTexto
cielo	T.cieloSuave	T.cieloLinea	T.cieloTexto
chiva	T.chivaSuave	T.chivaLinea	T.chivaTexto
neutro	T.niebla	T.linea	T.piedra

CONFIG_ACCION — comentario: «Cada acción tiene tono e icono; el tono agrupa (entrar, crear, revisar) y el icono distingue, así dos acciones del mismo color no se confunden.»

Acción (backend)	Tono	Icono	Etiqueta
LOGIN	verde	IconCandado	Ingreso
LOGIN_FAILED	alerta	IconProhibido	Ingreso fallido
REGISTER	cielo	IconPersona	Registro
CREATE_TRIP	chiva	IconMapa	Viaje creado
CREATE_OFFER	chiva	IconPrecio	Oferta enviada
VERIFY_DOCUMENT	verde	IconClipboard	Documento verificado
REGISTER_DRIVER	cielo	IconAuto	Registro de conductor

FILTROS — los botones de píldora del panel de filtros:

`{ value: '', label: 'Todos' }`, `LOGIN` → Ingresos, `LOGIN_FAILED` → Ingresos fallidos, `REGISTER` → Registros, `CREATE_TRIP` → Viajes, `CREATE_OFFER` → Ofertas, `VERIFY_DOCUMENT` → Verificaciones.

Estilos de celda: `celda` (`padding: '9px 16px'`, `verticalAlign: 'middle'`) y `celdaDato` (`celda + T.dato`, `12.5px`, `letterSpacing: '.04em'`, `whiteSpace: 'nowrap'`).

Estados

Estado	Inicial
logs	<code>[]</code>
cargando	<code>true</code>
filtroAccion	<code>''</code> — filtro del servidor
busqueda	<code>''</code> — filtro de cliente

Más `const { token } = useContext(AuthContext);`, `useNavigate()` y `useTema()`.

Función y endpoint

```
const params = new URLSearchParams({ limit: 200 });
if (accion) params.append('action', accion);
fetch(`${API_BASE_URL}/admin/logs?${params}`, { headers: { Authorization: `Bearer ${token}`,
'ngrok-skip-browser-warning': 'true' } }})
```

GET /admin/logs?limit=200[&action=<ACCION>]. Ante cualquier fallo, el `catch` hace `setLogs([])` en **silencio**: no hay estado de error visible, la tabla simplemente aparece vacía.

`useEffect(() => { if (token) cargarLogs(filtroAccion); }, [token, filtroAccion])` — cambiar el filtro dispara una petición nueva.

Dos filtros superpuestos

1. **filtroAccion** — viaja al servidor como parámetro `action`.
2. **busqueda** — filtra en cliente sobre lo ya cargado:

```
const logsFiltrados = logs.filter(l => {
  if (!busqueda) return true;
  const texto = `${l.detail} ${l.action} ${l.ip_address} ${l.user_id}`.toLowerCase();
  return texto.includes(busqueda.toLowerCase());
});
```

Es decir: busca en detalle, acción, IP e identificador de usuario, sin distinguir mayúsculas.

Contadores: `logs.reduce((acc, l) => { acc[l.action] = (acc[l.action] || 0) + 1; return acc; }, {})` — se calculan sobre `logs` (la respuesta del servidor), **no** sobre `logsFiltrados`.

formatearFecha(fecha) — `new Date(fecha).toLocaleString('es-CO', { day: '2-digit', month: 'short', year: 'numeric', hour: '2-digit', minute: '2-digit', second: '2-digit' })`, con `catch` que devuelve la cadena original.

Estructura visual

1. **Cabecera oscura** (`T.monte`, `padding: '16px 40px'`): botón «Volver al panel» con `IconFlechaIzq` (→ `/admin/conductores`), `<LogoWordmark alto={113} oscuro />` y el `<h1>` «Bitácora de auditoría»; a la derecha, `<BotonTema ... compacto />` y un botón verde «Actualizar» / «Actualizando».
2. **Rotulo** «Historial de acciones del sistema».
3. **Tarjetas resumen** — una por cada entrada de `CONFIG_ACCION`, en una rejilla `repeat(auto-fit, minmax(160px, 1fr))`. Cada tarjeta es un botón que **filtra la tabla por su acción**, y vuelve a pulsarse para quitar el filtro (`setFiltroAccion(activa ? '' : accion)`). Cuando está activa adopta el fondo y el borde de su tono.
4. **Barra de filtros y búsqueda** — campo con `IconLupa` y placeholder «Buscá en la bitácora», la fila de píldoras de `FILTROS` (la activa se pinta con fondo `T.tinta` y texto `T.papel`) y, alineado a la derecha, el recuento «`{n}` registro» / «registros» (pluralización con `logsFiltrados.length !== 1`).
5. **Tabla** — tres estados: cargando (con `IconReloj`), vacía (`IconBandeja` + «No hay registros que mostrar.») o la tabla. Comentario: «filas densas y desplazamiento horizontal propio, para que la tabla no empuje el ancho de la página» — de ahí el envoltorio con `overflowX: 'auto'` y `minWidth`:

'860px' en la `<table>`. – Columnas: **ID, Acción, Usuario, Detalle, IP, Fecha**. Los encabezados van en `T.dato`, 11.5 px, mayúsculas, `letterSpacing: '.16em'`. – Filas cebra: `i % 2 === 0 ? T.papel : T.niebla`. Clave `log.log_id`. – ID se muestra como `#{log.log_id}`; Usuario como `#{log.user_id}` o `'-'`; Detalle truncado con elipsis, `maxWidth: '320px'`, con el texto completo en el atributo `title`; IP y Fecha con guion largo si faltan. – Una acción desconocida cae en `{ tono: 'neutro', Ico: IconChincheta, label: log.action }` — se muestra la clave cruda del backend.

Perfil público del conductor

Ambos archivos implementan la misma historia de usuario (**HU38**) con dos formatos distintos, y ambos consumen la misma forma de datos.

Diferencia entre los dos

	<code>PerfilConductorPagina.jsx</code>	<code>PerfilConductorPublico.jsx</code>
Formato	Página completa con su propia ruta	Modal superpuesto
Ruta	<code>/conductor/:driverId</code> (dentro de <code>RutaPrivada</code>)	Ninguna — se monta desde otro componente
Cómo obtiene los datos	Los pide él mismo con <code>fetch</code>	Los recibe por props (datos, cargando)
Props	Ninguna	<code>{ abierto, cargando, datos, onCerrar }</code>
Animación	Ninguna	<code>framer-motion</code> (<code>AnimatePresence</code> , <code>motion.div</code>)
Iconos de comodidades	No — solo texto	Sí — cada comodidad con su icono
Sello de verificado	Píldora horizontal en la cabecera	Sello circular rotado <code>-11deg</code> que se desborda de la cabecera
Uso actual en el código	Enrutado en <code>App.jsx</code>	No se importa en ningún archivo

Comentario en `PerfilConductorPagina`: «HU38 — Perfil público del conductor como página completa (no una tarjeta flotando sobre fondo gris): la cabecera ocupa todo el ancho como en cualquier página del sitio, y el contenido corre debajo en una columna editorial. Sin emojis decorativos — solo tipografía, iconos de trazo, y el dorado reservado para lo verificado.»

Comentario en `PerfilConductorPublico`: «HU38 — Perfil público del conductor: el pasajero lo abre tocando el nombre/foto del conductor en una tarjeta de oferta. Formato "pase de abordar" (coherente con ser una app de transporte) — sello dorado para experiencia verificada, distinto del verde de "conectado/aprobado" que ya usa el resto de la app.»

La lectura razonable es que la versión de página **reemplazó** a la modal, que quedó sin usar. Los comentarios no lo dicen explícitamente.

PerfilConductorPagina.jsx

Constantes locales: `FOREST = 'var(--t-monte)'`, `GOLD = 'var(--t-chiva)'`,
`BRAND_GREEN = 'var(--t-ruta)'` (esta última **nunca se usa**).

ETIQUETAS_COMODIDAD — pares `[campo, etiqueta]`, sin icono:

Campo	Etiqueta
<code>tiene_ac</code>	Aire acondicionado
<code>tiene_wifi</code>	WiFi
<code>tiene_bano</code>	Baño
<code>tiene_musica</code>	Música
<code>tiene_maletero_amplio</code>	Maletero amplio
<code>tiene_sillas_bebe</code>	Sillas para bebé
<code>acepta_mascotas</code>	Acepta mascotas

Iconos locales: `IconPersona({ size = 34 })` e `IconCheck({ size = 14, color = '#fff' })` — SVG escritos a mano, **sin usar el componente `Icono`** del sistema de diseño. Nótese que `IconPersona` colisiona de nombre con el exportado por `diseño.jsx`, aunque aquí no se importa aquel.

Estados: `cargando (true)`, `error (false)`, `datos (null)`. Obtiene `driverId` de `useParams()`, `navigate` de `useNavigate()` y `token` de `AuthContext`.

Endpoint:

```
GET {API_BASE_URL}/drivers/{driverId}/public-profile
Authorization: Bearer <token>
ngrok-skip-browser-warning: true
```

El efecto depende de `[token, driverId]` y sale sin hacer nada si falta cualquiera de los dos. Cualquier respuesta no-`ok` o excepción activa `error`.

`volver()` — `window.open() ? window.close() : navigate('/dashboard')`. Es decir: si la página se abrió en una pestaña nueva desde otra ventana, la cierra; si no, vuelve al dashboard.

Tres estados de render:

1. **Cargando** — pantalla centrada con el texto «Cargando perfil...».
2. **Error o sin datos** — «No pudimos cargar este perfil.» / «Verifica tu conexión e intenta de nuevo.» y un botón «← Volver».
3. **Perfil**.

Qué muestra el perfil:

- **Cabecera a todo el ancho** con fondo `radial-gradient(circle at 12% -20%, rgba(34,197,94,0.35), transparent 55%), linear-gradient(155deg, #0a3d1f, var(--t-monte) 68%)`; dentro, una columna de `maxWidth: '840px'`. Contiene el botón «← Volver» y la palabra **TURIFY** en Syne.

- **Identidad:** avatar circular de 104 px (`datos.profile_photo_url` o el `IconPersona` local a 42 px), `datos.full_name` en Syne 34 px peso 800.
- **Vehículo** (si `datos.vehiculo`): `{categoria}` · Placa `{plate}` · `{capacity}` puestos.
- **Calificación:** si `datos.rating_avg != null`, `IconEstrella` con relleno y `Number(datos.rating_avg).toFixed(1)`, seguido de `{datos.rating_count}` calificaciones; si no, «Sin calificaciones aún».
- **Píldora «Conductor verificado»** (solo si `datos.conductor_verificado`): fondo `rgba(234,179,8,0.14)`, borde `rgba(234,179,8,0.4)`, con un disco dorado (`radial-gradient` de `#fde047` a `var(--t-chiva)` a `var(--t-chiva-texto)`) que contiene el `IconCheck` en `#3d2500`.
- **Tarjeta editorial** que sube sobre la cabecera (`margin: '-40px auto 0'`, `borderRadius: '18px'`), con filas de tipo etiqueta/valor separadas por `1px solid var(--t-niebla-2)`:
- **Empresa afiliada** → `datos.empresa_afiliada.name` (solo si existe).
- **Experiencia verificada** → `{datos.years_experience}` años (solo si `conductor_verificado` y `years_experience != null`).
- **Viajes completados** → `datos.viajes_completados` (siempre).
- **Comodidades** (solo si hay `vehiculo`) — píldoras de texto para cada campo de `ETIQUETAS_COMODIDAD` que sea verdadero en `datos.vehiculo`.
- **Nota final** (si `conductor_verificado`): borde izquierdo dorado de 3 px, titular «Experiencia verificada por Turify» y el texto «El administrador confirmó los años de experiencia de este conductor a partir de su RUNT.»

Importa un `@import` de Syne propio dentro de un `<style>`, en lugar de montar `<EstilosBase />`.

PerfilConductorPublico.jsx

Props:

Prop	Tipo	Función
abierto	booleano	Gobierna el <code>AnimatePresence</code> ; si es falso no se renderiza nada
cargando	booleano	Muestra el estado de carga
datos	objeto null	El perfil ya obtenido por el componente padre
onCerrar	función	Se llama al pulsar el velo o la ×

No hace ninguna petición: **no tiene endpoint propio**. El componente padre (que no existe en el código actual) sería quien llame a `/drivers/{id}/public-profile`.

ETIQUETAS_COMODIDAD — aquí son **tripletas** `[campo, etiqueta, Icono]`: `tiene_ac / IconAire`, `tiene_wifi / IconWifi`, `tiene_bano / IconBano`, `tiene_musica / IconMusica`, `tiene_maletero_amplio / IconMaleta`, `tiene_sillas_bebe / IconBebe`, `acepta_mascotas / IconMascota`. Las etiquetas de texto son idénticas a las de la versión página.

Estructura:

- Dentro de `<AnimatePresence>`, dos `motion.div`: un velo `rgba(5,14,5,0.6)` con `backdropFilter: 'blur(4px)'` (`zIndex: 4000`) que cierra al pulsarse, y el panel (`zIndex: 4001`) de 480 px (`maxWidth: '94vw'`, `maxHeight: '90vh'`, `overflowY: 'auto'`, `borderRadius: '26px'`) que entra con `{ opacity: 0, y: 28, scale: 0.96 }` → `{ opacity: 1, y: 0, scale: 1 }` en un `tween` de 0.24 s.

- Si `cargando || !datos`: « Cargando perfil...» (el único emoji que queda en estos archivos, contra la norma declarada del sistema).
- **Cabecera «pase de abordar»** con el mismo doble gradiente (aquí `circle at 15% -10%` y `var(--t-monte) 65%`): botón `×` circular arriba a la derecha, `TURIFY` en Syne y una píldora con borde «Perfil de conductor». Debajo, avatar de 84 px, nombre a 27 px, línea de vehículo y calificación (misma lógica `rating_avg != null`).
- **Sello dorado** (si `conductor_verificado`): círculo de 78 px en `position: absolute` con `bottom: '-32px'`, `transform: 'rotate(-11deg)'`, sombra `0 10px 22px rgba(161,98,7,0.4), 0 0 0 5px #fff`, con `IconVisto` y el texto «VERIFI-CADO» partido en dos líneas.
- **Separador** `borderTop: '2px dashed var(--t-linea)'` — el troquel del pase de abordar. Las filas del detalle también usan `1px dashed`.
- **Detalle**: mismas tres filas (Empresa afiliada — aquí con `IconEmpresa` junto al nombre —, Experiencia verificada, Viajes completados), las píldoras de comodidades **con icono**, y una caja final en tonos chiva (`var(--t-chiva-suave) / var(--t-chiva-linea)`) con `IconGorro`, el titular «Experiencia verificada por Turify» y el mismo texto sobre el RUNT.

Campos del perfil que consumen ambos (la forma que devuelve `GET /drivers/{id}/public-profile`):

`full_name`, `profile_photo_url`, `rating_avg`, `rating_count`, `conductor_verificado`, `years_experience`, `viajes_completados`, `empresa_afiliada.name`, y `vehiculo` con `categoria`, `plate`, `capacity` más los siete booleanos de comodidad.

PARTE 7

Observaciones y deuda técnica

Lo que se encontró de paso: código muerto, contradicciones entre comentario y código, y huecos conocidos.

Backend · modelo de datos y núcleo

Código duplicado / muerto

1. **ServiceRequestCreate** está definida dos veces en `schemas.py` (líneas 73 y 101). La segunda definición sobrescribe por completo a la primera, así que la clase de las líneas 73–99 —con su validador completo de fechas— es **código muerto**: nunca se usa.
2. **El validador efectivo perdió la validación de fechas.** En la segunda `ServiceRequestCreate`, `validate_dates` contiene únicamente el comentario `# ... (mantén tu lógica de validación de fechas anterior)` y la comprobación de `adults_count`. Es decir: pese al nombre del método, **no se valida que `departure_time` sea futura, ni que `return_time` exista y sea posterior en viajes `ROUND_TRIP`**. Esas reglas solo viven en la definición sobrescrita.
3. **ServiceRequestResponse, DriverResponse, OfferRead, UserProfileResponse y RoleEnum no se referencian en ningún router.** `OfferRead` y `UserProfileResponse` parecen escritas para `GET /api/service-requests/{request_id}/offers` y `GET /users/me/profile` respectivamente, pero ninguno de esos endpoints declara `response_model`.
4. **El router `routes.py` no se incluye en `main.py`.** Define `APIRouter(prefix="/api/routes", tags=["Routes"])` con el endpoint `POST /api/routes/compute` (`response_model=ComputeRouteResponse`, integración con Google Routes API), pero `main.py` solo importa e incluye `auth`, `drivers`, `service_requests` y `admin`. **Ese endpoint no está expuesto en la API.**
5. **Importaciones sin usar en `main.py`:** `Request`, `SessionLocal` y `registrar_log` se importan pero no se utilizan. También hay imports sin usar en `models.py`: `Index` y `JSON` (se usa `JSONNB` del dialecto Postgres, no `JSON`). En `security.py` se importa `schemas` sin usarlo.

Comentarios que contradicen el código

1. `get_current_user` dice **"Buscamos al usuario en MySQL"** (comentario del paso 3), pero la base es **PostgreSQL** (Supabase). Coherente con esto, `requirements.txt` incluye `PyMySQL==1.2.0`,

dependencia de MySQL que ya no corresponde a la arquitectura actual (`psycopg2-binary` es el driver real). Probable residuo de una migración MySQL → Postgres.

2. **UserCreate.phone_number** lleva el comentario «Ahora es obligatorio según tu SQL», mientras que `UserResponse.phone_number` es `Optional[str] = None` «porque hay usuarios antiguos sin el dato». Ambas cosas conviven, pero el modelo `User.phone_number` sí es `nullable=False`, así que en la práctica no debería haber usuarios sin teléfono creados por esta API.

Inconsistencias de esquema/modelo

1. **Campos del modelo sin exposición ni uso aparente:** `User.age` existe en el modelo y en `UserProfileResponse` (que no se usa) pero no en `UserResponse`. `ServiceRequest.infants_count`, `num_pets`, `wait_time_hours`, `num_days`, `is_peak_hour`, `is_high_season`, `suggested_price*`, `price_explanation`, `intermediate_stops` no aparecen en ningún schema Pydantic. Los modelos `PriceHistory`, `TripStop` y `Vehicle.vehicle_year` tampoco tienen esquema asociado.
2. **ServiceRequestCreate.tipo_via es Optional[str], no un Enum**, aunque el modelo tiene un `Enum('PAVIMENTADA', 'DESTAPADA', 'MIXTA', name='via_type')`. Se documenta con un comentario en vez de tipado, así que cualquier cadena arbitraria pasa la validación Pydantic y fallará recién en la base de datos. Lo mismo con `TripPassengerItem.document_type: str = 'CC'` frente al enum `doc_id_type`, y con `ResolveOfferCreate.action: str` frente a los valores `'ACCEPT' | 'REJECT'` del comentario.
3. **VehicleSettingsResponse declara categoria: str y tarifa_km_rango: list[float], que no existen como columnas** en el modelo `Vehicle`. Como el schema tiene `from_attributes = True`, esos valores deben ser inyectados manualmente por el endpoint (`drivers.py` define un mapa de categorías y rangos de tarifa por km en las líneas 43 y siguientes). Si el endpoint no los aporta, la validación del `response_model` falla.
4. **PriceHistory.tipo_via no tiene default**, a diferencia de `ServiceRequest.tipo_via` que sí tiene `default='PAVIMENTADA'`. Además ambos reutilizan el mismo nombre de enum Postgres `via_type`, lo cual es correcto pero implica que un cambio de valores afecta a las dos tablas.
5. **Falta de relationship en la mayoría de modelos.** Solo `AffiliatedCompany`, `User`, `Document` y `Vehicle` declaran relaciones. `ServiceRequest`, `DriverOffer`, `TripPassenger`, `Notification`, `Rating`, `TripStop` y `PriceHistory` tienen `ForeignKey` pero ninguna `relationship`, por lo que toda navegación entre ellas debe hacerse con `JOIN` manuales en los routers.
6. **Rating no restringe score a 1–5 a nivel de base de datos** (`Column(Integer, nullable=False)` sin `CheckConstraint`); el rango solo se valida en `RatingCreate` con `Field(..., ge=1, le=5)`. Tampoco hay restricción única (`request_id`, `rater_id`) que impida calificar dos veces el mismo viaje.

Seguridad

1. **SECRET_KEY tiene un fallback en texto plano** (`"una_clave_muy_secreta_para_desarrollo"`). Si la variable de entorno no está definida en producción, la aplicación arranca igual firmando tokens con una clave pública conocida.
2. **CORS con allow_origins=["*"] junto a allow_credentials=True** es una combinación que los navegadores rechazan (con credenciales, el origen comodín no es válido) y que además es permisiva de más para producción. La solicitud original pedía "orígenes permitidos": aquí es literalmente `["*"]`, no hay lista blanca.

3. **`datetime.utcnow()` está deprecado** en Python 3.12+ (`create_access_token`); además, `ServiceRequestCreate.validate_dates` (la versión muerta) compara `datetime.now()` naive contra un `departure_time` que puede venir con zona horaria, lo que lanzaría `TypeError` si esa validación llegara a ejecutarse.
4. **`Base.metadata.create_all(bind=engine)` en producción:** no hay migraciones (no aparece Alembic en `requirements.txt`). `create_all` solo crea tablas nuevas, nunca altera columnas existentes, por lo que cualquier campo añadido a un modelo ya desplegado no llegará a la base de datos.

Versiones sospechosas en `requirements.txt`

1. **`starlette==1.1.0` y `reportlab==5.0.1`** no corresponden a versiones publicadas de esas librerías en el rango habitual (Starlette va por la serie 0.x; ReportLab por la 4.x). `pycparser==3.0` y `cryptography==48.0.0` también están muy por delante de lo esperable. Vale la pena verificar que el archivo se generó con un `pip freeze` real y que estas versiones se resuelven al instalar.
2. **`PyMySQL==1.2.0`** es dependencia muerta (ver punto 6), y **`supabase==2.15.1`** (cliente HTTP de Supabase) coexiste con el acceso directo por SQLAlchemy/psycopg2 — habría que confirmar si se usa realmente (por ejemplo para Storage de archivos) o si es otro residuo.agentId: ac3dffff1ed359c804 (use SendMessage with to: 'ac3dffff1ed359c804', summary: '<5-10 word recap>' to continue this agent) subagent_tokens: 92067 tool_uses: 11 duration_ms: 247410

Backend · endpoints

Código muerto y contradicciones dentro de los cuatro archivos

1. **`POST /users/register` devuelve el objeto ORM completo, sin `response_model`.** Eso incluye `password_hash` en la respuesta HTTP del registro. Es el único endpoint del proyecto que expone el hash. Compárese con `GET /users/me`, que sí declara `schemas.UserResponse` y por tanto lo recorta.
2. **`upload_to_supabase (drivers.py): el docstring contradice al código.`** Dice “*devuelve la URL firmada (expira en 15 min)*”, pero el código llama `create_signed_url(file_path, expires_in=31536000)`, es decir un año — coherente con el comentario de la línea siguiente (`# URL firmada con 1 año de validez para documentos legales`) pero no con el docstring.
3. **`app/audit.py` documenta la acción `REGISTER_DRIVER` en su docstring (“Conductor envía documentos”), pero `POST /drivers/register-details` no llama a `registrar_log` en ningún punto.** Esa acción nunca aparece en la tabla `AuditLog`, y por tanto tampoco en `GET /admin/logs`. Lo mismo ocurre con `POST /drivers/upload-runt`, `PATCH /drivers/vehicle` y `PATCH /drivers/location`, que tampoco auditan.
4. **`GET /drivers/earnings`: el docstring está desactualizado.** Dice “*La ‘calificación promedio’ del criterio de aceptación queda pendiente: todavía no existe HU46 (Calificaciones bidireccionales / SCRUM-194), así que el frontend debe mostrar ‘Próximamente’ para ese dato por ahora*”, pero justo debajo el mismo endpoint calcula `calificacion_promedio` y `calificacion_cantidad` con datos reales de `Rating` (bajo el comentario `# HU46 – ahora que existen calificaciones reales (Rating), las usamos aquí`).
5. **`create_service_request (service_requests.py)` usa `RADIO_VISIBILIDAD_KM`, `_distancia_km`, `_match_comodidades` y `crear_notificacion` antes de que estén definidas en el archivo.** Funciona porque Python resuelve los nombres globales en tiempo de ejecución, no en tiempo de definición,

pero dificulta la lectura: la constante que se asigna en la línea 86 está definida en la línea 192, y `crear_notificacion` en la 1022.

6. **`schemas.ServiceRequestCreate` está definida dos veces** (líneas 73 y 101 de `schemas.py`). La segunda definición **sobrescribe silenciosamente a la primera**, así que el `model_validator` de la primera (con las validaciones de fecha y de `ROUND_TRIP`) es código muerto: nunca se ejecuta. El validador vigente es el segundo, que además tiene el comentario `# ... (mantén tu lógica de validación de fechas anterior)` en lugar de esa lógica — es decir, **el esquema vigente no valida fechas**; esas validaciones existen únicamente duplicadas a mano dentro de `create_service_request`.
7. **`_match_comodidades(db, conductor, service_request)` recibe `db` como primer parámetro y hace una consulta a `Vehicle` por cada conductor evaluado.** En `create_service_request` se llama dentro de la clave de `sorted()`, y en `GET /pending` una vez por solicitud: es un patrón N+1 de consultas.
8. **Notificaciones asimétricas en el flujo de negociación:** – `POST /{request_id}/offers` **no notifica al pasajero** de que llegó una oferta, aunque el enum `Notification.type` incluye un valor `'NEW_OFFER'` que **no se usa en ningún punto del código**. – `PATCH /offers/{offer_id}/resolve` con `ACCEPT` rechaza en cascada las demás ofertas pero **no notifica a esos conductores**, mientras que `PATCH /offers/{offer_id}/accept` sí lo hace en un caso equivalente.
9. **`resolve_counter_offer` no comprueba que `service_request` no sea `None`** antes de usar `service_request.status` y `service_request.passenger_id`. Si la solicitud asociada hubiera desaparecido, el endpoint fallaría con un `AttributeError` no manejado (500 sin cabeceras CORS, el mismo síntoma que el comentario de `/drivers/earnings` describe como problema ya vivido).
10. **Comentarios de ruta desalineados con el prefijo real** en `service_requests.py`: los comentarios dicen `# SCRUM-78 – PATCH /api/offers/{offer_id}/counter-offer` y `# SCRUM-81 – PATCH /api/offers/{offer_id}/accept`, pero como el router tiene `prefix="/api/service-requests"`, las rutas reales son `/api/service-requests/offers/{offer_id}/counter-offer` y `/api/service-requests/offers/{offer_id}/accept`.
11. **Comentarios huérfanos / mal colocados en `service_requests.py`:**
 - Línea 458-459: `# SCRUM-77 – GET /api/service-requests/{request_id}/offers` – El pasajero ve todas las ofertas de su viaje con datos del conductor está colocado inmediatamente antes de `get_driver_trip_history`, que no es ese endpoint (la definición real de `get_offers_for_request` está en la línea 777).
 - Línea 774-775:
 - `# HU17 – PATCH /api/service-requests/{request_id}/start` – Conductor inicia el viaje: `ASSIGNED → IN_PROGRESS` precede a `get_offers_for_request`, no a `start_trip`.
 - Líneas 1161-1162: `# SCRUM-83 – GET /api/service-requests/driver/my-offers` – El conductor ve sus negociaciones activas (SCRUM-83) queda suelto, sin ninguna función debajo (el endpoint real está en la línea 513).
 - Línea 328: `# [...] esto NO no depende de ningún estado de conexión` — doble negación por error de tipeo.
12. **Duplicación de la tabla de categorías de vehículo.** `drivers.RANGOS_CATEGORIA` / `drivers.calcular_categoria` y `service_requests._RANGOS_CATEGORIA_VEHICULO` / `service_requests._categoria_vehiculo` mantienen los mismos umbrales por separado; el propio

comentario lo admite ("*misma tabla de rangos que app/routers/drivers.py*"). Si se cambia una, la otra queda desincronizada.

13. **Duplicación de la lógica de calificaciones.** El mismo agregado `avg(Rating.score) / count(Rating.rating_id)` filtrado por `rated_id` está escrito cuatro veces: en `auth._resumen_calificaciones`, en `service_requests._promedio_calificacion`, e inline dentro de `drivers.get_driver_earnings` y `drivers.get_driver_public_profile`.
14. **GET /admin/logs no acota limit.** Un `?limit=1000000` es aceptado tal cual.
15. **app/routers/routes.py existe pero no está registrado.** Define `router = APIRouter(prefix="/api/routes", tags=["Routes"])` con `POST /api/routes/compute` (integración con Google Routes API, HU08), pero `app/main.py` solo importa e incluye `auth`, `drivers`, `service_requests`, `admin`. **Ese endpoint es inalcanzable;** el frontend calcula la ruta por su cuenta (`src/Dashboard.jsx` menciona la Routes API en un comentario).

Endpoints sin consumidor en el frontend

Buscando las rutas en `turify-frontend/src`, estos endpoints del backend no tienen ninguna llamada:

- `GET /drivers/my-documents` — el admin usa `/admin/drivers/pending`, que ya trae los documentos embebidos, y el conductor usa `/users/me/profile` o `/drivers/registration-status`.
- `GET /admin/drivers/{user_id}/documents` — redundante con la relación `documents` de `/admin/drivers/pending`.
- `GET /admin/documents/{document_id}/file` — el proxy de descarga no se invoca; el frontend abre directamente la `file_url` firmada de Supabase.
- `GET /api/service-requests/demand-zones` — la HU23 (mapa de zonas de demanda) no tiene pantalla construida.
- `GET /api/service-requests/{request_id}/status` — el estado se obtiene a través de `/assigned` y `/driver/my-offers`.
- `GET /api/service-requests/{request_id}/rating` (el GET) — el frontend usa el campo `ya_califico` que ya viene precalculado en `/assigned`, `/driver/history` y `/driver/my-offers`.
- `POST /api/routes/compute` — además de no estar registrado en `main.py`, tampoco se llama.

Huecos de autorización que conviene conocer

- **POST /api/service-requests/ no valida el rol:** un usuario con rol `DRIVER` puede publicar viajes como pasajero.
- **POST /drivers/register-details no valida el rol:** un `ADMIN` podría enviar documentos de conductor.
- **GET /api/service-requests/{request_id}/status acepta a cualquier DRIVER**, no solo al asignado al viaje.
- **PATCH /api/service-requests/{request_id}/cancel y PATCH /offers/{offer_id}/counter-offer no exceptúan al ADMIN**, a diferencia de `/offers/{request_id}/offers` (GET), `/offers/{request_id}/receipt` y `/offers/{request_id}/passengers` (GET), que sí lo hacen. El criterio de "el admin puede ver/hacer todo" no es uniforme en el router.
- **POST /offers/{request_id}/offers no impide ofertas duplicadas** del mismo conductor sobre el mismo viaje: puede crear tantas filas `DRIVER_OFFERED` como quiera, y todas aparecerán en la lista que ve el pasajero. `agentId: a8f02abeffa286598` (use `SendMessage with to: 'a8f02abeffa286598'`, `summary:`

'<5-10 word recap>' to continue this agent) subagent_tokens: 153667 tool_uses: 17 duration_ms: 506669

Frontend · pantalla del pasajero

Código muerto o sin uso

1. **SkeletonTarjetaConfirmado** se importa desde `./Skeleton` y nunca se usa.
2. **MarcaTurify** se importa desde `./diseno` y nunca se usa (solo se usa `LogoWordmark`).
3. **trazarRutaConductor** (líneas 337–346) está definida y jamás se llama.
4. **inputStyle** (línea 929) se define y nunca se aplica.
5. **oferta.vehiculo** se renderiza en la línea 1792, pero `cargarMisViajes` **nunca crea ese campo** al mapear las ofertas → siempre `undefined` (div vacío).
6. **oferta.created_at** se comprueba en la línea 1834, pero tampoco se mapea → el bloque "Oferta enviada ..." nunca se muestra y, por tanto, **tiempoRelativo es efectivamente código muerto**.
7. El campo **estado** se calcula tanto para las ofertas (`estado: o.status`) como para los viajes (`'Oferta recibida' / 'Buscando conductor'`) y **nunca se lee** en el render.
8. `handleAceptarOferta(viajeId, ofertaId)` recibe `viajeId` y no lo usa.
`handleUbicacionActual({ texto, coords })` desestructura `texto` y no lo usa.

Bugs probables

1. **Fugas de suscripción de Supabase.** En los efectos de las líneas 508 y 825, el `return () => supabase.removeChannel(channel)` está **dentro del `.then()` del import dinámico**, no en el cuerpo del `useEffect`. React nunca recibe esa función de limpieza, así que los canales `notif-dashboard-*` y `viajes-pasajero-*` no se cierran (a diferencia del efecto de tracking, que sí lo hace bien con el patrón `cancelado + array canales`).
2. **fuecEnviado no persiste.** Se calcula solo en memoria y el backend no informa si ya se registraron ocupantes, así que tras un refresco de página el botón vuelve a decir "Registrar ocupantes del viaje" aunque ya se hayan enviado.
3. **handleRechazarOferta es puramente cosmético:** no llama a ningún endpoint, y el siguiente `cargarMisViajes()` (Realtime o polling) hace reaparecer la oferta rechazada.
4. **marcarTodasLeidas dispara N peticiones PATCH** en paralelo, una por notificación no leída; no hay endpoint masivo.
5. La dependencia del efecto de tracking es `[JSON.stringify(...)]`, un patrón que funciona pero que ESLint marca y que recalcula el filtro y la serialización en cada render.
6. `BOUNDS_ANTIOQUIA` está **dentro** del componente `Dashboard` (se recrea en cada render) y además está **duplicado** literalmente en `InputDireccion.jsx`.
7. `SelectorPasajero` se define dentro del cuerpo de `Dashboard`, por lo que React lo trata como un tipo de componente nuevo en cada render (remonta el subárbol en vez de actualizarlo).
8. En `handleContinuarConDireccionManual`, la rama del destino lee `coordsBuffer.current` sin comprobar que tenga contenido; si `lat1/lon1` fueran `undefined` se escribiría `origen: { lat: undefined, lng: undefined }` en `datosMapa`.
9. `geocodificar` devuelve `{ lat, lon }` (con `lon`) mientras que el resto del archivo trabaja con `{ lat, lng }`; la conversión se hace a mano en cada punto de uso, lo que es fácil de confundir.

10. En el mini mapa de tracking se usa `window.google.maps.SymbolPath.CIRCLE` directamente en el `icon`; está protegido por `mapsLoaded` en la condición del bloque, pero es un acceso a `window.google` sin comprobación propia.
11. El botón submit del formulario **no se deshabilita** mientras `cargandoMapa` es `true`; solo cambia el texto, así que se puede reenviar la búsqueda.

Inconsistencias de estilo y contenido

1. **BRAND_GREEN se declara en la línea 14, antes del `import { T, ... }` de la línea 16.** Funciona por el hoisting de los imports ES, pero rompe la lectura y es frágil.
2. En `Dashboard.jsx` `BRAND_GREEN = T.ruta`; en `InputDireccion.jsx` y `SelectorFechaHora.jsx` es la cadena literal `'var(--t-ruta)'`. Mismo resultado, tres declaraciones distintas.
3. **Voseo mezclado con tuteo** en la interfaz: "Tocá en el mapa...", "Tocá el mapa...", "probá con:" conviven con "Ingresa el precio...", "Completa nombre y...", "Busca una ruta..."
4. **Colores hexadecimales sueltos** conviviendo con los tokens `var(--t-*): #C2410C` (badge de notificaciones), `#2563eb`, `#1e40af`, `#3730a3`, `#e0e7ff`, `#ddd6fe`, `#d97706`, `#f59e0b`, `#08210F`, `rgba(34,197,94,...)` en el modal FUEC y en las clases `.fuec-*`. En `SelectorFechaHora.jsx`, además, bordes fijos `#e8e4db`, `#E2E4DC`, `#EDEEE8` y el trazo `#8a8578` de los SVG, que no se adaptan al tema oscuro.
5. `onMouseLeave` de las sugerencias en `InputDireccion` restaura `'#fff'` en vez de `var(--t-papel)`, así que en tema oscuro la fila queda blanca tras pasar el ratón.
6. En el `<style>` del Dashboard hay una regla `@import url('https://fonts.googleapis.com/css2?...')` **colocada en medio del bloque** (línea 1065). Según la especificación CSS, `@import` debe ir antes de cualquier otra regla, por lo que el navegador la ignora; además el archivo define `@keyframes girar` en ese bloque mientras los componentes usan `animation: 't-girar ...'` (con prefijo `t-`, presumiblemente definido en `EstilosBase`), es decir, hay dos nombres de animación de giro distintos conviviendo.
7. **Superposiciones inconsistentes:** la mayoría de modales usan `position: 'absolute'` con `height: '100vh'` (contraoferta, calificar, error de dirección, paneles laterales), mientras que el modal FUEC usa `position: 'fixed'` con `inset: 0`. Con `absolute`, si el contenedor tiene desplazamiento el backdrop puede desalinearse.
8. El texto del modal de resumen dice "Solo se te notificarán ofertas de conductores cuyo vehículo cumpla lo que marques aquí.", pero los comentarios del código (HU55/HU26) y el render de las ofertas dicen exactamente lo contrario: el filtro es flexible, se muestran todas las ofertas y solo se avisa qué comodidades faltan. **La interfaz se contradice a sí misma.**
9. `tiene_estimacion_peajes: false` queda como marcador de una integración futura (Routes API). Los comentarios sobre peajes en `trazarRutaConCoords` y en `crearViaje` dan razones distintas para no pedirlos (uno dice que requiere facturación aparte desde el backend, el otro que Google no cubre Colombia).
10. `PerfilDrawer` se monta siempre (recibe `abierto`), a diferencia del resto de superposiciones, que se montan condicionalmente dentro de `AnimatePresence`.
11. La rama de `ADMIN` devuelve `<AdminConductores />` **sin** `ToastContainer`, mientras que la de `DRIVER` sí lo incluye: un `toast` disparado en la vista de administrador no se vería.
12. `nombreDelPunto` usa la API de promesas del Geocoder (`await geocoder.geocode(...)`) mientras que `geocodificar` y `geocodificarInverso` usan la API de callbacks envuelta a mano en `new Promise`. Tres formas de hacer lo mismo en el mismo archivo.

13. El límite de 44 asientos está codificado directamente (≥ 44) en dos sitios: `actualizarPasajeros` y `SelectorPasajero`, sin una constante compartida.
14. En `SelectorFechaHora`, el filtro de horas $h * 60 + 45 \geq \text{horaMinimaHoy}$ deja seleccionable la hora mínima aunque después el filtro de minutos pueda dejar la lista de minutos **vacía** (si `horaMinimaHoy` cae, por ejemplo, en el minuto 50 de esa hora); en ese caso el `<select>` de minutos se queda sin opciones.
15. La errata `mismodDia` (por `mismoDia`) se repite en tres usos.

Cosas que el código no permite determinar

- El contrato exacto de los endpoints (campos devueltos por `/pending`, `/offers`, `/assigned`, forma de comodidades, significado preciso de `recomendado` o `conductores_notificados`) solo se infiere del consumo en el frontend; no está documentado aquí.
- El header `'ngrok-skip-browser-warning': 'true'` sugiere que el backend se expone por ngrok en desarrollo, pero eso no se confirma en este archivo.
- No queda claro por qué `estado` (de viajes y de ofertas) se calcula si nunca se lee; puede ser un remanente de una versión anterior de la interfaz. `agentId: a914ee57a8a0259ff` (use `SendMessage with to: 'a914ee57a8a0259ff', summary: '<5-10 word recap>' to continue this agent`)
`subagent_tokens: 151927 tool_uses: 11 duration_ms: 390067`

Frontend · pantallas del conductor

PanelConductor.jsx

1. **Restos del estado "conectarse" eliminado.** Varios comentarios y textos siguen refiriéndose a un modo conectado/desconectado que ya no existe: – Comentario de la cabecera (línea ~628): "...y el control de conexión, que es lo único que el conductor toca antes de empezar a trabajar" — ese control ya no está. – Comentario del mapa (línea ~1298): "Mientras estás desconectado no se ven solicitudes puntuales (eso solo aparece al conectarte), pero sí las zonas con más flujo de pedidos" — no hay estado de desconexión ni se pintan "zonas de demanda"; solo hay pines por solicitud. – Comentario de `cargarSolicitudes`: menciona "conectarse" como uno de los disparadores de la carga visible. – `actualizarUbicacionBackend` se documenta como "Envía `current_lat/current_lng/is_online`", pero ningún llamador envía `is_online`.
2. **Fuga de suscripción de Supabase.** En el efecto #1, el `return () =>`
`supabase.removeChannel(channel)` está dentro del callback del `.then()` de la importación dinámica, no en el cuerpo del `useEffect`. React nunca recibe esa función de limpieza, así que el canal `notif-conductor-<user_id>` no se desuscribe al desmontar ni cuando cambian `token` / `usuario`; se pueden acumular canales.
3. **estadoConfig es código muerto:** se define en la línea 565 y no se invoca en ninguna parte. Además contiene una clave duplicada en la entrada `RECHAZADO: { ..., Ico: IconEquis, Ico: IconEquis, label: 'Rechazado' }` (JavaScript se queda con la segunda; es inofensivo pero delata un copiar-pegar).
4. **cfgViaje es código muerto:** dentro del `map` de la pestaña "Ofertas" se construye el objeto `cfgViaje` con `ASSIGNED / IN_PROGRESS / COMPLETED` y nunca se lee; los estados del viaje se renderizan con bloques `if` sueltos.

5. **esRechazado no se usa:** se calcula (`viaje.status === 'REJECTED'`) pero nunca se consulta en el JSX.
6. **Importaciones sin uso:** `Rotulo` e `IconRecibo` se importan de `./diseno` y no aparecen en ninguna parte del archivo.
7. **Inconsistencia entre la oferta optimista y el render.** `enviarOferta` inserta en `viajesActivos` un objeto con la clave `estado: 'ESPERANDO'`, pero la pestaña "Ofertas" lee `viaje.status` (valores `DRIVER_OFFERED`, `PASSENGER_COUNTER_OFFERED`, `ACCEPTED`, `REJECTED`). Ese elemento cae, por tanto, en el fallback `{ bg: 'var(--t-niebla-2)', color: 'var(--t-piedra)', label: viaje.status }` y muestra `undefined` como etiqueta hasta el siguiente `cargarViajesActivos()`. Además, ese objeto no lleva `offer_id`, que es lo que se usa como `key` del `map`.
8. **Comentario de sección mal ubicado:** sobre el bloque `{pestañaActiva === 'historial' && ...}` (línea 1135) el comentario dice `{/* PESTAÑA MI VEHÍCULO – HU55 (SCRUM-207) */}`. La pestaña de vehículo empieza más abajo, en la línea 1198, sin comentario propio. También hay dos comentarios encadenados sobre la pestaña de ofertas (`PESTAÑA VIAJES ACTIVOS - SCRUM-89` y `PESTAÑA MIS OFERTAS – SCRUM-83` y `SCRUM-84`).
9. **Contadores de pestaña fijos en 0:** `historial`, `ganancias` y `vehiculo` declaran `count: 0`, así que su burbuja nunca aparece. Es intencional o pendiente, no se puede saber desde el archivo.
10. **Pines del mapa sin filtrar:** `solicitudesConUbicacion` deriva de `solicitudes`, no de `solicitudesFiltradas`. Con filtros activos, el mapa muestra viajes que la lista ha ocultado.
11. **Peticiones en ráfaga en marcarTodasLeídas:** dispara un `PATCH` por notificación no leída, sin agrupar ni esperar. Con muchas notificaciones puede ser costoso.
12. **marcarLeida no comprueba res.ok:** marca como leída en el estado local aunque el servidor haya fallado.
13. **Colores fuera del sistema de tokens:** `'#C2410C'` (errores del radar, ganancias y vehículo), `'#3b82f6'` y `'#2563eb'` (contraoferta e "Iniciar viaje"), `'#f59e0b'` (pines), `'#e0e7ff'` y `'#fff'` en varios sitios se escriben literalmente en vez de usar `T.*` o `var(--t-*)`; con el tema oscuro pueden quedar desalineados (por ejemplo `backgroundColor: '#fff'` en el cuerpo de las tarjetas del radar y en las notificaciones leídas).
14. **Redundancia en la carga de viajesActivos:** el efecto #6 (al cambiar a la pestaña "Ofertas") y el efecto #10 (sondeo de fondo cada 15 s) llaman al mismo endpoint; al entrar en la pestaña se dispara una petición extra.
15. **Reportes de ubicación superpuestos:** durante un viaje `IN_PROGRESS` conviven el intervalo de 20 s (efecto #3) y el de 3 s (efecto #11); ambos hacen `PATCH /drivers/location`.
16. **Voseo mezclado.** La interfaz alterna registros: "Recibís", "Activá tu ubicación", "Finalizó", "Volvé a entrar" (voseo rioplatense) con "Elige de 1 a 5 estrellas", "Verifica los valores", "Aún no has enviado ofertas", "Cuéntanos más" (tuteo). No es un fallo funcional, pero es una inconsistencia de redacción.
17. **clave FUEC:** el modal de ocupantes se rotula "Documento de viaje" y las clases CSS se llaman `.fuec-input` / `.fuec-select`, pero esas dos clases **no se aplican a ningún elemento** del archivo; son estilos huérfanos, probablemente de un formulario FUEC que ya no está.

PerfilDrawer.jsx

1. **No existe subida de foto de perfil.** El archivo solo muestra `perfil.profile_photo_url`. Si la referencia de repaso menciona "subida de foto" en este componente, la funcionalidad no está aquí (sí está la carga inicial en `FormularioConductor.jsx` mediante el `DropZone` `profile_photo`).

2. **Orden de declaración confuso.** `IconIdea` e `IconEmpresa` se definen en las líneas 2–3, *antes* del `import { ..., Icono, ... } from './diseno'` de la línea 10. Funciona porque los `import` se elevan y los iconos solo se evalúan al renderizar, pero se lee como un error.
3. **Manejo de errores desparejo.** `guardarTelefono` usa `alert()` del navegador (el resto del proyecto ya migró a toasts o banners); `cargarPerfil` solo escribe en la consola si la respuesta no es `ok`, dejando el cajón en blanco sin explicación; `guardarTelefono` lanza `new Error()` sin mensaje, de modo que el `detail` del backend se pierde.
4. **La sección "Seguridad" no exige perfil.** Se renderiza con `!cargando && seccion === 'seguridad'`, sin `&& perfil`, a diferencia de "perfil" e "historial". No rompe nada (solo usa `passwordForm`), pero es una asimetría.
5. **Texto de "ofertas activas" solo para pasajeros** y en voseo ("Tenés N ofertas activas"), mientras que otras partes del mismo cajón tutean ("Aún no has solicitado viajes", "No tienes documentos registrados").
6. **ESTADO_VIAJE.PENDING incluye Ico: IconReloj**, pero el fallback del historial (`{ color: 'var(--t-piedra)', label: v.status }`) no trae `Ico`; por eso el JSX comprueba `est.Ico &&`. Correcto, pero conviene notarlo al leer.
7. **Claves de documento sin tildes:** `'Licencia de Conduccion'`, `'Tarjeta de operacion'`, `'Tecnomecanica'`, `'Seguros Contractual y extracontractual'`. Coinciden con las de `FormularioConductor.jsx` salvo en los textos mostrados: aquí `'Tecnomecanica'` se muestra como "Tecnomecánica" y `'Seguros...'` como "Seguros", mientras que en el formulario son "Revisión Tecnomecánica" y "Seguros (Contractual / Extracontractual)". Dos diccionarios paralelos (`LABEL_DOC` y `ETIQUETA_DOCUMENTO`) que pueden divergir.
8. **ICONO_DOC incluye 'RUNT', ETIQUETA_DOCUMENTO del formulario no.** Coherente con que el `RUNT` se sube solo desde aquí, pero significa que en `VistaEstadoDocumentos` un documento `RUNT` se mostraría con su tipo crudo.
9. **El bloque de RUNT usa una IIFE dentro del JSX** (`{(() => { ... })() }`). Funciona, pero es un patrón que dificulta la lectura; podría ser una variable derivada antes del `return`.
10. **El botón "Cerrar sesión" no envuelve el icono y el texto en un contenedor flex**, a diferencia del resto del archivo; el `IconSalir` queda pegado al texto sin `gap`.

FormularioConductor.jsx

1. **Lista de empresas codificada en duro:** `<option value="1">Departour</option>` y `<option value="2">Transporte Real</option>`. Los ids 1 y 2 se envían tal cual al backend. Si aparece una tercera empresa hay que tocar el código.
2. **documentos_rechazados puede no existir en el flujo de error.** Si `consultarEstado` falla, `estadoData` queda en `null` y se muestra el formulario sin banner de rechazo; el usuario puede reenviar sin saber qué le rechazaron.
3. **Los documentos no son obligatorios.** Los cinco `DropZone` no tienen `required` (un `input type="file"` oculto con `required` daría problemas de accesibilidad, pero no hay validación JS que lo sustituya). Se puede enviar la solicitud sin adjuntar nada; el rechazo llega solo desde el servidor. La barra de progreso sugiere lo contrario.
4. **Sin validación de tipo ni de tamaño de archivo.** El `DropZone` no declara `accept` y no comprueba el MIME. Se puede soltar cualquier archivo (por ejemplo un `.zip`) y se enviará. Compárese con `PerfilDrawer.jsx`, donde el campo del `RUNT` sí declara `accept=".pdf,.jpg,.jpeg,.png,.webp"`.

5. **El drop acepta múltiples archivos pero solo se guarda el primero:** `onDrop` pasa el `FileList` completo y `handleInputConductor` toma `files[0]`. Silenciosamente ignora el resto.
6. **`errorData` puede fallar.** En `enviarFormularioConductor`, `await respuesta.json()` se llama sin `catch` sobre una respuesta de error; si el servidor devuelve un cuerpo vacío o HTML (por ejemplo un 502 del proxy), el `.json()` lanza un `SyntaxError` y el mensaje que ve el usuario es el del parseo, no el del error real. Los `cargar*` de `PanelConductor.jsx` sí blindan esto con `.catch(() => ({}))`.
7. **Booleanos como texto en el `FormData`.** `formData.append('tiene_ac', formConductor.tiene_ac)` envía los strings `"true"` / `"false"`. Depende del backend interpretarlos; no se puede verificar desde el frontend.
8. **`estadoData` sintético tras el envío.** Se fija `{ estado: 'PENDIENTE', mensaje: ..., documentos: null }` sin volver a consultar el servidor; la clave `mensaje` **no la lee** `VistaEstadoDocumentos` (que usa su propio `config[estado].descripcion`), así que es un campo muerto.
9. **Cabecera se declara después de su primer uso** (`VistaEstadoDocumentos` la usa en la línea 154 y se define en la 198). Funciona, pero conviene reordenar.
10. **`acepta_mascotas` sin cargo asociado.** En el registro se pregunta si acepta mascotas, pero no el `cargo_mascota`; ese campo solo aparece en la pestaña "Vehículo" de `PanelConductor.jsx`. Tampoco se pregunta `acepta_menores_2_anos` ni `capacidad_real`, que sí existen en el `PATCH /drivers/vehicle`.
11. **`BRAND_GREEN` es `T.ruta` (valor literal), no `'var(--t-ruta)'`** como en los otros dos archivos. En una interfaz con tema claro/oscuro esto significa que el verde de este formulario no reacciona al cambio de tema (aunque `T` puede ya resolver variables CSS; eso habría que confirmarlo en `diseño.jsx`, que no forma parte de este encargo).
12. **Sin `useToast`.** A diferencia de `PanelConductor.jsx`, aquí todos los mensajes son banners en línea. Consistente dentro del archivo, pero inconsistente con el resto de la aplicación.

Transversal

- **Ninguna petición usa `AbortController`:** si el componente se desmonta mientras hay un `fetch` en vuelo, el `setState` posterior queda huérfano.
- **La cabecera `'ngrok-skip-browser-warning': 'true'` está presente en todas las llamadas**, lo que sugiere que el backend se expone por un túnel ngrok en desarrollo; en producción es inocua pero innecesaria.
- **Los tres archivos usan estilos en línea casi por completo**, con solo pequeños bloques `<style>` para pseudo-clases (`:focus`, `:hover`, `::placeholder`) y media queries, que no pueden expresarse en línea. `agentId: acdb0689fe7e2e20e` (use `SendMessage` with to: `'acdb0689fe7e2e20e'`, summary: `'<5-10 word recap>'` to continue this agent) `subagent_tokens: 167676` `tool_uses: 9` `duration_ms: 452805`

Frontend · sistema de diseño y pantallas auxiliares

Duplicados de archivo por diferencia de mayúsculas. El directorio `src/` contiene pares que solo se distinguen por la caja de una letra, y con contenidos y fechas distintas:

- `AdminConductores.jsx` (2 sep 20:11) y `Adminconductores.jsx` (2 sep 06:24) — **mismo tamaño exacto**, 32 969 bytes.
- `PanelConductor.jsx` y `Panelconductor.jsx` — ambos 95 753 bytes.

- `AuthProvider.jsx` y `Authprovider.jsx` — ambos 1 793 bytes.
- `InputDireccion.jsx` (11 943 bytes) e `Inputdireccion.jsx` (11 234 bytes) — **estos sí difieren**.

En Linux son archivos distintos; en macOS o Windows (sistemas de archivos insensibles a mayúsculas) esto es una fuente segura de conflictos e importaciones ambiguas. Conviene eliminar las copias en minúscula.

Componentes y exportaciones sin usar:

- `PerfilConductorPublico.jsx` no se importa en ningún archivo del proyecto. Es código muerto salvo que se planee reintroducirlo.
- `Dato` (de `diseño.jsx`) no se usa en ninguna parte: donde hace falta un dato monoespaciado, cada pantalla escribe `fontFamily: T.dato` a mano (`AdminConductores` para el teléfono, `AdminLogs` en `celdaDato`, `Registro` para el teléfono).
- `IconSenal` está exportado y no se usa en ningún archivo.
- `SkeletonBlock` está exportado y no se usa en ningún archivo.
- `MarcaTurify` se **importa** en `Dashboard.jsx` pero no aparece en su JSX: `import` sin uso.
- `LogoMonograma` y `LogoBifurcacion` se importan en `Login.jsx` solo como alternativas documentadas (`// Logo en uso. Alternativas: LogoMonograma | LogoBifurcacion`); nunca se renderizan. Es intencionado, pero un linter lo marcará.
- `BRAND_GREEN` en `PerfilConductorPagina.jsx` se define y nunca se usa.
- El `export default` de `Skeleton.jsx` (objeto con los cinco componentes) no lo importa nadie.

Props recibidas y no usadas: `VisorPDF` y `ModalDocumento` en `AdminConductores.jsx` reciben `token` (y `VisorPDF` también `documentId`) sin utilizarlos. Probablemente son restos de una versión que descargaba el documento autenticado en vez de enlazar la URL pública de Cloudinary.

Inconsistencias de tokenización:

- La variante `inactivo` de `Boton` escribe `'var(--t-niebla-2)'` literal en vez de `T.niebla2`.
- `BotonCentrarMapa` usa `borderRadius: '10px'` en vez de `T.rControl`, que vale exactamente eso.
- `PerfilConductorPagina.jsx` y `PerfilConductorPublico.jsx` casi no usan `T`: escriben `'var(--t-...)'` a mano y repiten `"'DM Sans', system-ui, sans-serif"` en vez de `T.ui`. `PerfilConductorPagina` importa de `diseño` únicamente `IconEstrella`.
- No hay tokens de sombra: cada `boxShadow` es un `rgba(...)` literal e irrepetible.
- Varios valores `rgba` de marca están cableados al **modo claro**: `rgba(22,163,74,.12)` (el foco de los campos en `Login` y `Registro`) es `#16A34A`, que en modo oscuro es `#22C55E`. Igual con `rgba(34,197,94,0.35)` en los gradientes de perfil y `rgba(34,197,94,0.2)` en `SkeletonDashboard`.

Emojis: el sistema declara «sin emojis» y `PerfilConductorPagina` lo repite en su comentario, pero `PerfilConductorPublico` conserva `Cargando perfil...`. El botón de cerrar de `Toast`, `PerfilConductorPublico` y `ModalDocumento` usa el carácter `×` en lugar de `IconEquis` (en `ModalDocumento` sí usa `IconEquis`).

IconEmpresa está en el bloque «Comodidades del vehículo» de `diseño.jsx`, pero dibuja un edificio y se usa para la empresa afiliada, no para una comodidad. Está agrupado donde no corresponde.

Duplicación de `IconIdea`. El icono existe exportado en `diseño.jsx`, pero `InputDireccion.jsx` y `PerfilDrawer.jsx` lo **redefinen localmente** con exactamente el mismo `path`, en lugar de importarlo.

Voseo mezclado en el copy. La interfaz alterna tuteo y voseo, a veces en la misma pantalla: `Login` dice «Entrá a tu cuenta», «Vos decidís», pero también «Verifica que el backend esté activo»; `PerfilConductorPagina` dice «Verifica tu conexión». No es un error técnico, pero es una inconsistencia de tono visible.

Detalles funcionales que conviene conocer:

- `AuthProvider` considera **vigente** un token sin claim `exp` (`if (!decoded?.exp) return false`).
- `Login.jsx` guarda el token en `localStorage` **y además** llama a `onLoginSuccess`, que a su vez hace que `iniciarSesion` vuelva a guardarlo. Escritura duplicada, inofensiva pero redundante.
- El valor del `AuthContext.Provider` y la función `alternar` de `useTema` se recrean en cada render (sin `useMemo` / `useCallback`), lo que provoca re-renders de todo el árbol consumidor.
- `RutaAdmin` devuelve `null` mientras `usuario` carga: la pantalla queda en blanco, sin indicador. Si `/users/me` falla por red sin que el token esté expirado, `usuario` se queda en `null` para siempre y la ruta de admin nunca se resuelve.
- Tras iniciar sesión, `App.jsx` navega siempre a `/dashboard`, también para un ADMIN, que debe llegar a `/admin/conductores` manualmente.
- No hay ruta comodín `*`: una URL inexistente renderiza una página vacía.
- `AdminLogs` traga los errores de red en silencio (`catch { setLogs([]); }`), de modo que un backend caído se ve igual que una bitácora vacía.
- `AdminConductores` actualiza `conductores` de forma optimista tras verificar un documento en vez de recargar; el `useEffect` con dependencia `[conductores]` re-sincroniza el conductor seleccionado. Funciona, pero acopla dos efectos.
- El `useEffect` de `ToastItem` tiene array de dependencias vacío mientras usa `duration`, `id` y `onRemove`: es intencional (el toast se configura una sola vez), pero React Hooks lo señalará.
- `getPDFcomoImagen` y `getUrlPreview` implementan lógicas de transformación de Cloudinary parcialmente solapadas y se aplican en cadena (`ModalDocumento` calcula `urlPreview` con la segunda y se la pasa a `VisorPDF`, que le aplica la primera). Funciona porque ambas son idempotentes, pero está enredado.
- `Registro.jsx` pasa siempre `false` como argumento `hayError` al campo *Teléfono*, así que nunca podría marcarse en rojo (coherente con que no se valida).
- `useTema` se invoca por separado en cada pantalla; no hay estado de tema compartido en React. Funciona porque la fuente de verdad real es el atributo `data-tema` del `<html>` más `localStorage`, pero `Login` y `Registro` no montan `BotonTema` ni llaman a `useTema`: si el usuario nunca ha guardado preferencia, esas dos pantallas quedan a merced de `prefers-color-scheme` sin que el atributo esté puesto. `agentId: ae15158b751884653` (use `SendMessage` with to: 'ae15158b751884653', summary: '<5-10 word recap>' to continue this agent) `subagent_tokens: 172957` `tool_uses: 21` `duration_ms: 450013`